

LÊ TRỌNG AN

KỸ THUẬT CƠ ĐIỆN TỬ

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC PHENIKAA**



ĐỒ ÁN TỐT NGHIỆP

**THIẾT KẾ HỆ THỐNG IOT CHO ỨNG DỤNG
QUẢN LÝ
PHƯƠNG TIỆN GIAO THÔNG TRONG LĨNH
VỰC
CHO THUÊ XE TỰ LÁI**

Sinh viên: Lê Trọng An

Mã số sinh viên: 21010389

Khóa: K15

Ngành: Kỹ thuật Cơ Điện tử

Hệ: Đại học chính quy

Giảng viên hướng dẫn: TS. Nguyễn Đức Nam

Hà Nội – 2026

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC PHENIKAA**



ĐỒ ÁN TỐT NGHIỆP

**THIẾT KẾ HỆ THỐNG IOT CHO ỨNG DỤNG QUẢN LÝ
PHƯƠNG TIỆN GIAO THÔNG TRONG LĨNH VỰC
CHO THUÊ XE TỰ LÁI**

Sinh viên: Lê Trọng An

Mã số sinh viên: 21010389

Khóa: K15

Ngành: Kỹ thuật Cơ Điện tử

Hệ: Đại học chính quy

Giảng viên hướng dẫn: TS. Nguyễn Đức Nam

Hà Nội – 2026

PHENIKAA UNIVERSITY
FACULTY OF MECHANICAL ENGINEERING AND
MECHATRONICS



MEM710071 – CAPSTONE DESIGN 2
DESIGN OF AN IOT SYSTEM FOR VEHICLE
MANAGEMENT
IN SELF-DRIVE CAR RENTAL SERVICES

Semester: I Academic year: 2025–2026

Project Team:

Student name	Student ID	Cohort
Le Trong An	21010389	K15

Course Instructor: Dr. Nguyen Duc Nam

This report is submitted to the Faculty of Mechanical Engineering and Mechatronics,
School of Engineering, Phenikaa University in partial fulfillment of the requirements
of the Graduation Thesis course MEM710071.

HANOI – 2026

ĐẠI HỌC PHENIKAA
KHOA CƠ KHÍ – CƠ ĐIỆN TỬ



MEM710071
ĐỒ ÁN TỐT NGHIỆP

**THIẾT KẾ HỆ THỐNG IOT CHO ỨNG DỤNG QUẢN LÝ
PHƯƠNG TIỆN GIAO THÔNG TRONG LĨNH VỰC CHO
THUÊ XE TỰ LÁI**

Học kỳ I Năm học 2025–2026

Họ và tên sinh viên	Mã số sinh viên	Lớp
Lê Trọng An	21010389	K15- KTCĐT2

Giảng viên hướng dẫn: TS. Nguyễn Đức Nam

HÀ NỘI, 2026

NHẬN XÉT ĐỒ ÁN TỐT NGHIỆP CỦA GIẢNG VIÊN HƯỚNG DẪN

Giảng viên hướng dẫn: TS. Nguyễn Đức Nam

Khoa: Cơ khí – Cơ Điện tử.

Tên đề tài: Thiết kế hệ thống IoT cho ứng dụng quản lý phương tiện giao thông trong lĩnh vực cho thuê xe tự lái.

Sinh viên thực hiện: Lê Trọng An.

Lớp: K15-KTCĐT2.

NỘI DUNG NHẬN XÉT

I. Nhận xét Đồ án tốt nghiệp:

- Nhận xét về hình thức:
- Tính cấp thiết của đề tài:
- Mục tiêu của đề tài:
- Nội dung của đề tài:
- Tài liệu tham khảo:
- Phương pháp nghiên cứu:
- Tính sáng tạo và ứng dụng:

II. Nhận xét về tinh thần và thái độ làm việc của sinh viên:

III. Kết quả đạt được:

IV. Kết luận: Đồng ý cho bảo vệ: ☐ Không đồng ý cho bảo vệ: ☐

Hà Nội, ngày ..., tháng ... năm 20...

GIẢNG VIÊN HƯỚNG DẪN

(Ký, ghi rõ họ tên)

TS. Nguyễn Đức Nam

NHẬN XÉT ĐỒ ÁN TỐT NGHIỆP CỦA GIẢNG VIÊN PHẢN BIỆN

Giảng viên phản biện: Theo quyết định phân công phản biện của Khoa

Tên đề tài: Thiết kế hệ thống IoT cho ứng dụng quản lý phương tiện giao thông trong lĩnh vực cho thuê xe tự lái.

Sinh viên thực hiện: Lê Trọng An

Lớp: K15-KTCĐT2

Giảng viên hướng dẫn: TS. Nguyễn Đức Nam

NỘI DUNG NHẬN XÉT

I. Nhận xét Đồ án tốt nghiệp:

- Bố cục, hình thức trình bày:
- Đảm bảo tính cấp thiết, hiện đại, không trùng lặp:
- Nội dung:
- Mức độ thực hiện:

II. Kết quả đạt được:

III. Ưu nhược điểm:

IV. Kết luận: Đồng ý cho bảo vệ: ☐ Không đồng ý cho bảo vệ: ☐

Hà Nội, ngày ..., tháng ...năm 20...

GIẢNG VIÊN PHẢN BIỆN

(Ký, ghi rõ họ tên)

Theo quyết định phân công phản biện của Khoa

BIÊN BẢN ĐÁNH GIÁ ĐỒ ÁN TỐT NGHIỆP

I. Thông tin chung

- Thời gian: từ ...giờ ...đến ...giờ ...ngày .../.../20...
- Địa điểm:
- Họ và tên sinh viên: Lê Trọng An Mã số SV: 21010389
- Lớp: K15-KTCĐT2 Ngành: Kỹ thuật Cơ Điện tử Khóa: K15
- Tên đề tài: Thiết kế hệ thống IoT cho ứng dụng quản lý phương tiện giao thông trong lĩnh vực cho thuê xe tự lái.
- Giảng viên hướng dẫn: TS. Nguyễn Đức Nam

II. Thành phần hội đồng

- Chủ tịch
- Thư ký
- Giảng viên phản biện
- Ủy viên 1

III. Tổng hợp câu hỏi của thành viên hội đồng:

IV. Tổng hợp nội dung trả lời của sinh viên:

V. Nội dung đánh giá của Hội đồng

- Ý nghĩa của đồ án:
- Về nội dung, kết cấu của đồ án:
- Phương pháp nghiên cứu:
- Các kết quả nghiên cứu đạt được:

VI. Điểm kết luận: (Bằng chữ:)

CHỦ TỊCH HỘI ĐỒNG
(Ký, ghi rõ họ tên)

THƯ KÝ
(Ký, ghi rõ họ tên)

GIẢI TRÌNH CÁC CHỈNH SỬA

(Nếu có)

Chưa phát sinh nội dung cần giải trình.

LỜI CAM ĐOAN

Tên tôi là: Lê Trọng An.

Mã sinh viên: 21010389

Lớp: K15-KTCĐT2

Ngành: Kỹ thuật Cơ Điện tử.

Tôi đã thực hiện đồ án tốt nghiệp với đề tài: Thiết kế hệ thống IoT cho ứng dụng quản lý phương tiện giao thông trong lĩnh vực cho thuê xe tự lái.

Tôi xin cam đoan đây là đề tài nghiên cứu của riêng tôi và được sự hướng dẫn của TS. Nguyễn Đức Nam.

Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa được công bố dưới bất kỳ hình thức nào. Nếu phát hiện có bất kỳ hình thức gian lận nào, tôi xin hoàn toàn chịu trách nhiệm trước pháp luật.

Hà Nội, ngày ..., tháng ...năm 20...

GIẢNG VIÊN HƯỚNG DẪN

(Ký, ghi rõ họ tên)

SINH VIÊN

(Ký, ghi rõ họ tên)

TS. Nguyễn Đức Nam

Lê Trọng An

TÓM TẮT ĐỒ ÁN TỐT NGHIỆP - ABSTRACT

Sự phát triển nhanh của dịch vụ cho thuê xe tự lái tại Việt Nam khiến nhu cầu giám sát phương tiện không còn dừng ở việc biết xe đang ở đâu, mà còn ở việc hiểu xe đang vận hành ra sao và có dấu hiệu bất thường hay không. Từ nhu cầu đó, đồ án này trình bày quá trình thiết kế và triển khai một hệ thống IoT giám sát phương tiện theo hướng end-to-end, bao gồm thiết bị gắn trên xe, hạ tầng cloud và bảng điều khiển web, nhằm hỗ trợ doanh nghiệp theo dõi vị trí, nắm bắt trạng thái kỹ thuật và phản ứng sớm với các rủi ro vận hành.

Ở tầng thiết bị, hệ thống được xây dựng trên bo mạch PCB tự thiết kế. Trong đó, ESP32-S3 giữ vai trò bộ điều khiển trung tâm; SIM7600CE-T đảm nhiệm truyền dữ liệu 4G và định vị vệ tinh GNSS; LIS3DSH theo dõi rung động để nhận biết chuyển động khi xe đỗ; còn adapter vgate iCar Pro cho phép đọc dữ liệu OBD2 từ xe qua BLE. Kiến trúc nguồn được tổ chức theo nhiều nhánh để thiết bị vẫn duy trì hoạt động khi xe tắt máy, đồng thời hạn chế ảnh hưởng tới ắc quy chính.

Ở tầng phần mềm, dữ liệu từ thiết bị được gửi bằng MQTT 5.0 tới EMQX. Có thể hiểu MQTT là một cơ chế nhắn tin nhẹ, phù hợp với thiết bị IoT cần gửi nhiều bản tin nhỏ qua mạng di động. Sau đó, MQTT Bridge tiếp nhận và phân luồng dữ liệu tới từng hệ lưu trữ phù hợp: PostgreSQL cho dữ liệu quan hệ và nhật ký OTA, VictoriaMetrics cho dữ liệu chuỗi thời gian, VictoriaLogs cho nhật ký sự kiện. API server được xây dựng bằng Express.js và TypeScript, đóng vai trò kết nối giữa thiết bị, dữ liệu lưu trữ và giao diện khai thác.

Bảng điều khiển web được phát triển bằng Next.js 15 và React 19, hiển thị bản đồ, biểu đồ và trạng thái phương tiện gần như theo thời gian thực thông qua Socket.IO. Nhờ đó, người vận hành không chỉ quan sát vị trí xe mà còn có thể theo dõi lịch sử hành trình, cảnh báo và trạng thái thiết bị trên cùng một giao diện thống nhất.

Kết quả đạt được là một hệ thống IoT giám sát phương tiện hoàn chỉnh từ phần cứng đến phần mềm, có khả năng theo dõi vị trí thời gian thực, đọc dữ liệu chẩn đoán OBD2, thiết lập hàng rào địa lý, phát sinh cảnh báo tự động, cập nhật firmware từ xa và hỗ trợ quản lý đội xe. Quan trọng hơn, hệ thống cho thấy một hướng triển khai phù hợp với bối cảnh doanh nghiệp cho thuê xe tự lái trong nước: chi phí hợp lý, có thể tùy biến và đủ dư địa để mở rộng trong các giai đoạn sau.

Từ khóa: IoT, giám sát phương tiện, GPS, OBD2, MQTT, ESP32-S3, thời gian thực, hàng rào địa lý

Abstract (English)

As the self-drive car rental market in Vietnam grows, the need for remote vehicle monitoring and real-time fleet management becomes increasingly practical for operators. This

thesis presents the design and development of an end-to-end IoT vehicle tracking system, covering the onboard device, cloud services, and operational dashboard for real-time tracking, diagnostic data collection, and alerting.

On the hardware side, the tracker is implemented as a custom integrated PCB rather than an assembly of off-the-shelf development boards. The main board integrates ESP32-S3 (MCU), SIMCom SIM7600CE-T (LTE+GNSS), LIS3DSH (IMU), DS3231M (RTC), W25Q128 (SPI Flash), and power blocks based on MP2482, TPS54231, AP2112, SX1308, and TP4056. The modem is configured to Auto mode (AT+CNMP=2) so it can switch between LTE/UMTS/GSM automatically, uses the default APN internet, and delivers GNSS data via AT+CGNSINF (with optional NMEA streaming through AT+CGNSTST) on the same UART1 channel. The device reads engine diagnostic data via OBD2 through a vgate iCar Pro adapter over Bluetooth Low Energy (BLE), where this adapter is an external peripheral outside the main PCB. The power architecture uses a 5V main rail, an approximately 4V modem rail, a 3.3V logic rail, and a 18650 1S backup cell to maintain operation when the vehicle engine is off.

On the software side, device data is transmitted through MQTT 5.0 via EMQX with per-device ACL authorization. The MQTT Bridge validates and routes incoming messages to specialized storage systems: PostgreSQL for relational data and OTA update logs, VictoriaMetrics for time-series telemetry, and VictoriaLogs for operational event logs. The API server is built with Express.js and TypeScript following Domain-Driven Design (DDD), using database-backed session token authentication and providing OTA orchestration endpoints for devices.

The web interface is developed using Next.js 15 and React 19, providing a real-time dashboard with Leaflet maps, ECharts visualizations, and WebSocket connectivity via Socket.IO for instant vehicle position and status updates. The entire system is containerized and deployed using Docker, ensuring consistency between development and production environments.

The result is a complete end-to-end IoT vehicle tracking system, from hardware to software, capable of real-time location tracking, OBD2 diagnostic data retrieval, geofencing, automated alerts, remote firmware updates, and fleet-management support for self-drive car rental services.

Keywords: IoT, vehicle tracking, GPS, OBD2, MQTT, ESP32-S3, real-time, geofencing

LỜI CẢM ƠN - ACKNOWLEDGEMENTS

Trước tiên, tôi xin gửi lời cảm ơn chân thành và sâu sắc nhất đến TS. Nguyễn Đức Nam — người đã trực tiếp hướng dẫn, định hướng và tận tình hỗ trợ tôi trong suốt quá trình thực hiện đề án tốt nghiệp này. Những góp ý chuyên môn và sự động viên của thầy là nguồn động lực quan trọng giúp tôi hoàn thành đề tài.

Tôi xin trân trọng cảm ơn quý thầy cô trong Khoa Cơ khí - Cơ Điện tử, Trường Kỹ thuật - Đại học Phenikaa đã truyền đạt những kiến thức nền tảng vững chắc trong suốt quá trình học tập, tạo điều kiện thuận lợi để tôi có thể áp dụng vào thực tiễn thông qua đề án này.

Tôi cũng xin gửi lời cảm ơn đến gia đình, bạn bè và các đồng nghiệp đã luôn ủng hộ, chia sẻ và đồng hành cùng tôi trong suốt thời gian thực hiện đề tài.

Mặc dù đã nỗ lực hoàn thiện đề án trong phạm vi thời gian và kiến thức cho phép, nội dung chắc chắn vẫn còn những thiếu sót. Tôi rất mong nhận được những ý kiến góp ý từ quý thầy cô và hội đồng để tiếp tục hoàn thiện đề tài.

Xin chân thành cảm ơn!

Hà Nội, ngày ...tháng ...năm 20...

SINH VIÊN THỰC HIỆN

Lê Trọng An

MỤC LỤC - TABLE OF CONTENT

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN	i
NHẬN XÉT CỦA GIẢNG VIÊN PHẢN BIỆN	iii
BIÊN BẢN ĐÁNH GIÁ ĐỒ ÁN TỐT NGHIỆP	iv
GIẢI TRÌNH CÁC CHỈNH SỬA	v
LỜI CAM ĐOAN	vi
TÓM TẮT ĐỒ ÁN TỐT NGHIỆP - ABSTRACT	vii
LỜI CẢM ƠN - ACKNOWLEDGEMENTS	ix
MỤC LỤC - TABLE OF CONTENT	x
DANH MỤC BẢNG - LIST OF TABLES	xv
DANH MỤC HÌNH ẢNH VÀ ĐỒ THỊ - LIST OF PICTURES AND GRAPHS	xviii
DANH MỤC TỪ VIẾT TẮT - LIST OF ABBREVIATION	xx
CHƯƠNG 1. GIỚI THIỆU DỰ ÁN – PROJECT OVERVIEW	1
1.1. Đặt vấn đề / Bối cảnh của dự án	1
1.1.1. Bối cảnh thị trường cho thuê xe tự lái tại Việt Nam	1
1.1.2. Thách thức kỹ thuật	1
1.1.3. Động lực thực hiện dự án	2
1.2. Mục tiêu và phạm vi của dự án	2
1.2.1. Mục tiêu tổng quát	2
1.2.2. Phạm vi dự án	3
1.3. Các tiêu chí cần đạt được của dự án	5
1.3.1. Tiêu chí phần cứng	5
1.3.2. Tiêu chí firmware	5
1.3.3. Tiêu chí cloud/backend	6
1.3.4. Tiêu chí frontend	6
1.4. Phương pháp tiếp cận thiết kế kỹ thuật	7
1.4.1. Phương pháp luận tổng thể	7
1.4.2. Kiến trúc hệ thống	8
1.4.3. Chiến lược quản lý năng lượng	10
1.4.4. Công nghệ sử dụng	11

1.5. Kết quả và khuyến nghị	12
1.5.1. Kết quả đạt được	12
1.5.2. Khuyến nghị và hướng phát triển	13
Kết luận chương 1	15
CHƯƠNG 2. PHÂN TÍCH VẤN ĐỀ KỸ THUẬT	16
2.1. Mô tả vấn đề – Problem statement	16
2.1.1. Bối cảnh thực tế	16
2.1.2. Xác định vấn đề kỹ thuật	16
2.1.3. Mục tiêu kỹ thuật	17
2.2. Bối cảnh và cơ sở kỹ thuật – Background and Technical reviews	18
2.2.1. Tổng quan về hệ thống IoT trong giám sát phương tiện	18
2.2.2. Các giải pháp giám sát phương tiện hiện có	18
2.2.3. So sánh giao thức truyền thông IoT	19
2.2.4. So sánh vi điều khiển (MCU)	20
2.2.5. So sánh cơ sở dữ liệu	22
2.3. Yêu cầu kỹ thuật và các tiêu chuẩn thiết kế – Design criteria and Constraints	23
2.3.1. Yêu cầu chức năng	23
2.3.2. Yêu cầu phi chức năng	24
2.3.3. Ràng buộc thiết kế	25
2.3.4. Tiêu chuẩn thiết kế áp dụng	25
2.4. Yêu cầu từ các bên liên quan – Stakeholder requirements	26
2.4.1. Xác định các bên liên quan	26
2.4.2. Ma trận yêu cầu các bên liên quan	26
2.4.3. Phân tích chi tiết yêu cầu	27
2.4.4. Ma trận truy xuất yêu cầu - chức năng	28
Kết luận chương 2	29
CHƯƠNG 3. CÁC GIẢI PHÁP THIẾT KẾ – DESIGN SOLUTIONS	30
3.1. Phân tích và đề xuất giải pháp phần cứng – Hardware analysis and solution	30
3.1.1. Phân tích và lựa chọn phần cứng	30
3.1.2. Giải pháp phần cứng	38
3.2. Phân tích và đề xuất giải pháp Firmware – Firmware analysis and solution	49
3.2.1. Phân tích và lựa chọn giải pháp Firmware	49
3.2.2. Giải pháp Firmware	51
3.3. Phân tích và đề xuất kiến trúc Backend/Cloud – Backend/cloud analysis and solution	72
3.3.1. Phân tích và lựa chọn kiến trúc Cloud	72
3.3.2. Giải pháp Backend & Cloud	74
3.4. Phân tích và đề xuất công nghệ Frontend – Frontend analysis and solution	88

3.4.1. Phân tích và lựa chọn công nghệ Frontend	88
3.4.2. Giải pháp Frontend	89
3.5. Phân tích, đánh giá và lựa chọn phương án khả thi – Analysis, Evaluation and Selection	103
3.6. Tối ưu phương án thiết kế – The optimal solution	103
Kết luận chương 3	104

CHƯƠNG 4. TRIỂN KHAI GIẢI PHÁP VÀ KẾT QUẢ – IMPLEMENTATION AND RESULTS 105

4.1. Thiết kế chi tiết giải pháp – Detailed design solution	105
4.1.1. Thiết kế chi tiết phần cứng	105
4.1.2. Triển khai firmware, cloud và giao diện điều khiển	106
4.2. Chế tạo và lắp ráp hệ thống – Manufacture and Assembly	107
4.2.1. Lắp ráp mạch điện tử (Electronics Assembly)	107
4.2.2. Lắp đặt trong xe (Vehicle Installation)	107
4.2.3. Triển khai Firmware	107
4.2.4. Triển khai hệ thống Cloud	108
4.2.5. Triển khai Frontend Dashboard (Frontend Dashboard Implementation)	109
4.2.6. Cấu hình giám sát hệ thống (Monitoring & Observability)	109
4.2.7. Checklist hardening trước khi vận hành production	110
Tóm tắt	111
4.3. Đo lường và kết quả – Measurements and Results	111
4.3.1. Thiết lập môi trường thử nghiệm	111
4.3.2. Kết quả kiểm thử phần cứng	114
4.3.3. Kết quả kiểm thử firmware	121
4.3.4. Kết quả kiểm thử hệ thống Cloud	128
4.3.5. Kiểm thử tích hợp toàn hệ thống (End-to-End Integration Test)	133
4.3.6. Tổng hợp và so sánh với chỉ tiêu thiết kế	138
4.3.7. Đánh giá độ ổn định vận hành thực tế	143
4.3.8. Kết luận chương 4	143
Tài liệu tham khảo Chương 4 (Phần D)	144

CHƯƠNG 5. ĐÁNH GIÁ VÀ KHUYẾN NGHỊ – EVALUATION AND RECOMMENDATIONS 145

5.1. Đánh giá hiệu năng	145
5.1.1. Đánh giá hiệu năng phần cứng	145
5.1.2. Đánh giá hiệu năng firmware	146
5.1.3. Đánh giá hiệu năng hệ thống đám mây (Cloud)	148
5.1.4. Đánh giá hiệu năng giao diện người dùng (Frontend)	149

5.2. Đánh giá kinh tế và môi trường	149
5.2.1. Phân tích chi phí phần cứng	150
5.2.2. Phân tích chi phí hạ tầng đám mây	151
5.2.3. Lợi thế từ công nghệ mã nguồn mở	151
5.2.4. Đánh giá tác động môi trường	152
5.3. Đánh giá rủi ro và biện pháp giảm thiểu	152
5.3.1. Ma trận rủi ro	152
5.3.2. Phân tích chi tiết các rủi ro chính	154
5.3.3. Tổng hợp mức độ rủi ro	155
5.4. Khuyến nghị cho tương lai	156
5.4.1. Giai đoạn 2: Mở rộng tính năng (Phase 2)	156
5.4.2. Ứng dụng trí tuệ nhân tạo và học máy (AI/ML)	156
5.4.3. Ứng dụng di động (Mobile Application)	157
5.4.4. Tính toán biên (Edge Computing)	157
5.4.5. Triển khai sản xuất (Production Deployment)	158
5.4.6. Phần cứng phiên bản 2 (Hardware v2)	158
5.4.7. Tóm tắt lộ trình phát triển	159
Kết luận chương 5	159

CHƯƠNG 6. PHẢN HỒI VÀ BÀI HỌC KINH NGHIỆM – REFLECTION AND LESSONS LEARNED

	161
6.1. Ứng dụng kiến thức kỹ thuật – Application of prior coursework	161
6.1.1. Vi xử lý và Vi điều khiển	161
6.1.2. Mạng máy tính và IoT	161
6.1.3. Cơ sở dữ liệu	161
6.1.4. Lập trình hướng đối tượng	161
6.1.5. Điện tử số và Analog	161
6.1.6. Kỹ thuật phần mềm	161
6.2. Giải quyết các vấn đề kỹ thuật phức tạp – Complex engineering problems	162
6.2.1. Vấn đề 1: Phân tích bản tin OBD2 đa khung qua BLE	162
6.2.2. Vấn đề 2: Đường ống dữ liệu thời gian thực với đảm bảo phân phối	162
6.2.3. Vấn đề 3: Quản lý năng lượng với nhiều nguồn cấp	162
6.2.4. Vấn đề 4: Kiến trúc đám mây có khả năng mở rộng cho IoT	163
6.3. Tác động đạo đức và xã hội – Ethical and social impacts	163
6.3.1. Quyền riêng tư và bảo vệ dữ liệu cá nhân	163
6.3.2. An toàn phương tiện và trách nhiệm kỹ thuật	163
6.3.3. Tác động xã hội tích cực	163
6.3.4. Những lo ngại cần xem xét	164
6.4. Tổng kết và bài học kinh nghiệm – Reflection and Lessons Learned	164

6.4.1. Tầm quan trọng của thiết kế kiến trúc trước khi lập trình	164
6.4.2. Độ phức tạp của hệ thống IoT nhiều tầng	164
6.4.3. Sức mạnh của hệ sinh thái mã nguồn mở	164
6.4.4. Kiểm thử ở mọi tầng là bắt buộc	164
6.4.5. Tổng kết cá nhân	164
Kết luận chương 6	165
Tài liệu tham khảo Chương 6	165
TÀI LIỆU TRÍCH DẪN - REFERENCES	166
[A] Tài liệu tiếng Việt	166
[B] Tài liệu tiếng Anh (Books & Papers)	166
[C] Tài liệu kỹ thuật (Datasheets & Technical Documents)	167
[D] Tài liệu trực tuyến (Online Resources)	168
PHỤ LỤC - APPENDICES	172
PHỤ LỤC 1: BÁO CÁO TÀI CHÍNH - FINANCE REPORT	173
1.1. Bảng kê chi phí linh kiện (Bill of Materials - BOM)	173
1.2. Chi phí hạ tầng cloud (ước tính hàng tháng)	173
1.3. Tổng hợp chi phí dự án	174
PHỤ LỤC 2: CÁC TIÊU CHUẨN THIẾT KẾ - STANDARDS	175
2.1. Tiêu chuẩn OBD2 (SAE J1979 / ISO 15031–5)	175
2.2. Tiêu chuẩn MQTT 5.0 (OASIS)	175
2.3. Bảo mật thông tin (ISO 27001 - tham khảo)	175
2.4. Thiết kế REST API (RFC 7231 và best practices)	175
PHỤ LỤC 3: KẾ HOẠCH THỰC HIỆN - ASSIGNMENT AND TIMELINES	176
3.1. Phân chia giai đoạn dự án	176
3.2. Biểu đồ Gantt (Gantt Chart)	176
3.3. Chi tiết công việc theo giai đoạn	176
PHỤ LỤC 4: TÀI LIỆU BỔ TRỢ - SUPPORTING MATERIALS	177
4.1. Mã nguồn dự án (Source Code Repository)	177
4.2. Tham chiếu cơ sở dữ liệu (Database Schema)	177
4.3. Tài liệu API (API Documentation)	177
4.4. Tài liệu cấu hình môi trường (Environment Configuration)	177
4.5. Hướng dẫn cài đặt và chạy hệ thống	177

DANH MỤC BẢNG - LIST OF TABLES

1.1	Tóm tắt phạm vi dự án theo tầng (layer)	4
1.3.1A	Tổng hợp thông tin về STT, Tiêu chí, Mục tiêu cụ thể	5
1.2A	Tiêu chí firmware	6
1.2B	Tiêu chí cloud/backend	6
1.2C	Tiêu chí frontend	7
1.3	Tổng hợp công nghệ sử dụng trong dự án	11
2.1	So sánh các giải pháp giám sát phương tiện	18
2.2	So sánh các giao thức truyền thông IoT	19
2.3	So sánh các vi điều khiển cho ứng dụng IoT tracker	20
2.4	So sánh các giải pháp cơ sở dữ liệu cho hệ thống IoT	22
2.3.4A	Tổng hợp chuẩn và quy ước kỹ thuật áp dụng trong hệ thống	26
2.5	Ma trận yêu cầu các bên liên quan	27
2.6	Ma trận truy xuất yêu cầu các bên liên quan với chức năng hệ thống	29
3.1	Đối chiếu yêu cầu kỹ thuật khi lựa chọn vi điều khiển	32
3.2	Thông số kỹ thuật ESP32-S3 được chọn	34
3.3	Đối chiếu các phương án LTE/GNSS theo tác động tích hợp	35
3.4	Thông số kỹ thuật phương án SIM7600CE-T	37
3.5	So sánh phương pháp kết nối OBD2 theo thông số triển khai	38
3.6	Chiến lược kết nối BLE OBD2 theo chế độ hoạt động	40
3.7	Thông số kỹ thuật cảm biến LIS3DSH	41
3.8	Thông số khối LDO 3.3V	44
3.9	Thông số khối Buck 5V	45
3.10	Thông số khối Buck 3.8V/4V cho modem	45
3.11	Thông số khối Boost 5V dự phòng	45
3.12	Bảng trạng thái chuyển nguồn và cảnh báo	45
3.13	Thông số khối sạc pin	46
3.14	Thông số kỹ thuật pin dự phòng 18650 1S	46
3.14A	Bảng tổng hợp linh kiện hệ thống tracker (BOM)	48
3.15	Tổng hợp chi phí theo nhóm	49
3.5A	So sánh các phương án nền tảng firmware	50
3.2.2A	Tổng hợp thông tin về Lớp, Thành phần chính, Vai trò	52
3.2.2B	Tổng hợp thông tin về Tình huống lỗi, Xử lý, Thời gian timeout . .	58
3.2.2C	Tổng hợp thông tin về Bước, Lệnh AT, Mục đích chính	59
3.2.2D	Tổng hợp thông tin về Tình huống, Biện pháp	60
3.2.2E	Tổng hợp thông tin về GPIO, Chức năng, Hướng	60
3.2.2F	Tổng hợp thông tin về Điều kiện, Trạng thái charger, Lý do	63
3.2.2G	Tổng hợp thông tin về Chế độ, Trạng thái ESP32, Trạng thái modem	64

3.2.2H	Tổng hợp thông tin về Topic, Hướng, QoS	66
3.2.2I	Tổng hợp thông tin về Lệnh, Mục đích, Tham số chính	66
3.2.2J	Tổng hợp thông tin về Trạng thái, Mã, Mô tả	67
3.2.2K	Tổng hợp thông tin về Cảm biến, Phương pháp hiệu chuẩn, Lưu trữ	72
3.15A	So sánh các phương án kiến trúc Cloud	73
3.16	Phân bổ tài nguyên Docker cho các dịch vụ	76
3.17	So sánh các MQTT Broker phổ biến	78
3.18A	So sánh các phương án kiến trúc lưu trữ dữ liệu	80
3.18	So sánh các cơ sở dữ liệu trong hệ thống	81
3.19	Các nhóm bảng chính trong PostgreSQL	82
3.20	So sánh VictoriaMetrics và InfluxDB	83
3.21	So sánh framework Backend	84
3.22	Tóm tắt các nhóm API Endpoints chính	85
3.23	So sánh JWT và Session-based authentication	87
3.23A	So sánh các phương án công nghệ frontend tổng thể	88
3.24	So sánh framework Frontend	90
3.25	Chiến lược quản lý trạng thái	91
3.26	So sánh thư viện bản đồ	92
3.2.4A	Tổng hợp thông tin về Cột, Mô tả, Chức năng	96
3.27	Ma trận các trang chức năng chính	97
3.2.4B	Tổng hợp thông tin về Component, File, Chức năng	99
3.28	Breakpoint và tương ứng giao diện	101
3.29	Ma trận đánh giá tổng hợp các phương án thiết kế	103
4.12	Thiết bị đo lường sử dụng trong thử nghiệm phần cứng	111
4.13	Cấu hình môi trường thử nghiệm phần mềm	113
4.14	Kết quả đo dòng tiêu thụ năng lượng theo chế độ hoạt động	115
4.15	Thời lượng pin dự phòng theo kịch bản sử dụng (ước tính theo base- line 18650)	117
4.16	Kết quả kiểm thử nhiệt độ hoạt động	118
4.17	Kết quả kiểm thử tương thích OBD2 trên các thương hiệu xe	120
4.18	Kết quả kiểm thử mạch LVD	120
4.19	Kết quả đo thời gian kết nối BLE OBD2	121
4.20	Thời gian phản hồi OBD2 PID	121
4.21	Thời gian bắt vệ tinh GPS (TTFF)	122
4.22	Độ chính xác vị trí GPS	124
4.23	Độ trễ truyền dữ liệu MQTT qua mạng 4G	124
4.24	Kết quả kiểm thử máy trạng thái firmware	125
4.25	Kết quả kiểm thử reconnect MQTT	126
4.25A	Đối chiếu luồng OTA theo mã nguồn triển khai	127

4.26	Kết quả đo hiệu suất API Backend	128
4.27	Độ trễ end-to-end (Device -> Cloud -> Browser)	129
4.28	Thông lượng ghi cơ sở dữ liệu	130
4.29	Kết quả kiểm thử tải đồng thời	130
4.30	Kết quả đánh giá hiệu suất Frontend (Lighthouse)	132
4.31	Kết quả kiểm thử luồng dữ liệu chính	133
4.32	Kết quả kiểm thử cảnh báo Geofence	135
4.33	Kết quả kiểm thử lệnh điều khiển từ xa	135
4.34	Kết quả kiểm thử mất kết nối và phục hồi	136
4.35	Tổng hợp kết quả đo lường phần cứng so với chỉ tiêu thiết kế . . .	138
4.36	Tổng hợp kết quả đo lường firmware so với chỉ tiêu thiết kế . . .	140
4.37	Tổng hợp kết quả đo lường Cloud/Backend so với chỉ tiêu thiết kế	140
4.38	Tổng hợp kết quả đo lường Frontend so với chỉ tiêu thiết kế . . .	141
4.39	Bảng tổng hợp tổng thể — So sánh kết quả với chỉ tiêu thiết kế . .	141
4.40	Chỉ số ổn định vận hành thực tế	143
5.1	Đánh giá hiệu năng quản lý năng lượng	146
5.2	Đánh giá hiệu năng firmware	147
5.3	Đánh giá hiệu năng tổng hợp hệ thống đám mây	149
5.4	Chi phí BOM thiết bị tracker	150
5.5	So sánh chi phí với giải pháp thương mại	150
5.6	Chi phí hạ tầng đám mây theo quy mô	151
5.7	Thang đo xác suất và tác động	153
5.8	Ma trận đánh giá rủi ro và biện pháp giảm thiểu	153
5.9	Tổng hợp mức độ rủi ro theo phân loại	155
5.10	Lộ trình phát triển khuyến nghị	159

DANH MỤC HÌNH ẢNH VÀ ĐỒ THỊ - LIST OF PICTURES AND GRAPHS

1.1	Sơ đồ tổng quan vấn đề và giải pháp đề xuất	2
1.2	Quy trình phát triển dự án theo các giai đoạn	8
1.3	Kiến trúc tổng thể hệ thống IoT Vehicle Tracking	9
1.4	Sơ đồ chuyển đổi giữa các chế độ năng lượng	11
1.5	Lộ trình phát triển dự án theo các giai đoạn (Roadmap)	14
2.1	Sơ đồ kiến trúc dữ liệu của hệ thống - Data Architecture Diagram .	23
3.1	Sơ đồ khối tổng thể hệ thống tracker	30
3.1a	Phân rã chi tiết các khối chức năng của tracker	31
3.2	Sơ đồ kết nối BLE giữa ESP32-S3 và vgate iCar Pro	40
3.3	Sơ đồ kết nối LIS3DSH với ESP32-S3 qua I2C	42
3.3a	Sơ đồ chi tiết chân kết nối LIS3DSH với ESP32-S3	43
3.4	Sơ đồ khối hệ thống quản lý nguồn	44
3.4a	Kiến trúc nguồn và phân phối điện áp trong hệ thống	44
3.5	Sơ đồ kiến trúc phân lớp của firmware	53
3.6	Sơ đồ tương tác giữa các FreeRTOS task	54
3.7	Lưu đồ thuật toán luồng hoạt động chính của firmware	55
3.8	Lưu đồ thuật toán quy trình kết nối và đọc dữ liệu BLE OBD2 . . .	57
3.9	Lưu đồ điều phối modem SIM7600CE-T theo trạng thái mạng và chế độ vận hành	62
3.10	Lưu đồ thuật toán điều khiển power path	63
3.11	Sơ đồ máy trạng thái của thiết bị theo dõi	69
3.12	Sơ đồ kiến trúc tổng quan hệ thống Cloud	75
3.12a	Luồng dữ liệu chi tiết từ thiết bị đến dashboard	75
3.13	Sơ đồ chiến lược lưu trữ kép	81
3.14	Sơ đồ quan hệ cơ sở dữ liệu (ER Diagram)	83
3.14a	Luồng điều phối OTA giữa Backend, EMQX, thiết bị và MQTT Bridge	86
3.15	Sơ đồ kiến trúc Feature-Sliced Design của ứng dụng Frontend . . .	93
3.15a	Luồng dữ liệu một chiều giữa Next.js, Query, Zustand và Backend	94
3.16	Biểu đồ trình tự (Sequence Diagram) luồng xác thực người dùng .	94
3.17	Giao diện trang Dashboard tổng quan	95
3.18	Giao diện trang quản lý phương tiện	97
3.19a	Phân rã lớp bản đồ React Leaflet và kênh Socket.IO	98
3.19	Kiến trúc tích hợp bản đồ thời gian thực	98
3.20	Giao diện trang bản đồ thời gian thực với các marker xe	100
3.21	Wireframe bố cục giao diện Dashboard	101

3.22	Các mẫu thiết kế UI của hệ thống - Data Table, Form, Charts	102
3.23	Sơ đồ kiến trúc tổng thể phương án tối ưu	103
4.29	Bố trí thiết bị đo lường trong phòng thí nghiệm	112
4.30	Thiết bị tracker được lắp đặt trên xe thử nghiệm	113
4.31	Sơ đồ môi trường thử nghiệm tổng thể	114
4.32	Đồ thị dòng tiêu thụ theo thời gian trong một chu kỳ hoạt động hoàn chỉnh (Driving -> Parking -> Alert -> Parking)	116
4.33	Đồ thị điện áp pin dự phòng theo thời gian theo baseline 18650 trong kịch bản tracking liên tục	118
4.34	Đồ thị dòng tiêu thụ theo nhiệt độ môi trường	119
4.35	Biểu đồ phân bố thời gian kết nối BLE OBD2 (20 lần đo)	123
4.36	Đồ thị thời gian bắt vệ tinh GPS trong các kịch bản (Cold/Warm/Hot Start)	123
4.37	Biểu đồ phân bố độ trễ MQTT trong điều kiện mạng 4G ổn định .	125
4.38	Biểu đồ thời gian phản hồi API (P50, P95, P99) cho các endpoint chính	129
4.39	Đồ thị phân bố độ trễ end-to-end (1000 bản tin mẫu)	129
4.40	Đồ thị hiệu suất hệ thống theo số lượng thiết bị đồng thời	131
4.41	Kết quả Lighthouse Performance Audit của trang Dashboard	132
4.42	Screenshot giao diện Dashboard hiển thị vị trí xe đang di chuyển trên bản đồ (Leaflet)	134
4.43	Screenshot biểu đồ dữ liệu OBD2 thời gian thực (ECharts) — RPM, Speed, Coolant Temp	135
4.44	Screenshot cảnh báo Geofence trên giao diện Dashboard với bản đồ hiển thị vùng cảnh báo	137
4.45	Screenshot giao diện gửi lệnh điều khiển từ Dashboard và kết quả phản hồi từ thiết bị	137
4.46	Screenshot hành trình trên bản đồ, thể hiện giai đoạn gián đoạn và tiếp tục cập nhật sau khi phục hồi kết nối	138
4.47	Biểu đồ radar so sánh chỉ tiêu thiết kế và kết quả đạt được (spider chart)	142

DANH MỤC TỪ VIẾT TẮT - LIST OF ABBREVIATION

Từ viết tắt	Tiếng Anh	Tiếng Việt
ADC	Analog-to-Digital Converter	Bộ chuyển đổi tương tự sang số
API	Application Programming Interface	Giao diện lập trình ứng dụng
BLE	Bluetooth Low Energy	Bluetooth năng lượng thấp
BOM	Bill of Materials	Danh sách linh kiện
CI/CD	Continuous Integration / Continuous Deployment	Tích hợp liên tục / Triển khai liên tục
CORS	Cross-Origin Resource Sharing	Chia sẻ tài nguyên nguồn chéo
DDD	Domain-Driven Design	Thiết kế hướng miền
Docker	Docker Container Platform	Nền tảng container hóa ứng dụng
EMQX	Erlang MQTT Broker	Máy chủ MQTT dựa trên Erlang
ESP32	Espressif System-on-Chip 32-bit	Vi điều khiển SoC 32-bit của Espressif
FreeRTOS	Free Real-Time Operating System	Hệ điều hành thời gian thực mã nguồn mở
GPIO	General Purpose Input/Output	Cổng vào/ra đa năng
GNSS	Global Navigation Satellite System	Hệ thống vệ tinh dẫn đường toàn cầu
GPS	Global Positioning System	Hệ thống định vị toàn cầu
HTTP	Hypertext Transfer Protocol	Giao thức truyền siêu văn bản
HTTPS	Hypertext Transfer Protocol Secure	Giao thức truyền siêu văn bản bảo mật
IMU	Inertial Measurement Unit	Đơn vị đo lường quán tính
IoT	Internet of Things	Internet vạn vật
JWT	JSON Web Token	Mã thông báo web dạng JSON
LVD	Low Voltage Disconnect	Ngắt điện áp thấp
MCU	Microcontroller Unit	Vi điều khiển
MQTT	Message Queuing Telemetry Transport	Giao thức truyền thông tin hàng đợi

Từ viết tắt	Tiếng Anh	Tiếng Việt
OBD2	On-Board Diagnostics II	Chẩn đoán trên xe thể hệ 2
ORM	Object-Relational Mapping	Ánh xạ đối tượng - quan hệ
PCB	Printed Circuit Board	Bảng mạch in
REST	Representational State Transfer	Kiến trúc truyền trạng thái đại diện
RTOS	Real-Time Operating System	Hệ điều hành thời gian thực
SoC	System on Chip	Hệ thống trên chip
SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc
TLS	Transport Layer Security	Bảo mật tầng vận chuyển
UART	Universal Asynchronous Receiver-Transmitter	Bộ thu phát bất đồng bộ đa năng
WebSocket	WebSocket Protocol	Giao thức kết nối hai chiều thời gian thực

CHƯƠNG 1. GIỚI THIỆU DỰ ÁN – PROJECT OVERVIEW

1.1. Đặt vấn đề / Bối cảnh của dự án

1.1.1. Bối cảnh thị trường cho thuê xe tự lái tại Việt Nam

Những năm gần đây, thị trường cho thuê xe tự lái tại Việt Nam tăng trưởng rõ rệt, đặc biệt tại TP. Hồ Chí Minh, Hà Nội và Đà Nẵng. Theo báo cáo ngành vận tải [1], nhu cầu thuê xe tăng trung bình 15–20% mỗi năm nhờ sự phục hồi của du lịch nội địa, nhu cầu di chuyển linh hoạt và xu hướng chia sẻ phương tiện. Với doanh nghiệp vận hành đội xe, tốc độ tăng trưởng này mang lại cơ hội kinh doanh nhưng cũng kéo theo một yêu cầu rất thực tế: phải biết xe đang ở đâu, đang được sử dụng như thế nào và có an toàn hay không mà không thể chỉ dựa vào các cuộc gọi xác nhận. Khi quy mô đội xe mở rộng, áp lực giám sát vận hành và bảo toàn tài sản tăng lên nhanh chóng. Trong thực tế, các doanh nghiệp cho thuê xe tự lái hiện nay thường gặp bốn nhóm khó khăn sau:

- **Quản lý thủ công kém hiệu quả:** Nhiều doanh nghiệp quy mô vừa và nhỏ vẫn quản lý xe bằng sổ sách, gọi điện xác nhận hoặc dựa vào sự chủ động của khách hàng. Cách làm này không cung cấp thông tin thời gian thực về vị trí và trạng thái phương tiện [2].
- **Rủi ro mất cắp và sử dụng sai mục đích:** Khi không có hệ thống giám sát, xe có thể bị sử dụng vượt phạm vi địa lý đã thỏa thuận, chạy quá số km quy định, hoặc trong trường hợp xấu nhất là bị chiếm đoạt. Việc phát hiện các tình huống này thường bị trễ, gây thiệt hại lớn về tài sản [3].
- **Thiếu dữ liệu chẩn đoán kỹ thuật:** Doanh nghiệp khó theo dõi tình trạng kỹ thuật của xe từ xa, nên việc bảo trì thường mang tính bị động và chỉ được thực hiện khi sự cố đã xuất hiện. Điều này làm tăng chi phí sửa chữa và rút ngắn tuổi thọ phương tiện.
- **Giải pháp thương mại đắt đỏ:** Các hệ thống GPS tracking thương mại hiện có trên thị trường (như Vietmap, iTracking) thường có chi phí cao — bao gồm phí thiết bị, phí dịch vụ hàng tháng, và phí tích hợp — không phù hợp với các doanh nghiệp quy mô nhỏ với ngân sách hạn chế [4].

1.1.2. Thách thức kỹ thuật

Để giải quyết những khó khăn trên, hệ thống không thể chỉ là một thiết bị định vị đơn giản. Bài toán thực tế là phải dung hòa nhiều yêu cầu kỹ thuật vốn dễ xung đột với nhau: thiết bị phải tiết kiệm điện nhưng vẫn luôn sẵn sàng, phải truyền dữ liệu liên tục nhưng vẫn chịu được mất sóng, và phải đủ đơn giản để triển khai thực tế với chi phí hợp lý. Bốn thách thức nổi bật là:

Thứ nhất, bài toán tiêu thụ năng lượng. Nếu thiết bị theo dõi hoạt động liên tục ở trạng thái công suất cao, ắc quy xe có thể bị hao đáng kể chỉ sau vài tuần, từ đó ảnh hưởng trực tiếp đến khả năng khởi động [5]. Vì vậy, hệ thống phải biết khi nào cần hoạt động đầy đủ và khi nào có thể chuyển sang chế độ tiết kiệm điện.

Thứ hai, giám sát khi xe đỗ. Khi xe tắt máy (IGN OFF), thiết bị không thể duy trì mọi khối chức năng như lúc xe đang chạy. Tuy nhiên, nó vẫn phải phát hiện được các chuyển động bất thường, chẳng hạn xe bị dắt đi hoặc bị cầu kéo. Đây là bài toán cân bằng giữa độ nhạy phát hiện và mức tiêu thụ năng lượng [6].

Thứ ba, độ tin cậy của kết nối. Hệ thống hoạt động trong môi trường di động nên kết nối 4G/LTE có thể gián đoạn bất kỳ lúc nào. Điều người quản lý cần không chỉ là "có mạng thì gửi được", mà là khi mất mạng thiết bị vẫn không làm mất dữ liệu quan trọng và có thể đồng bộ lại khi kết nối được phục hồi.

Thứ tư, xử lý dữ liệu thời gian thực. Khi số lượng xe tăng lên hàng chục hoặc hàng trăm, hệ thống backend phải tiếp nhận liên tục dữ liệu vị trí, thông số OBD2 và trạng thái cảm biến với độ trễ thấp, đồng thời vẫn giữ giao diện giám sát dễ quan sát và dễ thao tác cho người quản lý.



Hình 1.1: Sơ đồ tổng quan vấn đề và giải pháp đề xuất

1.1.3. Động lực thực hiện dự án

Nói cách khác, mục tiêu của đề tài không chỉ là làm ra một thiết bị biết gửi tọa độ GPS. Điều cần xây dựng là một hệ thống đủ tròn vẹn để người vận hành đội xe có thể nhìn thấy trạng thái phương tiện, hiểu nguyên nhân của các cảnh báo và đưa ra quyết định kịp thời. Từ nhu cầu thực tiễn đó, dự án "IoT Vehicle Tracking System" được triển khai nhằm xây dựng một giải pháp đồng bộ, bao gồm thiết bị phần cứng IoT, firmware nhúng, hệ thống cloud và giao diện web. Mục tiêu là tạo ra một hệ thống có chi phí hợp lý, dễ tùy biến và có thể mở rộng theo quy mô doanh nghiệp cho thuê xe tự lái tại Việt Nam.

1.2. Mục tiêu và phạm vi của dự án

1.2.1. Mục tiêu tổng quát

Mục tiêu chính của dự án là thiết kế và hiện thực một hệ thống IoT theo dõi phương tiện hoàn chỉnh, bao gồm cả phần cứng lẫn phần mềm, phục vụ cho bài toán quản lý đội xe cho thuê tự lái. Các mục tiêu dưới đây được xác định từ góc nhìn của đơn vị vận hành

đội xe: họ cần thông tin đủ nhanh để can thiệp, nhưng cũng cần hệ thống đủ bền bỉ để chạy lâu dài trên xe. Hệ thống cần đáp ứng các yêu cầu sau:

1. **Theo dõi vị trí thời gian thực (real-time tracking):** Cung cấp vị trí GPS của xe với tần suất cập nhật 5–30 giây khi xe đang di chuyển, hiển thị trên bản đồ trực tuyến.
2. **Giám sát trạng thái kỹ thuật qua OBD2:** Kết nối với cổng chẩn đoán OBD-II của xe thông qua giao thức Bluetooth Low Energy (BLE) để đọc các thông số như tốc độ, vòng tua máy, nhiệt độ động cơ, và mã lỗi chẩn đoán (DTC).
3. **Cảnh báo thông minh:** Tự động phát hiện và gửi thông báo khi có các sự kiện bất thường như: xe vượt ra khỏi vùng địa lý cho phép (hàng rào địa lý - geofencing), vượt tốc độ quy định, chuyển động bất thường khi xe đang đỗ, hoặc mất kết nối thiết bị.
4. **Tối ưu hóa năng lượng:** Đảm bảo thiết bị hoạt động liên tục mà không làm cạn ắc quy xe, với thời lượng pin dự phòng đủ để hoạt động độc lập trong trường hợp mất nguồn chính.
5. **Giao diện quản lý trực quan:** Cung cấp bảng điều khiển (dashboard) web cho phép người quản lý đội xe theo dõi vị trí, xem lịch sử hành trình, quản lý cảnh báo, và tạo báo cáo.

1.2.2. Phạm vi dự án

Để đề tài không dàn trải, phạm vi nghiên cứu được giới hạn theo từng tầng của hệ thống, từ đối tượng ứng dụng đến phần cứng, firmware và cloud. Cách xác định phạm vi này giúp người đọc thấy rõ đâu là phần đã được triển khai trong đồ án và đâu là nội dung được giữ lại cho các giai đoạn phát triển sau.

Đối tượng ứng dụng:

- Loại xe: Ô tô con (xe 4 chỗ, sedan, SUV) với hệ thống điện 12V hoặc 24V DC
- Ắc quy phổ thông: 40–70 Ah (trung bình 45–60 Ah)
- Đối tượng sử dụng: Các công ty cho thuê xe tự lái quy mô vừa và nhỏ tại Việt Nam

Phạm vi phần cứng:

- Thiết bị tracker IoT sử dụng vi điều khiển ESP32-S3
- Modem LTE + GNSS SIMCom SIM7600CE-T (Auto mode LTE/UMTS/GSM, APN mặc định internet) kết nối trực tiếp qua UART1 để cung cấp cả dữ liệu 4G và GNSS
- Adapter OBD2 BLE vgate iCar Pro (đọc dữ liệu chẩn đoán xe qua Bluetooth)
- Cảm biến gia tốc LIS3DSH (IMU) để phát hiện chuyển động và rung
- Pin dự phòng 18650 1S với mạch sạc và bảo vệ

Phạm vi firmware:

- Phát triển trên nền tảng ESP-IDF với hệ điều hành thời gian thực FreeRTOS
- Giao tiếp BLE với adapter OBD2
- Truyền dữ liệu qua giao thức MQTT 5.0
- Quản lý năng lượng đa chế độ (driving, parking, alert)

Phạm vi cloud/backend:

- EMQX MQTT Broker làm trung tâm tiếp nhận dữ liệu từ thiết bị
- MQTT Bridge (dịch vụ độc lập) xử lý và phân phối dữ liệu
- PostgreSQL lưu trữ dữ liệu quan hệ (người dùng, xe, khách hàng, cảnh báo)
- VictoriaMetrics lưu trữ dữ liệu chuỗi thời gian (telemetry)
- VictoriaLogs lưu trữ nhật ký sự kiện
- Express.js API server với kiến trúc Domain-Driven Design
- Next.js 15 frontend với bản đồ thời gian thực (Leaflet), biểu đồ (ECharts)

Giới hạn phạm vi nghiên cứu:

- Không xử lý OBD-II diagnostics phức tạp (chỉ đọc các thông số cơ bản)
- Không phát hiện va chạm (IMU chỉ dùng cho phát hiện chuyển động/rung)
- Không điều khiển động cơ xe (tắt bơm xăng, khóa xe) — chỉ gửi cảnh báo
- Không bao gồm ứng dụng di động ban đầu (dự kiến Phase 2 với Flutter WebView)

Bảng 1.1: Tóm tắt phạm vi dự án theo tầng (layer)

Tầng (Layer)	Công nghệ chính	Phạm vi
Phần cứng	ESP32-S3, SIMCom SIM7600CE-T (LTE + GNSS), vgate iCar Pro, LIS3DSH	Thiết kế PCB, chế tạo và tích hợp thiết bị hoàn chỉnh
Firmware	ESP-IDF, FreeRTOS, MQTT 5.0	Lập trình nhúng đầy đủ
MQTT Broker	EMQX 5.x	Cấu hình và triển khai
Backend	Express.js, TypeScript, PostgreSQL, VictoriaMetrics	Phát triển API và xử lý dữ liệu
Frontend	Next.js 15, React 19, Leaflet, ECharts	Giao diện quản lý web
Infrastructure	Docker, Nginx Proxy Manager, Grafana	Triển khai và giám sát

1.3. Các tiêu chí cần đạt được của dự án

Dự án đặt ra các tiêu chí cụ thể (success criteria) cho từng tầng của hệ thống. Các tiêu chí này không chỉ dùng để đánh giá kết quả cuối cùng, mà còn đóng vai trò như một cam kết thiết kế xuyên suốt quá trình thực hiện: mỗi quyết định chọn linh kiện, tổ chức firmware hay xây dựng hạ tầng đều phải quay lại trả lời câu hỏi liệu nó có giúp hệ thống vận hành đủ ổn định, đủ tiết kiệm năng lượng và đủ hữu ích cho bài toán quản lý xe hay không.

1.3.1. Tiêu chí phần cứng

Ở tầng phần cứng, điều quan trọng nhất không phải là ”gắn được nhiều khối chức năng”, mà là thiết bị có thể tồn tại bền bỉ trong môi trường trên xe: đủ tiết kiệm điện, đủ ổn định về nhiệt và đủ an toàn với ắc quy.

Bảng 1.3.1A: Tổng hợp thông tin về STT, Tiêu chí, Mục tiêu cụ thể

STT	Tiêu chí	Mục tiêu cụ thể
1	Tiêu thụ điện chế độ ngủ sâu (deep sleep)	< 500 μ A
2	Tiêu thụ điện chế độ hoạt động	< 250 mA (trung bình)
3	Thời gian thức dậy từ deep sleep	< 3 giây
4	Thời lượng pin dự phòng (18650 1S)	$\geq$ 2.5 giờ tracking liên tục; $\geq$ 24 giờ cảnh báo gián đoạn (mục tiêu theo baseline 18650)
5	Ngưỡng chuyển nguồn bảo vệ ắc quy	Profile 12V: $OFF = 12.0$ V, $ON = 12.2$ V; Profile 24V: $OFF = 24.0$ V, $ON = 24.4$ V
6	Nhiệt độ hoạt động	-10 C đến +60 C

1.3.2. Tiêu chí firmware

Ở tầng firmware, tiêu chí không chỉ dừng ở việc đọc được cảm biến hay gửi được dữ liệu, mà nằm ở khả năng điều phối đúng nhịp giữa các khối để thiết bị phản ứng phù hợp với từng trạng thái vận hành của xe.

Bảng 1.2A: Tiêu chí firmware

STT	Tiêu chí	Mục tiêu cụ thể
1	Tần suất gửi GPS khi lái xe	5–30 giây (cấu hình được)
2	Tần suất heartbeat khi đỗ xe	10–30 phút
3	Thời gian kết nối OBD2 BLE	< 10 giây
4	Độ chính xác vị trí GNSS	< 5 mét (điều kiện thoáng)
5	Xử lý mất kết nối	Tự động kết nối lại; có phương án bổ sung buffer cục bộ
6	Phát hiện chuyển động bất thường	Độ nhạy IMU cấu hình được

1.3.3. Tiêu chí cloud/backend

Khi chuyển lên cloud/backend, trọng tâm đánh giá đổi sang khả năng tiếp nhận dữ liệu liên tục, phản hồi nhanh và bảo vệ dữ liệu khi số lượng thiết bị tăng lên theo quy mô đội xe.

Bảng 1.2B: Tiêu chí cloud/backend

STT	Tiêu chí	Mục tiêu cụ thể
1	Độ trễ xử lý MQTT message	< 500 ms (end-to-end)
2	Số lượng thiết bị đồng thời	>= 50 thiết bị (đã kiểm thử), dư địa tới khoảng 100 thiết bị
3	Độ sẵn sàng hệ thống (uptime)	>= 99.5%
4	Thời gian phản hồi API	< 200 ms (p95)
5	Lưu trữ dữ liệu telemetry	>= 90 ngày
6	Bảo mật xác thực	Phiên đăng nhập bằng opaque token lưu trong DB, token được băm SHA-256

1.3.4. Tiêu chí frontend

Cuối cùng, frontend là nơi toàn bộ nỗ lực kỹ thuật được chuyển thành trải nghiệm quan sát và ra quyết định cho người vận hành, nên tiêu chí ở đây tập trung vào tính cập nhật, dễ theo dõi và dễ sử dụng trong bối cảnh thực tế.

Bảng 1.2C: Tiêu chí frontend

STT	Tiêu chí	Mục tiêu cụ thể
1	Cập nhật bản đồ thời gian thực	< 2 giây độ trễ
2	Hỗ trợ trình duyệt	Chrome, Firefox, Edge (phiên bản mới nhất)
3	Responsive design	Desktop và tablet
4	Hiển thị cảnh báo	Thông báo tức thời qua WebSocket

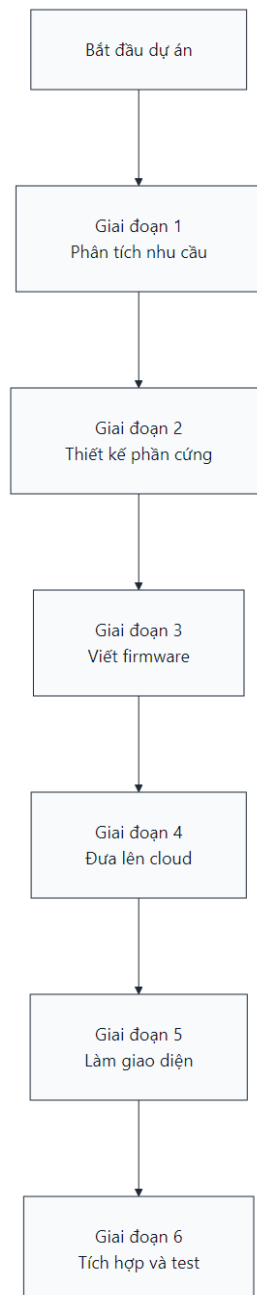
1.4. Phương pháp tiếp cận thiết kế kỹ thuật

1.4.1. Phương pháp luận tổng thể

Vì đề tài đi qua cả mạch nguồn, firmware nhúng, cloud lẫn dashboard, rủi ro lớn nhất không nằm ở việc từng khối chạy riêng lẻ, mà ở việc chúng có còn phối hợp đúng khi ghép thành một hệ thống hoàn chỉnh hay không. Từ đó, dự án chọn phương pháp **thiết kế từ dưới lên (bottom-up design)** kết hợp **phát triển lặp (iterative development)**: kiểm chứng trước những giả định rủi ro nhất ở tầng thấp, rồi mới ghép dần lên các tầng trên. Cách tiếp cận này giúp mỗi giai đoạn tạo ra một kết quả có thể đo và kiểm tra được, đồng thời giảm nguy cơ chỉ đến cuối dự án mới lộ ra xung đột về nguồn, giao tiếp hay hiệu năng. Cụ thể, tiến trình được tổ chức như sau:

1. **Giai đoạn 1 — Nghiên cứu và thiết kế phần cứng:** Xây nền cho hệ thống bằng cách khảo sát linh kiện, đối chiếu datasheet, thiết kế sơ đồ nguyên lý và tính toán nguồn. Mục tiêu của giai đoạn này là bảo đảm thiết bị có thể tồn tại bền bỉ trong môi trường trên xe trước khi nói đến các tính năng ở tầng trên.
2. **Giai đoạn 2 — Phát triển firmware:** Biến sơ đồ phần cứng thành hành vi vận hành thật trên ESP-IDF, bao gồm giao tiếp BLE OBD2, UART modem, I2C IMU, quản lý năng lượng và truyền dữ liệu MQTT. Đây là giai đoạn quyết định thiết bị có “phản ứng đúng lúc” hay không.
3. **Giai đoạn 3 — Xây dựng hạ tầng cloud:** Chuẩn bị lớp tiếp nhận và lưu trữ dữ liệu bằng cách triển khai EMQX, PostgreSQL, VictoriaMetrics và các dịch vụ phụ trợ trên Docker, đồng thời xác định cấu trúc dữ liệu và kết nối mạng.
4. **Giai đoạn 4 — Phát triển backend API:** Chuyển dữ liệu thô thành nghiệp vụ có thể truy vấn và điều phối, thông qua REST API, xử lý domain và kênh WebSocket thời gian thực.
5. **Giai đoạn 5 — Phát triển frontend:** Biến dữ liệu kỹ thuật thành thông tin dễ đọc đối với người vận hành bằng dashboard web, bản đồ trực tuyến và biểu đồ trực quan.

6. Giai đoạn 6 — Tích hợp và kiểm thử: Ghép toàn bộ các lớp lại thành một chuỗi hoàn chỉnh từ xe đến dashboard, sau đó kiểm thử, đo đạc và tối ưu những điểm còn nghẽn.

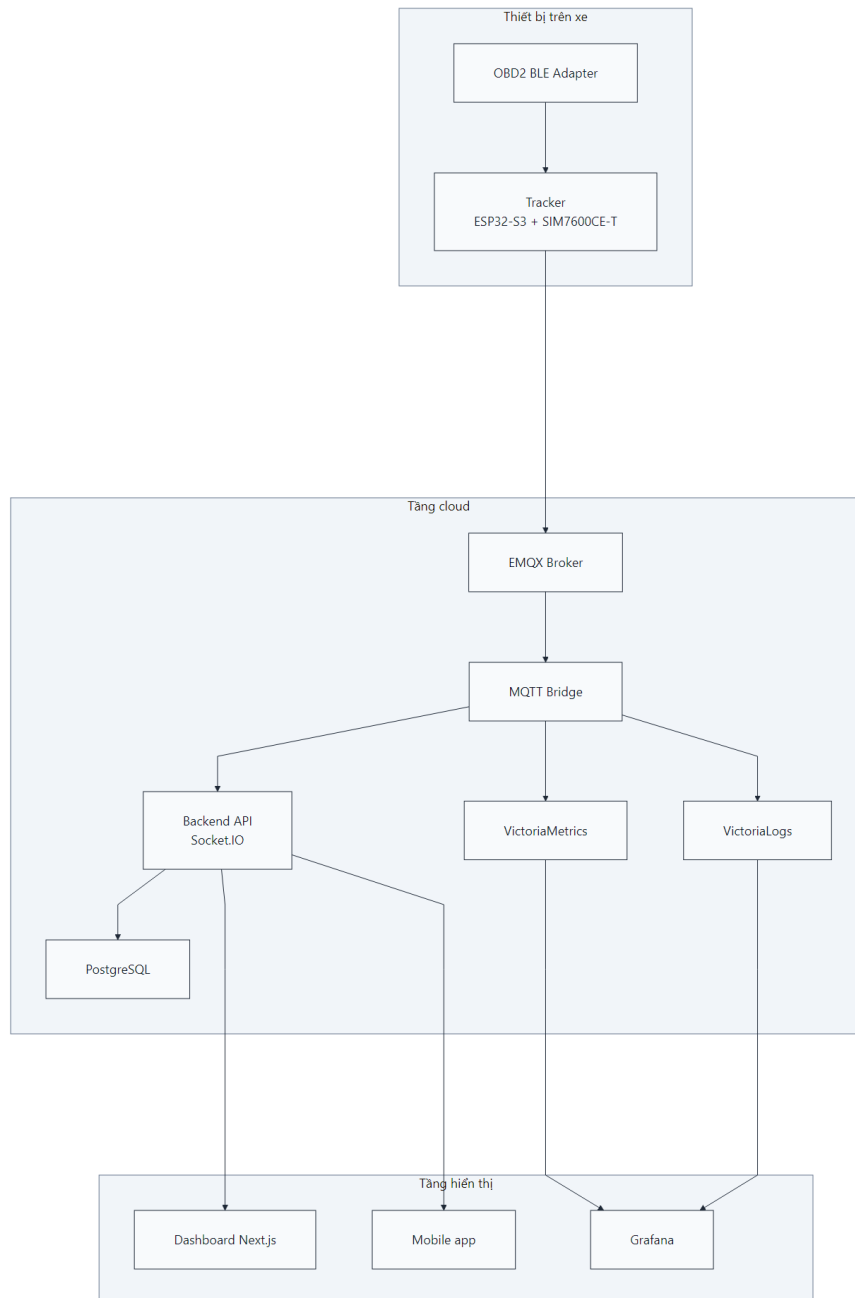


Hình 1.2: Quy trình phát triển dự án theo các giai đoạn

1.4.2. Kiến trúc hệ thống

Nhìn theo hành trình của một bản tin, hệ thống khởi đầu ở thiết bị gắn trên xe, đi qua lớp tiếp nhận dữ liệu thời gian thực trên cloud, rồi mới đến lớp nghiệp vụ và lớp giao diện cho người quản lý. Góc nhìn này giúp lý giải tự nhiên vì sao kiến trúc được tổ chức theo dạng phân tầng (layered architecture): thiết bị thu thập và gửi dữ liệu; lớp trung gian

tiếp nhận, phân luồng và lưu trữ; lớp ứng dụng phân tích, quản lý và trình bày thông tin cho người dùng. Trong triển khai thực tế, tracker (ESP32-S3, SIM7600CE-T, LIS3DSH, vgate iCar Pro) kết nối MQTT lên EMQX; MQTT Bridge tiếp nhận và phân luồng dữ liệu sang VictoriaMetrics, VictoriaLogs và Backend API; lớp ứng dụng phía trên phục vụ dashboard web Next.js và hệ quan trắc Grafana. Ứng dụng di động được giữ ở phạm vi phát triển giai đoạn sau để tránh làm dàn trải đồ án.



Hình 1.3: Kiến trúc tổng thể hệ thống IoT Vehicle Tracking

Chiến lược dữ liệu (Data Strategy):

Chiến lược dữ liệu được xây theo cùng logic phân tầng này. Lý do là dữ liệu trong hệ

thông không đồng nhất: có loại cần tính nhất quán và quan hệ chặt chẽ, có loại cần ghi liên tục với tần suất cao, và có loại chủ yếu phục vụ truy vết sự cố. Vì vậy, mỗi nhóm dữ liệu được đưa về đúng nơi phù hợp với đặc tính truy vấn và mục đích sử dụng.

- **PostgreSQL:** Lưu trữ dữ liệu quan hệ có cấu trúc như người dùng, xe, khách hàng, hành trình, cảnh báo, vùng địa lý (geofences) và lệnh điều khiển. Đây là lớp dữ liệu cần quan hệ rõ ràng và ràng buộc chặt chẽ.
- **VictoriaMetrics:** Lưu trữ dữ liệu chuỗi thời gian (time-series) như tọa độ GPS, dữ liệu OBD2 và chỉ số cảm biến. Nhóm dữ liệu này cần ghi nhiều, đọc nhanh theo trục thời gian và lưu dài hạn với chi phí hợp lý.
- **VictoriaLogs:** Lưu trữ nhật ký sự kiện, audit trail và các bản ghi hoạt động của thiết bị để phục vụ quan sát hệ thống và truy vết sự cố.

1.4.3. Chiến lược quản lý năng lượng

Thiết kế năng lượng của tracker thực chất là bài toán quyết định lúc nào thiết bị cần hoạt động đầy đủ, lúc nào chỉ nên “canh chừng”, và lúc nào phải ưu tiên bảo vệ nguồn xe. Nếu xử lý không khéo, hệ thống sẽ rơi vào hai cực đoan: theo dõi rất sát nhưng làm hao ắc quy, hoặc tiết kiệm điện quá mức nên bỏ lỡ sự kiện quan trọng. Vì vậy, chiến lược quản lý năng lượng đa chế độ (multi-mode power management) được xây quanh các trạng thái vận hành thật của xe, để mỗi thời điểm chỉ bật những khối thật sự cần thiết:

Chế độ 1 — Driving Mode (IGN ON, ưu tiên độ đầy đủ dữ liệu):

- Hệ thống hoạt động toàn bộ công suất
- Gửi vị trí GPS định kỳ (5–30 giây)
- Sạc pin dự phòng từ nguồn xe
- Kết nối server liên tục để theo dõi thời gian thực

Chế độ 2 — Parking Mode (IGN OFF, ưu tiên bảo toàn năng lượng):

- Vi điều khiển ESP32-S3 vào chế độ ngủ sâu (deep sleep)
- Tiêu thụ cực thấp ($< 500 \mu A$)
- Thức dậy định kỳ (10–30 phút) để gửi heartbeat
- IMU (LIS3DSH) hoạt động độc lập, cảnh rung ở ngưỡng đã cấu hình

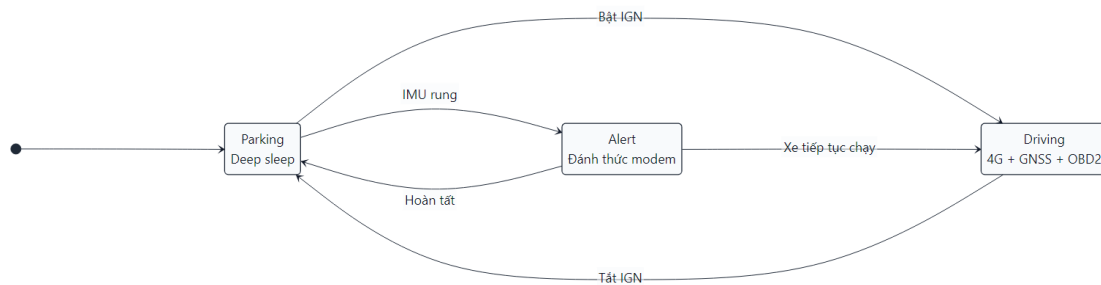
Chế độ 3 — Alert Mode (ưu tiên phản ứng nhanh khi có sự cố):

- IMU đánh thức ESP32 ngay lập tức qua interrupt
- Bật 4G + GPS, gửi cảnh báo ưu tiên lên server
- Tiếp tục theo dõi liên tục cho đến khi được xác nhận an toàn
- Sử dụng pin dự phòng nếu nguồn chính bị cắt

Bên cạnh đó, hệ thống còn có mạch chuyển nguồn dự phòng với cơ chế hysteresis để tránh đóng/ngắt liên tục khi điện áp dao động quanh ngưỡng. Thiết bị tự động chuyển

sang pin dự phòng khi điện áp ắc quy tụt xuống dưới mức an toàn theo từng profile: 12V dùng $Switch_{OFF} = 12.0\text{ V}$, $Switch_{ON} = 12.2\text{ V}$; 24V dùng $Switch_{OFF} = 24.0\text{ V}$, $Switch_{ON} = 24.4\text{ V}$, qua đó bảo vệ ắc quy không bị rút cạn quá mức [7].

Nhờ cách tổ chức này, năng lượng không còn là bài toán tối ưu từng linh kiện riêng lẻ, mà trở thành một phần của logic vận hành toàn hệ thống.



Hình 1.4: Sơ đồ chuyển đổi giữa các chế độ năng lượng

1.4.4. Công nghệ sử dụng

Bảng 1.3: Tổng hợp công nghệ sử dụng trong dự án

Thành phần	Công nghệ	Phiên bản	Vai trò
Vi điều khiển	ESP32-S3	—	MCU chính, xử lý và điều khiển
LTE + GNSS	SIMCom SIM7600CE-T (LTE + GNSS tích hợp)	—	Truyền dữ liệu 4G và định vị GPS
OBD2 Adapter	vgate iCar Pro	BLE 4.0	Đọc dữ liệu chẩn đoán xe
Cảm biến gia tốc	LIS3DSH	—	Phát hiện chuyển động và rung
Pin dự phòng	18650 1S Li-ion	3500 mAh	Nguồn điện dự phòng
Framework firmware	ESP-IDF	5.x	Phát triển firmware nhúng
RTOS	FreeRTOS	—	Hệ điều hành thời gian thực
MQTT Broker	EMQX	5.x	Tiếp nhận dữ liệu IoT
API Server	Express.js + TypeScript	—	REST API backend

Bảng 1.3: Tổng hợp công nghệ sử dụng trong dự án (tiếp theo)

Thành phần	Công nghệ	Phiên bản	Vai trò
Cơ sở dữ liệu quan hệ	PostgreSQL	16	Lưu trữ dữ liệu có cấu trúc
Cơ sở dữ liệu chuỗi thời gian	VictoriaMetrics	—	Lưu trữ telemetry
Nhật ký sự kiện	VictoriaLogs	—	Lưu trữ logs
Frontend	Next.js 15 + React 19	—	Giao diện web
Bản đồ	Leaflet	—	Hiển thị bản đồ thời gian thực
Biểu đồ	ECharts	—	Trực quan hóa dữ liệu
WebSocket	Socket.IO	4.8	Giao tiếp thời gian thực
Container hóa	Docker + Docker Compose	—	Triển khai dịch vụ
Giám sát	Grafana + VictoriaMetrics, VictoriaLogs	—	Dashboard giám sát hạ tầng

1.5. Kết quả và khuyến nghị

1.5.1. Kết quả đạt được

Nhìn theo cách đó, các kết quả đạt được không nên hiểu như một danh sách các khối đã lắp xong, mà như bằng chứng cho thấy chuỗi giá trị từ thiết bị đến dashboard đã được hình thành. Có thể tóm lược theo bốn tầng chính như sau:

Về phần cứng:

- Hoàn thiện thiết bị tracker IoT với bo mạch riêng do nhóm thiết kế, sử dụng ESP32-S3-WROOM-1 làm vi điều khiển trung tâm, tích hợp modem LTE + GNSS SIMCom SIM7600CE-T, cảm biến gia tốc LIS3DSH, khối nguồn đa rail, pin dự phòng 18650 1S và kết nối OBD2 BLE qua adapter vgate iCar Pro.
- Quá trình lựa chọn linh kiện bám theo datasheet, độ ổn định chất lượng và khả năng cung ứng trong nước để thuận lợi cho việc chế tạo, kiểm thử và thay thế khi cần.
- Hệ thống quản lý năng lượng đa chế độ hoạt động hiệu quả, với mức tiêu thụ điện ngủ sâu đạt yêu cầu ($< 500 \mu A$), đảm bảo không làm cạn ắc quy xe trong quá trình sử dụng bình thường.
- Mạch Low Voltage Disconnect (LVD) bảo vệ ắc quy xe hiệu quả, tự động ngắt khi điện áp tụt dưới ngưỡng an toàn.

Về firmware:

- Phát triển thành công firmware trên ESP-IDF với FreeRTOS, hiện thực đầy đủ các module: giao tiếp BLE OBD2, điều khiển modem UART, đọc cảm biến IMU, quản lý năng lượng, và truyền dữ liệu MQTT.
- Cơ chế phát hiện chuyển động bất thường qua IMU hoạt động chính xác, có khả năng đánh thức hệ thống từ chế độ ngủ sâu trong vòng < 3 giây.

Về cloud/backend:

- Xây dựng thành công hạ tầng cloud bao gồm EMQX MQTT Broker, dịch vụ MQTT Bridge, PostgreSQL, VictoriaMetrics và VictoriaLogs, toàn bộ được container hóa bằng Docker.
- Backend API (Express.js + TypeScript) với kiến trúc Domain-Driven Design, cung cấp các endpoint quản lý xe, thiết bị, telemetry, cảnh báo, geofences, và xác thực người dùng.
- Hệ thống đã kiểm thử ở mức 50+ thiết bị đồng thời với độ trễ end-to-end < 500 ms, đồng thời cho thấy còn dư địa mở rộng ổn định tới khoảng 100 thiết bị.

Về frontend:

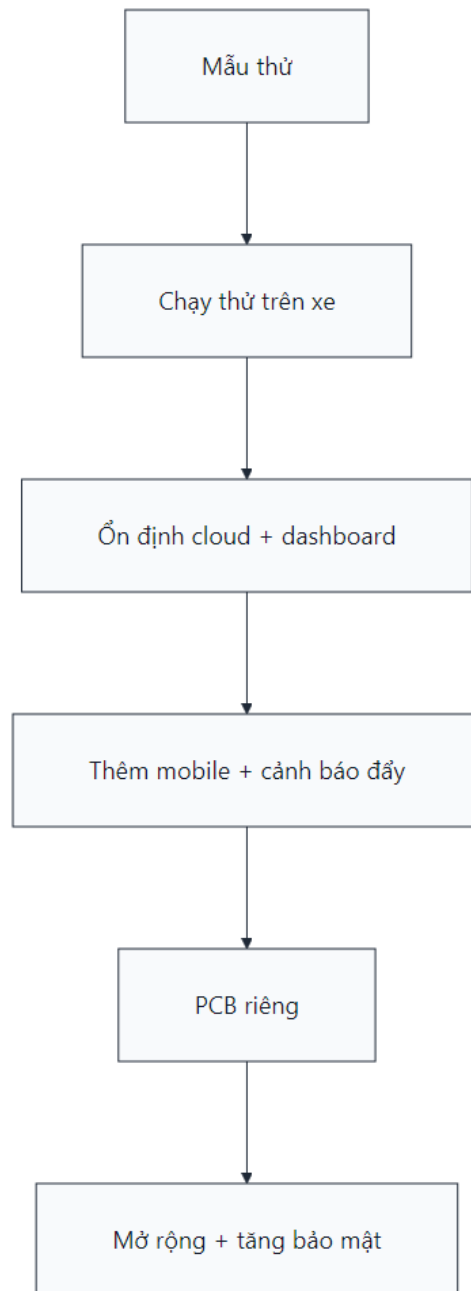
- Giao diện web quản lý (Next.js 15) với bản đồ thời gian thực (Leaflet), biểu đồ phân tích (ECharts), hệ thống cảnh báo trực tuyến, và bảng điều khiển tổng quan (dashboard).
- Cập nhật vị trí xe trên bản đồ với độ trễ < 2 giây thông qua WebSocket (Socket.IO).

1.5.2. Khuyến nghị và hướng phát triển

Trên cơ sở các kết quả đã đạt được và các giới hạn hiện hữu, các hướng phát triển tiếp theo được sắp xếp theo logic mở rộng từ việc làm chắc hệ lõi đến việc mở rộng giá trị sử dụng của hệ thống:

1. **Ứng dụng di động (Phase 2):** Phát triển ứng dụng Flutter WebView Hybrid cho phép người quản lý giám sát đội xe trên điện thoại di động, nhận thông báo push notification khi có cảnh báo [8].
2. **Tăng cường bảo mật:** Triển khai TLS/SSL cho kết nối MQTT trong môi trường production, áp dụng MQTT ACL chi tiết hơn, và bổ sung cơ chế mã hóa dữ liệu end-to-end giữa thiết bị và server.
3. **Tối ưu hóa hiệu năng:** Nghiên cứu và áp dụng các kỹ thuật nén dữ liệu telemetry (mã hóa sai khác, protobuf) để giảm băng thông 4G, đặc biệt khi vận hành đội xe lớn (> 500 xe).
4. **Tích hợp trí tuệ nhân tạo:** Áp dụng các mô hình machine learning để phân tích hành vi lái xe (driving behavior analysis), dự đoán bảo trì (predictive maintenance), và phát hiện bất thường (anomaly detection) từ dữ liệu telemetry.

5. **Mở rộng phạm vi OBD2:** Hỗ trợ đọc thêm nhiều thông số chẩn đoán xe, bao gồm mã lỗi DTC (Diagnostic Trouble Codes), dữ liệu động cơ nâng cao, và tích hợp với các loại xe điện (EV).
6. **Cải thiện khả năng mở rộng (scalability):** Nghiên cứu kiến trúc microservices với message queue (Apache Kafka hoặc RabbitMQ) để xử lý luồng dữ liệu lớn hơn khi số lượng thiết bị tăng lên hàng ngàn.
7. **Tối ưu PCB cho sản xuất:** Nâng thiết kế PCB hiện tại theo hướng DFM/EMI, giảm kích thước, tăng độ bền cơ khí và chuẩn bị cho sản xuất hàng loạt.



Hình 1.5: Lộ trình phát triển dự án theo các giai đoạn (Roadmap)

Kết luận chương 1

Chương 1 đã trả lời câu hỏi vì sao đề tài này cần được thực hiện và hệ thống cần đạt được điều gì trong bối cảnh thực tế của doanh nghiệp cho thuê xe tự lái. Từ bối cảnh thị trường, nhu cầu vận hành đến các thách thức về năng lượng, kết nối và dữ liệu, chương đã thiết lập bộ mục tiêu, phạm vi và tiêu chí đánh giá cho toàn bộ đồ án. Nếu Chương 1 là phần đặt vấn đề, thì Chương 2 sẽ chuyển sang câu hỏi quan trọng hơn: cần giải quyết chính xác những bài toán kỹ thuật nào để biến mục tiêu đó thành một hệ thống khả thi.

CHƯƠNG 2. PHÂN TÍCH VẤN ĐỀ KỸ THUẬT

Nếu Chương 1 đặt ra bối cảnh và mục tiêu của đề tài, thì Chương 2 đi sâu vào phần cốt lõi hơn: bóc tách bài toán kỹ thuật thật sự của hệ thống. Trọng tâm của chương không chỉ là liệt kê công nghệ, mà là làm rõ vì sao mỗi quyết định về phần cứng, truyền thông, lưu trữ và kiến trúc đều phải xuất phát từ nhu cầu vận hành thực tế của một hệ thống giám sát phương tiện.

2.1. Mô tả vấn đề – Problem statement

2.1.1. Bối cảnh thực tế

Trong bối cảnh ngành cho thuê ô tô và quản lý đội xe tại Việt Nam tiếp tục mở rộng, nhu cầu giám sát phương tiện theo thời gian thực đã trở thành yêu cầu thiết yếu đối với doanh nghiệp vận tải. Theo thống kê của Bộ Giao thông Vận tải, số lượng ô tô cá nhân và thương mại tăng trung bình 10–12% mỗi năm trong giai đoạn 2020–2025, kéo theo áp lực ngày càng lớn về quản lý, an ninh và tối ưu vận hành [1]. Tuy nhiên, điều doanh nghiệp cần không chỉ là dữ liệu vị trí rời rạc, mà là khả năng nhìn thấy trạng thái phương tiện đủ sớm để đưa ra quyết định vận hành. Khoảng cách giữa nhu cầu này và cách làm hiện tại thể hiện ở các hạn chế sau:

- **Giám sát thủ công:** Tài xế báo cáo vị trí qua điện thoại, không có dữ liệu liên tục, không thể xác minh chính xác hành trình. Phương pháp này phụ thuộc hoàn toàn vào yếu tố con người, dễ xảy ra sai sót và gian lận.
- **Thiết bị GPS đơn giản:** Chỉ cung cấp tọa độ vị trí, không đọc được dữ liệu động cơ (tốc độ, vòng tua, nhiên liệu), không hỗ trợ phát hiện bất thường khi xe đậu. Thiếu khả năng tích hợp sâu với hệ thống quản lý.
- **Giải pháp thương mại (fleet management):** Chi phí cao (50–200 USD/thiết bị + phí dịch vụ hàng tháng), phụ thuộc vào nhà cung cấp nước ngoài, khó tùy biến theo nhu cầu cụ thể của doanh nghiệp Việt Nam.

2.1.2. Xác định vấn đề kỹ thuật

Từ bối cảnh thực tiễn đó, bài toán của đề tài có thể quy về bốn câu hỏi kỹ thuật nền tảng. Đây cũng là bốn câu hỏi chi phối gần như mọi quyết định ở các chương sau, từ lựa chọn phần cứng đến thiết kế cloud:

Vấn đề 1: Quản lý năng lượng trong môi trường ô tô

Thiết bị theo dõi phải hoạt động liên tục 24/7 trong môi trường ô tô, nơi nguồn điện luôn bị ràng buộc bởi dung lượng ắc quy. Khi xe tắt máy (IGN OFF), thiết bị vẫn lấy điện từ ắc quy 12V hoặc 24V của xe; nếu duy trì chế độ theo dõi liên tục, ắc quy có thể cạn sau vài tuần và làm xe không khởi động được [2]. Vì vậy, câu hỏi không đơn thuần là ”thiết bị có hoạt động được không”, mà là ”thiết bị có hoạt động đủ thông minh để không gây

phiên toái cho chính chiếc xe mà nó đang giám sát hay không”.

Vấn đề 2: Giám sát khi xe đậu

Khi xe tắt máy, hệ thống cần tiết kiệm điện nhưng vẫn phát hiện được các chuyển động bất thường như bị trộm hoặc bị kéo cầu. Vì vậy, thiết bị phải có cơ chế đánh thức nhanh khi phát hiện sự kiện, đồng thời duy trì dòng tiêu thụ ở mức rất thấp. Nói cách khác, khi xe ”im lặng” thì thiết bị cũng phải ”ngủ”, nhưng vẫn phải tỉnh dậy đủ nhanh khi có sự cố.

Vấn đề 3: Truyền dữ liệu qua mạng di động

Hệ thống phải truyền liên tục dữ liệu GPS và OBD2 qua mạng 4G/LTE với độ trễ thấp, đồng thời duy trì cơ chế reconnect ổn định khi mất sóng. Bên cạnh đó, kiến trúc truyền thông cũng cần chứa dư địa để bổ sung lưu đệm cục bộ nếu triển khai ở quy mô lớn hơn. Việc lựa chọn giao thức phù hợp như MQTT, HTTP hay CoAP không chỉ là quyết định công nghệ, mà còn ảnh hưởng trực tiếp đến hiệu suất và độ tin cậy toàn hệ thống.

Vấn đề 4: Kiến trúc đám mây có khả năng mở rộng

Hệ thống cần hỗ trợ nhiều phương tiện đồng thời, cập nhật vị trí theo thời gian thực qua WebSocket, bảo đảm an toàn thông qua xác thực phiên và xử lý được luồng dữ liệu lớn đến từ nhiều thiết bị IoT. Với người sử dụng cuối, điều này được cảm nhận rất đơn giản: bản đồ phải cập nhật kịp thời, cảnh báo phải đến đúng lúc, và dữ liệu phải nhất quán ngay cả khi số lượng xe tăng lên.

2.1.3. Mục tiêu kỹ thuật

Từ bốn vấn đề nền tảng trên, đồ án rút ra bốn nhóm mục tiêu kỹ thuật. Mỗi nhóm vừa trả lời một nhu cầu vận hành cụ thể của doanh nghiệp cho thuê xe, vừa trở thành tiêu chí để sàng lọc và lựa chọn giải pháp ở Chương 3:

1. Thiết kế thiết bị tracker tích hợp GNSS và OBD2 dựa trên vi điều khiển ESP32-S3, đồng thời tối ưu tiêu thụ năng lượng khi xe đậu (mục tiêu dòng deep sleep toàn thiết bị dưới 500 μ A trong phiên bản phần cứng hiện tại và tiếp tục giảm ở các vòng tối ưu PCB tiếp theo).
2. Xây dựng cơ chế quản lý nguồn thông minh với pin dự phòng và mạch Low Voltage Disconnect (LVD) để thiết bị bám được trạng thái xe nhưng không làm ảnh hưởng đến khả năng khởi động của phương tiện.
3. Thiết kế đường truyền dữ liệu sử dụng MQTT qua mạng 4G/LTE, ưu tiên tự phục hồi kết nối và chứa sẵn phương án lưu đệm khi mất mạng kéo dài.
4. Phát triển nền tảng đám mây có khả năng mở rộng, bao gồm API server, cơ sở dữ liệu quan hệ, cơ sở dữ liệu chuỗi thời gian và giao diện web theo dõi thời gian thực.

2.2. Bối cảnh và cơ sở kỹ thuật – Background and Technical reviews

2.2.1. Tổng quan về hệ thống IoT trong giám sát phương tiện

Internet of Things (IoT) là mô hình cho phép các thiết bị vật lý kết nối với nhau và với hạ tầng đám mây thông qua Internet. Ở góc nhìn đơn giản, một hệ thống IoT giám sát phương tiện luôn phải đi qua ba bước: thu dữ liệu trên xe, truyền dữ liệu lên mạng, rồi biến dữ liệu đó thành thông tin mà người quản lý có thể đọc và hành động. Tương ứng với ba bước đó, hệ thống thường được tổ chức thành ba lớp chính [3]:

- **Lớp cảm biến và thu thập dữ liệu (Perception Layer):** Bao gồm các cảm biến GPS/GNSS, gia tốc kế (IMU), giao diện OBD2, và các cảm biến môi trường. Lớp này chịu trách nhiệm thu thập dữ liệu thô từ phương tiện.
- **Lớp mạng và truyền thông (Network Layer):** Sử dụng các giao thức truyền thông như MQTT, HTTP, CoAP qua hạ tầng mạng di động (4G/LTE) để truyền dữ liệu từ thiết bị lên đám mây.
- **Lớp ứng dụng và xử lý (Application Layer):** Bao gồm các dịch vụ đám mây (MQTT broker, API server, cơ sở dữ liệu) và giao diện người dùng (web dashboard, ứng dụng di động).

Trong triển khai của đề tài, kiến trúc ba lớp trên được bổ sung thêm một lớp đệm dữ liệu cục bộ tại thiết bị bằng **microSD SDMMC 4-bit + FATFS**. Có thể hình dung lớp này như một "vùng nhớ chờ", dùng để giữ lại dữ liệu telemetry khi mạng 4G/LTE gián đoạn, sau đó phát lại (replay) theo đúng thứ tự khi kết nối phục hồi. Cách tổ chức này giúp tăng độ liên tục dữ liệu trong vận hành thực tế mà không làm thay đổi kiến trúc cloud ở tầng trên.

2.2.2. Các giải pháp giám sát phương tiện hiện có

Bảng 2.1: So sánh các giải pháp giám sát phương tiện

Tiêu chí	Giám sát thủ công	GPS Tracker đơn giản	Fleet Management thương mại	Hệ thống đề xuất
Chi phí thiết bị	Không	500.000–1.500.000 VND	1.000.000–5.000.000 VND	1.514.000 VND
Phí dịch vụ hàng tháng	Không	50.000–100.000 VND	200.000–500.000 VND	Chi phí 4G SIM (70.000 VND)
Độ chính xác vị trí	Thấp (báo cáo thủ công)	Trung bình (GPS)	Cao (GPS + A-GPS)	Cao (GNSS đa hệ thống)

Bảng 2.1: So sánh các giải pháp giám sát phương tiện (tiếp theo)

Tiêu chí	Giám sát thủ công	GPS Tracker đơn giản	Fleet Management thương mại	Hệ thống đề xuất
Dữ liệu động cơ (OBD2)	Không	Không	Có (tùy model)	Có (BLE OBD2)
Phát hiện bất thường khi đậu	Không	Hạn chế	Có	Có (IMU + deep sleep)
Khả năng tùy biến	Không áp dụng	Thấp	Thấp (phụ thuộc vendor)	Cao (mã nguồn mở)
Quản lý năng lượng	Không áp dụng	Cơ bản	Tốt	Tốt (đa chế độ + pin dự phòng)
Tích hợp hệ thống	Không	Hạn chế (API riêng)	API do vendor	API mở (REST + WebSocket)

Bảng so sánh cho thấy phương án đề xuất giữ được các năng lực quan trọng của hệ thống thương mại như độ chính xác vị trí, khả năng đọc OBD2 và phát hiện bất thường, nhưng chi phí đầu tư thấp hơn và mức độ tùy biến cao hơn. Với bài toán của doanh nghiệp cho thuê xe trong nước, đây là hướng lựa chọn phù hợp hơn về cả kỹ thuật lẫn chi phí vận hành.

2.2.3. So sánh giao thức truyền thông IoT

Việc lựa chọn giao thức truyền thông không thể chỉ hiểu là chọn một “chuẩn phổ biến”. Với hệ thống theo dõi phương tiện, đây là quyết định chi phối trực tiếp lượng dữ liệu phải truyền, khả năng chịu mất sóng và cách backend mở rộng về sau [4].

Bảng 2.2: So sánh các giao thức truyền thông IoT

Tiêu chí	MQTT	HTTP/REST	CoAP
Mô hình truyền thông	Publish–Subscribe	Request/Response	Request/Response
Giao thức tầng giao vận	TCP	TCP	UDP
Overhead header	2 bytes (tối thiểu)	Hàng trăm bytes	4 bytes
Chất lượng dịch vụ (QoS)	3 mức (0, 1, 2)	Không có	2 mức (Confirmable/Non-confirmable)
Tiêu thụ băng thông	Thấp	Cao	Thấp

Bảng 2.2: So sánh các giao thức truyền thông IoT (tiếp theo)

Tiêu chí	MQTT	HTTP/REST	CoAP
Hỗ trợ bi-directional	Có (Subscribe)	Không (cần polling hoặc WebSocket)	Có (Observe)
Phù hợp cho IoT	Rất tốt	Trung bình	Tốt
Độ phổ biến	Rất cao	Rất cao	Trung bình
Hỗ trợ EMQX Broker	Có (native)	Có (qua plugin)	Hạn chế

Phân tích lựa chọn: Kết quả so sánh cho thấy MQTT phù hợp hơn không phải vì nó “cao cấp” hơn HTTP hay CoAP, mà vì cách nó trao đổi dữ liệu gần với nhịp vận hành của hệ thống: thiết bị gửi nhiều bản tin nhỏ qua mạng di động không phải lúc nào cũng ổn định, đồng thời đôi lúc cần nhận lệnh ngược từ server. Cụ thể:

- **Overhead thấp:** Header tối thiểu 2 bytes, giảm băng thông và tiết kiệm năng lượng cho thiết bị IoT chạy bằng pin.
- **Mô hình Publish/Subscribe:** Cho phép nhiều thành phần nhận dữ liệu cùng lúc mà không làm thiết bị trên xe phải gửi lặp đi lặp lại cho từng nơi nhận. Điều này phù hợp với kiến trúc có dashboard, dịch vụ xử lý và các kênh cảnh báo cùng khai thác một nguồn dữ liệu.
- **QoS linh hoạt:** Hệ thống không cần mọi bản tin đều được bảo đảm ở cùng một mức. QoS 0 phù hợp với dữ liệu telemetry tần suất cao như vị trí GPS, trong khi QoS 1 phù hợp hơn với cảnh báo và lệnh điều khiển vì các bản tin này cần được đảm bảo tới nơi ít nhất một lần.
- **Retain Message và Last Will:** Hai cơ chế này giúp hệ thống phản ánh trạng thái online/offline của thiết bị rõ ràng hơn, từ đó người vận hành có thể phân biệt giữa “xe không có dữ liệu mới” và “thiết bị vừa mất kết nối”.

2.2.4. So sánh vi điều khiển (MCU)

Bảng 2.3: So sánh các vi điều khiển cho ứng dụng IoT tracker

Tiêu chí	ESP32-S3	STM32L4	Raspberry Pi Zero 2W
Kiến trúc	Xtensa LX7, dual-core, 240 MHz	ARM Cortex-M4, 80 MHz	ARM Cortex-A53, quad-core, 1 GHz

Bảng 2.3: So sánh các vi điều khiển cho ứng dụng IoT tracker (tiếp theo)

Tiêu chí	ESP32-S3	STM32L4	Raspberry Pi Zero 2W
RAM	512 KB SRAM + 8 MB PSRAM	256 KB SRAM	512 MB DRAM
BLE	BLE 5.0 (tích hợp)	Không (cần module ngoài)	BLE 5.0 (tích hợp)
Wi-Fi	802.11 b/g/n (tích hợp)	Không	802.11 b/g/n (tích hợp)
Tiêu thụ deep sleep	10–15 μ A	1–2 μ A	Không hỗ trợ deep sleep
Giá thành (VND)	80.000–150.000	150.000–300.000	400.000–600.000
Framework phát triển	Arduino / ESP-IDF	STM32CubeIDE / Mbed	Linux / Python
Cộng đồng hỗ trợ	Rất lớn	Lớn	Rất lớn
Độ phù hợp cho tracker	Cao	Trung bình	Thấp

Phân tích lựa chọn: Nếu chỉ nhìn riêng thông số low-power, STM32L4 có lợi thế đáng kể. Tuy nhiên, khi đặt vào bối cảnh toàn hệ thống - nơi thiết bị phải đồng thời xử lý BLE OBD2, modem, IMU và logic trạng thái - ESP32-S3 cho thấy điểm cân bằng tốt hơn giữa tích hợp, chi phí và độ phức tạp triển khai. Các lý do chính là:

- **BLE 5.0 tích hợp:** Kết nối trực tiếp với OBD2 adapter (vgate iCar Pro) mà không cần module BLE ngoài, giảm độ phức tạp phần cứng và chi phí.
- **Chi phí thấp:** Rẻ hơn 30–50% so với STM32L4 với cùng khả năng xử lý, phù hợp với ngân sách đồ án.
- **Hệ sinh thái phát triển phong phú:** Hỗ trợ ESP-IDF (chính thức) và Arduino framework, cộng đồng lớn, nhiều thư viện có sẵn cho MQTT, BLE, GNSS. Điều này làm giảm đáng kể thời gian thử sai khi hiện thực đồ án.
- **Deep sleep 10–15 μ A ở mức MCU:** Mặc dù cao hơn STM32L4 (1–2 μ A), mức này vẫn là nền tảng phù hợp cho một thiết kế low-power. Trong hệ thống hoàn chỉnh, dòng ngủ sâu còn phụ thuộc mạch nguồn, cảm biến và linh kiện phụ trợ, nên chênh lệch ở mức riêng MCU không phải là yếu tố duy nhất quyết định tuổi thọ pin toàn thiết bị.

2.2.5. So sánh cơ sở dữ liệu

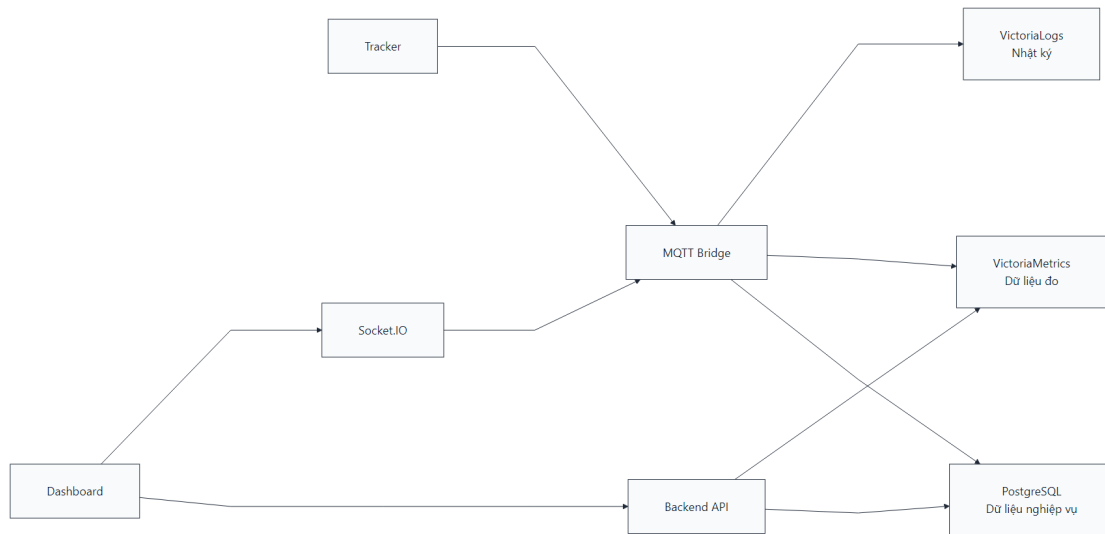
Bảng 2.4: So sánh các giải pháp cơ sở dữ liệu cho hệ thống IoT

Tiêu chí	PostgreSQL + VictoriaMetrics	MongoDB + InfluxDB	TimescaleDB
Dữ liệu quan hệ (users, vehicles)	Rất tốt (PostgreSQL)	Trung bình (MongoDB)	Tốt (PostgreSQL core)
Dữ liệu chuỗi thời gian (telemetry)	Rất tốt (VictoriaMetrics)	Tốt (InfluxDB)	Tốt (TimescaleDB extension)
Hiệu suất ghi (write)	Cao (VM: hàng triệu điểm/giây)	Cao (InfluxDB: hàng triệu điểm/giây)	Trung bình
Nén dữ liệu	Rất tốt (VM: 10–70x nén)	Tốt (InfluxDB: TSM)	Tốt (PostgreSQL compression)
Tài nguyên tiêu thụ	Thấp (VM: 1–2 GB RAM)	Cao (InfluxDB: 4+ GB RAM)	Trung bình
Ngôn ngữ truy vấn	SQL (PG) + MetricsQL (VM)	MongoDB Query + Flux/InfluxQL	SQL
Tích hợp Grafana	Có (native)	Có	Có
Giấy phép	Apache 2.0	SSPL (MongoDB) + MIT (InfluxDB OSS)	Apache 2.0 (Community)
Độ phức tạp vận hành	Trung bình (2 hệ thống riêng)	Cao (2 hệ thống khác nhau)	Thấp (1 hệ thống)

Phân tích lựa chọn: Với hệ thống này, không phải mọi dữ liệu đều giống nhau. Thông tin người dùng, phương tiện hay hành trình cần cấu trúc quan hệ chặt chẽ; trong khi dữ liệu vị trí và cảm biến lại là dữ liệu biến đổi liên tục theo thời gian. Vì vậy, sự kết hợp PostgreSQL + VictoriaMetrics được chọn vì:

- **Phân tách trách nhiệm rõ ràng:** PostgreSQL xử lý dữ liệu quan hệ như người dùng, phương tiện, cảnh báo và geofence; VictoriaMetrics xử lý dữ liệu chuỗi thời gian như tọa độ GPS, dữ liệu OBD2 và telemetry cảm biến. Mỗi hệ thống làm tốt phần việc mà nó được thiết kế để phục vụ.
- **Hiệu suất và tài nguyên:** VictoriaMetrics có tỷ lệ nén dữ liệu rất cao (10–70x) và tiêu thụ ít RAM hơn InfluxDB, phù hợp với server có tài nguyên hạn chế.

- **Tương thích Grafana:** Cả hai đều hỗ trợ Grafana native, thuận tiện cho việc xây dựng dashboard giám sát thống nhất mà không cần thêm công cụ trung gian.



Hình 2.1: Sơ đồ kiến trúc dữ liệu của hệ thống - Data Architecture Diagram

2.3. Yêu cầu kỹ thuật và các tiêu chuẩn thiết kế – Design criteria and Constraints

2.3.1. Yêu cầu chức năng

Những yêu cầu chức năng dưới đây đánh dấu bước chuyển từ “bài toán” sang “năng lực hệ thống”. Có thể xem chúng như năm lời hứa vận hành tối thiểu mà hệ thống phải thực hiện được để người quản lý thật sự dùng được trong bối cảnh đội xe vận hành ngoài thực tế.

FC-01: Theo dõi vị trí thời gian thực

- Gửi vị trí GPS định kỳ (5–30 giây khi lái xe, 10–30 phút khi đậu xe)
- Độ chính xác vị trí: dưới 5 mét trong điều kiện trời quang
- Hỗ trợ đa hệ thống định vị (GPS, GLONASS, BeiDou)

FC-02: Đọc dữ liệu động cơ qua OBD2

- Kết nối BLE với OBD2 adapter (vgate iCar Pro)
- Đọc trạng thái IGN (bật/tắt máy), RPM, tốc độ, nhiên liệu
- Tần suất đọc: 5–30 giây khi xe chạy (mặc định 5 giây)

FC-03: Phát hiện bất thường khi đậu xe

- IMU (LIS3DSH) phát hiện chuyển động/rung bất thường
- Đánh thức ESP32 từ deep sleep trong vòng 100ms
- Gửi cảnh báo ưu tiên (priority alert) lên server

FC-04: Quản lý năng lượng thông minh

- Chuyển đổi giữa 3 chế độ: Lái xe (Active) - Đậu xe (Sleep) - Cảnh báo (Alert)
- Pin dự phòng tự động tiếp quản theo profile nguồn: hệ 12V tại ngưỡng $Switch_{OFF} = 12.0\text{ V}$, hệ 24V tại ngưỡng $Switch_{OFF} = 24.0\text{ V}$
- Có phương án dự phòng suy luận trạng thái IGN qua ADC khi kết nối OBD2 BLE không sẵn sàng

FC-05: Giao diện web giám sát

- Bản đồ thời gian thực (Leaflet) hiển thị vị trí tất cả phương tiện
- Biểu đồ dữ liệu OBD2 (ECharts) - tốc độ, RPM, nhiên liệu
- Hệ thống cảnh báo và thông báo (alert notifications)
- Quản lý phương tiện, tài xế, khách hàng, hành trình

Năm nhóm chức năng trên bao quát trọn chuỗi nhu cầu cốt lõi của hệ thống: biết xe đang ở đâu, hiểu xe đang như thế nào, phát hiện khi có bất thường, không làm phiền hệ thống điện của xe và trình bày thông tin ở dạng mà người vận hành có thể đọc nhanh, xử lý nhanh.

2.3.2. Yêu cầu phi chức năng

Nếu yêu cầu chức năng trả lời “hệ thống phải làm được gì”, thì yêu cầu phi chức năng trả lời “hệ thống phải làm tốt tới mức nào”. Đây là lớp tiêu chí cho biết nguyên mẫu có đủ độ chín để bước ra môi trường vận hành thật hay chưa, đặc biệt khi phải theo dõi nhiều xe và chạy liên tục.

NFR-01: Hiệu suất (Performance)

- Độ trễ end-to-end (thiết bị đến dashboard): dưới 3 giây trong điều kiện mạng bình thường
- API response time: dưới 200ms cho 95% request
- Hỗ trợ đồng thời tối thiểu 50 phương tiện kết nối MQTT, với dư địa mở rộng ổn định tới khoảng 100 thiết bị

NFR-02: Độ tin cậy (Reliability)

- Tự động kết nối lại (reconnect) khi mạng phục hồi
- Giữ được cấu hình và trạng thái quan trọng qua NVS; buffer telemetry cục bộ là hướng mở rộng của thiết bị
- MQTT QoS 1 cho các tin nhắn cảnh báo (đảm bảo gửi ít nhất một lần)

NFR-03: Bảo mật (Security)

- Xác thực phiên bằng opaque session token; chỉ lưu hash SHA-256 của token trong cơ sở dữ liệu
- MQTT ACL (Access Control List) phân quyền theo từng thiết bị
- Mã hóa TLS cho kết nối MQTT và HTTPS trong môi trường sản xuất

NFR-04: Khả năng mở rộng (Scalability)

- Kiến trúc microservices với Docker container độc lập
- MQTT broker (EMQX) hỗ trợ phân cụm (clustering) để mở rộng
- Cơ sở dữ liệu chuỗi thời gian (VictoriaMetrics) tối ưu cho luồng dữ liệu lớn

NFR-05: Khả năng bảo trì (Maintainability)

- Mã nguồn tổ chức theo mô hình Domain-Driven Design (DDD)
- API RESTful với tài liệu Swagger/OpenAPI tự động
- Logging tập trung với VictoriaLogs và dashboard Grafana

2.3.3. Ràng buộc thiết kế

Các ràng buộc sau đây giới hạn không gian thiết kế của đề tài. Chúng buộc nhóm phải chọn phương án cân bằng giữa tính khả thi học thuật, khả năng triển khai thực tế trên xe và ngân sách sẵn có.

Ràng buộc phần cứng:

- Điện áp đầu vào: 12V hoặc 24V DC từ ắc quy xe (dao động phụ thuộc cấu hình hệ thống điện)
- Nhiệt độ hoạt động: -10 độ C đến 70 độ C (môi trường trong xe ô tô)
- Rung động và sốc: Chịu được rung động liên tục khi xe vận hành trên đường xá
- Kích thước: Đủ nhỏ để lắp đặt kín đáo trong xe (không lớn hơn 120x80x40 mm)

Ràng buộc phần mềm:

- Backend framework: Express.js + TypeScript (yêu cầu từ công nghệ hiện có của nhóm)
- Frontend framework: Next.js 15 + React 19 (yêu cầu học thuật)
- MQTT Broker: EMQX (giấy phép mã nguồn mở, hỗ trợ rules engine)
- Triển khai: Docker Compose trên máy chủ Linux (mỗi dịch vụ có docker-compose.yml riêng)

Ràng buộc về chi phí:

- Tổng chi phí phần cứng: dưới 2.000.000 VND cho một bộ tracker
- Ưu tiên các linh kiện có datasheet rõ ràng, chất lượng ổn định và nguồn cung trong nước để tiếp cận
- Ưu tiên các giải pháp mã nguồn mở để giảm chi phí giấy phép

2.3.4. Tiêu chuẩn thiết kế áp dụng

Bảng dưới đây tổng hợp các chuẩn kỹ thuật và quy ước triển khai được dùng làm nền cho hệ thống. Điểm quan trọng không phải là gom thật nhiều chuẩn, mà là chọn đúng

những chuẩn thực sự chi phối khả năng tương thích giữa thiết bị, mạng truyền và ứng dụng.

Bảng 2.3.4A: Tổng hợp chuẩn và quy ước kỹ thuật áp dụng trong hệ thống

Chuẩn / quy ước	Mô tả	Áp dụng trong hệ thống
IEEE 802.15.1 (Bluetooth)	Chuẩn BLE 4.0/5.0	Kết nối OBD2 adapter
3GPP LTE Cat-4	Chuẩn 4G/LTE	Truyền dữ liệu qua modem SIM7600CE-T (Cat-4)
NMEA 0183	Chuẩn dữ liệu GPS	Phân tích tọa độ từ khối GNSS tích hợp
SAE J1979 (OBD2)	Chuẩn chẩn đoán động cơ	Đọc dữ liệu qua OBD2 BLE adapter
MQTT v5.0 (OASIS)	Giao thức IoT messaging	Truyền dữ liệu thiết bị - server
REST (RFC 7231)	Kiến trúc API	Backend API server
Opaque session token + MQTT ACL	Cơ chế xác thực và phân quyền	Xác thực phiên người dùng, phân quyền truy cập thiết bị theo ACL MQTT

2.4. Yêu cầu từ các bên liên quan – Stakeholder requirements

2.4.1. Xác định các bên liên quan

Hệ thống IoT Vehicle Tracking System liên quan đến nhiều nhóm sử dụng với nhu cầu khác nhau. Nếu không tách rõ ngay từ đầu, hệ thống rất dễ rơi vào tình trạng mạnh về mặt kỹ thuật nhưng lệch khỏi nhu cầu vận hành. Vì vậy, phần này xác định ai là người hưởng lợi trực tiếp, ai là người vận hành, ai là người bị tác động, từ đó giữ trọng tâm thiết kế vào những nhu cầu thực sự cần giải quyết.

2.4.2. Ma trận yêu cầu các bên liên quan

Bảng 2.5: Ma trận yêu cầu các bên liên quan

STT	Bên liên quan	Vai trò	Yêu cầu chính	Mức độ ưu tiên
1	Công ty cho thuê xe	Chủ sở hữu hệ thống	Giám sát vị trí tất cả xe theo thời gian thực; Nhận cảnh báo khi xe đi ra khỏi vùng cho phép (geofence); Báo cáo hành trình, quãng đường, nhiên liệu; Giảm thiểu thời gian xe đậu không sử dụng	Cao
2	Tài xế / Người thuê xe	Người vận hành xe	Không bị giám sát quá mức (quyền riêng tư); Thiết bị không ảnh hưởng đến vận hành xe; Không rút cạn ắc quy xe	Trung bình
3	Quản trị viên hệ thống	Người quản lý kỹ thuật	Giao diện quản lý dễ sử dụng; Hệ thống ổn định, ít lỗi; Có khả năng theo dõi trạng thái thiết bị (online/offline); Quản lý người dùng và phân quyền	Cao
4	Đội bảo trì	Kỹ thuật viên	Thiết bị dễ lắp đặt và bảo trì; Chẩn đoán lỗi từ xa (remote diagnostics); Cảnh báo bảo trì định kỳ (battery low, thiết bị offline)	Trung bình
5	Khách hàng (người thuê xe cuối)	Người dùng gián tiếp	Quá trình thuê xe nhanh gọn; Tin tưởng vào độ bảo mật của dữ liệu cá nhân; Nhận thông báo về trạng thái đơn thuê	Thấp (Phase 2)

2.4.3. Phân tích chi tiết yêu cầu

Từ ma trận trên, có thể thấy không phải mọi bên liên quan đều đòi hỏi cùng một loại giá trị. Nhóm sử dụng chính quan tâm đến hiệu quả vận hành và an ninh; nhóm quản trị quan tâm đến tính ổn định và khả năng kiểm soát; còn nhóm bảo trì cần hệ thống dễ chẩn đoán và dễ can thiệp khi có sự cố.

Công ty cho thuê xe (Stakeholder chính)

Đây là nhóm đối tượng trọng tâm của hệ thống. Các yêu cầu của nhóm này tập trung vào

ba khía cạnh:

1. *Giám sát và an ninh*: Theo dõi vị trí phương tiện 24/7, phát hiện và cảnh báo ngay khi có bất thường (xe bị di chuyển trái phép, vượt qua geofence, tốc độ vượt ngưỡng). Hệ thống cần gửi thông báo qua nhiều kênh (web dashboard, Telegram bot, email).
2. *Báo cáo và phân tích*: Xuất báo cáo hành trình chi tiết (quãng đường, thời gian, điểm dừng), thống kê nhiên liệu tiêu thụ, đánh giá hành vi lái xe (tốc độ trung bình, số lần phanh gấp). Các báo cáo này giúp tối ưu hóa chi phí vận hành và bảo trì.
3. *Quản lý đội xe*: Quản lý trạng thái từng phương tiện (đang cho thuê, đang bảo trì, sẵn sàng), lịch bảo trì định kỳ dựa trên số km hoặc thời gian, lịch sử cho thuê và doanh thu theo phương tiện.

Quản trị viên hệ thống

Yêu cầu của quản trị viên tập trung vào khả năng vận hành và giám sát hệ thống:

1. *Dashboard tổng quan*: Hiển thị trạng thái tất cả thiết bị (online/offline, mức pin, cường độ tín hiệu), số lượng phương tiện đang hoạt động, cảnh báo chưa xử lý.
2. *Quản lý thiết bị*: Thêm/xóa/sửa thông tin thiết bị tracker, cấp phát thiết bị cho phương tiện, gửi lệnh điều khiển từ xa (reset, cập nhật cấu hình, yêu cầu vị trí).
3. *Quản lý người dùng*: Tạo tài khoản, phân quyền (admin, manager, viewer), theo dõi lịch sử đăng nhập và thao tác (audit log).

Đội bảo trì

Yêu cầu của đội bảo trì hướng đến việc giảm thời gian và chi phí bảo trì:

1. *Chẩn đoán từ xa*: Xem trạng thái thiết bị (mức pin, nhiệt độ, cường độ tín hiệu 4G) từ xa, phát hiện và cảnh báo khi thiết bị gặp lỗi (mất kết nối kéo dài, pin yếu).
2. *Hướng dẫn lắp đặt*: Tài liệu hướng dẫn lắp đặt chi tiết, danh sách kiểm tra (checklist) khi lắp đặt mới hoặc bảo trì.

Nhìn tổng thể, ba nhóm yêu cầu này kéo hệ thống về một điểm cân bằng khá rõ: phải đủ mạnh để quản lý đội xe, nhưng cũng phải đủ gọn và ổn định để lắp đặt, vận hành và bảo trì trên thực địa. Đây cũng là cơ sở để ma trận truy xuất ở mục tiếp theo ánh xạ yêu cầu định tính sang các chức năng cụ thể của hệ thống.

2.4.4. Ma trận truy xuất yêu cầu - chức năng

Bảng 2.6: Ma trận truy xuất yêu cầu các bên liên quan với chức năng hệ thống

Yêu cầu bên liên quan	FC-01 (Vị trí)	FC-02 (OBD2)	FC-03 (Bắt thường)	FC-04 (Năng lượng)	FC-05 (Web)
Giám sát vị trí 24/7	X			X	X
Cảnh báo geofence	X				X
Báo cáo nhiên liệu		X			X
Phát hiện trộm xe			X	X	X
Không rút cạn ắc quy				X	
Quản lý thiết bị từ xa					X
Chẩn đoán lỗi từ xa	X	X		X	X

Ma trận truy xuất trên cho thấy các chức năng hệ thống (FC-01 đến FC-05) bao phủ toàn bộ yêu cầu của các bên liên quan chính. Trong đó, FC-04 (Quản lý năng lượng) và FC-05 (Giao diện web) là hai chức năng được nhắc đến nhiều nhất, qua đó khẳng định tầm quan trọng của thiết kế quản lý năng lượng thông minh và giao diện giám sát thân thiện.

Kết luận chương 2

Chương 2 đã chuyển bài toán ở mức ”ý tưởng” thành các ràng buộc kỹ thuật cụ thể: thiết bị phải tiết kiệm điện ra sao, truyền dữ liệu thế nào, lưu trữ ở đâu và kiến trúc cần mở rộng theo hướng nào. Thông qua quá trình khảo sát và đối sánh các giải pháp hiện có, chương đã làm rõ tiêu chí lựa chọn thay vì chỉ liệt kê công nghệ. Đây chính là cơ sở để Chương 3 bước sang phần quyết định quan trọng tiếp theo: chọn phương án thiết kế nào là phù hợp nhất với bài toán đã được phân tích.

CHƯƠNG 3. CÁC GIẢI PHÁP THIẾT KẾ – DESIGN SOLUTIONS

Sau khi xác định rõ yêu cầu và ràng buộc ở Chương 2, Chương 3 tập trung vào câu hỏi ”chọn gì và vì sao chọn như vậy”. Trình bày trong chương này không chỉ nêu phương án cuối cùng, mà còn giải thích quá trình cân nhắc giữa nhiều lựa chọn khác nhau ở bốn tầng: phần cứng thiết bị theo dõi, firmware nhúng trên vi điều khiển, hệ thống backend/cloud và giao diện người dùng. Nhờ đó, người đọc có thể theo dõi logic thiết kế theo từng bước thay vì chỉ nhìn thấy kết quả cuối cùng.

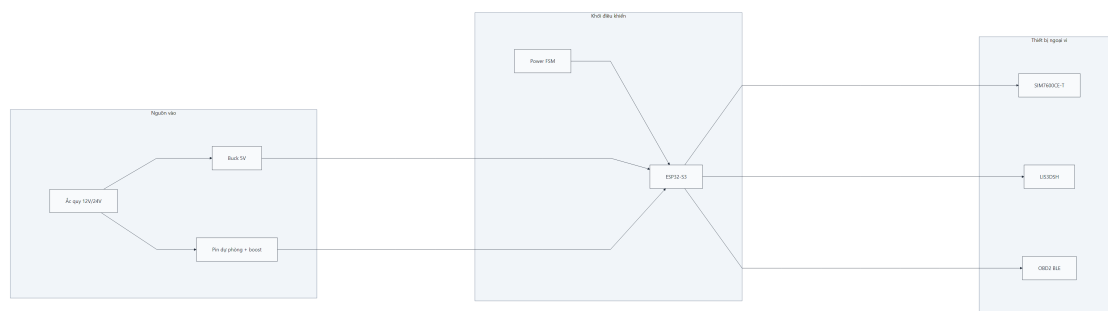
3.1. Phân tích và đề xuất giải pháp phần cứng – Hardware analysis and solution

3.1.1. Phân tích và lựa chọn phần cứng

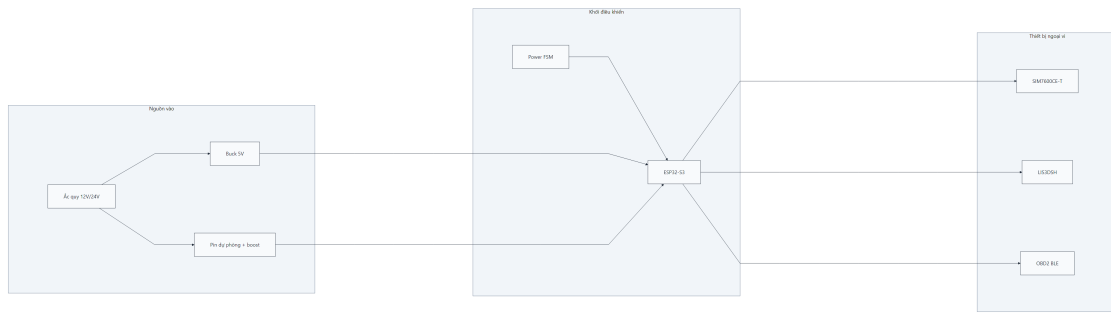
Phần này trình bày quá trình phân tích, lựa chọn và thiết kế các thành phần phần cứng cho thiết bị theo dõi xe IoT. Mục tiêu không chỉ là tạo ra một tracker ”đủ chức năng” trên lý thuyết, mà là một thiết bị có thể lắp lên xe, vận hành ổn định và còn chừa dư địa cho bảo trì, mở rộng. Vì vậy, mỗi quyết định kỹ thuật đều được soi chiếu dưới bốn góc nhìn: độ tin cậy, khả năng hoạt động liên tục với nguồn dự phòng, mức độ tích hợp và mức độ phù hợp với ngân sách.

Quy ước thuật ngữ trong phần cứng: tài liệu dùng cụm ”khối phần cứng/khối chức năng” cho các thành phần tích hợp trên PCB; từ ”module” chỉ dùng khi nói về module phần mềm (firmware/cloud) hoặc khi trích dẫn nguyên văn tên sản phẩm của nhà sản xuất.

3.1.1.1. Sơ đồ khối hệ thống Hệ thống tracker được tổ chức theo kiến trúc khối chức năng với năm khối chính, trong đó ESP32-S3 đóng vai trò điều phối trung tâm. Nếu nhìn dưới góc độ người sử dụng, năm khối này tương ứng với năm câu hỏi rất thực tế: thiết bị lấy dữ liệu xe ở đâu, xác định vị trí bằng cách nào, phát hiện rung/chuyển động ra sao, gửi dữ liệu về máy chủ như thế nào và lấy điện từ đâu để duy trì hoạt động. Sơ đồ khối tổng thể được trình bày như sau:



Hình 3.1: Sơ đồ khối tổng thể hệ thống tracker



Hình 3.1a: Phân rã chi tiết các khối chức năng của tracker

Ở tầng thiết bị, khối OBD2 cung cấp trạng thái vận hành của xe; khối LTE/GNSS chịu trách nhiệm truyền dữ liệu và định vị; khối IMU duy trì khả năng giám sát khi xe tắt máy; khối nguồn đảm bảo thiết bị chuyển đổi giữa nguồn chính và nguồn dự phòng an toàn; còn khối vi điều khiển là nơi tổng hợp toàn bộ tín hiệu để ra quyết định theo trạng thái. Nhờ cách tách này, tracker không còn được nhìn như một bo mạch rời rạc, mà như một chuỗi chức năng nối tiếp từ thu thập dữ liệu, xử lý cục bộ, truyền tin cho đến bảo vệ nguồn.

Hệ thống vận hành theo ba chế độ chính: (1) chế độ lái xe — khi động cơ bật (IGN ON), các khối chức năng cần thiết được kích hoạt; (2) chế độ đỗ xe — khi động cơ tắt (IGN OFF), ESP32 chuyển sang deep sleep và chỉ IMU LIS3DSH duy trì giám sát chuyển động; (3) chế độ cảnh báo — khi IMU ghi nhận chuyển động bất thường, hệ thống tự đánh thức và gửi cảnh báo qua 4G.

Luồng dữ liệu được tổ chức theo ba nhánh: OBD2 (RPM, tốc độ, nhiên liệu) đọc qua BLE từ adapter vgate iCar Pro; dữ liệu GNSS được cung cấp trực tiếp bởi khối modem tích hợp SIMCom SIM7600CE-T (GNSS nội bộ) trên cùng UART với LTE; và dữ liệu chuyển động thu từ IMU LIS3DSH qua I2C. ESP32-S3 tổng hợp, đóng gói và truyền toàn bộ dữ liệu lên máy chủ qua MQTT thông qua kết nối 4G/LTE của SIM7600CE-T.

Để tăng độ liên tục dữ liệu khi mạng gián đoạn, luồng telemetry được bổ sung thêm nhánh lưu đệm cục bộ trên microSD (SDMMC 4-bit + FATFS). Dữ liệu được ghi theo thứ tự bản ghi, sau đó replay lên MQTT khi kết nối phục hồi.

3.1.1.2. Phân tích và lựa chọn vi điều khiển (MCU) Việc lựa chọn vi điều khiển (MCU) là quyết định thiết kế quan trọng nhất, ảnh hưởng trực tiếp đến khả năng tích hợp các khối ngoại vi, mức tiêu thụ năng lượng, và độ phức tạp phát triển firmware. Ba ứng viên được đưa vào quá trình đánh giá là ESP32-S3 của Espressif, STM32L4 của STMicroelectronics, và nRF52840 của Nordic Semiconductor.

a) Các ứng viên **ESP32-S3** là vi điều khiển dual-core Xtensa LX7 hoạt động tới 240 MHz, tích hợp sẵn WiFi 802.11 b/g/n và Bluetooth LE 5.0 [19]. Thiết bị có 512 KB

SRAM on-chip, dùng flash ngoài, hỗ trợ nhiều giao tiếp ngoại vi (UART/I2C/SPI/ADC) và dòng deep-sleep mức μA tùy cấu hình [19], [20].

STM32L4 (đại diện bởi STM32L476 trong so sánh) là dòng MCU low-power của ST, dùng nhân ARM Cortex-M4 tới 80 MHz và không tích hợp BLE/Wi-Fi [53], [54]. Dòng này nổi bật với các mức dòng ngủ sâu rất thấp theo cấu hình nguồn (shutdown/standby/STOP2) [53].

nRF52840 là MCU của Nordic với nhân ARM Cortex-M4F 64 MHz, hỗ trợ Bluetooth LE 5.x và 802.15.4 [55], [56]. Thiết bị có 1 MB Flash, 256 KB RAM, hỗ trợ UART/UARTE và dòng System OFF mức dưới μA tùy điều kiện cấu hình [55], [56].

b) Đối chiếu yêu cầu thiết kế với thông số phần cứng Thay vì chấm điểm tổng quát, Bảng 3.1 đối chiếu trực tiếp các ràng buộc kỹ thuật của tracker với thông số và tác động tích hợp của từng MCU.

Bảng 3.1: Đối chiếu yêu cầu kỹ thuật khi lựa chọn vi điều khiển

Yêu cầu của tracker	ESP32-S3	STM32L4	nRF52840
Kết nối OBD2 BLE với vgate iCar Pro	BLE 5.0 tích hợp, tương thích ngược BLE 4.0	Không tích hợp BLE, phải thêm module ngoài	BLE 5.0 tích hợp
Cấu hình UART của hệ thống	3 UART controllers (đủ cho modem + debug, còn dư cho mở rộng) [19], [20]	Nhiều serial instance trong dòng STM32L476 (USART, UART, LPUART), đáp ứng yêu cầu số cổng [53], [54]	UARTE hoặc UART thường dùng 2 instance, ít dư địa nếu cần giữ debug UART riêng [55], [56]
Tài nguyên xử lý cho OBD2 + modem + IMU	Dual-core 240 MHz, 512 KB SRAM [19]	Cortex-M4 tới 80 MHz, 128 KB SRAM trên STM32L476RE [54]	Cortex-M4F 64 MHz, 256 KB RAM [55], [56]
Dòng ngủ sâu/siêu thấp công suất	Deep-sleep mức μA tùy cấu hình RTC/IO [19], [20]	STOP2 cỡ 1.1 μA (theo điều kiện datasheet), standby/shutdown cỡ nA [53], [54]	System OFF mức dưới μA tùy RAM retention/GPIO [55]
Framework triển khai	Arduino IDE + ESP-IDF [47]	STM32Cube/HAL [53]	nRF Connect SDK [57]

*Bảng 3.1: Đối chiếu yêu cầu kỹ thuật khi lựa chọn vi điều khiển
(tiếp theo)*

Yêu cầu của tracker	ESP32-S3	STM32L4	nRF52840
Phần cứng bổ sung để đạt đủ chức năng	Không cần thêm radio BLE	Cần thêm BLE module nếu vẫn giữ OBD2 BLE (STM32L476 không tích hợp BLE) [53], [54]	Không cần BLE module, nhưng cần xử lý bài toán thiếu serial cho debug
Chi phí triển khai phần cứng tham chiếu	100.000–200.000 VND (mặt bằng BOM dự án)	150.000–300.000 VND (mặt bằng BOM dự án)	Thường cao hơn ESP32-S3 và ít phổ biến hơn trong bối cảnh dự án
Tác động tới sơ đồ hiện tại	Giữ nguyên sơ đồ BLE + modem UART, thuận lợi cho tích hợp GNSS trong modem	Tăng phần cứng BLE ngoài và công đoạn tích hợp ban đầu	Có thể phải chuyển debug sang USB/SWO để nhường UART cho chức năng chính

c) Phân tích các yếu tố quan trọng BLE tích hợp: Đây là ràng buộc rất thực tế vì tracker dùng adapter OBD2 vgate iCar Pro theo chuẩn BLE 4.0. Nếu MCU không có BLE tích hợp, thiết kế sẽ phải bổ sung thêm một module riêng. Điều đó không chỉ làm tăng BOM, mà còn khiến sơ đồ mạch, nguồn cấp và firmware phức tạp hơn ngay từ đầu. Ở khía cạnh này, ESP32-S3 và nRF52840 thuận lợi hơn STM32L4.

Ngân sách UART: Trong kiến trúc hiện tại, UART không phải tài nguyên "thừa". Hệ thống cần ít nhất một kênh cho modem và một kênh cho debug, đồng thời vẫn nên chừa dư địa cho mở rộng hoặc chẩn đoán khi triển khai thực tế. ESP32-S3 đáp ứng tốt nhu cầu này với 3 UART phần cứng; nRF52840 hạn chế hơn; còn STM32L4 có thể đáp ứng nhưng phải đánh đổi bằng việc bổ sung BLE ngoài.

Tiêu thụ deep sleep: STM32L4 và nRF52840 có lợi thế rõ ràng về dòng ngủ sâu theo tài liệu hãng. Tuy nhiên, trong một tracker hoàn chỉnh, MCU không phải thành phần duy nhất tiêu thụ năng lượng. Modem LTE, GNSS, chu kỳ wake-up và các linh kiện phụ trợ cũng góp phần đáng kể vào ngân sách điện năng. Vì vậy, chênh lệch ở mức riêng MCU có ý nghĩa, nhưng không đủ để bù cho việc tăng độ phức tạp tích hợp [19], [53], [54], [55].

Độ phức tạp triển khai: ESP32-S3 có lợi thế thực tế vì vừa có BLE sẵn, vừa có hệ sinh thái ESP-IDF/Arduino IDE quen thuộc, phù hợp cho firmware phải đồng thời xử lý BLE

OBD2, AT command modem, I2C IMU và deep sleep. STM32L4 mạnh về low power nhưng kéo theo thêm công việc tích hợp BLE. nRF52840 phù hợp nếu hệ thống ưu tiên radio BLE là chính, nhưng dư địa UART cho kịch bản hiện tại hạn chế hơn.

d) Kết luận lựa chọn Trên cơ sở đối chiếu trực tiếp các yêu cầu của hệ thống, **ESP32-S3** được chọn làm vi điều khiển chính cho tracker. Nói ngắn gọn, đây là phương án cân bằng tốt nhất giữa ”đủ mạnh để làm được việc” và ”đủ gọn để triển khai thực tế”. Quyết định này dựa trên các điểm chốt sau:

- Đáp ứng đầy đủ kiến trúc hiện tại: 1 kết nối BLE cho OBD2, UART cho modem và một đường debug riêng, đồng thời vẫn còn dư địa UART cho mở rộng.
- BLE 5.0 tích hợp giúp bỏ hẳn module BLE ngoài, giảm BOM và rút ngắn công đoạn tích hợp.
- Tài nguyên xử lý 240 MHz dual-core và 512 KB SRAM đủ để chạy đồng thời BLE, modem, IMU và state machine.
- Chi phí triển khai phần cứng với ESP32-S3 thấp hơn STM32L4 khoảng 30–50% theo mặt bằng linh kiện đang dùng trong đồ án.
- Dòng deep-sleep của ESP32-S3 cao hơn STM32L4 theo tài liệu hãng, nhưng vẫn nằm trong giới hạn chấp nhận được của bài toán pin backup khi xét toàn bộ duty-cycle của modem, GNSS và các chu kỳ wake-up [19], [53], [54].

Bảng 3.2: Thông số kỹ thuật ESP32-S3 được chọn

Thông số	Giá trị
CPU	Dual-core Xtensa LX7, tối đa 240 MHz [19]
RAM on-chip	512 KB SRAM [19]
Flash	Flash ngoài (biến thể WROOM thường 4–16 MB) [19], [20]
Bluetooth	Bluetooth LE 5.0, tương thích ngược BLE 4.0 cho OBD2 adapter [19]
WiFi	802.11 b/g/n (không dùng trong luồng chính của tracker) [19]
UART	3 UART controllers [19], [20]
I2C	2 I2C controllers [19], [20]
ADC	SAR ADC 12-bit, tối đa 20 kênh (theo package) [19]
GPIO	Tối đa 45 GPIO (theo package) [19]
Dòng deep-sleep	Mức μ A tùy cấu hình RTC domain và IO [19], [20]
Biến thể linh kiện	ESP32-S3-WROOM-1
Phương án tích hợp	Tích hợp trực tiếp vào bo mạch chính

3.1.1.3. Phân tích và lựa chọn modem truyền thông Modem truyền thông đóng vai trò then chốt trong hệ thống vì nó đồng thời giải quyết hai nhu cầu thiết yếu nhất của tracker: gửi dữ liệu về máy chủ qua mạng di động và cung cấp vị trí của xe thông qua GNSS. Với bài toán này, lựa chọn modem không chỉ ảnh hưởng đến tốc độ hay thông số kỹ thuật, mà còn tác động trực tiếp đến cách đi dây, cách tổ chức firmware và mức độ phức tạp của toàn bộ bo mạch. Trong kiến trúc hiện tại, hệ thống sử dụng **SIM7600CE-T tích hợp LTE + GNSS** nhằm giảm số lượng phần cứng rời, đơn giản hóa đi dây và đồng bộ với firmware thực tế.

a) Các ứng viên Ba phương án được đưa vào đánh giá gồm hai phương án tách rời (**SIMCom A7670C + u-blox NEO-M8N**, **Quectel EC200U-CN + NEO-M8N**) và một phương án tích hợp (**SIMCom SIM7600CE-T**).

SIMCom A7670C + u-blox NEO-M8N là phương án tách rời: A7670C phụ trách LTE/AT command, NEO-M8N phụ trách GNSS/NMEA. Theo trang sản phẩm chính thức của SIMCom, A7670C nằm trong nhóm LTE Cat-1 và dùng nguồn 3.4–4.2 V [58], [62].

Quectel EC200U-CN + NEO-M8N có cấu trúc gần giống phương án A7670C + NEO-M8N. Tuy nhiên dòng EC200U được hãng mô tả là Cat-1 bis, GNSS là tính năng tùy chọn theo biến thể/cấu hình và I/O logic có ràng buộc riêng, nên mức độ phù hợp với cấu hình đang triển khai thấp hơn [59], [61].

SIMCom SIM7600CE-T là phương án tích hợp GNSS trong modem, giúp đi dây gọn hơn và giảm số lượng phần cứng rời [60].

b) Đối chiếu kiến trúc LTE/GNSS theo ràng buộc tích hợp Vấn đề cốt lõi của tracker không chỉ là modem có lên mạng được hay không, mà còn là phương án LTE/GNSS ảnh hưởng thế nào tới sơ đồ UART, hồ sơ tiêu thụ nguồn và mức độ thuận lợi khi phát triển firmware. Bảng 3.3 dưới đây đối chiếu trực tiếp các điểm khác biệt này.

Bảng 3.3: Đối chiếu các phương án LTE/GNSS theo tác động tích hợp

Yêu cầu tích hợp	A7670C + NEO-M8N	EC200U-CN + NEO-M8N	SIM7600CE-T
Số khối phần cứng rời	2 khối rời: A7670C (LTE) + NEO-M8N (GNSS) [58], [62]	2 khối rời: EC200U-CN (LTE) + NEO-M8N (GNSS) [59], [61]	1 khối tích hợp LTE + GNSS (SIM7600CE-T) [60]

Bảng 3.3: Đối chiếu các phương án LTE/GNSS theo tác động tích hợp (tiếp theo)

Yêu cầu tích hợp	A7670C + NEO-M8N	EC200U-CN + NEO-M8N	SIM7600CE-T
LTE category / tốc độ	Cat-1, tối đa 10 Mbps DL / 5 Mbps UL [58]	Cat-1, tối đa 10 Mbps DL / 5 Mbps UL [59], [61]	Cat-4, tối đa 150 Mbps DL / 50 Mbps UL [60]
GNSS tích hợp trong modem	Không tích hợp GNSS trên A7670C, cần GNSS ngoài [58], [62]	GNSS tùy chọn theo variant/cấu hình dòng EC200U [59], [61]	Có GNSS tích hợp trong module [60]
Điện áp cấp nguồn modem	3.4–4.2 V (typ. 3.8 V) [58]	3.3–4.3 V (typ. 3.8 V) [61]	3.4–4.2 V (typ. 3.8 V) [60]
Đường dữ liệu vị trí trong kiến trúc đề tài	NMEA/UBX từ NEO-M8N qua UART riêng [62], [63]	NMEA/UBX từ NEO-M8N qua UART riêng [62], [63]	GNSS AT tích hợp trong cùng modem [60]
Bật/tắt LTE và GNSS độc lập	Có, do tách 2 module/2 rail nguồn	Có, nếu triển khai tách tương tự A7670C+NEO-M8N	Không tách hoàn toàn do GNSS đi cùng modem
Tác động khi reset modem LTE	Có thể reset modem và giữ logic GNSS riêng	Tương tự phương án tách nếu GNSS dùng module riêng	Reset modem thường kéo theo gián đoạn GNSS
Mức phụ thuộc giữa các chức năng firmware	Thấp, tách rõ modem_lte và gnss	Thấp, nhưng phải đổi tập lệnh modem	Cao hơn do LTE/GNSS chung khối tích hợp
Tác động tới BOM phần cứng thực tế	Bộ modem + GNSS đang chốt 330.000–500.000 VND (BOM dự án)	Chưa phải cấu hình BOM chính của repo	Giảm số khối phần cứng rời, thuận lợi cho đi dây và lắp ráp

c) Kết luận lựa chọn SIMCom SIM7600CE-T được lựa chọn làm kiến trúc truyền thông chính vì đây là phương án giúp thiết kế tổng thể gọn hơn và dễ triển khai hơn, dù không phải lúc nào cũng linh hoạt nhất ở từng thành phần riêng lẻ. Các lý do cụ thể gồm:

- Tích hợp LTE và GNSS trong cùng một module giúp giảm số lượng phần cứng rời và rút gọn đường kết nối UART.
- Giữ nguyên sơ đồ chân đang triển khai (UART1 trên GPIO16/17, PWR-KEY trên

GPIO26), không cần thay đổi pin mapping phần cứng.

- Luồng AT được chuẩn hóa với Auto mode (AT+CNMP=2), kiểm tra AT+CEREG? trước AT+CGACT=1, 1, và APN mặc định internet, phù hợp với firmware hiện tại.
- GNSS tích hợp được điều khiển trực tiếp bằng AT+CGNSPWR và đọc dữ liệu qua AT+CGNSINF, thống nhất với module firmware đang vận hành.
- Phù hợp với phương án BOM phần cứng đang áp dụng cho bo mạch đã chế tạo, thuận lợi cho triển khai và đối chiếu tài liệu kỹ thuật.

Bảng 3.4: Thông số kỹ thuật phương án SIM7600CE-T

Thông số	Giá trị
Kiến trúc	SIM7600CE-T tích hợp LTE Cat-4 và GNSS trong cùng module [60]
LTE category / tốc độ	Cat-4, tối đa 150 Mbps downlink / 50 Mbps uplink [60]
GNSS	GNSS tích hợp (GPS/GLONASS/BeiDou/Galileo), điều khiển bằng AT+CGNSPWR, đọc dữ liệu bằng AT+CGNSINF [60]
GNSS tích hợp trong modem	Có [60]
Giao tiếp với MCU	1 UART (GPIO16/17) cho toàn bộ AT LTE/GNSS + GPIO26 điều khiển PWR-KEY [60]
Giao thức dữ liệu	LTE dùng AT command cho PDP/MQTT; GNSS lấy dữ liệu qua AT+CGNSINF và có thể stream NMEA bằng AT+CGNSTST khi cần [60]
Điện áp modem	3.4–4.2 V, điển hình 3.8 V [60]
Luồng attach LTE	AT+CNMP=2 (auto mode), cấu hình AT+CGDCONT=1, "IP", "internet", kiểm tra AT+CEREG? trước AT+CGACT=1, 1
Đồng bộ GNSS	Bật/tắt GNSS bằng AT+CGNSPWR=1/0, đọc dữ liệu bằng AT+CGNSINF
Điện áp cấp nguồn	3.4–4.2 V, điển hình 3.8 V [60]
Ý nghĩa tích hợp	Giảm số linh kiện rời, đơn giản hóa đi dây và đồng bộ với firmware thực tế
Ghi chú	APN mặc định sử dụng trong hệ thống là internet; pin mapping phần cứng giữ nguyên

3.1.2. Giải pháp phần cứng

Trên cơ sở các lựa chọn phần cứng đã phân tích ở mục 3.1.1, phần này đi vào cách nhóm hiện thực chúng thành kiến trúc tracker hoàn chỉnh, từ kết nối OBD2 và IMU đến khối nguồn và BOM tham chiếu.

3.1.2.1. Thiết kế khối thu thập dữ liệu OBD2

a) Lựa chọn phương pháp kết nối OBD2 Thu thập dữ liệu từ ECU xe qua cổng OBD2 (On-Board Diagnostics II) là một yêu cầu cốt lõi của thiết kế tracker vì đây là nguồn thông tin giúp hệ thống đi xa hơn một thiết bị định vị thông thường. Có hai hướng tiếp cận chính: sử dụng adapter OBD2 có dây (wired ELM327 qua UART) hoặc adapter OBD2 không dây (BLE ELM327). Nhóm lựa chọn kết nối không dây qua BLE vì cách làm này phù hợp hơn với cách bố trí thiết bị thực tế trên xe và với tài nguyên phần cứng của tracker:

Bảng 3.5: So sánh phương pháp kết nối OBD2 theo thông số triển khai

Tiêu chí kỹ thuật	OBD2 có dây (ELM327 UART)	OBD2 không dây (BLE ELM327 / vgate iCar Pro)
Kết nối vật lý tracker ↔ OBD2	Cần dây kéo cố định từ cổng OBD2 đến tracker	Không cần dây giữa tracker và cổng OBD2
Giao thức tới MCU	UART TTL	BLE 4.0 GATT; vgate iCar Pro là bản Bluetooth 4.0 BLE [29], [64]
Tài nguyên MCU tiêu tốn	Chiếm thêm 1 UART phần cứng trong lúc kết nối	Tận dụng BLE tích hợp của ESP32-S3, giữ UART cho modem/GNSS/debug
Giao thức OBD-II hỗ trợ	Phụ thuộc adapter ELM327 cụ thể	Vgate công bố hỗ trợ J1850 PWM/VPW, ISO9141-2, ISO14230-4, ISO15765-4 CAN, SAE J1939 CAN [64]
Hành vi sleep của adapter	Phụ thuộc adapter có dây cụ thể	Vendor công bố tự sleep sau khoảng 30 phút khi xe tắt máy [64]
Hành vi wake	Phụ thuộc adapter và đường cấp nguồn	Vendor quảng bá tự khởi động khi ignition ON; cần hiểu là hành vi phụ thuộc xe [64]
Dòng tiêu thụ adapter	Phụ thuộc adapter; không dùng 1 số cứng nếu không có datasheet đúng mẫu	Trang hãng được kiểm tra không công bố dòng active/standby, nên không chốt số mA trong luận văn [64]

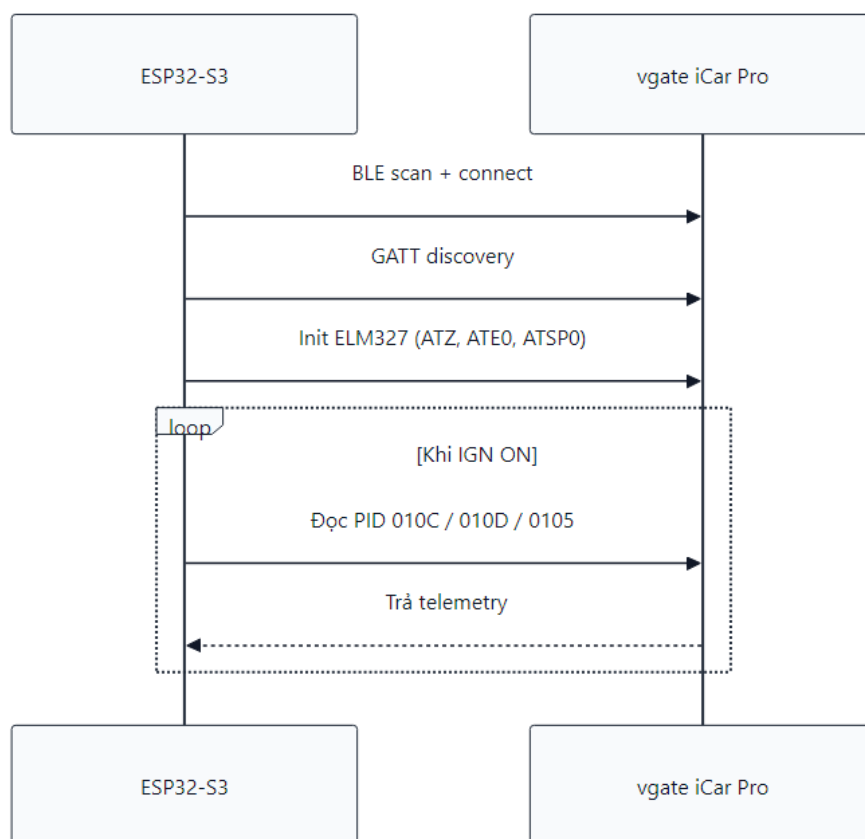
Bảng 3.5: So sánh phương pháp kết nối OBD2 theo thông số triển khai (tiếp theo)

Tiêu chí kỹ thuật	OBD2 có dây (ELM327 UART)	OBD2 không dây (BLE ELM327 / vgate iCar Pro)
Hành vi khi MCU deep sleep	UART không duy trì truyền dữ liệu khi MCU ngủ sâu	BLE link cũng không duy trì qua deep sleep; cần reconnect sau wake
Tác động lắp đặt thực địa	Tracker thường bị ràng buộc gần cổng OBD2	Tracker có thể đặt lệch vị trí OBD2 trong cabin nếu vẫn trong vùng BLE
Bảo trì/thay adapter giữa các xe	Cần tháo dây liên quan đến tracker	Chỉ cần rút/cắm adapter và cấu hình lại kết nối nếu cần

Kết luận: Phương pháp BLE được chọn vì phù hợp hơn với kiến trúc hiện tại của tracker: tận dụng BLE tích hợp của ESP32-S3, không chiếm thêm UART phần cứng, giảm ràng buộc đi dây, đồng thời vgate iCar Pro đã có công bố rõ về BLE 4.0, auto-sleep và danh sách giao thức OBD-II. Với các thông số hãng chưa công bố (như dòng active/standby), báo cáo chỉ ghi nhận là chưa có số liệu chính thức thay vì đưa ra giá trị ước lượng.

b) Lựa chọn adapter OBD2 BLE: vgate iCar Pro vgate iCar Pro là adapter OBD2 sử dụng Bluetooth Low Energy (BLE) 4.0, tương thích với BLE 5.0 của ESP32-S3. Adapter này được chọn dựa trên các tiêu chí sau:

- **Tương thích giao thức:** Hỗ trợ tập lệnh ELM327 qua BLE GATT characteristics, cho phép đọc đa dạng dữ liệu từ ECU bao gồm trạng thái động cơ (IGN), số vòng quay (RPM), tốc độ xe, mức nhiên liệu, nhiệt độ động cơ, và mã lỗi chẩn đoán (DTC) [7].
- **Khả năng kết nối:** BLE 4.0 của adapter tương thích ngược với BLE 5.0 của ESP32-S3 [19], [64]. Vendor cũng công bố cơ chế tự sleep sau khoảng 30 phút khi xe tắt máy và tự khởi động lại theo ignition, phù hợp cho kịch bản tracker trên xe [64].
- **Tiêu thụ năng lượng:** Ở thời điểm rà soát nguồn, trang hãng được kiểm tra không công bố dòng active/standby chính thức cho vgate iCar Pro BLE [64]. Vì vậy báo cáo chỉ ghi nhận đây là thông số chưa được công bố đầy đủ, không đưa ra giá trị mA cố định khi chưa có datasheet chính thức đúng mẫu.
- **Phổ biến và giá hợp lý:** Ở lần rà soát thị trường Việt Nam ngày 12/04/2026, các mức giá tham khảo ghi nhận cho vgate iCar Pro BLE nằm quanh khoảng 367.000–609.000 VND tùy kênh bán lẻ và phiên bản.



Hình 3.2: Sơ đồ kết nối BLE giữa ESP32-S3 và vgate iCar Pro

c) Chiến lược kết nối theo chế độ hoạt động Một ràng buộc thiết kế quan trọng là ESP32-S3 không thể duy trì kết nối BLE trong trạng thái deep sleep. Vì vậy, kết nối OBD2 không được xem là một kênh "luôn bật", mà chỉ hoạt động khi nó thực sự tạo ra giá trị dữ liệu cho từng trạng thái vận hành của xe:

Bảng 3.6: Chiến lược kết nối BLE OBD2 theo chế độ hoạt động

Chế độ	Cần BLE OBD2?	Lý do
Lái xe (IGN ON)	Có	Đọc dữ liệu xe (RPM, tốc độ, nhiên liệu) liên tục
Đỗ xe (IGN OFF)	Không	Không cần dữ liệu OBD2, chỉ cần IMU phát hiện chuyển động
Cảnh báo	Tùy chọn	Có thể kết nối để xác nhận IGN, hoặc chỉ dùng IMU + GPS

Khi xe chạy (IGN ON), ESP32-S3 duy trì kết nối BLE liên tục với adapter để đọc dữ liệu OBD2 định kỳ mỗi 5–30 giây. Khi xe đỗ (IGN OFF), kết nối BLE được ngắt trước khi ESP32 vào deep sleep, vì ở thời điểm này mục tiêu ưu tiên không còn là lấy dữ liệu động cơ, mà là giữ mức tiêu thụ thấp nhất trong khi vẫn duy trì khả năng phát hiện xâm nhập. IMU LIS3DSH đảm nhiệm việc giám sát đo độc lập và chỉ đánh thức hệ thống khi có tín hiệu đáng ngờ. Cách tổ chức này giúp giảm đáng kể năng lượng tiêu thụ ở pha đỗ xe mà không làm mất đi chức năng cảnh báo cốt lõi.

Trong trường hợp adapter OBD2 không kết nối được (timeout 10 giây, retry 2–3 lần), hệ thống tự động chuyển sang phương pháp dự phòng (fallback) là đo điện áp ắc quy qua ADC để phát hiện trạng thái IGN. Phương pháp này kém chính xác hơn nhưng đảm bảo hệ thống vẫn hoạt động bình thường.

3.1.2.2. Thiết kế khối cảm biến chuyển động IMU

a) Vai trò của IMU trong hệ thống Cảm biến đo quán tính (IMU - Inertial Measurement Unit) đóng vai trò then chốt trong việc phát hiện chuyển động của xe khi đang đỗ, cho phép hệ thống phát hiện các tình huống bất thường như rung xe (cố gắng mở cửa), kéo xe, hoặc cầu xe. Đặc biệt, IMU cho phép ESP32-S3 ở trạng thái deep sleep liên tục và chỉ thức dậy khi có chuyển động thực sự, giúp tiết kiệm năng lượng đáng kể.

b) Lựa chọn cảm biến: LIS3DSH LIS3DSH của STMicroelectronics là cảm biến gia tốc 3 trục (3-axis accelerometer) low-power được lựa chọn cho hệ thống. Các đặc tính kỹ thuật chính được tóm tắt trong bảng sau:

Bảng 3.7: Thông số kỹ thuật cảm biến LIS3DSH

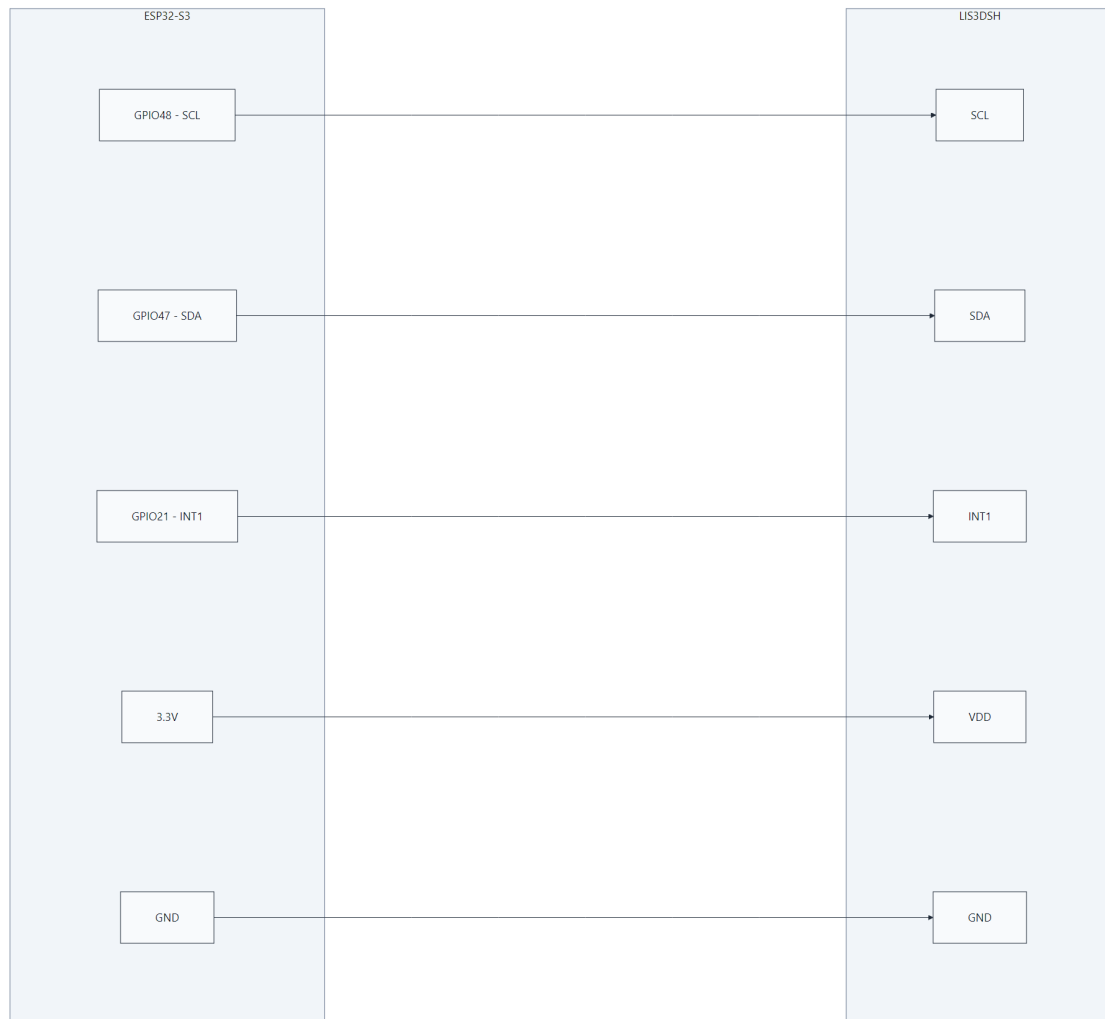
Thông số	Giá trị
Loại	3-axis Digital Accelerometer
Điện áp hoạt động	1.8–3.6 V
Giao tiếp	I2C (400 kHz) hoặc SPI (10 MHz)
Phạm vi đo	+/-2g, +/-4g, +/-8g, +/-16g (có thể điều chỉnh)
Độ phân giải	16-bit
Tiêu thụ (Active)	10–50 μ A
Tiêu thụ (Low-power)	2–5 μ A
Wake-up interrupt	Có (ngưỡng có thể cấu hình)

Bảng 3.7: Thông số kỹ thuật cảm biến LIS3DSH (tiếp theo)

Thông số	Giá trị
Package	LGA-16 (3x3x1 mm)

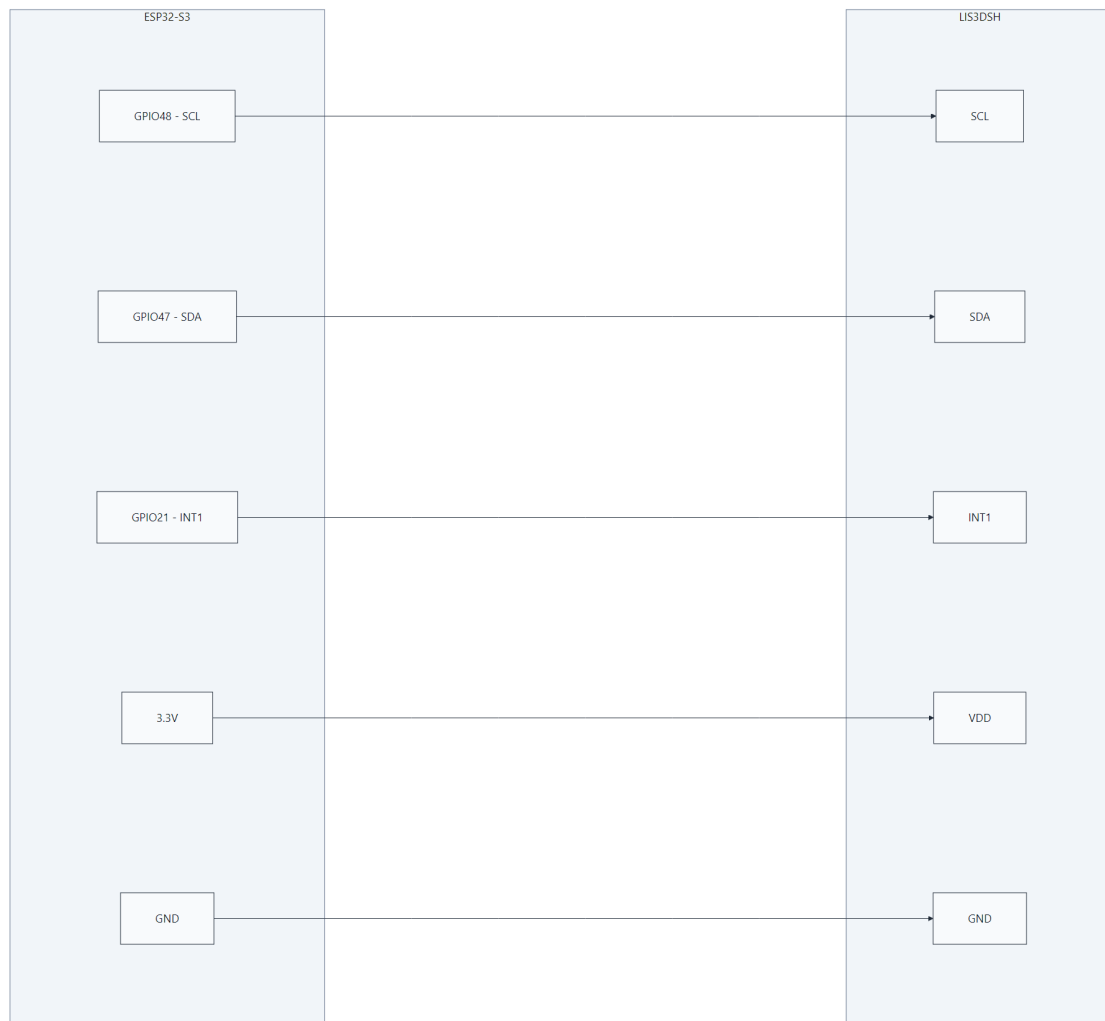
c) Nguyên lý hoạt động trong hệ thống LIS3DSH được cấu hình ở chế độ low-power với tần suất lấy mẫu 1 Hz ($ODR = 1 \text{ Hz}$) và chỉ bật chức năng motion detection. Khi phát hiện gia tốc vượt ngưỡng 0.2g trên bất kỳ trục nào (X, Y, Z), cảm biến tự động phát tín hiệu interrupt đến chân GPIO của ESP32-S3 thông qua chân INT1, đánh thức vi điều khiển từ trạng thái deep sleep [9].

Ngưỡng gia tốc 0.2g được chọn để cân bằng giữa độ nhạy và khả năng chống báo giả. Giá trị này đủ lớn để loại bỏ rung nhẹ từ môi trường (gió, xe cộ đi ngang) nhưng đủ nhỏ để phát hiện các chuyển động thực sự như rung của xe hoặc di chuyển.



Hình 3.3: Sơ đồ kết nối LIS3DSH với ESP32-S3 qua I2C

Cấu hình I2C sử dụng địa chỉ 0x18, tốc độ 400 kHz, với hai điện trở kéo lên 10 kΩ trên



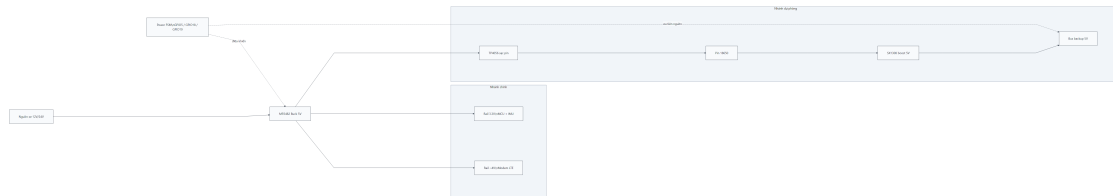
Hình 3.3a: Sơ đồ chi tiết chân kết nối LIS3DSH với ESP32-S3

các net IMU-SCL và IMU-SDA. Ở runtime hiện tại, bus I2C được ánh xạ về GPIO48 (SCL) và GPIO47 (SDA), chân INT1 đi vào GPIO21 để đánh thức ESP32-S3, còn INT2 được giữ lại cho mở rộng sau này.

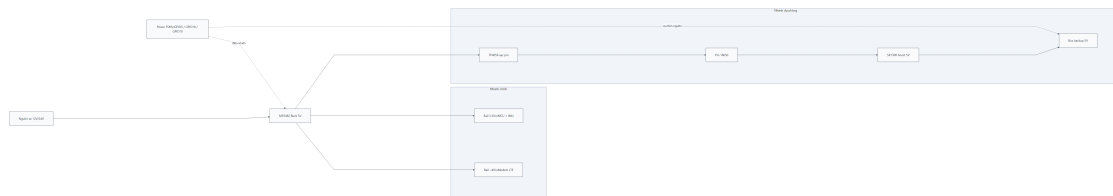
d) Ưu điểm của giải pháp

- **Tiết kiệm năng lượng tốt nhất:** Dòng tiêu thụ chỉ 2–5 μA ở chế độ low-power, trong khi vẫn duy trì khả năng phát hiện chuyển động. ESP32-S3 không cần chạy liên tục để kiểm tra cảm biến.
- **Phát hiện chuyển động độc lập:** LIS3DSH tự xử lý việc phát hiện chuyển động bằng phần cứng, chỉ gửi interrupt khi có sự kiện — không phụ thuộc vào CPU của ESP32.
- **Hệ sinh thái phát triển phong phú:** Nhiều thư viện sẵn có cho Arduino và ESP-IDF; IC LIS3DSH hoặc cảm biến tương đương đều dễ tích hợp vào sơ đồ phần cứng của thiết bị.

3.1.2.3. Thiết kế hệ thống quản lý nguồn Hệ thống quản lý nguồn là khối phức tạp nhất trong thiết kế phần cứng, vì nó phải đồng thời giải quyết ba bài toán vốn dễ xung đột với nhau: tạo đúng các mức điện áp cho từng tải, duy trì đường nguồn dự phòng khi cần và bảo vệ ắc quy xe khỏi bị rút cạn quá mức. Vì vậy, khối này được tách thành năm nhánh chức năng để người đọc dễ theo dõi vai trò của từng đường nguồn thay vì chỉ nhìn một sơ đồ điện phức tạp.



Hình 3.4: Sơ đồ khối hệ thống quản lý nguồn



Hình 3.4a: Kiến trúc nguồn và phân phối điện áp trong hệ thống

a) LDO 3.3V cho ESP32-S3 (AP2112-3.3) Nhánh 3.3V dùng **AP2112-3.3** như một LDO để hạ từ **bus 5V chính** xuống 3.3V, cấp cho ESP32-S3 và các tải logic 3.3V. Cách tổ chức này phù hợp với kiến trúc nguồn thực tế của bo mạch: MP2482 đảm nhiệm bước hạ áp đầu tiên từ 12–24V xuống 5V, còn AP2112-3.3 đảm nhiệm bước ổn áp cuối cho miền logic.

Bảng 3.8: Thông số khối LDO 3.3V

Khối	IC	Input	Output	Tải
LDO 3.3V	AP2112-3.3	5V	3.3V	ESP32-S3

b) Buck 5V bus chính (MP2482) Nhánh buck 5V dùng **MP2482** để tạo **bus 5V chính** từ nguồn 12–24V của xe. Bus 5V này là nguồn trung gian cho các khối phụ trợ và đầu vào mạch sạc pin.

Bảng 3.9: Thông số khối Buck 5V

Khối	IC	Input	Output	Tải
Buck 5V	MP2482	12–24V	5V	Bus 5V chính

c) Buck 3.8V/4V cho modem (TPS54231) Do modem SIM7600CE-T làm việc trên miền nguồn thấp áp, hệ thống bố trí nhánh buck riêng dùng **TPS54231** để hạ 12–24V xuống khoảng **4V** cấp cho modem.

Bảng 3.10: Thông số khối Buck 3.8V/4V cho modem

Khối	IC	Input	Output	Tải
Buck 3.8V	TPS54231	12–24V	4V	SIM7600CE-T

d) Boost 5V từ pin dự phòng (SX1308) và Power Path Khi ắc quy xe yếu, nguồn dự phòng lấy từ pin 18650 1S (đánh định 3.7V). Khối **SX1308** tăng áp lên 5V để duy trì cấp nguồn cho hệ thống.

Bảng 3.11: Thông số khối Boost 5V dự phòng

Khối	IC	Input	Output	Tải
Boost 5V	SX1308	Battery 3.7V	5V	Backup từ pin

Power path runtime sử dụng **diode OR** giữa nhánh 5V chính (MP2482) và nhánh 5V dự phòng (SX1308), kết hợp điều khiển GPIO để bật/tắt các nhánh liên quan. Mục tiêu là chuyển nguồn mượt, không reset hệ thống khi đổi nguồn.

e) Giám sát điện áp và LVD LVD dùng ngưỡng profile kép theo firmware:

- **Profile 12V:** $Switch_{OFF} = 12.0\text{ V}$, $Switch_{ON} = 12.2\text{ V}$
- **Profile 24V:** $Switch_{OFF} = 24.0\text{ V}$, $Switch_{ON} = 24.4\text{ V}$

Bảng 3.12: Bảng trạng thái chuyển nguồn và cảnh báo

Chế độ	IGN	U_{batt}	Nguồn tracker	Sạc pin	Cảnh báo
Xe chạy	ON	Profile 12V: $U_{batt} \geq 13.0\text{ V}$; Profile 24V: $U_{batt} \geq 26.0\text{ V}$	Ắc quy	Có	Không

Bảng 3.12: Bảng trạng thái chuyển nguồn và cảnh báo (tiếp theo)

Chế độ	IGN	U_{batt}	Nguồn tracker	Sạc pin	Cảnh báo
Xe đỗ	OFF	Profile 12V: $U_{batt} > 12.0\text{ V}$; Profile 24V: $U_{batt} > 24.0\text{ V}$	Ắc quy	Không	Không
Ắc quy yếu	OFF	Profile 12V: $U_{batt} \leq 12.0\text{ V}$; Profile 24V: $U_{batt} \leq 24.0\text{ V}$	Pin 18650 1S	Không	Cảnh báo chuyển nguồn
Ắc quy phục hồi	OFF	Profile 12V: $U_{batt} \geq 12.2\text{ V}$; Profile 24V: $U_{batt} \geq 24.4\text{ V}$	Ắc quy	Không	Cảnh báo phục hồi

Ngoài kênh ADC, tín hiệu trạng thái LVD từ khối LVD phân cứng được đưa về **GPIO19 (LVD_STATUS)** để giám sát nhanh. Quy ước runtime: **GPIO19 HIGH = low-voltage**, **GPIO19 LOW = bình thường**.

f) Mạch sạc pin 1S (TP4056) Khối sạc dùng **TP4056**, nhận **5V** từ nhánh **MP2482** và sạc pin 18650 1S theo chuẩn **4.2V/1S**.

Bảng 3.13: Thông số khối sạc pin

Khối	IC	Input	Output	Tải
Sạc pin	TP4056	5V từ MP2482	4.2V	Pin 18650 1S

Dòng sạc TP4056 được thiết lập theo điện trở PROG và giới hạn nhiệt của mạch, vì vậy không dùng một giá trị dòng cố định cho mọi điều kiện vận hành.

g) Pin dự phòng 18650 1S Li-ion Pin 18650 1S Li-ion được chọn làm nguồn dự phòng với cấu hình 1 cell đơn giản, kết hợp khối sạc TP4056, khối boost SX1308 và mạch bảo vệ pin 1S trong kiến trúc nguồn của hệ thống.

Bảng 3.14: Thông số kỹ thuật pin dự phòng 18650 1S

Thông số	Giá trị
Loại	Li-ion 18650 1S
Dung lượng	3500 mAh @ 3.7V
Năng lượng	12.95 Wh
Điện áp	3.0–4.2V (nominal 3.7V)

Bảng 3.14: Thông số kỹ thuật pin dự phòng 18650 1S (tiếp theo)

Thông số	Giá trị
Số chu kỳ	500–1000 chu kỳ (80% capacity)

Tính toán thời gian hoạt động từ pin backup:

Các giá trị dưới đây là ước tính theo cùng giả định tổn hao của phương án gốc, với dung lượng hiệu quả xấp xỉ 3150 mAh (tương đương 90% dung lượng danh định), không phải số đo thực nghiệm mới trên cell 18650.

Chế độ heartbeat (đỗ xe, chu kỳ 15 phút):

- Dòng trung bình: $I_{\text{avg}} = \frac{250 \text{ mA} \times 15 \text{ s} + 3 \text{ mA} \times 885 \text{ s}}{900 \text{ s}} = 7.1 \text{ mA}$
- Dung lượng hiệu quả (sau tổn hao): 3150 mAh
- Thời gian hoạt động: $T = \frac{3150}{7.1} = 444 \text{ giờ} \approx 18.5 \text{ ngày}$

Chế độ heartbeat (đỗ xe, chu kỳ 30 phút):

- Dòng trung bình: $\approx 5 \text{ mA}$
- Thời gian hoạt động: $T = \frac{3150}{5} = 630 \text{ giờ} \approx 26 \text{ ngày}$

Chế độ cảnh báo (track liên tục):

- Dòng trung bình: $\approx 250 \text{ mA}$
- Thời gian hoạt động: $T = \frac{3150}{250} = 12.6 \text{ giờ}$

Giá trị 12.6 giờ ở đây là phép tính lý tưởng theo tải giả định 250 mA tại mức cell; khi quy đổi sang tải hệ thống thực tế có hao hụt boost và các pha truyền dữ liệu đỉnh cao hơn, thời lượng tracking liên tục trong Chương 4 được lấy theo khoảng bảo thủ hơn là 2.6–3.0 giờ.

Kết quả quy đổi cho thấy baseline pin 18650 1S vẫn đủ khả năng duy trì hoạt động tracker trong nhiều ngày ở chế độ heartbeat và trên 12 giờ ở chế độ cảnh báo liên tục theo mô hình lý tưởng; trong vận hành toàn hệ thống, các giá trị bảo thủ hơn ở Chương 4 phù hợp hơn cho mục tiêu duy trì giám sát tạm thời khi ắc quy xe yếu, nhưng vẫn cần được kiểm thử thực nghiệm lại nếu chuyển sang mẫu pin mới trong pha chế tạo tiếp theo [14].

3.1.2.4. Bảng tổng hợp linh kiện (Bill of Materials)

Bảng 3.14A: Bảng tổng hợp linh kiện hệ thống tracker (BOM)

STT	Thành phần	Đơn vị	SL	Giá tham chiếu T4/2026 (VND)	Ghi chú
1	ESP32-S3-WROOM-1 (N16R8)	Cái	1	95.000	MCU trung tâm của bo mạch
2	IC cảm biến LIS3DSH	Cái	1	40.000	Cảm biến gia tốc 3 trục
3	vgate iCar Pro (OBD2 BLE)	Cái	1	367.000	BLE 4.0, ELM327 compatible
4	SIMCom SIM7600CE-T	Bộ	1	650.000	Modem LTE + GNSS tích hợp
5	Pin 18650 1S Li-ion 3500mAh	Cái	1	79.000	Loại có protection board
6	IC sạc TP4056 + mạch phụ trợ	Cái	1	25.000	Input 5V, output 4.2V, dòng theo PROG
7	Mạch LDO AP2112-3.3	Cái	1	5.000	5V -> 3.3V cấp ESP32-S3
8	Mạch buck MP2482 5V	Cái	1	17.000	Bus 5V chính
9	Mạch buck TPS54231 4V	Cái	1	9.000	12-24V -> 4V cấp modem SIM7600CE-T
10	Mạch boost SX1308 5V backup	Cái	1	12.000	Backup từ pin 18650 1S
11	Khối giám sát LVD + ADC	Cái	1	8.000	Giám sát LVD (GPIO19)
12	Diode Schottky (SS54/SS34)	Cái	2	2.500	Diode-OR power path
13	Mạch bảo vệ pin 1S (BMS)	Cái	1	12.000	Bảo vệ pin 18650 1S
14	Điện trở (10kOhm, 2.2kOhm)	Gói	1	5.000	Voltage divider, pull-up
15	Tụ điện (100uF, 220uF)	Gói	1	5.000	Lọc nhiễu, decoupling

*Bảng 3.14A: Bảng tổng hợp linh kiện hệ thống tracker (BOM)
(tiếp theo)*

STT	Thành phần	Đơn vị	SL	Giá tham chiếu T4/2026 (VND)	Ghi chú
16	Connector, header pin	Gói	1	55.000	Header, jack OBD2 và đầu nối nguồn
17	PCB 2 lớp (50x50 mm)	Cái	1	20.000	Bo mạch theo layout của đề tài
18	Vỏ bảo vệ (tùy chọn)	Cái	1	25.000	Nhựa hoặc kim loại
19	Dây nối, cáp, phụ kiện	-	-	60.000	Dây điện, anten 4G/GNSS, khay SIM
20	Linh kiện phụ trợ khác	-	-	20.000	Fuse, switch, LED

Bảng 3.15: Tổng hợp chi phí theo nhóm

Nhóm	Hạng mục	Chi phí (VND)
A	Thành phần chính (MCU, cảm biến, modem, OBD2, pin)	1.231.000
B	Quản lý nguồn (AP2112-3.3, MP2482, TPS54231, SX1308, TP4056, diode OR, khối LVD, pin 1S)	93.000
C	Kết nối, RF và linh kiện phụ trợ	145.000
D	PCB và vỏ (tùy chọn)	45.000
	Tổng cộng	1.514.000

Theo mặt bằng giá tham chiếu thị trường Việt Nam được rà soát ngày 12/04/2026, tổng chi phí vật tư cho một bộ tracker hoàn chỉnh vào khoảng **1.514.000 VND**. Mức này vẫn nằm dưới ngưỡng chi phí mục tiêu của đề án, đồng thời phản ánh thực tế hơn so với mặt bằng giá linh kiện ở thời điểm đầu năm.

3.2. Phân tích và đề xuất giải pháp Firmware – Firmware analysis and solution

3.2.1. Phân tích và lựa chọn giải pháp Firmware

3.2.1.1. Đặt vấn đề cho giải pháp Firmware Nếu xem phần cứng là “cơ thể” của thiết bị, thì firmware là lớp biến tập linh kiện đó thành một hệ thống biết *phản ứng đúng lúc*. Người dùng không trực tiếp nhìn thấy firmware, nhưng việc vị trí có cập nhật đều, cảnh

báo có đến kịp hay không, hay ắc quy xe có bị ảnh hưởng hay không đều do lớp này quyết định. Vì nằm ở điểm giao nhau giữa phần cứng, truyền thông và yêu cầu nghiệp vụ, bài toán firmware không dừng ở chuyện “cho thiết bị chạy được”, mà phải bảo đảm thiết bị chạy đúng, ổn định và tiết kiệm năng lượng trong môi trường thực tế.

- **Đa nhiệm thời gian thực:** thu thập cảm biến, điều khiển modem, truyền dữ liệu và xử lý cảnh báo phải chạy song song, độ trễ thấp.
- **Quản lý năng lượng nghiêm ngặt:** chuyển trạng thái linh hoạt giữa active/sleep/deep sleep theo điều kiện vận hành xe.
- **Giao tiếp đa giao thức:** BLE (OBD2), UART (modem), MQTT/HTTP (cloud), I2C/ADC/GPIO (ngoại vi phần cứng).
- **Độ bền vận hành cao:** có cơ chế retry, fallback và phục hồi sau mất kết nối mạng hoặc lỗi ngoại vi.

3.2.1.2. So sánh các phương án nền tảng Firmware

Bảng 3.5A: So sánh các phương án nền tảng firmware

Phương án	Mô tả	Ưu điểm	Hạn chế	Mức phù hợp
PA-FW1: Arduino Core + Superloop	Vòng lặp chính tuần tự, xử lý tác vụ theo polling	Dễ bắt đầu, ít cấu hình	Khó mở rộng đa nhiệm thực sự, khó tối ưu deep sleep phức tạp, quản lý lỗi hạn chế	Trung bình
PA-FW2: ESP-IDF + FreeRTOS (Đã chọn)	Kiến trúc task và event, driver chính thức Espressif	Đa nhiệm tốt, hỗ trợ power management sâu, tích hợp NimBLE / modem / UART / I2C ổn định	Độ phức tạp cao hơn Arduino	Cao
PA-FW3: Zephyr RTOS trên ESP32	RTOS đa nền tảng, kiến trúc phân lớp	Tính chuẩn hóa tốt, khả năng mở rộng dài hạn	Hệ sinh thái ESP32 chuyên biệt và tài liệu thực chiến BLE-OBD2/modem ít hơn ESP-IDF	Trung bình

3.2.1.3. Chọn giải pháp Firmware Từ kết quả so sánh, đề án chọn **PA-FW2: ESP-IDF + FreeRTOS** làm nền tảng firmware chính. Đây không phải là lựa chọn “mạnh nhất

trên giấy”, mà là lựa chọn bám sát nhất cách thiết bị phải vận hành ngoài thực tế: nhiều ngoại vi, nhiều trạng thái, yêu cầu tiết kiệm điện và cần dễ truy vết khi có lỗi.

Nền tảng này thuyết phục hơn ở bốn điểm chính:

- **Khớp yêu cầu điều phối thời gian thực của hệ thống:** ESP-IDF + FreeRTOS cho phép tổ chức state machine trung tâm, các driver bất đồng bộ và primitive đồng bộ ở mức khối chức năng. Nói cách khác, đây là nền tảng đủ linh hoạt để firmware vừa theo dõi nhiều tín hiệu cùng lúc vừa không làm luồng xử lý chính trở nên rối và khó kiểm soát.
- **Tối ưu cho ESP32-S3:** driver và power feature chính thức giúp giảm rủi ro tích hợp phần cứng, đồng thời rút ngắn đáng kể thời gian xử lý các lỗi phát sinh trong giai đoạn chế tạo và kiểm thử.
- **Phù hợp bài toán năng lượng:** hỗ trợ deep sleep/wakeup và điều khiển peripheral theo trạng thái, nên phù hợp với một thiết bị phải luôn cân bằng giữa khả năng giám sát và mức tiêu thụ điện.
- **Dễ kiểm soát chất lượng vận hành:** thuận lợi xây dựng state machine, retry strategy và fallback cho BLE/modem. Điều này đặc biệt quan trọng vì thiết bị không hoạt động trong phòng thí nghiệm lý tưởng mà trong môi trường xe có nhiễu, rung động và mất sóng bất chợt.

3.2.2. Giải pháp Firmware

Firmware là nơi toàn bộ quyết định vận hành của thiết bị được hiện thực hóa thành hành vi cụ thể: đọc cảm biến, đánh giá trạng thái xe, điều khiển modem, quản lý năng lượng và gửi dữ liệu lên máy chủ. Trên cơ sở phương án đã chọn ở mục 3.2.1, phần này trình bày cách nhóm biến các yêu cầu đó thành một kiến trúc firmware đủ rõ ràng để phát triển, đủ gọn để vận hành ổn định và đủ linh hoạt để mở rộng sau này.

3.2.2.1. Kiến trúc firmware và luồng hoạt động

a) Kiến trúc phân lớp (Layered Architecture) Để người đọc dễ hình dung, firmware được tách theo từng lớp trách nhiệm thay vì trình bày như một khối mã nguồn lớn. Cách chia này giúp trả lời rõ ba câu hỏi quan trọng: lớp nào làm việc với phần cứng, lớp nào giữ trạng thái vận hành, và lớp nào chịu trách nhiệm giao tiếp với hệ thống bên ngoài.

Bảng 3.2.2A: Tổng hợp thông tin về Lớp, Thành phần chính, Vai trò

Lớp	Thành phần chính	Vai trò
1	adc_reader, imu_lis3dh, modem_at, modem_lte, modem_gnss, ble_mgr, ble_obd	Trừu tượng hóa phần cứng và giao thức ngoại vi
2	power_mgr, nvs_config, app_state	Quản lý nguồn, lưu cấu hình NVS và duy trì ngữ cảnh RTC qua deep sleep
3	state_machine, command_handler	Điều phối trạng thái vận hành, lệnh điều khiển và luồng OTA
4	data_formatter, mqtt_client	Đóng gói payload JSON, publish/subscribe MQTT theo topic chuẩn của hệ thống
5	offline_queue, sd_log_store	Hàng đợi cục bộ trên microSD (enqueue, replay theo ACK/QoS, quota GC, metadata phiên ghi)

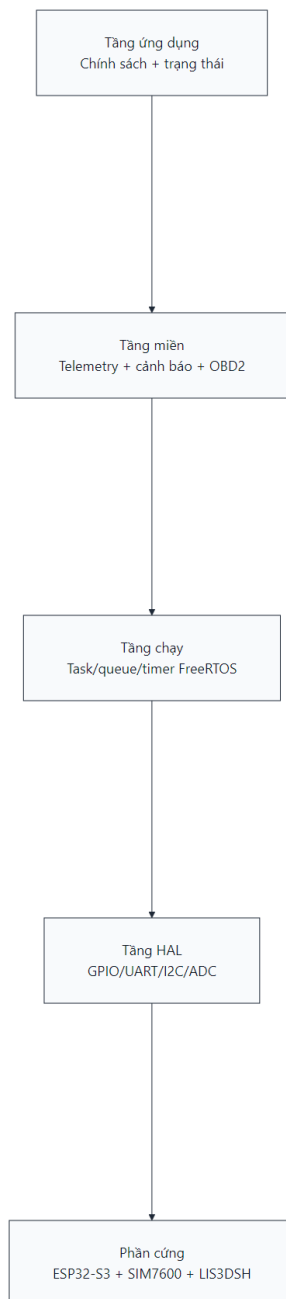
Trong triển khai hiện tại, firmware không tách thành nhiều tác vụ nghiệp vụ cấp cao như sơ đồ khái niệm ban đầu. Phần lớn quyết định vận hành được gom về một state machine trung tâm; các primitive FreeRTOS chỉ được dùng ở những nơi thật sự cần đồng bộ như NimBLE host task, BLE manager, BLE OBD và lớp AT command.

b) Cơ chế điều phối với FreeRTOS Điểm đáng chú ý trong triển khai này là FreeRTOS không được dùng theo hướng “chia nhỏ càng nhiều task càng tốt”. Thay vào đó, firmware giữ một luồng điều phối trung tâm trong `app_main()`, nạp cấu hình từ NVS, xác định trạng thái ban đầu theo nguyên nhân đánh thức rồi gọi `state_machine_run()` theo chu kỳ ngắn. Cách tổ chức này giúp logic điều khiển vẫn tập trung, dễ đọc và dễ debug, trong khi các cơ chế đồng bộ của FreeRTOS chỉ xuất hiện ở những nơi thật sự cần thiết.

```
void app_main(void) {
    ESP_ERROR_CHECK(nvs_config_init());

    config_t config = {0};
    ESP_ERROR_CHECK(nvs_config_load(&config));

    app_state_t state = APP_STATE_INIT;
```



Hình 3.5: Sơ đồ kiến trúc phân lớp của firmware

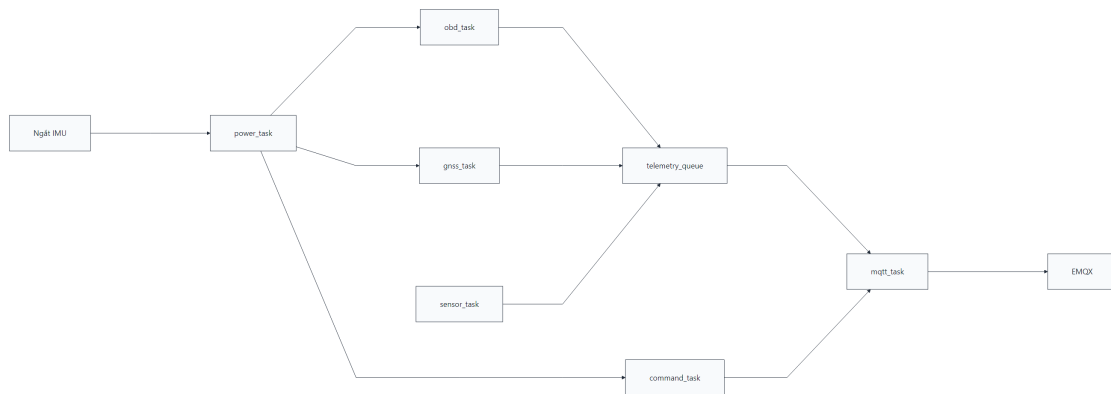
```
esp_sleep_wakeup_cause_t wakeup = esp_sleep_get_wakeup_cause();  
if (wakeup == ESP_SLEEP_WAKEUP_TIMER) {  
    state = APP_STATE_HEARTBEAT;  
} else if (wakeup == ESP_SLEEP_WAKEUP_EXT0) {  
    state = APP_STATE_ALARM;  
}  
  
ESP_ERROR_CHECK(state_machine_init(&config));
```

```

while (true) {
    state = state_machine_run(state);
    vTaskDelay(pdMS_TO_TICKS(100));
}
}

```

Các primitive FreeRTOS vẫn xuất hiện ở các mô-đun cần đồng bộ truy cập tài nguyên dùng chung. Cụ thể, NimBLE host chạy trong một task nền riêng; ble_mgr dùng queue và mutex để nhận kết quả khám phá/kết nối; ble_obd dùng semaphore cho phản hồi từ adapter; còn modem_at dùng mutex để tuần tự hóa lệnh AT trên UART chung.



Hình 3.6: Sơ đồ tương tác giữa các FreeRTOS task

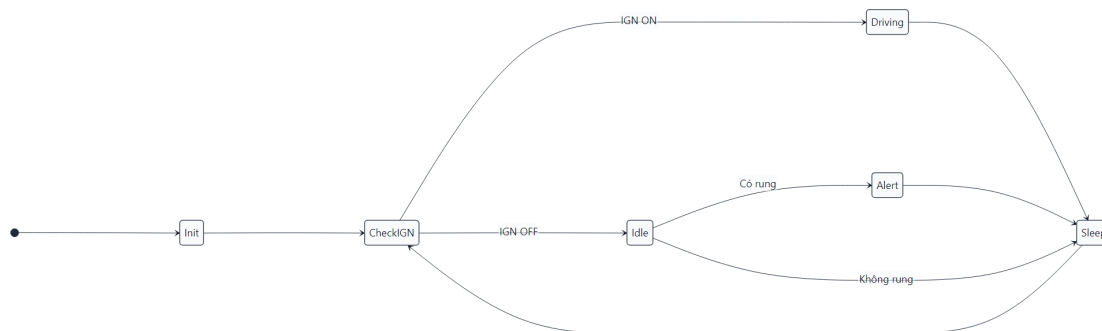
c) Luồng hoạt động cơ bản Nếu nhìn từ góc độ vận hành, firmware lặp lại một chu trình rất rõ: khởi tạo, nhận biết trạng thái xe, chọn chế độ phù hợp, thực hiện tác vụ và quay về trạng thái tiết kiệm năng lượng khi có thể. Luồng tổng thể được tổ chức theo trình tự sau:

1. **Khởi tạo ngoại vi:** Cấu hình và khởi động các peripheral gồm IMU (LIS3DSH qua I2C), modem SIM7600CE-T (UART1, Auto mode AT+CNMP=2, GNSS qua AT+CGNSINF), ADC (đọc điện áp), và BLE stack (NimBLE).
2. **Đọc trạng thái IGN và điện áp ắc quy:** Hệ thống ưu tiên đọc trạng thái động cơ (IGN) trực tiếp từ ECU qua OBD2 BLE. Nếu không kết nối được OBD2, hệ thống fallback sang đo điện áp ắc quy qua ADC theo profile: profile 12V dùng $IGN_{ON} \geq 13.0\text{ V}$ và $IGN_{OFF} \leq 12.0\text{ V}$; profile 24V dùng $IGN_{ON} \geq 26.0\text{ V}$ và $IGN_{OFF} \leq 24.0\text{ V}$.
3. **Quyết định chế độ hoạt động:** Dựa trên trạng thái IGN và dữ liệu cảm biến, firmware chuyển sang chế độ phù hợp (lái xe, đỗ xe, hoặc cảnh báo).
4. **Thực thi tác vụ trong từng chế độ:**
 - **Chế độ lái xe (Driving):** Kết nối OBD2, theo dõi liên tục, giữ BLE active, gửi

telemetry định kỳ.

- **Chế độ đỗ xe (Parked):** Không kết nối OBD2, chuyển sang deep sleep, gửi heartbeat định kỳ.
- **Chế độ cảnh báo (Alarm):** Gửi cảnh báo ngay lập tức, theo dõi liên tục, có thể kết nối OBD2 tùy chọn.

5. **Xử lý sau tác vụ:** Nếu IGN ON, giữ kết nối BLE và không deep sleep. Nếu IGN OFF, ngắt BLE và chuyển sang deep sleep để tiết kiệm năng lượng.



Hình 3.7: Lưu đồ thuật toán luồng hoạt động chính của firmware

3.2.2.2. Module giao tiếp Bluetooth OBD2

a) Tổng quan giao tiếp BLE OBD2 Module giao tiếp Bluetooth Low Energy (BLE) OBD2 cho phép thiết bị theo dõi kết nối không dây với adapter vgate iCar Pro — thiết bị OBD2 hỗ trợ BLE, được cắm trực tiếp vào cổng chẩn đoán OBD-II của xe. Firmware sử dụng NimBLE stack (tích hợp trong ESP-IDF) để triển khai giao tiếp BLE theo chuẩn GATT (Generic Attribute Profile).

Kiến trúc module BLE OBD2 được tổ chức theo bốn tầng logic: lớp điều phối ứng dụng trong `state_machine.c`, lớp giao thức OBD tại `ble_obd.c`, lớp quản lý BLE tại `ble_mgr.c` và lớp BLE stack của NimBLE. Cách tách lớp này giúp firmware cô lập xử lý PID OBD2 khỏi phần GATT/BLE thuần túy, thuận lợi cho debug và mở rộng.

b) Quy trình kết nối BLE OBD2 Quy trình kết nối BLE với adapter OBD2 diễn ra theo các bước sau:

1. **Kiểm tra địa chỉ BLE đã lưu:** Đọc địa chỉ MAC của vgate iCar Pro từ bộ nhớ flash (NVS). Nếu đã có địa chỉ, chuyển trực tiếp sang bước kết nối (reconnect nhanh trong 1–3 giây).
2. **Quét tìm thiết bị (Scan):** Nếu chưa có địa chỉ, thực hiện BLE scan để tìm adapter. Quá trình scan lọc thiết bị theo tên (device name) hoặc service UUID đặc trưng của OBD2 adapter.

3. **Kết nối GATT:** Thiết lập kết nối BLE với adapter, thực hiện GATT service discovery để tìm các characteristic cần thiết (TX và RX characteristic).

4. **Gửi lệnh khởi tạo ELM327:** Gửi chuỗi lệnh khởi tạo giao thức ELM327 qua BLE GATT write:

- ATZ — Reset adapter
- ATE0 — Tắt echo
- ATL0 — Tắt line feed
- ATS0 — Tắt khoảng trắng
- ATSP0 — Tự động phát hiện giao thức OBD2

5. **Đọc dữ liệu OBD2:** Gửi các lệnh PID (Parameter ID) để đọc thông số xe:

- 010C — RPM động cơ
- 010D — Tốc độ xe
- 0105 — Nhiệt độ nước làm mát
- 012F — Mức nhiên liệu
- AT IGN — Trạng thái động cơ (ignition)

```
/* Quy trình kết nối và đọc dữ liệu OBD2 */
void ble_obd2_task(void *param) {
    /* Bước 1: Đọc BLE address từ NVS */
    uint8_t saved_addr[6];
    bool has_saved = nvs_read_ble_address(saved_addr);

    /* Bước 2: Kết nối hoặc scan */
    if (has_saved) {
        ble_connect_direct(saved_addr); /* Reconnect nhanh */
    } else {
        ble_start_scan(BLE_SCAN_TIMEOUT_MS);
        ble_connect_first_obd2_device();
        nvs_save_ble_address(connected_addr);
    }

    /* Bước 3: Khởi tạo ELM327 */
    ble_obd_send_command("ATZ");
    ble_obd_send_command("ATE0");
    ble_obd_send_command("ATSP0");

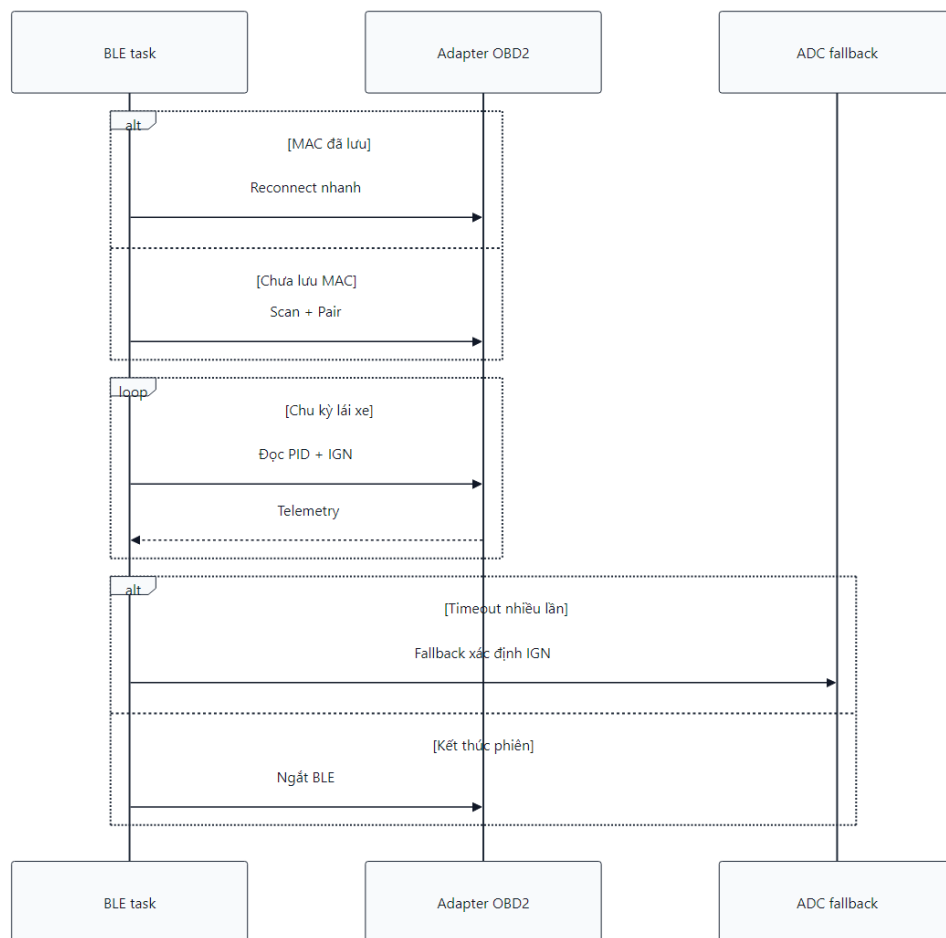
    /* Bước 4: Vòng lặp đọc dữ liệu */
    while (device_state == STATE_DRIVING) {
```

```

obd2_data_t data;
data.rpm    = ble_obd_read_pid(PID_RPM);
data.speed  = ble_obd_read_pid(PID_SPEED);
data.temp   = ble_obd_read_pid(PID_COOLANT_TEMP);
data.fuel    = ble_obd_read_pid(PID_FUEL_LEVEL);

xQueueSend(telemetry_queue, &data, pdMS_TO_TICKS(100));
vTaskDelay(pdMS_TO_TICKS(TRACKING_INTERVAL_MS));
}
}

```



Hình 3.8: Lưu đồ thuật toán quy trình kết nối và đọc dữ liệu BLE OBD2

c) Chiến lược kết nối theo chế độ hoạt động Việc kết nối BLE OBD2 được tối ưu hóa theo từng chế độ hoạt động của thiết bị để tiết kiệm năng lượng:

Chế độ lái xe (IGN ON): ESP32-S3 duy trì kết nối BLE liên tục với OBD2 adapter. Dữ liệu OBD2 được đọc định kỳ (mỗi 5–10 giây). Hệ thống chỉ sử dụng light sleep (giữ

BLE active) thay vì deep sleep.

Chế độ đỗ xe (IGN OFF): Không kết nối OBD2 adapter. Trạng thái IGN đã được xác nhận trước khi chuyển sang deep sleep. Việc không kết nối BLE khi đỗ xe giúp tiết kiệm đáng kể thời gian wake-up và năng lượng tiêu thụ.

Chế độ cảnh báo (Motion Detected): Kết nối OBD2 là tùy chọn. Hệ thống có thể chỉ sử dụng IMU và GPS để xác nhận chuyển động mà không cần dữ liệu OBD2.

Đặc biệt, khi ESP32-S3 chuyển sang deep sleep, kết nối BLE bị ngắt hoàn toàn. Mỗi lần wake-up, firmware cần thực hiện reconnect với adapter. Nhờ có việc lưu địa chỉ MAC vào flash, thời gian reconnect chỉ mất khoảng 1–3 giây, nhanh hơn đáng kể so với pairing lần đầu (3–10 giây).

d) Xử lý lỗi và cơ chế dự phòng (Fallback) Module BLE OBD2 được thiết kế với nhiều lớp xử lý lỗi:

Bảng 3.2.2B: Tổng hợp thông tin về Tình huống lỗi, Xử lý, Thời gian timeout

Tình huống lỗi	Xử lý	Thời gian timeout
Không kết nối được adapter	Retry 2–3 lần với delay 2 giây, sau đó fallback	10 giây
Mất kết nối giữa chừng	Reconnect 2–3 lần, nếu thất bại thì fallback	5 giây mỗi lần
Adapter không phản hồi lệnh	Retry 1–2 lần, nếu thất bại thì fallback	5 giây

Cơ chế dự phòng (Fallback): Khi không thể giao tiếp với OBD2 adapter, hệ thống chuyển sang phát hiện trạng thái động cơ bằng phương pháp đo điện áp ắc quy. Phương pháp này có độ chính xác thấp hơn OBD2 nhưng vẫn đảm bảo hệ thống hoạt động liên tục. Tiêu chí phân biệt theo profile: profile 12V dùng $IGN_{ON} \geq 13.0\text{ V}$ và $IGN_{OFF} \leq 12.0\text{ V}$; profile 24V dùng $IGN_{ON} \geq 26.0\text{ V}$ và $IGN_{OFF} \leq 24.0\text{ V}$.

Mọi lỗi kết nối OBD2 đều được ghi lại vào log để phục vụ việc giám sát và khắc phục sự cố từ xa.

3.2.2.3. Module điều khiển modem SIMCom

a) Kiến trúc modem SIMCom SIM7600CE-T (LTE + GNSS tích hợp) Trong kiến trúc hiện tại, modem SIMCom SIM7600CE-T đảm nhận toàn bộ luồng LTE và GNSS. ESP32-S3 giao tiếp với modem qua UART1 (GPIO16 TX, GPIO17 RX). Firmware sử

dùng một chuỗi lệnh AT chính xác để đảm bảo modem sẵn sàng trước khi bật PDP context và khởi động GNSS.

Bảng 3.2.2C: Tổng hợp thông tin về Bước, Lệnh AT, Mục đích chính

Bước	Lệnh AT	Mục đích chính
1	AT	Kiểm tra modem còn phản hồi
2	AT+CPIN?	Kiểm tra SIM card đã sẵn sàng
3	AT+CNMP=2	Đặt chế độ tự động LTE/UMTS/GSM
4	AT+CGDCONT=1 ,"IP","internet"	Cấu hình PDP context với APN mặc định internet
5	AT+CEREG?	Poll đến khi +CEREG: 0,1 (hoặc stat=5) trước khi kích hoạt PDP context
6	AT+CGACT=1,1	Bật PDP context sau khi đăng ký mạng thành công
7	AT+CGPADDR=1	Ghi nhận địa chỉ IP được cấp (dùng để log và kiểm tra)
8	AT+CGNSPWR=1	Bật GNSS tích hợp và đợi modem trả về fix vị trí
9	AT+CGNSINF	Đọc tọa độ GNSS hiện tại (vị trí, tốc độ, số vệ tinh); AT+CGNSTST chỉ dùng nếu cần dữ liệu NMEA hoàn toàn

Toàn bộ lệnh AT cho LTE và GNSS được thực hiện trên cùng UART1. Trình tự kiểm tra AT+CEREG? trước khi gọi AT+CGACT=1,1 giúp tránh kích hoạt PDP context khi modem chưa đăng ký mạng.

b) Trình tự khởi tạo LTE/GNSS trong firmware Ở tầng firmware, chuỗi lệnh trên không được gửi dồn trong một lần, mà được tách thành các bước kiểm tra có điều kiện: xác nhận modem phản hồi, xác nhận SIM sẵn sàng, chờ modem đăng ký mạng, sau đó mới bật PDP context và GNSS. Cách tổ chức này giúp quá trình khởi tạo dễ retry hơn và hạn chế lỗi dây chuyền khi điều kiện sóng di động thay đổi.

c) Quản lý kết nối và tiết kiệm điện

- **Khi hoạt động (driving mode):** Giữ PDP context mở và GNSS bật liên tục để gửi telemetry định kỳ và heartbeat.
- **Khi dừng (parking mode):** Firmware đóng PDP context (AT+CGACT=0,1), đưa modem vào AT+CSCLK=1 hoặc AT+CFUN=0, rồi chuyển ESP32-S3 sang deep sleep. Khi

wake-up, modem được đánh thức qua UART và thực hiện lại các lệnh AT+CFUN=1 + AT+CGACT=1, 1.

- **GNSS:** Sau khi AT+CGNSPWR=1, firmware tuần tự gọi AT+CGNSINF để lấy dữ liệu; AT+CGNSTST chỉ dùng khi cần stream NMEA để hỗ trợ debug hoặc phân tích phụ trợ.

d) Xử lý sự cố

Bảng 3.2.2D: Tổng hợp thông tin về Tình huống, Biện pháp

Tình huống	Biện pháp
Modem không phản hồi	Reset nhanh qua PWR-KEY (GPIO26) rồi bắt đầu lại chuỗi AT
Mất kết nối 4G	Đợi AT+CEREG? thành 1, gọi AT+CGACT=0, 1 rồi AT+CGACT=1, 1 để tái khởi động PDP
GNSS không fix dù đã bật	Gọi lại AT+CGNSINF, nếu vẫn thất bại thì bật lại GNSS (AT+CGNSPWR=0/1) rồi thử lại
Modem quá nóng	Đưa AT+CFUN=0 để tạm thời tắt RF và chờ nhiệt độ giảm trước khi tiếp tục

Các chế độ ngủ (AT+CSCLK=1, AT+CFUN=0) và wake-up bằng UART giữ cho modem tiêu thụ thấp mà vẫn có thể khởi động lại nhanh khi ESP32-S3 tỉnh dậy.

3.2.2.4. Module quản lý nguồn và GPIO

a) Sơ đồ chân GPIO Module quản lý nguồn sử dụng các chân GPIO của ESP32-S3 để điều khiển và giám sát hệ thống nguồn điện. Bảng sau liệt kê đầy đủ các chân GPIO được sử dụng:

Bảng 3.2.2E: Tổng hợp thông tin về GPIO, Chức năng, Hướng

GPIO	Chức năng	Hướng	Mô tả
2	IGN_IN	Input	Đọc trạng thái động cơ (GPIO hoặc OBD2)
4	U_BATT_ADC	Input	Đọc điện áp ắc quy (ADC 12-bit)
5	CHARGER_EN	Output	Điều khiển IC sạc TP4056
18	POWER_PATH_EN	Output	Chọn nguồn cấp (ắc quy hoặc pin dự phòng)

*Bảng 3.2.2E: Tổng hợp thông tin về GPIO, Chức năng, Hướng
(tiếp theo)*

GPIO	Chức năng	Hướng	Mô tả
19	LVD_STATUS	Input	Đọc trạng thái LVD từ khối LVD phần cứng
21	LIS3DSH_INT	Input	Ngắt từ cảm biến gia tốc IMU
47	LIS3DSH_SDA	I/O	Đường dữ liệu I2C
48	LIS3DSH_SCL	I/O	Đường clock I2C
16	MODEM_UART_TX	Output	UART TX đến modem
17	MODEM_UART_RX	Input	UART RX từ modem
26	MODEM_PWR-KEY	Output	Điều khiển nguồn modem

b) Điều khiển Power Path (Diode OR + EN) Hệ thống dùng power path theo **diode OR** giữa nhánh 5V chính (MP2482) và nhánh 5V backup (SX1308). Firmware điều khiển qua GPIO18 để ưu tiên nhánh nguồn phù hợp theo profile điện áp.

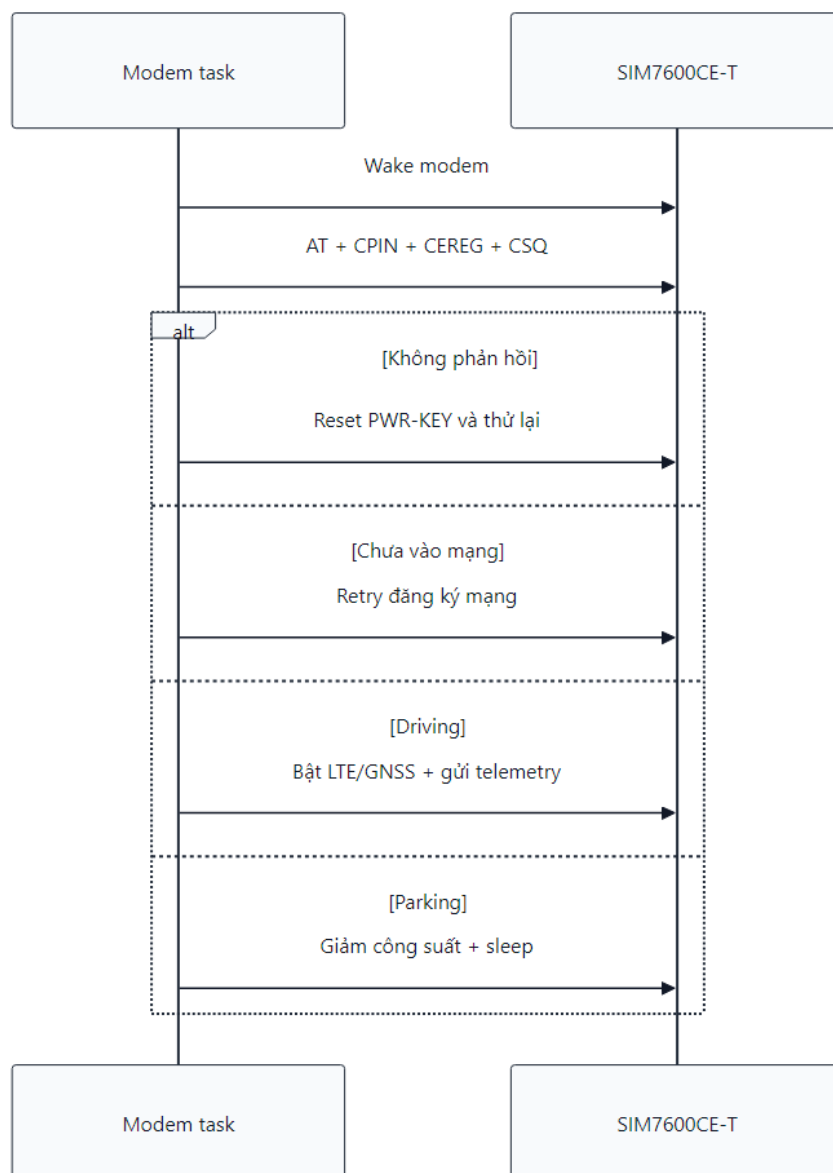
- **GPIO18 = LOW (0):** Ưu tiên nguồn ắc quy (nhánh MP2482 hoạt động)
- **GPIO18 = HIGH (1):** Giảm/khóa nhánh chính và cho phép nhánh backup qua SX1308 + diode OR

Logic điều khiển nguồn bám theo profile 12V/24V:

```
power_status_t power_get_status(float lvd_threshold_v, float lvd_hysteresis_v) {
    power_status_t status = {
        .voltage = adc_read_battery_voltage(),
        .is_low = false,
        .charger_on = gpio_get_level(PIN_CHARGER_EN) == 1,
    };

    if (status.voltage <= lvd_threshold_v) {
        s_voltage_low_latched = true;
    } else if (status.voltage >= lvd_hysteresis_v) {
        s_voltage_low_latched = false;
    }

    status.is_low = s_voltage_low_latched || power_is_low_voltage();
}
```

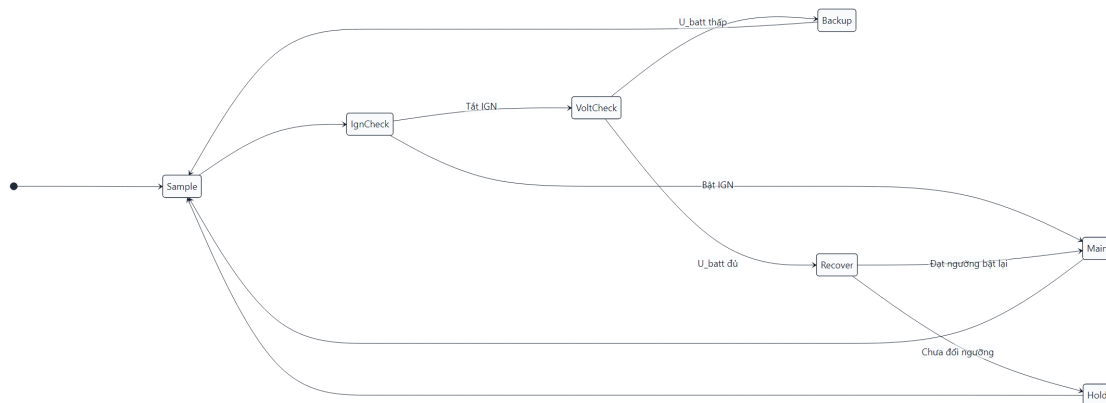


Hình 3.9: Lưu đồ điều phối modem SIM7600CE-T theo trạng thái mạng và chế độ vận hành

```

    return status;
}

```



Hình 3.10: Lưu đồ thuật toán điều khiển power path

c) Điều khiển IC sạc (TP4056) IC sạc TP4056 được điều khiển qua chân GPIO5 (CHARGER_EN). Logic sạc được thiết kế để bảo vệ cả ắc quy xe lẫn pin dự phòng:

Bảng 3.2.2F: Tổng hợp thông tin về Điều kiện, Trạng thái charger, Lý do

Điều kiện	Trạng thái charger	Lý do
IGN ON và U_{batt} vượt ngưỡng profile (ví dụ $> 12\text{ V}$ cho hệ 12V)	Enable (GPIO5 = HIGH)	Máy phát điện đang nạp, có thể sạc pin dự phòng
IGN OFF	Disable (GPIO5 = LOW)	Bảo vệ ắc quy không bị hao pin khi xe tắt máy
U_{batt} dưới ngưỡng profile (ví dụ $< 12\text{ V}$ cho hệ 12V)	Disable (GPIO5 = LOW)	Ắc quy yếu, không đủ năng lượng để sạc pin dự phòng

d) Giám sát điện áp và LVD Firmware đọc điện áp ắc quy qua kênh ADC 12-bit (GPIO4) với bộ chia áp để đưa điện áp ắc quy 12V hoặc 24V về dải đo của ADC (0–3.3V). Giá trị ADC được chuyển đổi sang điện áp thực thông qua công thức hiệu chuẩn (calibration).

Ngoài ra, hệ thống có thêm kênh giám sát LVD độc lập từ khối LVD phần cứng (GPIO19). Đây là kênh dự phòng cho ADC, cho phép kiểm tra nhanh trạng thái nguồn với cơ chế trễ (hysteresis): profile 12V dùng $Switch_{OFF} = 12.0\text{ V}$, $Switch_{ON} = 12.2\text{ V}$; profile 24V dùng $Switch_{OFF} = 24.0\text{ V}$, $Switch_{ON} = 24.4\text{ V}$. Khối LVD xử lý hysteresis ở mức phần cứng, tránh hiện tượng dao động (oscillation) quanh ngưỡng.

e) Các chế độ quản lý nguồn Firmware hỗ trợ ba chế độ năng lượng chính:

Bảng 3.2.2G: Tổng hợp thông tin về Chế độ, Trạng thái ESP32, Trạng thái modem

Chế độ	Trạng thái ESP32	Trạng thái modem	Trạng thái BLE	Dòng tiêu thụ
Normal (Driving)	Active	4G + GNSS active	Connected	150–250 mA
Light Sleep	Light sleep	Sleep (CSCLK)	Giữ kết nối	10–20 mA
Deep Sleep	Deep sleep	Deep sleep (CFUN=0)	Ngắt kết nối	< 2 mA

Chuyển đổi giữa các chế độ dựa trên trạng thái IGN và dữ liệu cảm biến, được điều phối bởi máy trạng thái chính (trình bày tại mục 3.2.2.5).

3.2.2.5. Định dạng dữ liệu và máy trạng thái

a) Định dạng dữ liệu MQTT Dữ liệu từ thiết bị được đóng gói theo định dạng JSON và truyền lên broker qua nhóm topic `v1/{device_id}/...`. Trong hệ thống hiện tại, thiết bị phát ba nhóm payload chính là `rawdata`, `status` và `events`; ngoài ra còn có luồng firmware để báo trạng thái OTA và luồng commands theo chiều ngược lại từ máy chủ.

Thông điệp Rawdata (dữ liệu telemetry chính)

Đây là gói dữ liệu được gửi định kỳ khi thiết bị đang hoạt động, phản ánh trạng thái GNSS, nguồn và cảm biến rung:

```
{
  "device_id": "TRACKER_001",
  "auth_token": "device_secret_token",
  "timestamp": 1710000000000,
  "uptime": 452130,
  "data": {
    "vibration": 128,
    "battery_top": 12.54,
    "battery_bot": 3.96,
    "latitude": 10.77653,
    "longitude": 106.70098,
    "speed": 42.5,
```

```

    "course": 181.2,
    "satellites": 11,
    "ignition": true,
    "error_code": 0
  }
}

```

Thông điệp Event (sự kiện/cảnh báo)

Được gửi khi firmware phát hiện một sự kiện bất thường hoặc muốn ghi nhận một mốc vận hành đáng chú ý:

```

{
  "device_id": "TRACKER_001",
  "auth_token": "device_secret_token",
  "event_type": "high_vibration",
  "code": 201,
  "message": "Vehicle movement detected while parked",
  "timestamp": 1710000005123
}

```

Thông điệp Firmware (trạng thái OTA)

Luồng firmware được dùng để ghi nhận vòng đời cập nhật firmware từ lúc gán tác vụ đến khi xác nhận thành công sau reboot hoặc rollback:

```

{
  "device_id": "TRACKER_001",
  "auth_token": "device_secret_token",
  "jobId": "ota_1710000000_ab12cd",
  "status": "confirming",
  "progress": 99,
  "targetVersion": "1.2.0",
  "currentVersion": "1.1.3",
  "partition": "ota_0"
}

```

Thông điệp Status (trạng thái hoạt động)

Được gửi khi thiết bị đổi trạng thái phiên làm việc hoặc phát heartbeat tại các mốc cần theo dõi:

```

{
  "device_id": "TRACKER_001",
  "auth_token": "device_secret_token",

```

```

    "status": "running",
    "session_id": 1024,
    "timestamp": 1710000010000
}

```

b) Thiết kế MQTT Topic Hệ thống sử dụng cấu trúc phân cấp MQTT topic để tách rõ luồng telemetry, trạng thái, sự kiện và OTA:

Bảng 3.2.2H: Tổng hợp thông tin về Topic, Hướng, QoS

Topic	Hướng	QoS	Mô tả
v1/{device_id}/rawdata	Device → Server	0	Payload telemetry chính; tần suất cao, chấp nhận mất vài mẫu
v1/{device_id}/status	Device → Server	1	Heartbeat, trạng thái phiên, online/offline
v1/{device_id}/events	Device → Server	1	Cảnh báo và sự kiện quan trọng
v1/{device_id}/firmware	Device → Server	1	Tiến trình OTA và kết quả cập nhật firmware
v1/{device_id}/commands	Server → Device	1	Lệnh điều khiển và cập nhật cấu hình từ máy chủ

Các lệnh điều khiển từ máy chủ hiện được firmware xử lý gồm:

Bảng 3.2.2I: Tổng hợp thông tin về Lệnh, Mục đích, Tham số chính

Lệnh	Mục đích	Tham số chính
update_config	Cập nhật cấu hình thiết bị	heartbeat_interval_s, tracking_interval_s
request_location	Yêu cầu gửi vị trí ngay ở chu kỳ kế tiếp	Không
enable_tracking	Bật hoặc tắt phát telemetry định kỳ	enabled (boolean)
reboot	Khởi động lại thiết bị	Không
ota_update	Kích hoạt cập nhật firmware OTA	jobId, version, url, size, sha256, force, confirmTimeoutSec

Bảng 3.2.2I: Tổng hợp thông tin về Lệnh, Mục đích, Tham số chính (tiếp theo)

Lệnh	Mục đích	Tham số chính
manual_rollback / ota_rollback	Yêu cầu quay về firmware trước đó	Không bắt buộc

c) Chiến lược xử lý mất kết nối Ở phạm vi triển khai hiện tại, firmware kết hợp hai cơ chế song song: (1) tự phục hồi kết nối MQTT và (2) hàng đợi cục bộ trên **microSD SDMMC 4-bit** để giữ dữ liệu khi mất mạng. Luồng dữ liệu được hiện thực qua cặp module `offline_queue` và `sd_log_store`: bản ghi được enqueue khi telemetry phát sinh (`offline_queue_enqueue()`), ghi xuống FATFS (`sd_log_store_append()`), sau đó replay theo thứ tự sequence khi mạng phục hồi (`offline_queue_replay_tick()`) và chốt ACK cho bản ghi quan trọng (`offline_queue_handle_publish_ack()`).

Thiết kế này cho phép tách bản ghi thành hai nhóm: **critical** (QoS 1, yêu cầu ACK) và **non-critical** (QoS 0, ưu tiên throughput), đồng thời áp dụng cơ chế quota + garbage collection (`sd_log_store_gc_if_needed()`) để tránh đầy thẻ. Trong phạm vi vận hành hiện tại, hệ thống giả định thẻ SD ở trạng thái **card-present-at-boot**; luồng hotplug runtime (rút/cắm nóng) không phải phạm vi mục tiêu.

/ Chiến lược hiện tại: enqueue local + replay khi MQTT sẵn sàng */*
`offline_queue_enqueue(type, payload, gps_fix, net_up);`

```

if (tracker_mqtt_is_connected()) {
    offline_queue_replay_tick();
}

```

d) Máy trạng thái thiết bị (Device State Machine) Máy trạng thái là cơ chế điều phối trung tâm của firmware, quyết định hành vi thiết bị tại từng thời điểm vận hành. Hệ thống định nghĩa bảy trạng thái chính:

Bảng 3.2.2J: Tổng hợp thông tin về Trạng thái, Mã, Mô tả

Trạng thái	Mã	Mô tả
INIT	0	Khởi tạo hệ thống, cấu hình ngoại vi
CHECK_IGN	1	Kiểm tra trạng thái động cơ
DRIVING	2	Chế độ lái xe - theo dõi liên tục
PARKED	3	Chế độ đỗ xe - tiết kiệm năng lượng

Bảng 3.2.2J: Tổng hợp thông tin về Trạng thái, Mã, Mô tả (tiếp theo)

Trạng thái	Mã	Mô tả
ALARM	4	Chế độ cảnh báo - phát hiện chuyển động bất thường
HEARTBEAT	5	Gửi tín hiệu heartbeat
SLEEP	6	Deep sleep - tiêu thụ năng lượng tối thiểu

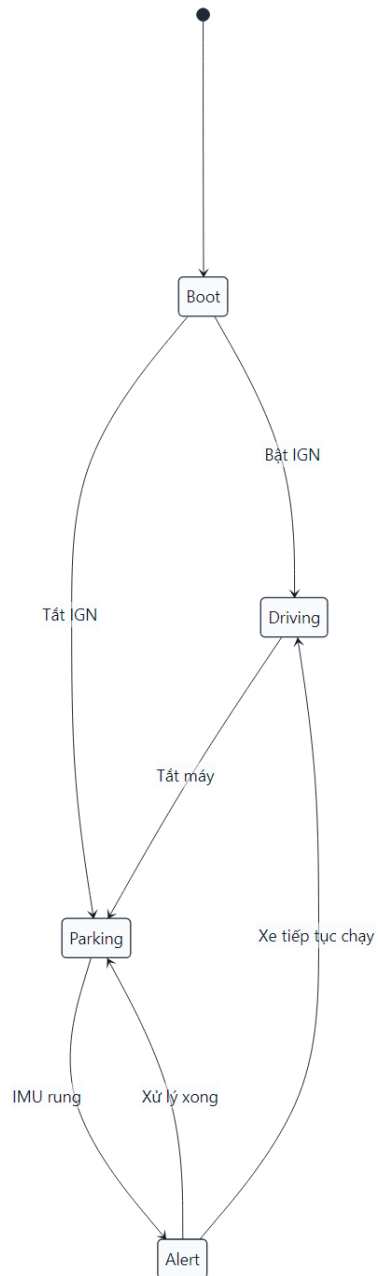
Bảng chuyển đổi trạng thái:

Trạng thái hiện tại	Sự kiện kích hoạt	Trạng thái tiếp theo
INIT	Khởi tạo hoàn tất	CHECK_IGN
CHECK_IGN	IGN = ON	DRIVING
CHECK_IGN	IGN = OFF	PARKED
DRIVING	IGN = OFF được phát hiện	PARKED
PARKED	IMU interrupt (motion detected)	ALARM
PARKED	Timer wake-up	HEARTBEAT
ALARM	Motion dừng, IGN = OFF	PARKED
ALARM	IGN = ON	DRIVING
HEARTBEAT	Gửi heartbeat xong	SLEEP
SLEEP	Timer wake-up hoặc IMU interrupt	CHECK_IGN

```
/* Cấu trúc máy trạng thái */
```

```
typedef enum {
    STATE_INIT,
    STATE_CHECK_IGN,
    STATE_DRIVING,
    STATE_PARKED,
    STATE_ALARM,
    STATE_HEARTBEAT,
    STATE_SLEEP
} device_state_t;

void state_machine_task(void *param) {
    device_state_t state = STATE_INIT;
```



Hình 3.11: Sơ đồ máy trạng thái của thiết bị theo dõi

```

while (1) {
    switch (state) {
    case STATE_INIT:
        init_peripherals();
        state = STATE_CHECK_IGN;
        break;

    case STATE_CHECK_IGN:
        if (check_ignition_status()) {

```

```

        state = STATE_DRIVING;
    } else {
        state = STATE_PARKED;
    }
    break;

case STATE_DRIVING:
    start_continuous_tracking();
    /* Chuyển sang PARKED khi phát hiện IGN OFF */
    EventBits_t bits = xEventGroupWaitBits(
        system_event_group, EVT_IGN_OFF,
        pdTRUE, pdFALSE, pdMS_TO_TICKS(1000));
    if (bits & EVT_IGN_OFF) {
        stop_ble_connection();
        state = STATE_PARKED;
    }
    break;

case STATE_PARKED:
    prepare_for_sleep();
    state = STATE_SLEEP;
    break;

case STATE_ALARM:
    send_alert_immediate();
    start_continuous_tracking();
    /* Đợi cho đến khi hết chuyển động */
    if (!is_motion_detected() && !check_ignition_status()) {
        state = STATE_PARKED;
    }
    break;

case STATE_HEARTBEAT:
    wakeup_modem();
    tracker_mqtt_connect();
    send_heartbeat();
    state = STATE_SLEEP;
    break;

```

```

    case STATE_SLEEP:
        shutdown_modem();
        configure_wakeup_sources();
        enter_deep_sleep();
        /* Sau khi wake-up, bắt đầu lại từ CHECK_IGN */
        state = STATE_CHECK_IGN;
        break;
    }
}
}

```

Lưu trạng thái qua deep sleep: Trước khi vào deep sleep, firmware lưu trạng thái hiện tại vào vùng nhớ RTC (RTC memory) — vùng nhớ đặc biệt của ESP32-S3 được giữ nguyên nội dung trong suốt quá trình deep sleep. Khi wake-up, firmware đọc trạng thái từ RTC memory để biết chế độ hoạt động trước đó và xử lý phù hợp.

3.2.2.6. Cấu hình hệ thống

a) Cấu trúc dữ liệu cấu hình Toàn bộ cấu hình của thiết bị được lưu trữ trong bộ nhớ flash (NVS — Non-Volatile Storage) của ESP32-S3 dưới dạng cấu trúc dữ liệu cố định:

```

typedef struct {
    char    device_id[16];          /* Mã định danh thiết bị */
    char    mqtt_broker[64];        /* Địa chỉ MQTT broker */
    uint16_t mqtt_port;            /* Cổng MQTT (mặc định 1883) */
    char    mqtt_username[32];      /* Tên đăng nhập MQTT */
    char    mqtt_password[32];      /* Mật khẩu MQTT */
    uint16_t heartbeat_interval;    /* Chu kỳ gửi heartbeat (giây) */
    uint16_t tracking_interval;     /* Chu kỳ gửi telemetry (giây) */
    char    obd2_ble_address[18];   /* Địa chỉ MAC của OBD2 adapter */
    float    switch_off;            /* Ngưỡng chuyển sang pin backup: 12.0V (12V) */
    float    switch_on;            /* Ngưỡng quay lại ắc quy: 12.2V (12V) hoặc */
    float    ign_on_threshold;      /* Ngưỡng suy luận IGN ON: >=13.0V (12V) hoặc */
    float    ign_off_threshold;     /* Ngưỡng suy luận IGN OFF: <=12.0V (12V) hoặc */
} config_t;

```

b) Quản lý cấu hình Cấu hình được quản lý theo ba cơ chế:

Cấu hình mặc định (Default Configuration): Được định nghĩa trong mã nguồn firmware, áp dụng khi thiết bị khởi động lần đầu hoặc khi NVS bị xóa. Bao gồm các giá trị an toàn như heartbeat_interval = 900 (15 phút), tracking_interval

= 10 (10 giây), và profile nguồn kép: profile 12V (switch_off=12.0, switch_on=12.2, ign_on_threshold=13.0, ign_off_threshold=12.0) hoặc profile 24V (switch_off=24.0, switch_on=24.4, ign_on_threshold=26.0, ign_off_threshold=24.0).

Cấu hình lưu trữ (Persistent Configuration): Lưu trong NVS, được tải khi thiết bị khởi động. Các thay đổi cấu hình từ máy chủ được lưu vào NVS để giữ nguyên sau khi reset hoặc mất điện.

Cập nhật cấu hình từ xa (Remote Configuration Update): Máy chủ gửi lệnh update_config qua MQTT topic v1/{device_id}/commands. Firmware phân tích lệnh, cập nhật các tham số tương ứng và lưu vào NVS. Thiết bị gửi xác nhận (ACK) về máy chủ sau khi cập nhật thành công.

c) Hiệu chuẩn cảm biến (Sensor Calibration) Để đảm bảo độ chính xác của phép đo, firmware hỗ trợ hiệu chuẩn ba loại cảm biến:

Bảng 3.2.2K: Tổng hợp thông tin về Cảm biến, Phương pháp hiệu chuẩn, Lưu trữ

Cảm biến	Phương pháp hiệu chuẩn	Lưu trữ
ADC (điện áp ác quy)	So sánh với đồng hồ vạn năng, tính hệ số hiệu chỉnh	NVS
IMU (LIS3DSH)	Đặt thiết bị trên mặt phẳng, đo offset các trục X/Y/Z	NVS
GNSS	Hiệu chỉnh offset vị trí (nếu cần)	NVS

Các giá trị hiệu chuẩn được lưu trữ trong NVS và được tải khi thiết bị khởi động, đảm bảo tính nhất quán của phép đo giữa các lần reset.

3.3. Phân tích và đề xuất kiến trúc Backend/Cloud – Backend/cloud analysis and solution

3.3.1. Phân tích và lựa chọn kiến trúc Cloud

3.3.1.1. Đặt vấn đề cho kiến trúc Cloud Khi dữ liệu rời khỏi thiết bị, trọng tâm không còn là “đọc được dữ liệu” mà là “biến bản tin đó thành thông tin hữu ích”. Ở tầng Backend/Cloud, một bản tin chỉ thật sự có giá trị khi nó được tiếp nhận ổn định, lưu đúng nơi, đẩy tới dashboard đúng lúc và vẫn truy vết được khi có sự cố. Vì vậy, kiến trúc cloud phải đồng thời giải quyết các bài toán sau:

- **Tiếp nhận dữ liệu liên tục từ nhiều thiết bị:** telemetry gửi theo chu kỳ ngắn, có thể xuất hiện burst khi thiết bị reconnect.

- **Cập nhật thời gian thực cho dashboard:** dữ liệu vị trí/trạng thái cần phản ánh gần như tức thời cho người vận hành.
- **Lưu trữ dữ liệu hỗn hợp:** vừa có dữ liệu chuỗi thời gian (GPS, nguồn, IMU; sẵn sàng mở rộng thêm OBD2), vừa có dữ liệu quan hệ nghiệp vụ (xe, người dùng, cảnh báo, geofence).
- **Chi phí và năng lực vận hành phù hợp đồ án:** ưu tiên self-hosted, dễ triển khai, dễ mở rộng theo từng dịch vụ.

3.3.1.2. So sánh các phương án kiến trúc Cloud

Bảng 3.15A: So sánh các phương án kiến trúc Cloud

Phương án	Mô tả ngắn	Ưu điểm	Hạn chế	Mức phù hợp
PA-A: Monolithic REST + 1 DB	Thiết bị gửi HTTP trực tiếp vào API server, lưu chung một cơ sở dữ liệu	Triển khai ban đầu đơn giản	Không tối ưu cho telemetry tần suất cao, khó mở rộng realtime, tải dồn lên API	Trung bình
PA-B: Managed Cloud Native	AWS IoT Core + Lambda + dịch vụ DB managed	Khả năng mở rộng cao, nhiều dịch vụ sẵn có	Chi phí vận hành cao, phụ thuộc nhà cung cấp, độ phức tạp hạ tầng vượt phạm vi đồ án	Trung bình
PA-C: Event-driven self-hosted	MQTT Broker + MQTT Bridge + dual database + API + WebSocket	Phù hợp IoT realtime, tách tải tốt, mở rộng theo dịch vụ, chi phí tự chủ	Cần chuẩn hóa message flow và quản lý nhiều dịch vụ Docker	Cao (Đã chọn)

3.3.1.3. Chọn giải pháp kiến trúc Cloud Đồ án chọn PA-C: kiến trúc phân tầng kết hợp hướng sự kiện (event-driven) với chuỗi xử lý:

Device -> EMQX -> MQTT Bridge -> PostgreSQL/

VictoriaMetrics/VictoriaLogs -> Backend API -> Frontend

Lựa chọn này xuất phát từ cách dữ liệu thực sự di chuyển trong hệ thống: thiết bị gửi bản tin liên tục, lớp ingest tiếp nhận và phân luồng, lớp lưu trữ ghi theo đúng đặc tính dữ liệu, còn lớp ứng dụng phục vụ người dùng và nghiệp vụ. Nói cách khác, kiến trúc được

chọn vì nó bám sát hành trình tự nhiên của dữ liệu thay vì dồn mọi trách nhiệm về một dịch vụ duy nhất.

Các lý do lựa chọn chính gồm:

- **Khớp bản chất dữ liệu IoT:** MQTT xử lý tốt mô hình pub/sub, giảm mức phụ thuộc trực tiếp giữa thiết bị và tầng ứng dụng. Điều này giúp thiết bị chỉ tập trung gửi dữ liệu, thay vì phải biết chi tiết ai sẽ nhận và xử lý dữ liệu đó ở phía sau.
- **Tối ưu hiệu năng theo vai trò:** Bridge xử lý ingestion, API tập trung nghiệp vụ, database tách theo loại dữ liệu. Mỗi lớp chỉ làm đúng việc mình mạnh nhất, nhờ đó hệ thống dễ giữ ổn định hơn khi tải tăng.
- **Dễ scale theo chiều ngang:** có thể nhân bản riêng broker/bridge/api khi số thiết bị tăng, thay vì phải mở rộng toàn bộ hệ thống một cách đồng loạt và tốn kém.
- **Phù hợp chi phí và phạm vi thực hiện:** self-hosted qua Docker Compose, không phụ thuộc nền tảng cloud thương mại, nên phù hợp với tính chất của một đồ án vừa cần triển khai thật, vừa cần kiểm soát tốt nguồn lực.

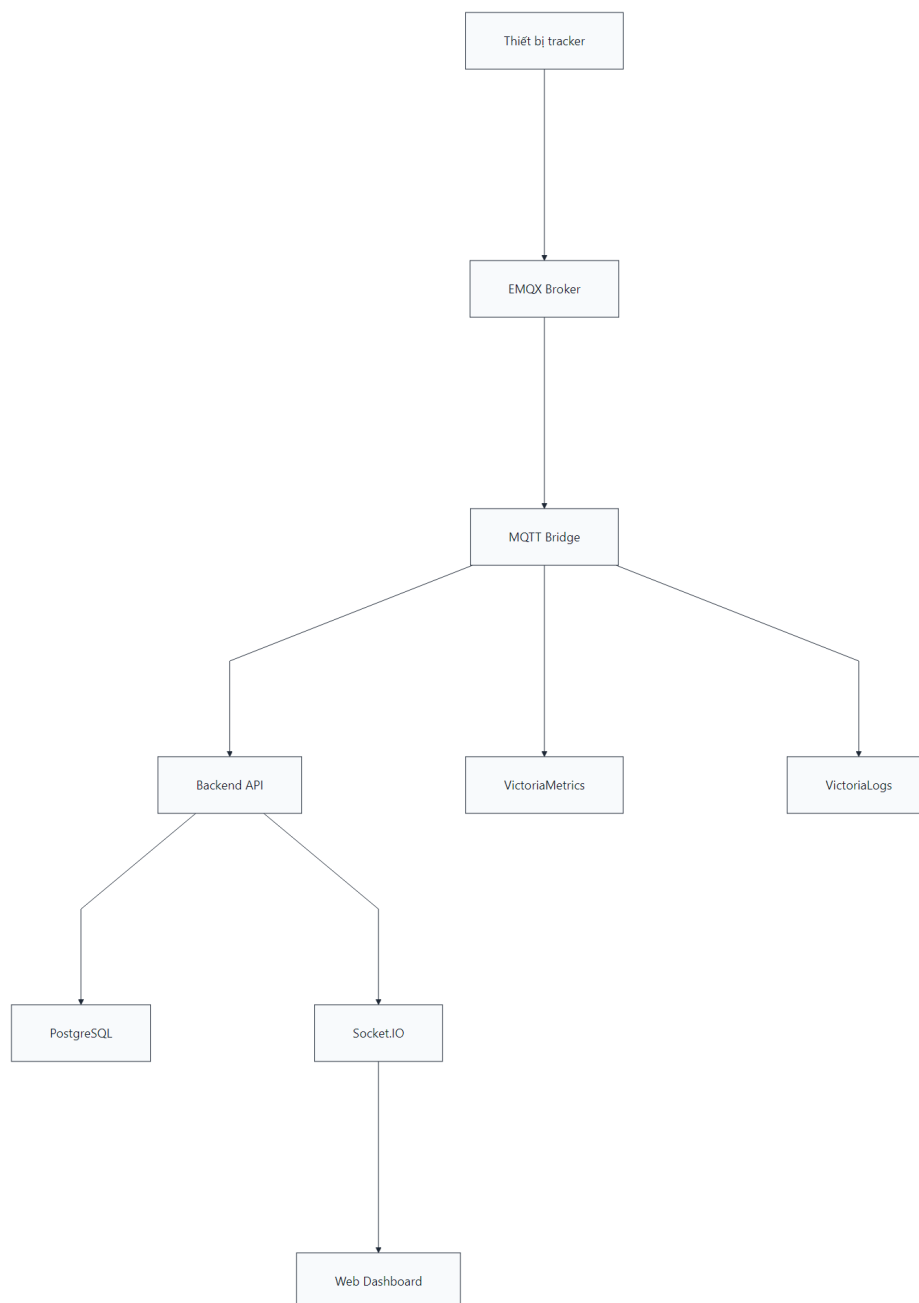
3.3.2. Giải pháp Backend & Cloud

Sau khi xác định phương án kiến trúc phù hợp, phần này đi vào cách triển khai cụ thể từng lớp của hệ thống Backend/Cloud. Cách trình bày được sắp xếp theo hành trình dữ liệu, từ lúc bản tin đi vào broker cho tới khi thông tin xuất hiện trên dashboard của người vận hành.

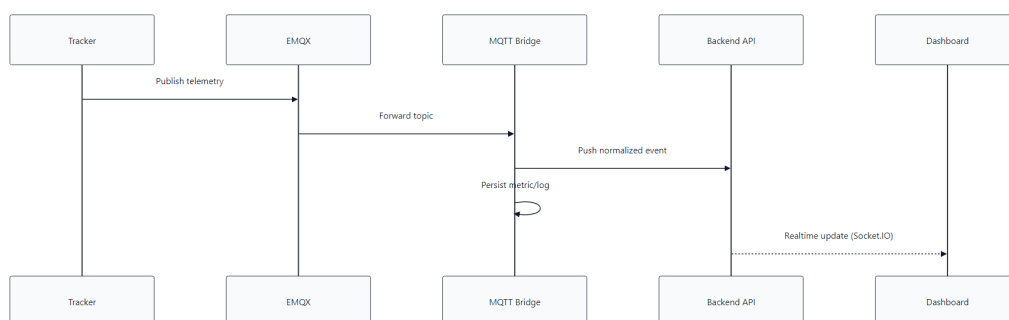
3.3.2.1. Kiến trúc tổng quan hệ thống Cloud (Cloud Architecture Overview)

a) Mô hình kiến trúc tổng thể Để người đọc dễ theo dõi, có thể hình dung hệ thống cloud như một chuỗi xử lý nhiều lớp: lớp đầu nhận dữ liệu, lớp giữa lưu và xử lý, lớp cuối cung cấp thông tin cho người dùng. Kiến trúc này vừa tận dụng được ưu thế của mô hình hướng sự kiện cho IoT, vừa giữ cho trách nhiệm của từng thành phần không bị chồng lấn. Cụ thể, hệ thống gồm các tầng chính sau:

- **Tầng thu thập dữ liệu (Data Ingestion Layer):** Tiếp nhận dữ liệu từ các thiết bị IoT thông qua giao thức MQTT, xử lý và phân phối đến các thành phần lưu trữ.
- **Tầng lưu trữ dữ liệu (Data Storage Layer):** Sử dụng chiến lược lưu trữ kép (dual database strategy) với cơ sở dữ liệu quan hệ và cơ sở dữ liệu chuỗi thời gian.
- **Tầng xử lý nghiệp vụ (Business Logic Layer):** API Server xử lý các yêu cầu từ Frontend, thực thi logic nghiệp vụ và quản lý trạng thái hệ thống.
- **Tầng giao tiếp thời gian thực (Real-time Communication Layer):** Cung cấp dữ liệu cập nhật trực tiếp đến người dùng thông qua giao thức WebSocket.



Hình 3.12: Sơ đồ kiến trúc tổng quan hệ thống Cloud



Hình 3.12a: Luồng dữ liệu chi tiết từ thiết bị đến dashboard

b) Luồng dữ liệu chính trong hệ thống Nếu mô tả bằng một “câu chuyện dữ liệu”, luồng chính của hệ thống diễn ra theo trình tự sau:

1. **Thiết bị IoT** gửi dữ liệu telemetry chính (GNSS, trạng thái nguồn, mức rung, ignition, uptime) lên EMQX Broker thông qua giao thức MQTT 5.0 theo topic có cấu trúc `v1/{device_id}/rawdata`.
2. **EMQX Broker** tiếp nhận và phân phối message đến các subscriber. Rules Engine của EMQX chủ yếu đóng vai trò lọc/bỏ trợ; phần lớn nghiệp vụ cảnh báo vẫn được xử lý tại MQTT Bridge.
3. **MQTT Bridge** (dịch vụ Node.js độc lập) subscribe các topic từ EMQX, xác thực dữ liệu đầu vào bằng Zod schema, sau đó thực hiện ghi kép (dual write):
 - Ghi dữ liệu chuỗi thời gian vào **VictoriaMetrics** (vị trí, tốc độ, điện áp, rung, ignition, uptime).
 - Ghi nhật ký sự kiện vào **VictoriaLogs** (kết nối/ngắt kết nối, lỗi, sự kiện MQTT).
 - Cập nhật trạng thái thiết bị vào **PostgreSQL** (trạng thái trực tuyến, phiên làm việc).
 - Phát sự kiện đến **Socket.IO** để cập nhật thời gian thực cho Dashboard.
4. **API Server** (Express.js) cung cấp REST API cho Frontend, truy vấn dữ liệu từ PostgreSQL và VictoriaMetrics, đồng thời gửi lệnh điều khiển ngược về thiết bị thông qua EMQX.
5. **Frontend** (Next.js) nhận dữ liệu cập nhật thời gian thực qua Socket.IO và dữ liệu lịch sử qua REST API.

c) Mô hình triển khai Docker Ở mức triển khai, kiến trúc trên được hiện thực bằng mô hình per-service Docker Compose theo quy ước IVM26. Mỗi dịch vụ có một file `docker-compose.yml` riêng và cùng tham gia mạng Docker `tracking-network` (external network). Cách tổ chức này giúp cấu trúc vận hành của đồ án bám sát cách các hệ thống thật được chia dịch vụ trong thực tế:

- **Độc lập khi triển khai:** Mỗi dịch vụ có thể khởi động, dừng hoặc cập nhật độc lập mà không ảnh hưởng đến các dịch vụ khác.
- **Quản lý tài nguyên riêng:** Mỗi container được giới hạn CPU và RAM cụ thể (ví dụ: Backend 512MB/1 CPU, MQTT Bridge 256MB/0.5 CPU).
- **Dễ dàng mở rộng:** Có thể nhân bản (scale) từng dịch vụ riêng lẻ theo nhu cầu.

Bảng 3.16: Phân bổ tài nguyên Docker cho các dịch vụ

Dịch vụ	Container	Port	Tài nguyên
Tracking_Backend	tracking-backend	4000	512MB, 1 CPU

Bảng 3.16: Phân bổ tài nguyên Docker cho các dịch vụ (tiếp theo)

Dịch vụ	Container	Port	Tài nguyên
Tracking_Frontend	tracking-frontend	4001	512MB, 1 CPU
Tracking_MqttBridge	tracking-mqtt-bridge	-	256MB, 0.5 CPU
Tracking_PostgreSQL	tracking-postgres	5432	1GB, 2 CPU
Tracking_EMQX	tracking-emqx	1883, 8883, 8083, 8084, 18083	1GB, 2 CPU
Tracking_VictoriaMetrics	tracking-victoriametrics	8428	512MB, 1 CPU
Tracking_VictoriaLogs	tracking-victorialogs	9428	512MB, 1 CPU
Tracking_Grafana	tracking-grafana	4002	Chưa cấu hình cứng

3.3.2.2. MQTT Broker - EMQX (Message Broker Selection & Configuration)

a) Vai trò của MQTT Broker MQTT Broker là điểm vào đầu tiên của dữ liệu ở phía cloud. Vai trò của nó có thể hiểu đơn giản là một “trạm trung chuyển tin nhắn”: thiết bị gửi bản tin đến broker, broker tiếp nhận và phân phối cho các thành phần cần xử lý tiếp theo. Nhờ mô hình **Publish-Subscribe (Pub/Sub)**, các thành phần trong hệ thống có thể giao tiếp bất đồng bộ mà không cần ghép chặt với nhau.

Trong hệ thống theo dõi phương tiện, cách tổ chức này đặc biệt quan trọng vì số lượng thiết bị có thể tăng dần theo thời gian, trong khi nhịp gửi dữ liệu lại thay đổi theo chế độ hoạt động của từng xe.

b) So sánh và lựa chọn MQTT Broker Để lựa chọn MQTT Broker phù hợp, đồ án đã đánh giá bốn giải pháp phổ biến trên thị trường:

Bảng 3.17: So sánh các MQTT Broker phổ biến

Tiêu chí	Mosquitto	EMQX	HiveMQ	VerneMQ
Kết nối đồng thời	1.000	100.000.000	200.000.000	10.000.000
Thông lượng message	40K msg/s	100K msg/s	200K msg/s	50K msg/s
Độ trễ (Latency)	0,25 ms	0,27 ms	<1 ms	2,1 ms
Sử dụng CPU	Rất thấp	Thấp (2%)	Thấp	Cao (10%)
Sử dụng RAM	254 MB	495 MB	Trung bình	1,2 GB
Hỗ trợ Clustering	Không	Có (20+ node)	Có	Có
Khả dụng cao (HA)	Không	Có	Có	Có
Độ phức tạp cài đặt	Rất dễ	Trung bình	Khó	Trung bình
Chi phí	Miễn phí	Miễn phí/Trả phí	Trả phí	Miễn phí/Trả phí
Cộng đồng	Lớn	Lớn	Trung bình	Nhỏ

Kết luận lựa chọn: EMQX Single Node

EMQX được lựa chọn làm MQTT Broker cho hệ thống vì đây là phương án giữ được thể cân bằng tốt giữa hiệu năng, khả năng mở rộng và mức độ phù hợp với nguồn lực triển khai của đề án. Các lý do chính gồm:

- **Hiệu năng phù hợp:** Theo tài liệu công bố của EMQX, broker đáp ứng tốt tải message lớn với độ trễ thấp, đủ cho quy mô triển khai hiện tại và còn dư địa mở rộng.
- **Khả năng mở rộng:** Hỗ trợ clustering nhiều node, thuận lợi nếu hệ thống tăng số lượng thiết bị trong giai đoạn sau.
- **Rules Engine tích hợp:** Thuận tiện cho các tác vụ lọc hoặc chuyển đổi dữ liệu đơn giản ngay tại broker; tuy nhiên trong kiến trúc hiện tại, đây không phải lớp xử lý nghiệp vụ chính.
- **Miễn phí và cộng đồng lớn:** Phiên bản Community cung cấp đầy đủ tính năng cần thiết cho đề án, có cộng đồng hỗ trợ tích cực.
- **Dashboard quản lý:** Cung cấp giao diện web quản lý tại port 18083, hỗ trợ giám sát kết nối, topic và message theo thời gian thực.

c) Cấu hình EMQX Rules Engine EMQX Rules Engine là lớp xử lý dữ liệu nhẹ ngay tại broker. Trong hệ thống hiện tại, phần lớn nghiệp vụ cảnh báo được xử lý ở MQTT Bridge; tuy vậy, Rules Engine vẫn phù hợp để lọc nhanh các điều kiện đơn giản hoặc đẩy

dữ liệu sang pipeline phụ mà không làm tăng tải cho backend.

Các rule minh họa phù hợp với payload hiện tại:

Rule 1 - Cảnh báo rung mạnh từ luồng sự kiện nội bộ:

SELECT

```
payload.device_id as device_id,  
payload.latitude as latitude,  
payload.longitude as longitude,  
payload.value as vibration,  
now() as timestamp
```

FROM "internal/events/device/alert"

WHERE payload.alert_type = 'high_vibration'

Rule 2 - Cảnh báo vượt tốc độ từ rawdata:

SELECT

```
payload.device_id as device_id,  
payload.data.speed as speed,  
payload.data.latitude as latitude,  
payload.data.longitude as longitude,  
now() as timestamp
```

FROM "v1+/rawdata"

WHERE payload.data.speed > 100

Rule 3 - Cảnh báo pin dự phòng thấp:

SELECT

```
payload.device_id as device_id,  
payload.data.battery_bot as battery_bot,  
payload.data.latitude as latitude,  
payload.data.longitude as longitude,  
now() as timestamp
```

FROM "v1+/rawdata"

WHERE payload.data.battery_bot < 3.5

Luồng xử lý cảnh báo:

1. Thiết bị phát bản tin lên EMQX qua các topic telemetry/sự kiện.
2. MQTT Bridge tiếp nhận, đánh giá rule và phát sự kiện vào internal/events/*.
3. Backend listener xử lý sự kiện, ghi PostgreSQL và phát Socket.IO để cập nhật dashboard theo thời gian thực.

Ngoài ra, hệ thống có thể mở rộng thêm rule cho geofence sơ cấp, thống kê traffic theo topic, hoặc phát hiện thiết bị im lặng bất thường để phục vụ vận hành.

3.3.2.3. Thiết kế cơ sở dữ liệu (Database Architecture Design) Đặt vấn đề lưu trữ: Khi đi sâu hơn vào tầng dữ liệu, có thể thấy hệ thống đang phục vụ hai nhu cầu rất khác nhau. Một mặt, telemetry cần được ghi nhanh và truy vấn theo thời gian. Mặt khác, dữ liệu nghiệp vụ đòi hỏi tính toàn vẹn, quan hệ rõ ràng và khả năng quản lý dài hạn. Nếu dùng một công cụ cho cả hai loại bài toán, hệ thống thường sẽ phải đánh đổi hiệu quả ở một trong hai phía.

Bảng 3.18A: So sánh các phương án kiến trúc lưu trữ dữ liệu

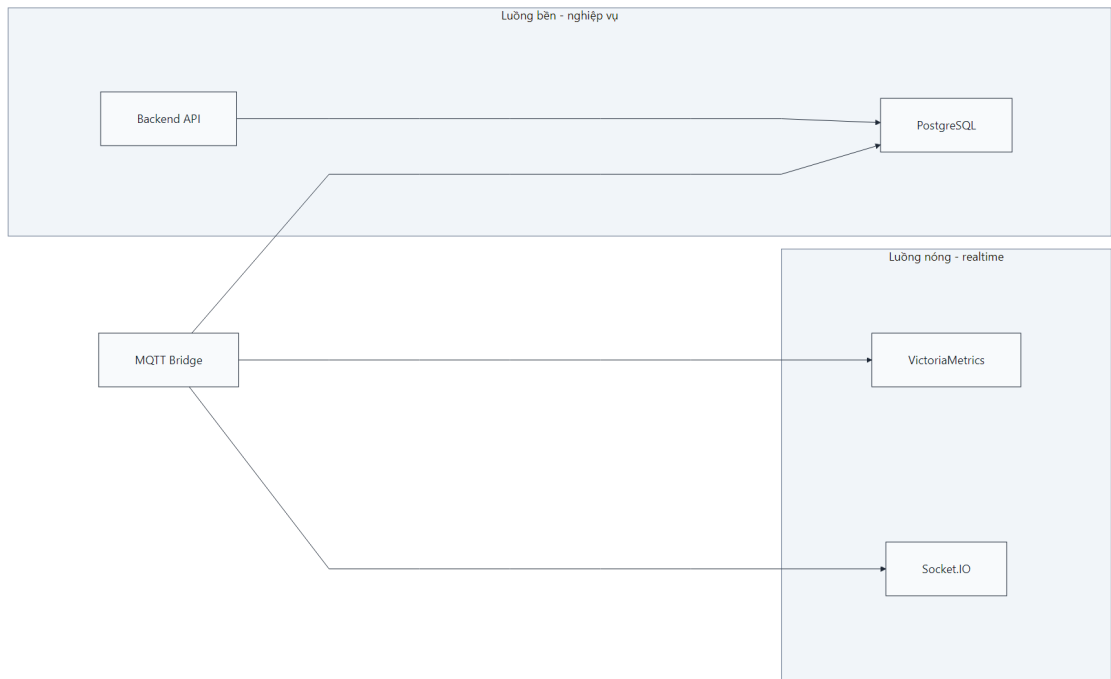
Phương án	Mô tả	Ưu điểm	Hạn chế	Mức phù hợp
PA-DB1: PostgreSQL duy nhất	Dùng PostgreSQL cho cả nghiệp vụ và telemetry	Đơn giản vận hành	Ghi telemetry lớn dễ ảnh hưởng truy vấn nghiệp vụ, chi phí index/partition cao	Trung bình
PA-DB2: Time-series DB duy nhất	Dùng CSDL chuỗi thời gian cho mọi loại dữ liệu	Tối ưu ghi dữ liệu cảm biến	Kém phù hợp dữ liệu quan hệ phức tạp, khó đảm bảo ràng buộc nghiệp vụ	Thấp
PA-DB3: Hybrid DB (Đã chọn)	PostgreSQL + VictoriaMetrics, VictoriaLogs	Mỗi loại dữ liệu dùng đúng công cụ, cân bằng hiệu năng và khả năng truy vấn	Tăng số thành phần cần vận hành	Cao

Kết luận lựa chọn: Chọn **PA-DB3 (hybrid database)** để tách trách nhiệm dữ liệu, giữ ổn định cho nghiệp vụ và tối ưu luồng telemetry theo thời gian thực.

a) Chiến lược lưu trữ kép (Dual Database Strategy) Hệ thống theo dõi phương tiện cần lưu trữ hai loại dữ liệu có đặc tính khác nhau căn bản:

- Dữ liệu thô (Raw Telemetry Data):** Bao gồm vị trí GPS, tốc độ, mức pin, dữ liệu OBD2, dữ liệu cảm biến gia tốc. Loại dữ liệu này có tần suất ghi rất cao (mỗi giây khi xe đang di chuyển), khối lượng lớn, nhưng chỉ cần lưu trữ trong thời gian ngắn (7–30 ngày) và chủ yếu phục vụ truy vấn theo chuỗi thời gian.
- Dữ liệu nghiệp vụ (Business Data):** Bao gồm thông tin xe, khách hàng, chuyến đi, cảnh báo, vi phạm, lệnh điều khiển. Loại dữ liệu này có tần suất ghi thấp, khối lượng nhỏ, nhưng cần lưu trữ lâu dài (6–12 tháng trở lên) và yêu cầu tính toàn vẹn dữ liệu quan hệ (ACID).

Do sự khác biệt cơ bản về đặc tính, hệ thống áp dụng chiến lược lưu trữ kép sử dụng ba cơ sở dữ liệu chuyên biệt:



Hình 3.13: Sơ đồ chiến lược lưu trữ kép

- **PostgreSQL (quan hệ):** lưu dữ liệu nghiệp vụ như vehicles, customers, trips, alerts, violations, commands, geofences.
- **VictoriaMetrics (chuỗi thời gian):** lưu dữ liệu telemetry mật độ cao như vị trí GPS, tốc độ, mức pin, ignition/uptime, dữ liệu IMU (retention khoảng 30 ngày).
- **VictoriaLogs (nhật ký):** lưu sự kiện thiết bị, lỗi, phiên làm việc, message MQTT và nhật ký kiểm toán (retention khoảng 7 ngày).

Bảng 3.18: So sánh các cơ sở dữ liệu trong hệ thống

Cơ sở dữ liệu	Mục đích	Ưu điểm chính
PostgreSQL 16	Dữ liệu quan hệ (nghiệp vụ)	ACID, relationships, truy vấn phức tạp
VictoriaMetrics	Dữ liệu chuỗi thời gian (telemetry)	Nhanh gấp 10 lần InfluxDB, PromQL, tiêu thụ RAM thấp
VictoriaLogs	Nhật ký tập trung (logging)	Tìm kiếm nhanh, lưu trữ nhỏ gọn, LogsQL

b) Cơ sở dữ liệu PostgreSQL - Schema quan hệ PostgreSQL 16 lưu trữ toàn bộ dữ liệu nghiệp vụ của hệ thống. Schema được chuẩn hóa đến dạng chuẩn thứ ba (3NF) và hỗ trợ soft delete cho các bảng quan trọng.

Các nhóm bảng chính (Phase 1 - khoảng 20 bảng):

Bảng 3.19: Các nhóm bảng chính trong PostgreSQL

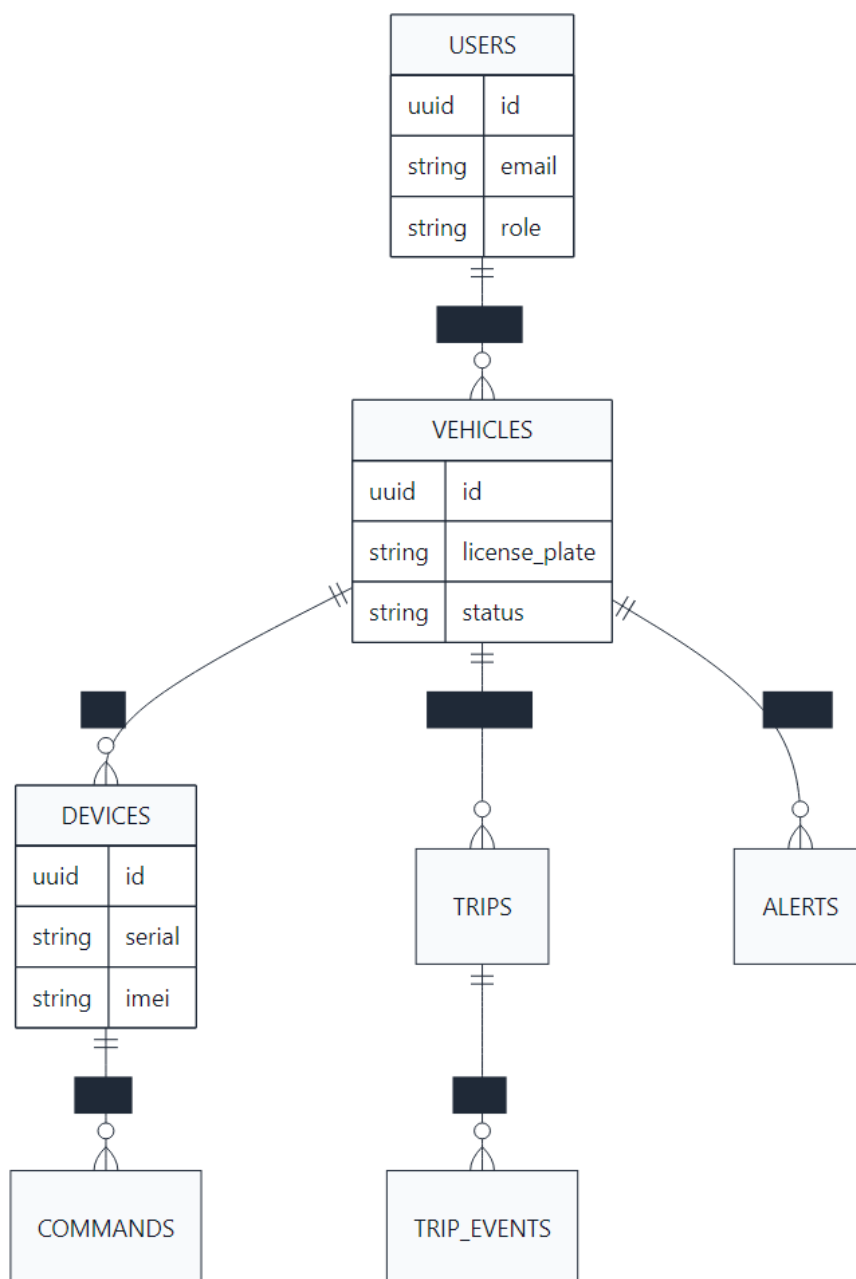
Nhóm bảng	Các bảng chính	Mô tả
Người dùng & Xác thực	users, user_sessions	Quản lý tài khoản, phiên đăng nhập
Phương tiện & Thiết bị	vehicles, devices, device_configurations	Thông tin xe và thiết bị tracker
Khách hàng	customers	Thông tin khách hàng thuê xe
Chuyến đi	trips, trip_events, stops	Hành trình, sự kiện và điểm dừng
Cảnh báo & Vi phạm	alerts, violations	Cảnh báo hệ thống và vi phạm giao thông
Vùng địa lý	geofences, vehicle_geofences	Vùng địa lý và gán kết với xe
Lệnh điều khiển	commands	Lệnh gửi đến thiết bị
Bảo trì	maintenance_records	Lịch sử bảo trì phương tiện
Nhật ký	connection_logs, device_status_history	Nhật ký kết nối và lịch sử trạng thái
Thông báo	notifications	Thông báo hệ thống
Nhiên liệu	fuel_records	Quản lý nhiên liệu

Sơ đồ quan hệ chính (ER Diagram - rút gọn):

- users 1:N vehicles, customers, notifications.
- vehicles 1:1 devices; 1:N trips, alerts, maintenance_records; N:N geofences (qua vehicle_geofences).
- customers 1:N trips.
- trips 1:N trip_events, stops, violations.
- devices 1:N commands, device_status_history, connection_logs.

Schema được thiết kế với khả năng mở rộng, các khóa ngoại và chỉ mục (index) đã được chuẩn bị sẵn để tích hợp thêm các bảng Phase 2 bao gồm: bookings (đặt xe), rental_contracts (hợp đồng thuê), payments (thanh toán), damage_reports (báo cáo hư hỏng) và reviews (đánh giá).

c) Cơ sở dữ liệu VictoriaMetrics - Schema chuỗi thời gian VictoriaMetrics là cơ sở dữ liệu chuỗi thời gian tương thích Prometheus, được lựa chọn thay thế InfluxDB nhờ hiệu năng vượt trội (nhanh hơn 10 lần), mức tiêu thụ RAM thấp hơn và tích hợp tốt với



Hình 3.14: Sơ đồ quan hệ cơ sở dữ liệu (ER Diagram)

Grafana.

Bảng 3.20: So sánh VictoriaMetrics và InfluxDB

Tiêu chí	InfluxDB	VictoriaMetrics (Đã chọn)
Ngôn ngữ truy vấn	Flux (phức tạp)	PromQL (đơn giản)
Hiệu năng	Tốt	Nhanh gấp 10 lần
Sử dụng RAM	Cao	Thấp
Nén dữ liệu	Tốt	Tốt hơn (gấp 10 lần)

Bảng 3.20: So sánh VictoriaMetrics và InfluxDB (tiếp theo)

Tiêu chí	InfluxDB	VictoriaMetrics (Đã chọn)
Tương thích Prometheus	Không	Có
Độ khó học	Cao (Flux)	Thấp (PromQL)
Tích hợp Grafana	Qua plugin	Hỗ trợ sẵn (Native)

Chính sách lưu trữ (Retention Policy): Dữ liệu thô được lưu trữ trong 30 ngày (-retentionPeriod=30d). Hệ thống hỗ trợ cấu hình downsampling thông qua vmagent với recording rules để tạo dữ liệu tổng hợp theo giờ/ngày, phục vụ báo cáo dài hạn mà không tốn dung lượng lưu trữ lớn.

d) VictoriaLogs - Cơ sở dữ liệu nhật ký VictoriaLogs được sử dụng làm hệ thống nhật ký tập trung (centralized logging), thay thế bộ công cụ ELK Stack truyền thống với ưu điểm nhẹ hơn và dễ triển khai. VictoriaLogs lưu trữ các loại nhật ký: sự kiện thiết bị, nhật ký lỗi, phiên làm việc, message MQTT, và nhật ký kiểm toán (audit trail). Chính sách lưu trữ: 7 ngày cho nhật ký thông thường.

3.3.2.4. API Server và kiến trúc phần mềm (API Server & Software Architecture)

a) Lựa chọn công nghệ API Server là tầng ứng dụng Backend cung cấp REST API, WebSocket và xử lý logic nghiệp vụ. Để lựa chọn framework phù hợp, đề án đã đánh giá ba phương án:

Bảng 3.21: So sánh framework Backend

Tiêu chí	Express + TypeScript	NestJS	Go + Gin
Độ khó học	Rất dễ	Trung bình	Khó
Linh hoạt	Rất cao	Trung bình	Cao
Hiệu năng	Cao	Cao	Rất cao
Lượng code mẫu (Boilerplate)	Ít	Nhiều	Trung bình
Hệ sinh thái IoT	Rất tốt	Tốt	Tốt
Hỗ trợ thời gian thực	Rất tốt	Rất tốt	Tốt

Kết luận lựa chọn: Node.js + Express + TypeScript

Lý do lựa chọn gồm: tính linh hoạt cao; đặc tính nhẹ và nhanh phù hợp với IoT; TypeScript giúp bảo đảm an toàn kiểu dữ liệu; Zod kết hợp xác thực runtime với type inference; và khả năng tổ chức code theo Domain-Driven Design.

b) Kiến trúc Domain-Driven Design (DDD) API Server được tổ chức theo kiến trúc Domain-Driven Design với ba tầng rõ ràng:

Tầng 1 - API Layer (Routes): Định nghĩa các route, ánh xạ HTTP method đến controller tương ứng.

Tầng 2 - Service Layer (Business Logic): Chứa toàn bộ logic nghiệp vụ, điều phối giữa các repository và dịch vụ bên ngoài.

Tầng 3 - Repository Layer (Data Access): Truy cập cơ sở dữ liệu, thực hiện các truy vấn SQL và trả về dữ liệu đã được xác thực kiểu.

Hệ thống bao gồm hơn 20 domain module, mỗi module có cấu trúc services/, repositories/, types/ riêng biệt, đảm bảo nguyên tắc Single Responsibility và dễ bảo trì.

c) MQTT Bridge - Dịch vụ độc lập MQTT Bridge đã được tách thành dịch vụ Node.js độc lập (Tracking_MqttBridge/) với package.json, docker-compose.yml, logger, connection pool và cấu hình riêng. Cách tách này giúp dịch vụ dễ mở rộng hơn, cô lập lỗi tốt hơn và giữ kiến trúc tổng thể gọn hơn.

3.3.2.5. Thiết kế API Endpoints (REST API Design)

a) Nguyên tắc thiết kế API được thiết kế tuân thủ các nguyên tắc RESTful: URL dạng số nhiều, versioning (/api/v1/), phân trang, định dạng lỗi thống nhất, và HTTP Status Code chuẩn.

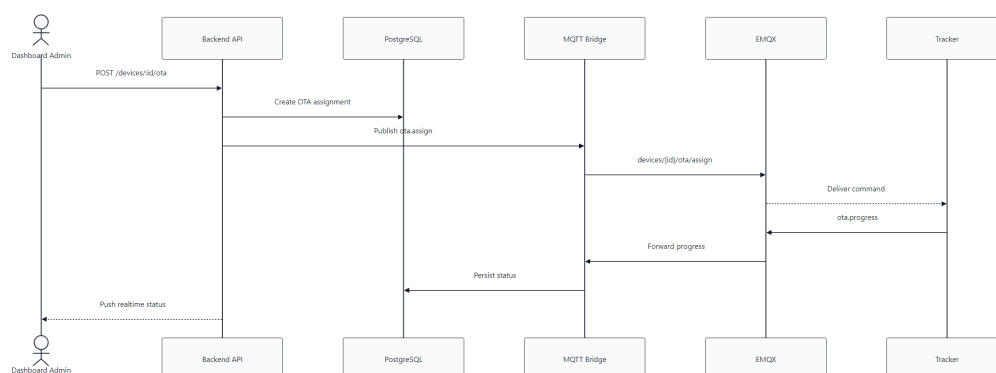
b) Bảng tóm tắt API Endpoints

Bảng 3.22: Tóm tắt các nhóm API Endpoints chính

Nhóm	Method	Endpoint	Mô tả	Auth
Auth	POST	/api/v1/auth/login	Đăng nhập	Không
	GET	/api/v1/auth/me	Thông tin người dùng hiện tại	Có
	POST	/api/v1/auth/logout	Đăng xuất	Có

Bảng 3.22: Tóm tắt các nhóm API Endpoints chính (tiếp theo)

Nhóm	Method	Endpoint	Mô tả	Auth
Devices	GET	/api/v1/devices	Danh sách thiết bị	Có
	POST	/api/v1/devices	Tạo thiết bị mới	Có
	POST	/api/v1/device s/:id/ota	Gửi lệnh OTA đến thiết bị	Có
	POST	/api/v1/device s/: id/ota/rollback	Yêu cầu rollback firmware	Có
Vehicles	GET	/api/v1/vehicles	Danh sách xe	Có
	POST	/api/v1/vehicles	Tạo xe mới	Có
IoT Data	POST	/api/v1/iot/data	Gửi dữ liệu cảm biến	Không
Telemetry	GET	/api/v1/device s/:id/telemetry	Dữ liệu telemetry theo thiết bị	Có
	GET	/api/v1/telemetry/history	Truy vấn lịch sử telemetry	Có
Dashboard	GET	/api/v1/dashboard/stats	Thống kê tổng quan	Có
Firmware	POST	/api/v1/firmware/upload	Tải lên firmware	Có
	GET	/api/v1/firmware/:id/download	Tải file firmware để OTA	Có



Hình 3.14a: Luồng điều phối OTA giữa Backend, EMQX, thiết bị và MQTT Bridge

Backend vẫn duy trì một số alias tương thích như /api/v1/device hoặc /api/v1/export, nhưng tài liệu ưu tiên dạng tài nguyên số nhiều (/devices,

/vehicles, /customers, /trips, /alerts, /geofences) để thống nhất cách đặt tên.

3.3.2.6. WebSocket và truyền dữ liệu thời gian thực (Real-time Communication)

Hệ thống sử dụng **Socket.IO 4.8** làm giải pháp truyền dữ liệu thời gian thực từ Backend đến Frontend. Ở phạm vi triển khai này, hệ thống định nghĩa năm namespace chính: /dashboard, /devices, /notifications, /exports và /firmware. Tất cả namespace đều đi qua middleware xác thực; riêng namespace /devices hỗ trợ join room theo từng thiết bị (device:{id}) để nhận sự kiện chi tiết.

Các sự kiện tiêu biểu gồm device:status, device:position, command:ack, stats:update, alert:new, geofence:enter, geofence:exit, export:ready và firmware:assignment. Trong triển khai hiện tại, namespace /firmware chủ yếu dùng để phát sự kiện gán hoặc cập nhật tác vụ OTA từ backend; còn trạng thái chi tiết của quá trình OTA được MQTT Bridge ghi vào firmware_update_log và VictoriaLogs để phục vụ truy vết, theo dõi và đồng bộ với giao diện quản trị.

3.3.2.7. Bảo mật hệ thống (Security Design)

a) Xác thực Session-based (Không dùng JWT) Hệ thống sử dụng cơ chế xác thực dựa trên phiên làm việc (session-based authentication) với *opaque session token* lưu trữ trong cơ sở dữ liệu, thay vì JSON Web Token (JWT) tự mang toàn bộ thông tin phiên. Token ở đây chỉ đóng vai trò khóa tham chiếu; backend mới là nơi lưu và kiểm tra trạng thái phiên, còn giá trị token được băm SHA-256 trước khi ghi xuống cơ sở dữ liệu.

Bảng 3.23: So sánh JWT và Session-based authentication

Tiêu chí	JWT	Session-based (Đã chọn)
Thu hồi token	Khó (cần blacklist)	Dễ (xóa khỏi DB)
Kiểm soát phiên	Hạn chế	Toàn quyền (xem, thu hồi mọi phiên)
Bảo mật	Token chứa thông tin nhạy cảm	Chỉ chứa ID ngẫu nhiên
Kích thước	Lớn (header + payload + signature)	Nhỏ (chỉ token ID)

b) Bảo mật tầng truyền tải Hệ thống bảo vệ tầng truyền tải bằng nhiều lớp bảo vệ: Helmet để thiết lập các header bảo mật, CORS để giới hạn origin hợp lệ, rate limiting để chặn lạm dụng theo IP, và cơ chế gắn Request-ID để truy vết luồng xử lý theo từng request.

c) Bảo mật MQTT Ở tầng MQTT, mỗi thiết bị được cấp ACL riêng và Client ID duy nhất, kết hợp xác thực bằng username/password. Cách làm này giúp cô lập phạm vi publish/subscribe của từng tracker và giảm rủi ro một thiết bị truy cập nhầm sang luồng dữ liệu của thiết bị khác.

d) Hệ thống thông báo cảnh báo Hệ thống hỗ trợ gửi thông báo qua Telegram Bot (thời gian thực) và Email SMTP (chỉ tiết). Các loại cảnh báo: xe di chuyển bất thường, vi phạm tốc độ, pin thấp, thiết bị mất kết nối, lịch bảo trì sắp đến.

3.4. Phân tích và đề xuất công nghệ Frontend – Frontend analysis and solution

3.4.1. Phân tích và lựa chọn công nghệ Frontend

3.4.1.1. Đặt vấn đề cho giải pháp Frontend Nếu cloud là nơi xử lý dữ liệu, thì frontend là nơi người vận hành thực sự “nhìn thấy” hệ thống đang hoạt động ra sao. Một dashboard tốt không chỉ hiển thị đủ thông tin, mà còn phải giúp người dùng nhận ra điều gì quan trọng, điều gì bất thường và cần hành động ở đâu. Vì vậy, bài toán thiết kế frontend cần đồng thời đáp ứng các yêu cầu sau:

- **Hiển thị thời gian thực ổn định:** bản đồ, trạng thái thiết bị và cảnh báo phải cập nhật liên tục với độ trễ thấp.
- **Khả năng mở rộng theo tính năng nghiệp vụ:** quản lý xe, chuyển đi, geofence, cảnh báo, báo cáo cần dễ mở rộng.
- **Hiệu năng và trải nghiệm người dùng:** tải trang nhanh, điều hướng mượt, responsive tốt trên desktop/mobile.
- **Dễ triển khai production:** build và đóng gói Docker thuận lợi cho mô hình vận hành đa dịch vụ.

3.4.1.2. So sánh các phương án công nghệ Frontend

Bảng 3.23A: So sánh các phương án công nghệ frontend tổng thể

Phương án	Mô tả stack	Ưu điểm	Hạn chế	Mức phù hợp
PA-FE1: Vite + React CSR	React + Vite + client-side routing	Build nhanh, cấu hình linh hoạt	SEO/first load kém hơn SSR, cần tự lắp ghép nhiều thành phần	Trung bình
PA-FE2: Nuxt (Vue)	Vue + Nuxt SSR/SSG	SSR tốt, cấu trúc rõ	Khác hệ sinh thái React đang dùng ở dự án, chi phí chuyển đổi cao	Trung bình

Bảng 3.23A: So sánh các phương án công nghệ frontend tổng thể (tiếp theo)

Phương án	Mô tả stack	Ưu điểm	Hạn chế	Mức phù hợp
PA-FE3: Next.js + React (Đã chọn)	Next.js 15 + React 19 + App Router	SSR/RSC tốt, hệ sinh thái lớn, tối ưu production, tích hợp realtime thuận lợi	Độ phức tạp framework cao hơn CSR thuần	Cao

3.4.1.3. Chọn giải pháp Frontend Đồ án chọn **PA-FE3: Next.js 15 + React 19** làm nền tảng frontend chính. Đây không phải là lựa chọn chạy theo xu hướng framework, mà là lựa chọn bám sát nhu cầu của một dashboard phải vừa “đọc nhanh” vừa “phản ứng nhanh” khi dữ liệu và cảnh báo thay đổi liên tục.

Các lý do lựa chọn gồm:

- **Phù hợp yêu cầu realtime + dashboard phức hợp:** kết hợp tốt giữa REST cache và WebSocket.
- **Hiệu năng tải trang tốt:** App Router và khả năng render phía server cải thiện trải nghiệm ban đầu.
- **Mở rộng tính năng thuận lợi:** dễ tổ chức theo module (Feature-Sliced Design).
- **Sẵn sàng triển khai production:** hỗ trợ build/standalone output và đóng gói Docker rõ ràng.

3.4.2. Giải pháp Frontend

Trên cơ sở phương án đã chọn ở mục 3.4.1, phần này trình bày cách nhóm biến stack công nghệ thành một giao diện để người vận hành nắm tình huống nhanh, thao tác gọn và vẫn theo kịp dữ liệu thời gian thực: từ lựa chọn công nghệ, tổ chức mã nguồn đến cách quản lý trạng thái và hiển thị dữ liệu.

3.4.2.1. Lựa chọn công nghệ Frontend (Technology Selection)

a) Framework ứng dụng web Để lựa chọn framework phù hợp, nhóm không dừng ở so sánh tên công nghệ, mà đặt câu hỏi thực tế hơn: framework nào giúp dashboard tải nhanh, tổ chức rõ và vẫn dễ mở rộng khi số lượng trang quản trị tăng lên. Từ đó, nhóm đối chiếu Next.js, Nuxt.js và Vite + React theo các tiêu chí hiệu năng, khả năng mở rộng và trải nghiệm phát triển.

Bảng 3.24: So sánh framework Frontend

Tiêu chí	Next.js 15	Nuxt.js 3	Vite + React
Ngôn ngữ cơ sở	React 19 + TypeScript	Vue 3 + TypeScript	React + TypeScript
Rendering	SSR, SSG, ISR, RSC	SSR, SSG, ISR	CSR (chỉ client-side)
App Router	Có (App Router với React Server Components)	Có (File-based routing)	Không có sẵn, cần thư viện bổ sung
SEO & First Load	Tốt (Server-side rendering)	Tốt	Kém (chỉ client-side rendering)
Hệ sinh thái	Rất lớn (React ecosystem)	Lớn (Vue ecosystem)	Lớn (React ecosystem)
Real-time support	Tốt (Socket.IO, Server Actions)	Tốt	Tốt
Hiệu năng	Cao (Turbopack, Partial Prerendering)	Cao (Nitro engine)	Cao (Vite bundler)
Docker deployment	Có sẵn Dockerfile, standalone output	Có sẵn	Cần cấu hình thủ công
Cộng đồng & tài liệu	Rất lớn	Lớn	Lớn

Kết luận lựa chọn: Next.js 15 được chọn làm framework chính vì nó tạo được thế cân bằng tốt giữa trải nghiệm của người dùng cuối và khả năng tổ chức mã nguồn của đội phát triển. Các lý do cụ thể gồm:

- **React Server Components (RSC):** Cho phép render một phần trên server, giảm kích thước JavaScript gửi về client và tăng tốc độ tải trang đầu tiên (First Contentful Paint).
- **App Router:** Cơ chế định tuyến dựa trên thư mục (file-based routing) đơn giản hóa việc tổ chức cấu trúc trang và layout lồng nhau (nested layouts).
- **Hệ sinh thái React:** Tận dụng được hệ sinh thái cực kỳ phong phú của React bao gồm TanStack Query, Zustand, Leaflet, ECharts và hàng ngàn thư viện bổ sung.
- **TypeScript tích hợp:** Hỗ trợ TypeScript từ gốc, đảm bảo an toàn kiểu dữ liệu xuyên suốt ứng dụng.
- **Sẵn sàng cho triển khai:** Có sẵn các cơ chế tối ưu cho môi trường sản xuất như Image Optimization, Code Splitting và Standalone Output cho Docker.

b) Thư viện giao diện (UI Component Library) Hệ thống sử dụng shadcn/ui kết hợp với Radix UI làm nền tảng xây dựng giao diện. Điểm đáng giá của lựa chọn này không nằm ở số lượng component dựng sẵn, mà ở khả năng giữ giao diện nhất quán trong khi nhóm vẫn kiểm soát được cách từng thành phần được triển khai trong dự án.

Đặc điểm nổi bật:

- **Radix UI Primitives:** Cung cấp các component cơ bản đã xử lý accessibility (ARIA attributes), keyboard navigation và focus management.
- **Tailwind CSS v4:** Hệ thống utility-first CSS với CSS Variables cho phép tùy chỉnh giao diện nhanh chóng mà không cần viết CSS truyền thống.
- **Class Variance Authority (CVA):** Quản lý các biến thể (variants) của component một cách có hệ thống.
- **Oklch Color Space:** Sử dụng hệ màu oklch hiện đại cho độ chính xác màu sắc cao hơn và hỗ trợ chuyển đổi Light/Dark theme mượt mà.

c) Quản lý trạng thái (State Management) Với một dashboard vừa có dữ liệu tĩnh, dữ liệu thời gian thực và trạng thái giao diện, dồn mọi thứ vào cùng một nơi quản lý sẽ làm mã nguồn nhanh chóng rối lên. Vì vậy, hệ thống áp dụng chiến lược tách biệt giữa trạng thái client và trạng thái server:

Bảng 3.25: Chiến lược quản lý trạng thái

Loại trạng thái	Thư viện	Mục đích	Ví dụ
Client state	Zustand 5	Trạng thái UI và authentication	Token phiên đăng nhập, trạng thái sidebar, theme
Server state	TanStack Query 5	Dữ liệu từ API server	Danh sách xe, chuyến đi, cảnh báo
URL state	nuqs	Trạng thái đồng bộ với URL	Bộ lọc, phân trang, tab đang chọn
Real-time state	Socket.IO Client	Dữ liệu thời gian thực	Vị trí xe, dữ liệu telemetry

Kết luận: Zustand được chọn vì API đơn giản, kích thước cực nhỏ (1.1 KB) và phù hợp với yêu cầu của hệ thống nơi token phiên đăng nhập chỉ lưu trong bộ nhớ (memory-only, không persist ra localStorage) nhằm tăng tính bảo mật.

d) Thư viện bản đồ

Bảng 3.26: So sánh thư viện bản đồ

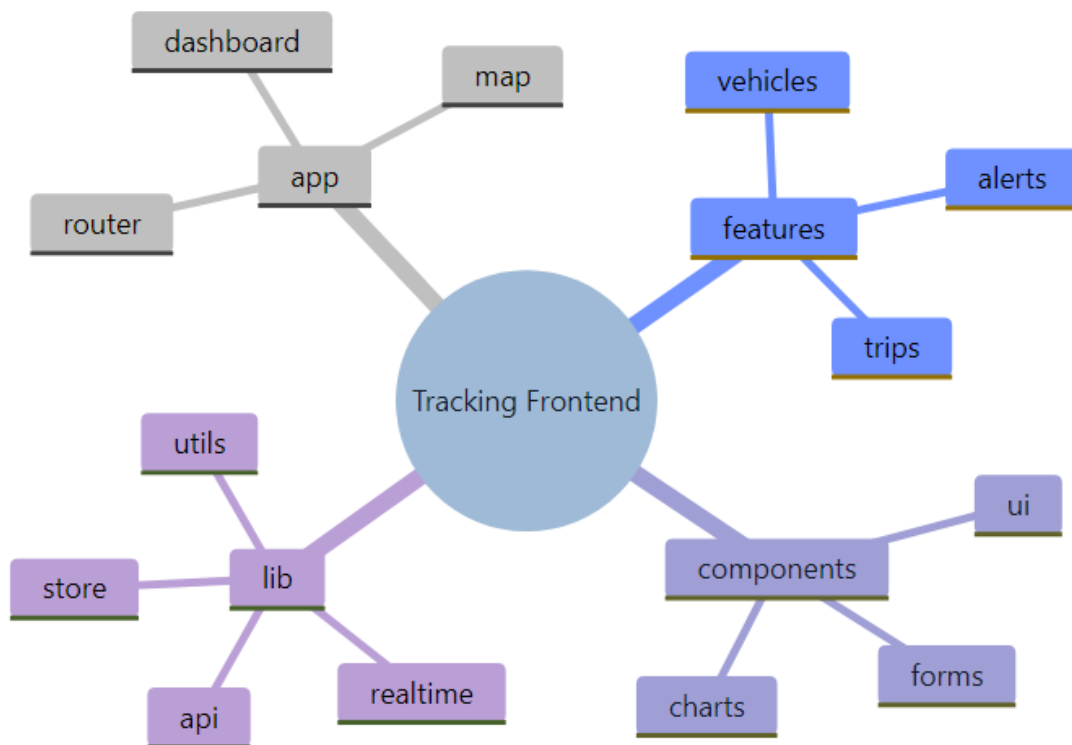
Tiêu chí	Leaflet.js	Mapbox GL JS	Google Maps JS API
Giấy phép	MIT (mã nguồn mở)	Proprietary (có free tier)	Proprietary (trả phí)
Chi phí	Miễn phí	Miễn phí đến 50K loads/tháng	0.007 USD/load sau 28K
Hiệu năng với nhiều marker	Tốt (với MarkerCluster)	Rất tốt (WebGL rendering)	Tốt
Tùy chỉnh marker	Cao	Cao	Trung bình
Offline/Self-hosted tiles	Hỗ trợ	Không	Không
React integration	react-leaflet (trưởng thành)	react-map-gl	@react-google-maps/api
Vẽ geofence	@geoman-io/leaflet-geoman-free	mapbox-gl-draw	Drawing Manager

Kết luận: Leaflet.js được chọn vì hoàn toàn miễn phí (MIT license), hỗ trợ tự host tile server cho môi trường không có Internet công cộng, và có hệ sinh thái plugin phong phú (MarkerCluster, Geoman cho vẽ geofence). Điều này đặc biệt phù hợp với hệ thống IoT có thể triển khai trong môi trường private network.

3.4.2.2. Kiến trúc ứng dụng web (Web Application Architecture)

a) Mô hình kiến trúc Feature-Sliced Design Ứng dụng Frontend được tổ chức theo mô hình Feature-Sliced Design (FSD), trong đó mỗi tính năng nghiệp vụ được đóng gói thành một module độc lập bao gồm components, hooks, types và utils. Cách tổ chức này giúp người đọc và người phát triển nhìn thấy ngay mối liên hệ giữa cấu trúc mã nguồn và các nghiệp vụ thực tế của hệ thống.

- `app/`: App Router của Next.js, bao gồm layout gốc và các route dashboard được bảo vệ.
- `features/`: các module nghiệp vụ theo FSD (vehicles, trips, alerts, map), mỗi module tách rõ components/hooks/types.
- `components/`: thư viện component dùng chung (UI, layout, map, charts, forms).
- `lib/`: hạ tầng ứng dụng (API client, realtime, state store, util).
- `types/`: các kiểu TypeScript dùng toàn cục.



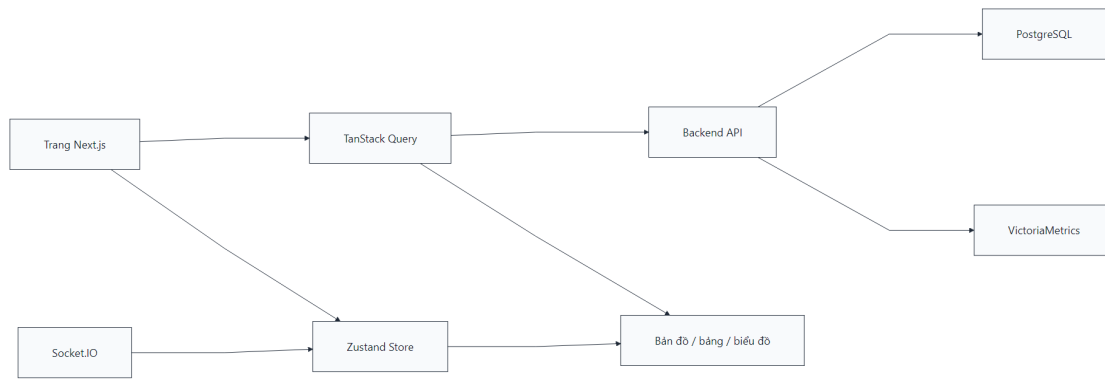
Hình 3.15: Sơ đồ kiến trúc Feature-Sliced Design của ứng dụng Frontend

Nguyên tắc tổ chức:

- **Co-location:** Các file liên quan đến một tính năng được đặt cùng thư mục, giảm thời gian tìm kiếm và bảo trì.
- **Encapsulation:** Mỗi feature module export một public API rõ ràng, các module khác chỉ truy cập thông qua hooks và components đã export.
- **Separation of Concerns:** Tách biệt rõ ràng giữa routing (app/), logic nghiệp vụ (features/), giao diện dùng lại (components/) và hạ tầng (lib/).

b) Luồng dữ liệu trong ứng dụng Luồng dữ liệu trong ứng dụng Frontend tuân theo mô hình một chiều (unidirectional data flow), đồng thời kết hợp ba nguồn dữ liệu chính:

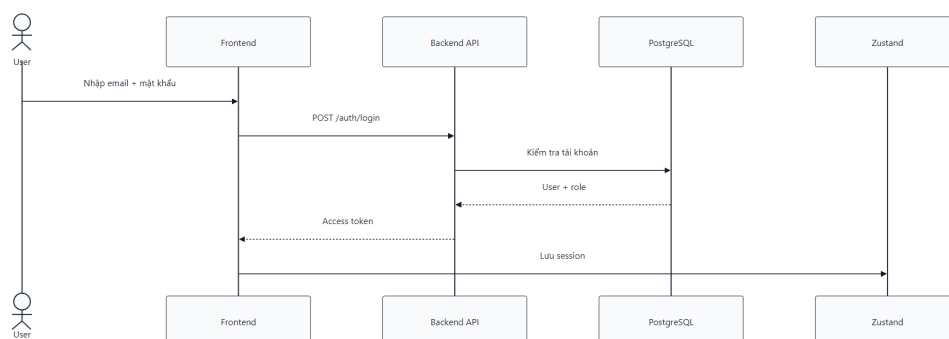
1. **REST API (TanStack Query):** Dữ liệu CRUD (danh sách xe, chuyến đi, cảnh báo) được lấy từ Backend qua HTTP và cache bởi TanStack Query với cơ chế stale-while-revalidate.
2. **Client State (Zustand):** Trạng thái giao diện như phiên đăng nhập, trạng thái sidebar, theme được quản lý tại client mà không cần gọi API.
3. **Real-time (Socket.IO):** Dữ liệu vị trí xe và telemetry được nhận theo thời gian thực qua WebSocket. Khi kết nối WebSocket đang hoạt động, TanStack Query sẽ tự động tắt polling để tránh trùng lặp dữ liệu.



Hình 3.15a: Luồng dữ liệu một chiều giữa Next.js, Query, Zustand và Backend

c) Cơ chế xác thực và bảo vệ route Hệ thống Frontend sử dụng *opaque session token* do Backend cấp, không dùng JWT tự mang thông tin phiên. Việc client gửi token trong header `Authorization: Bearer` chỉ là hình thức truyền token; về bản chất, đây vẫn là cơ chế session-based vì mọi quyền truy cập đều được quyết định sau khi server tra cứu phiên tương ứng:

1. **Đăng nhập:** Người dùng nhập email và mật khẩu, Backend tạo bản ghi phiên và trả về session token ngẫu nhiên.
2. **Lưu trữ token:** Token chỉ được lưu trong bộ nhớ Zustand (memory-only), không lưu vào localStorage hay sessionStorage, nhằm giảm thiểu rủi ro XSS.
3. **Gửi request:** Mọi request HTTP tự động đính kèm token vào header `Authorization: Bearer <token>`.
4. **AuthGuard:** Component AuthGuard bao bọc toàn bộ khu vực `/dashboard/*`, tự động chuyển hướng về trang đăng nhập nếu chưa xác thực hoặc token hết hạn.
5. **Gia hạn phiên:** Khi phiên sắp hết hạn, Frontend chủ động gọi cơ chế gia hạn để nhận token mới, nhờ đó người dùng không bị đăng nhập lại đột ngột trong lúc thao tác.

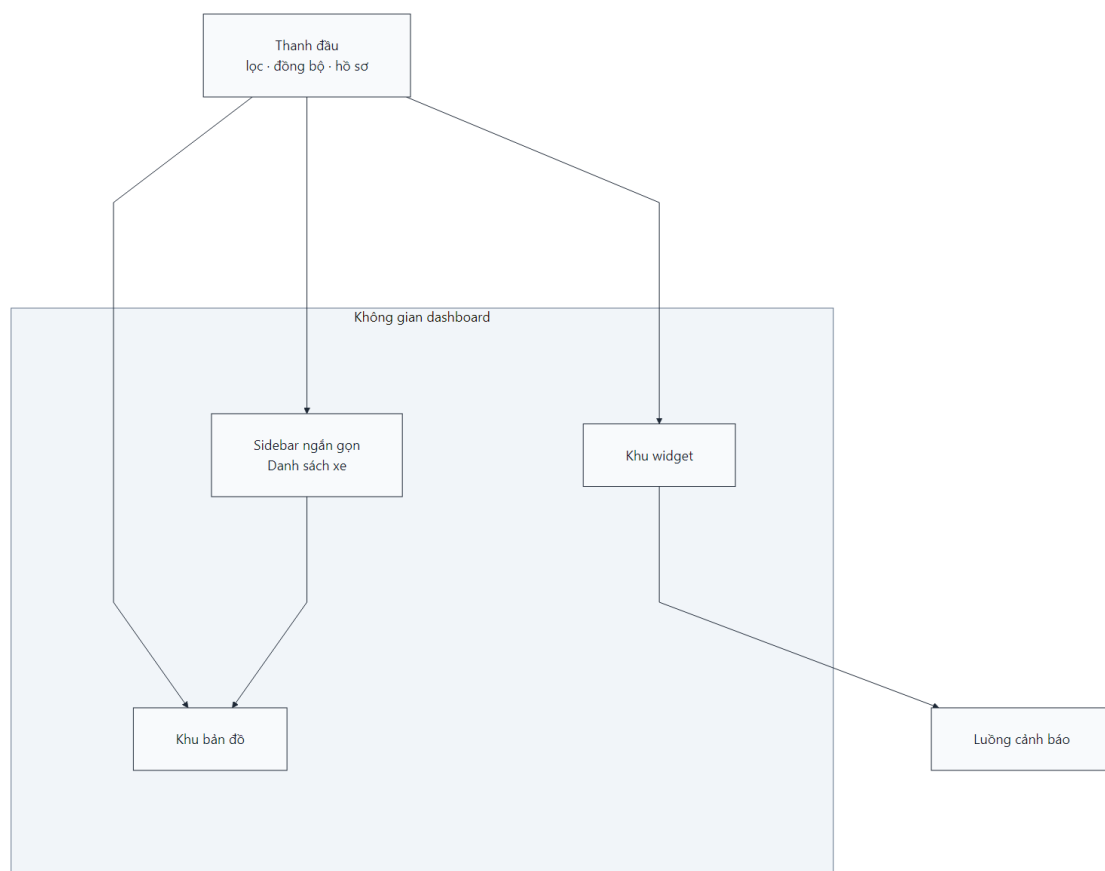


Hình 3.16: Biểu đồ trình tự (Sequence Diagram) luồng xác thực người dùng

3.4.2.3. Thiết kế các trang chức năng chính (Main Feature Pages)

a) Trang tổng quan (Dashboard) Trang Dashboard là điểm vào chính sau khi đăng nhập, cung cấp cái nhìn tổng thể về trạng thái hệ thống:

- **Stat Cards:** Hiển thị các chỉ số quan trọng: tổng số xe đang hoạt động, số chuyến đi trong ngày, số cảnh báo chưa xử lý, số vi phạm mới.
- **Bản đồ mini:** Hiển thị vị trí tất cả xe đang hoạt động trên bản đồ thu nhỏ.
- **Bảng cảnh báo gần đây:** Danh sách 10 cảnh báo mới nhất với badge mức độ nghiêm trọng (low, medium, high, critical).
- **Biểu đồ thống kê:** Biểu đồ số chuyến đi theo ngày và phân loại cảnh báo theo loại.



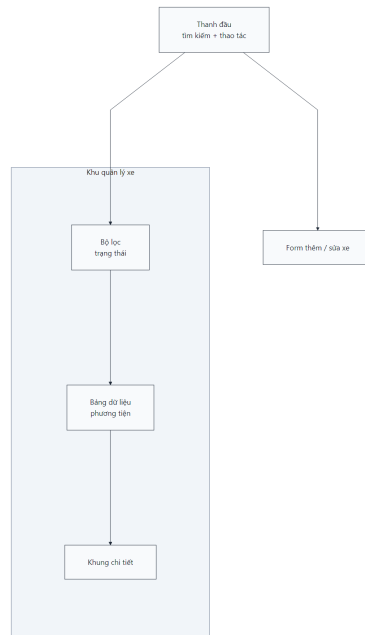
Hình 3.17: Giao diện trang Dashboard tổng quan

b) Quản lý phương tiện (Vehicle Management) Trang danh sách (/dashboard/vehicles):

Bảng 3.2.4A: Tổng hợp thông tin về Cột, Mô tả, Chức năng

Cột	Mô tả	Chức năng
Biển số xe	Số đăng ký phương tiện	Tìm kiếm
Hãng/Model	Thương hiệu và đời xe	Lọc
Trạng thái	Active, Inactive, Maintenance, Retired	Lọc theo trạng thái
Thiết bị	Mã thiết bị IoT gắn kèm	Liên kết
Lần cuối hoạt động	Thời điểm nhận dữ liệu gần nhất	Sắp xếp
Thao tác	Xem, Sửa, Xóa, Xem trên bản đồ	Nút hành động

Trang chi tiết (/dashboard/vehicles/[id]): Hiển thị thông tin đầy đủ của phương tiện bao gồm thông tin cơ bản, vị trí hiện tại trên bản đồ, trạng thái thiết bị IoT, các cảnh báo đang hoạt động, lịch sử chuyến đi và lịch sử bảo trì.



Hình 3.18: Giao diện trang quản lý phương tiện

c) Quản lý chuyến đi, cảnh báo và vùng địa lý Trang quản lý chuyến đi cung cấp lịch sử và chi tiết từng chuyến, gồm tuyến đường trên bản đồ, biểu đồ tốc độ theo thời gian và timeline sự kiện. Trang cảnh báo tổng hợp cảnh báo theo badge mức độ nghiêm trọng và hỗ trợ xử lý hàng loạt. Trang geofence cho phép vẽ hình tròn, đa giác hoặc hình chữ nhật trực tiếp trên bản đồ bằng Leaflet Geoman.

Bảng 3.27: Ma trận các trang chức năng chính

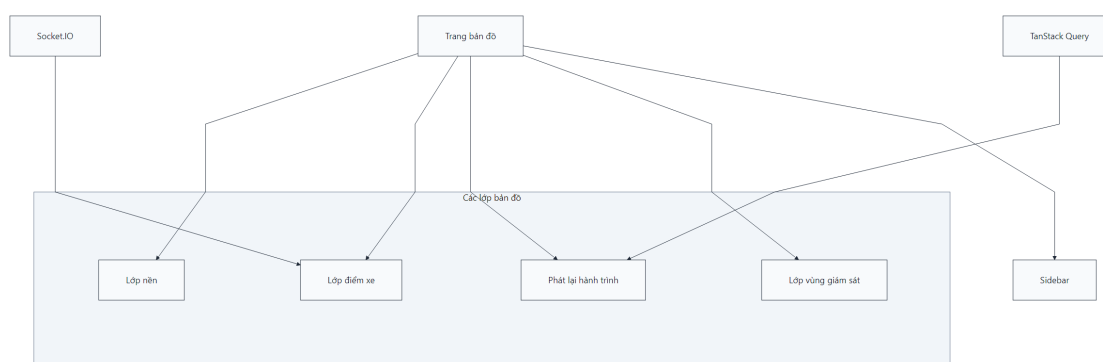
Trang	Đường dẫn	Chức năng chính	Real-time
Dashboard	/dashboard	Tổng quan hệ thống, stat cards, biểu đồ	Có (cập nhật số liệu)
Bản đồ	/dashboard/map	Theo dõi tất cả xe thời gian thực	Có (vị trí xe)
Phương tiện	/dashboard/vehicles	CRUD xe, trạng thái, lịch sử	Có (trạng thái xe)
Chuyến đi	/dashboard/trips	Lịch sử chuyến đi, tuyến đường	Không
Cảnh báo	/dashboard/alerts	Xem, xử lý cảnh báo	Có (cảnh báo mới)
Vi phạm	/dashboard/violations	Quản lý vi phạm tốc độ	Không

Bảng 3.27: Ma trận các trang chức năng chính (tiếp theo)

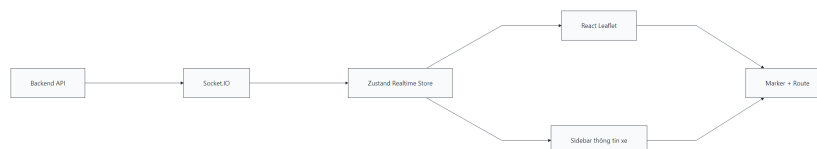
Trang	Đường dẫn	Chức năng chính	Real-time
Thiết bị	/dashboard/devices	Quản lý thiết bị IoT	Có (trạng thái thiết bị)
Vùng địa lý	/dashboard/geofences	Tạo, sửa geofence trên bản đồ	Không
Bảo trì	/dashboard/maintenance	Lịch sử bảo trì phương tiện	Không
Thông báo	/dashboard/notifications	Cấu hình Telegram, Email	Không
Cài đặt	/dashboard/settings	Hồ sơ, quản lý người dùng	Không

3.4.2.4. Tích hợp bản đồ thời gian thực (Real-time Map Integration)

a) Kiến trúc tích hợp bản đồ Trang bản đồ thời gian thực (/dashboard/map) là tính năng cốt lõi của hệ thống giám sát phương tiện. Kiến trúc tích hợp bản đồ bao gồm ba lớp chính:



Hình 3.19a: Phân rã lớp bản đồ React Leaflet và kênh Socket.IO



Hình 3.19: Kiến trúc tích hợp bản đồ thời gian thực

b) Cơ chế cập nhật vị trí thời gian thực Quy trình cập nhật vị trí xe trên bản đồ diễn ra như sau:

1. **Thiết bị IoT** gửi dữ liệu GPS qua MQTT đến EMQX Broker.
2. **MQTT Bridge** nhận và xử lý dữ liệu, lưu vào VictoriaMetrics, đồng thời chuyển tiếp đến Backend.
3. **Backend** phát sự kiện `vehicle:{id}:location` qua Socket.IO đến tất cả client đang subscribe.
4. **Frontend** nhận sự kiện, cập nhật tọa độ marker trên bản đồ. Sử dụng kỹ thuật nội suy (interpolation) để marker di chuyển mượt mà giữa hai điểm tọa độ liên tiếp.

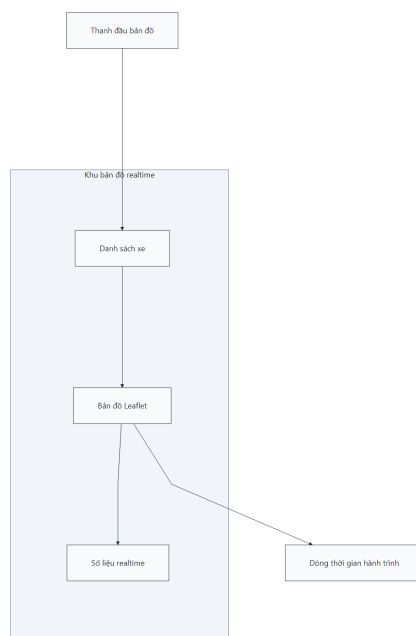
Tối ưu hiệu năng bản đồ:

- **Marker Clustering:** Sử dụng `react-leaflet-cluster` để gom nhóm các marker gần nhau khi zoom out, giảm số lượng DOM elements cần render.
- **Viewport-based rendering:** Chỉ render marker của các xe nằm trong vùng nhìn hiện tại của bản đồ.
- **Debounced updates:** Gom nhóm các cập nhật vị trí trong khoảng 100ms để tránh re-render quá nhiều lần.

c) **Các thành phần bản đồ** Hệ thống bao gồm bốn component bản đồ chuyên biệt:

Bảng 3.2.4B: Tổng hợp thông tin về Component, File, Chức năng

Component	File	Chức năng
VehicleMap	components /map/vehicle- map.tsx	Hiển thị tất cả xe trên bản đồ với marker và popup thông tin
RouteMap	components /map/route- map.tsx	Hiển thị tuyến đường chuyển đi với polyline và các điểm dừng
GeofenceMap	components /map/geofenc e-map.tsx	Vẽ và chỉnh sửa vùng địa lý (polygon, circle, rectangle)
MarkerPopup	components /map/marker- popup.tsx	Popup hiển thị thông tin xe khi click marker (biển số, tốc độ, trạng thái)



Hình 3.20: Giao diện trang bản đồ thời gian thực với các marker xe

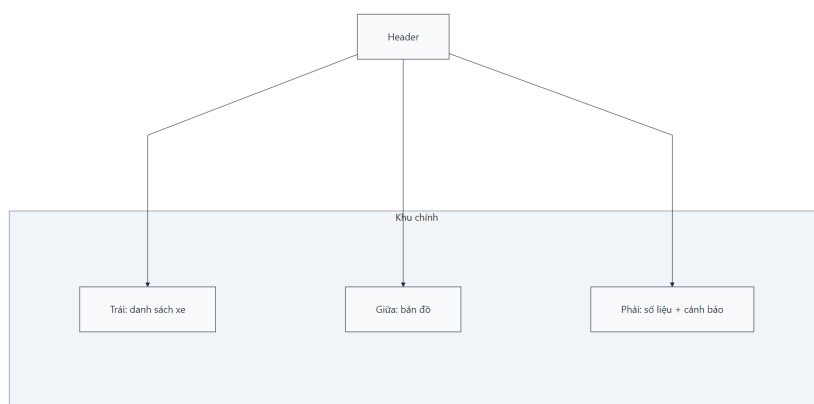
3.4.2.5. Thiết kế responsive và trải nghiệm người dùng (Responsive Design & UX)

a) Chiến lược responsive Trong phạm vi hiện tại, giao diện được ưu tiên cho desktop và tablet, nhưng vẫn giữ ba mức breakpoint chính để không phá vỡ trải nghiệm trên màn hình nhỏ:

Bảng 3.28: Breakpoint và tương ứng giao diện

Breakpoint	Kích thước	Giao diện	Điều chỉnh chính
Mobile	< 640px	Giao diện tối giản, một cột	Sidebar thành drawer, bảng thành danh sách card, bản đồ toàn màn hình
Tablet	640px - 1024px	Giao diện trung gian	Sidebar thu gọn (icon-only), bảng giữ nguyên với cuộn ngang
Desktop	> 1024px	Giao diện đầy đủ	Sidebar mở rộng, bảng đầy đủ cột, bản đồ chia đôi màn hình

b) Layout chính Giao diện Dashboard sử dụng bố cục (layout) gồm ba vùng chính: Header (breadcrumbs, search, user menu, theme toggle), Sidebar (navigation menu), và Main Content Area (page content, filters, data tables).



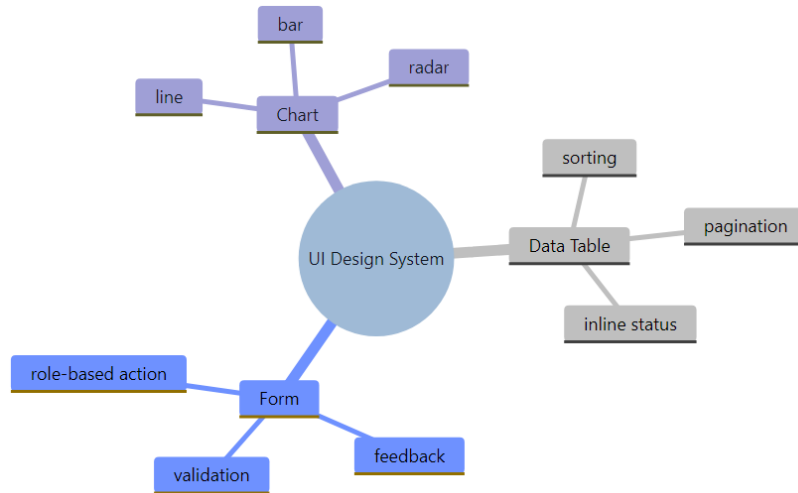
Hình 3.21: Wireframe bố cục giao diện Dashboard

c) Hệ thống theme Light/Dark Hệ thống hỗ trợ hai chế độ giao diện (Light và Dark) sử dụng CSS Variables với hệ màu oklch. Chuyển đổi được thực hiện qua thư viện next-themes và lưu trong cookie.

d) Các mẫu thiết kế giao diện (UI Patterns) Hệ thống áp dụng các mẫu thiết kế giao diện nhất quán xuyên suốt ứng dụng:

- **Bảng dữ liệu (Data Tables):** Sắp xếp, lọc, phân trang phía server, chọn nhiều dòng, xuất dữ liệu CSV/Excel.

- **Biểu mẫu (Forms):** React Hook Form + Zod validation, inline error messages, toast notifications (Sonner).
- **Bản đồ (Maps):** Custom markers với màu theo trạng thái, popup thông tin, drawing tools cho geofence.
- **Biểu đồ (Charts):** ECharts/Recharts cho telemetry, line chart tốc độ, gauge chart RPM, area chart nhiên liệu/nhiệt độ.



Hình 3.22: Các mẫu thiết kế UI của hệ thống - Data Table, Form, Charts

e) **Tối ưu trải nghiệm người dùng** Để nâng cao trải nghiệm người dùng, hệ thống áp dụng một số kỹ thuật:

- **Optimistic Updates:** Giao diện cập nhật ngay lập tức trước khi nhận phản hồi từ server.
- **Skeleton Loading:** Hiển thị khung xương thay vì spinner khi đang tải dữ liệu.
- **Page Transition Loader:** Thanh tiến trình ở đầu trang (sử dụng nextjs-toploader) khi chuyển trang.
- **Keyboard Shortcuts:** Hỗ trợ phím tắt cho các thao tác phổ biến (Cmd+K cho tìm kiếm).
- **Error Boundaries:** Mỗi khu vực chức năng được bao bọc bởi Error Boundary riêng.
- **Empty States:** Hiển thị trạng thái trống có hướng dẫn hành động khi chưa có dữ liệu.

3.5. Phân tích, đánh giá và lựa chọn phương án khả thi – Analysis, Evaluation and Selection

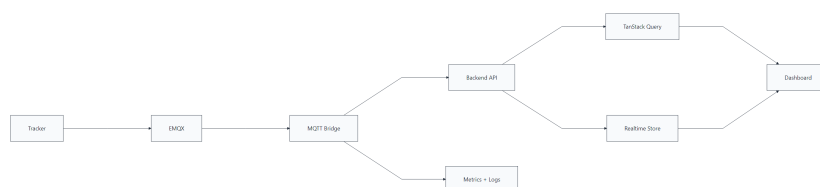
Dựa trên phân tích và đề xuất ở các mục 3.1 đến 3.4, bảng tổng hợp dưới đây đánh giá và so sánh các phương án đã xem xét:

Bảng 3.29: Ma trận đánh giá tổng hợp các phương án thiết kế

Tầng hệ thống	Phương án được chọn	Lý do chính
Vi điều khiển	ESP32-S3	BLE 5.0 tích hợp, dual-core, hỗ trợ AI
LTE + GNSS	SIMCom SIM7600CE-T	Modem tích hợp LTE + GNSS, giảm số lượng khối phần cứng rời
OBD2 Adapter	vgate iCar Pro (BLE)	Không cần dây, tương thích rộng
Cảm biến IMU	LIS3DSH	Siêu tiết kiệm điện, wake-on-motion
Buffer cục bộ	microSD SDMMC 4-bit + FATFS + replay	Duy trì liên tục telemetry khi mất mạng, phát lại theo thứ tự khi reconnect
MQTT Broker	EMQX	Rules Engine, ACL per device, clustering
Database quan hệ	PostgreSQL 16	Mature, PostGIS, open-source
Database time-series	VictoriaMetrics	Write throughput cao, PromQL compatible
API Server	Express.js + TypeScript	Nhẹ, linh hoạt,DDD architecture
Frontend	Next.js 15 + React 19	SSR/SSG, App Router, ecosystem
Bản đồ	Leaflet.js	Open-source, self-hosted tiles

3.6. Tối ưu phương án thiết kế – The optimal solution

Từ kết quả phân tích và đánh giá ở mục 3.5, phương án thiết kế tối ưu cho hệ thống IoT giám sát phương tiện được tổng hợp như sau:



Hình 3.23: Sơ đồ kiến trúc tổng thể phương án tối ưu

Kiến trúc phân tầng:

- **Tầng thiết bị (Device Layer):** ESP32-S3 + SIMCom SIM7600CE-T + vgate iCar Pro + LIS3DSH, quản lý nguồn thông minh với pin dự phòng 18650 1S

- **Tầng truyền thông (Communication Layer):** MQTT 5.0 qua 4G LTE, QoS phân tầng, kết hợp buffer cục bộ microSD và cơ chế replay khi kết nối phục hồi
- **Tầng xử lý (Processing Layer):** MQTT Bridge --> dual-write PostgreSQL + VictoriaMetrics, Express.js API (DDD)
- **Tầng trình bày (Presentation Layer):** Next.js 15, Leaflet maps, Socket.IO real-time, ECharts

Phương án này đáp ứng các yêu cầu kỹ thuật đã đặt ra ở Chương 2, đồng thời giữ chi phí phần cứng ở mức khoảng 1.514.000 VND cho mỗi thiết bị theo mặt bằng giá tham chiếu tháng 4/2026.

Kết luận chương 3

Chương 3 đã trả lời câu hỏi ”thiết kế hệ thống này như thế nào để vừa đáp ứng yêu cầu kỹ thuật, vừa khả thi khi triển khai”. Từ phần cứng, firmware đến backend/cloud và frontend, các phương án không chỉ được nêu ra mà còn được cân nhắc theo tiêu chí hiệu năng, chi phí, độ phức tạp và khả năng mở rộng. Nhờ đó, phương án cuối cùng ở Chương 3 không xuất hiện như một lựa chọn cảm tính, mà như kết quả của quá trình sàng lọc có cơ sở. Trên nền tảng đó, Chương 4 sẽ chuyển sang phần hiện thực: biến các lựa chọn thiết kế thành hệ thống vận hành được trong thực tế.

CHƯƠNG 4. TRIỂN KHAI GIẢI PHÁP VÀ KẾT QUẢ

– IMPLEMENTATION AND RESULTS

Nếu Chương 3 dừng ở mức lựa chọn phương án, thì Chương 4 trả lời câu hỏi liệu những lựa chọn đó có thực sự hoạt động khi đưa vào chế tạo, lập trình và tích hợp hay không. Nội dung chương đi từ phần cứng thiết bị, firmware, hạ tầng cloud, giao diện giám sát đến các kết quả đo lường sau khi lắp đặt hệ thống trên xe, qua đó giúp người đọc đối chiếu giữa ý tưởng thiết kế và kết quả triển khai thực tế.

4.1. Thiết kế chi tiết giải pháp – Detailed design solution

Phần này giữ nguyên trọng tâm của bản gốc nhưng lược bớt các sơ đồ phụ, bảng tra chi tiết và mô tả lắp. Mục tiêu là làm rõ hệ thống được hiện thực như thế nào ở ba lớp chính: phần cứng thiết bị, firmware nhúng và hạ tầng cloud/dashboard.

4.1.1. Thiết kế chi tiết phần cứng

Thiết bị tracker được xây dựng quanh ESP32-S3, modem SIM7600CE-T, adapter BLE OBD2 và cảm biến gia tốc LIS3DSH. Cách tổ chức này giúp một bo mạch duy nhất vừa đọc được dữ liệu vận hành từ xe, vừa lấy được vị trí GNSS, vừa gửi được telemetry qua mạng 4G mà không cần thêm bộ xử lý phụ. Về bản chất, thiết kế phần cứng ưu tiên ba yêu cầu: tích hợp gọn, dễ lắp trong xe và đủ ổn định để hỗ trợ nhiều chế độ nguồn khác nhau.

Ở mức liên kết chức năng, ESP32-S3 dùng UART để điều khiển modem, BLE để giao tiếp với adapter OBD2, I2C để làm việc với IMU và ADC để giám sát điện áp ắc quy. Kiến trúc nguồn gồm buck cho bus chính, LDO cho 3.3V, nhánh dự phòng dùng pin 18650 và mạch power path để chuyển đổi nguồn an toàn khi xe mất điện hoặc điện áp xuống thấp. Nhờ đó, thiết bị không chỉ hoạt động khi xe đang chạy mà còn có thể duy trì cảnh báo ngắn hạn ở trạng thái đỗ.

Khối	Phần tử chính	Vai trò trong hệ thống
Xử lý trung tâm	ESP32-S3	Điều phối trạng thái, đọc cảm biến, đóng gói dữ liệu và điều khiển toàn bộ thiết bị
Truyền thông và định vị	SIM7600CE-T	Kết nối 4G/MQTT, lấy vị trí GNSS, nhận lệnh điều khiển từ xa
Dữ liệu xe	vgate iCar Pro BLE OBD2	Đọc trạng thái khóa điện, tốc độ, RPM, nhiên liệu và các PID chẩn đoán cần thiết
Nguồn và dự phòng	Buck/LDO/Boost + pin 18650	Cấp nguồn ổn định, hỗ trợ chuyển nguồn và duy trì vận hành ngắn hạn khi mất nguồn xe

Bản gốc có trình bày đầy đủ sơ đồ nguyên lý, mapping GPIO, cấu trúc enclosure và checklist lắp đặt. Trong bản rút gọn này, các nội dung đó được gom thành nhận định kỹ thuật: thiết kế phần cứng đã đạt yêu cầu tích hợp trên xe thật, phù hợp với nguồn xe 12V/24V và tạo được nền tảng rõ ràng cho phần firmware quản lý năng lượng, kết nối và dữ liệu.

4.1.2. Triển khai firmware, cloud và giao diện điều khiển

Firmware được phát triển trên ESP-IDF 5.4 với FreeRTOS và NimBLE. Thay vì tách quá nhiều task ứng dụng, hệ thống dùng mô hình điều phối trung tâm kết hợp máy trạng thái *Driving*, *Parking*, *Alert*. Cách tổ chức này giúp kiểm soát tốt hơn luồng đọc dữ liệu, phát hiện chuyển động, chu kỳ gửi telemetry, ngủ sâu và đánh thức theo sự kiện. Những phần bất đồng bộ thật sự cần thiết, như BLE stack, đồng bộ AT command và phản hồi OBD2, mới được giao cho các cơ chế hàng đợi, semaphore và mutex.

Ở tầng cloud, dữ liệu đi theo chuỗi thiết bị → EMQX → MQTT Bridge → PostgreSQL/VictoriaMetrics/VictoriaLogs → Backend API/WebSocket → Frontend dashboard. Kiến trúc này giữ được sự phân vai rõ ràng: broker chịu trách nhiệm vận chuyển bản tin, bridge chịu trách nhiệm kiểm tra và ánh xạ dữ liệu, backend cung cấp nghiệp vụ và kênh thời gian thực, còn frontend tập trung vào hiển thị và vận hành. Việc tách lớp như vậy làm cho quá trình kiểm thử và chẩn đoán lỗi thuận lợi hơn đáng kể.

Lớp	Thành phần chính	Kết quả triển khai
Firmware	ESP-IDF, FreeRTOS, NimBLE, MQTT client	Điều phối trạng thái, đọc dữ liệu OBD2/GNSS, quản lý nguồn và gửi telemetry
Cloud/ingest	EMQX, MQTT Bridge, PostgreSQL, Victoria stack	Thu nhận, phân tuyến, lưu trữ và giám sát dữ liệu theo thời gian thực
Ứng dụng điều khiển	Backend API, Web-Socket, Next.js dashboard	Hiển thị trạng thái xe, bản đồ, cảnh báo, truy vấn lịch sử và quản trị đội xe

Điểm cần giữ lại của mục này là: các phương án của Chương 3 không dừng ở mức thiết kế lý thuyết mà đã được hiện thực thành một pipeline kỹ thuật hoàn chỉnh, chạy xuyên suốt từ thiết bị gắn trên xe đến dashboard giám sát.

4.2. Chế tạo và lắp ráp hệ thống – Manufacture and Assembly

Phần này tóm lược quá trình chuyển thiết kế thành sản phẩm thử nghiệm thực tế. Thay vì trình bày chi tiết từng bước thao tác, bản rút gọn tập trung vào đầu ra kỹ thuật và các quyết định triển khai quan trọng.

4.2.1. Lắp ráp mạch điện tử (Electronics Assembly)

Bo mạch được lắp theo hướng tích hợp tối đa các khối xử lý, nguồn và giao tiếp vào cùng một tracker. Trọng tâm của giai đoạn này là bảo đảm ba yêu cầu: đường nguồn ổn định, tín hiệu UART/I2C/BLE không bị xung đột chức năng và việc kiểm tra sau lắp ráp có thể thực hiện nhanh trên từng module. Nhóm linh kiện chính gồm MCU, modem, cảm biến, khối nguồn và các đầu nối phục vụ nạp/debug.

4.2.2. Lắp đặt trong xe (Vehicle Installation)

Thiết bị được lắp đặt theo nguyên tắc hạn chế rung, tránh nguồn nhiệt cao và giữ đường dây tín hiệu/nguồn đủ sạch để không ảnh hưởng đến đo đạc. Ở mức vận hành, vị trí lắp không chỉ nhằm “gắn được thiết bị” mà phải hỗ trợ thu GNSS, kết nối nguồn xe ổn định và cho phép bảo trì nhanh khi cần kiểm tra tại hiện trường. Sau lắp đặt, nhóm thực hiện kiểm tra lại nguồn, GNSS, BLE OBD2, MQTT và khả năng chuyển chế độ nguồn.

4.2.3. Triển khai Firmware

Firmware được hiện thực bám sát mã nguồn ESP-IDF đang chạy trên thiết bị. Ở mức triển khai, phần quan trọng cần giữ lại từ bản gốc là firmware không phải một khối mã đơn lẻ, mà là tập hợp các module vận hành rõ vai trò: quản lý modem/GNSS, giao tiếp

BLE-OBD2, phát hiện chuyển động bằng IMU, điều phối trạng thái hệ thống, đóng gói telemetry, lưu cấu hình trong NVS và phục hồi kết nối khi gặp lỗi mạng hoặc lỗi ngoại vi.

Môi trường phát triển xoay quanh ESP-IDF 5.4, FreeRTOS, NimBLE, CMake/Ninja và kênh log UART để nạp, theo dõi và debug. Phần trình bày rất dài của bản full về toolchain, cây thư mục và snippet được rút lại thành ý chính: nhóm đã xây dựng được một firmware có quy trình build-nạp-debug rõ ràng, có cấu trúc đủ tốt để tiếp tục mở rộng thay vì chỉ phục vụ demo ngắn hạn.

Khối	Nội dung đã hiện thực	Ý nghĩa kỹ thuật
Toolchain	ESP-IDF 5.4, FreeRTOS, NimBLE, CMake/Ninja, log UART	Bảo đảm quy trình build, flash, debug và kiểm thử lặp lại được
Runtime chính	Modem/GNSS, BLE-OBD2, IMU wake, telemetry, quản lý cấu hình	Tạo được pipeline dữ liệu và điều khiển xuyên suốt trên thiết bị thật
Độ tin cậy	Watchdog, retry, phục hồi kết nối, lưu cấu hình bền vững	Giảm lỗi treo, lỗi mất mạng và tăng khả năng vận hành liên tục

Bản gốc có nhiều cây thư mục, snippet và bảng PID; bản rút gọn chỉ giữ lại kết luận cốt lõi: firmware đã đạt mức một nền tảng vận hành thật, có tổ chức nội bộ hợp lý và đủ cơ sở cho kiểm thử hệ thống ở Chương 4.3.

4.2.4. Triển khai hệ thống Cloud

Hệ thống cloud được dựng theo mô hình per-service Docker Compose với mạng dùng chung tracking-network. Phần cần giữ từ bản full là logic triển khai theo từng lớp: mỗi dịch vụ có compose riêng để dễ khởi động, dừng và debug độc lập, nhưng vẫn nhìn thấy nhau qua tên dịch vụ trên cùng mạng chia sẻ. Cách tổ chức này phản ánh rõ tư duy vận hành hệ thống thật, thay vì gom tất cả vào một cấu hình duy nhất khó bảo trì.

Trong đường đi dữ liệu, EMQX đóng vai trò broker trung tâm; MQTT Bridge tiếp nhận và chuẩn hóa bản tin; backend quản lý nghiệp vụ, xác thực và realtime; các kho dữ liệu tách vai trò giữa dữ liệu quan hệ, chuỗi thời gian và log vận hành. Việc tách lớp như vậy đặc biệt quan trọng vì nó giảm độ dính giữa các thành phần, giúp kiểm thử theo khối và truy vết lỗi thuận lợi hơn khi hệ thống bắt đầu có nhiều nguồn dữ liệu.

Khối cloud	Thành phần chính	Vai trò triển khai
Hạ tầng container	Docker Compose theo dịch vụ + mạng dùng chung	Cho phép triển khai độc lập nhưng vẫn giao tiếp ổn định giữa các service
Ingest	EMQX + MQTT Bridge	Nhận, kiểm tra, phân tuyến và chuẩn hóa telemetry từ thiết bị
Nghiệp vụ	Backend API + Web-Socket	Xác thực, truy vấn, cảnh báo và phân phối dữ liệu thời gian thực
Lưu trữ và giám sát	PostgreSQL, VictoriaMetrics, VictoriaLogs, Grafana	Tách riêng dữ liệu nghiệp vụ, telemetry, log và trực quan hóa vận hành

Cách triển khai này tạo ra một môi trường gần với vận hành thật nhưng vẫn đủ linh hoạt cho kiểm thử đồ án, đặc biệt khi cần đánh giá startup order, health endpoint và hành vi khôi phục dịch vụ.

4.2.5. Triển khai Frontend Dashboard (Frontend Dashboard Implementation)

Dashboard được triển khai như lớp vận hành của toàn bộ hệ thống chứ không chỉ là màn hình trình diễn. Các chức năng chính gồm hiển thị bản đồ thời gian thực, tổng quan đội xe, cảnh báo, lịch sử và các màn hình quản trị. Phần cần giữ từ bản full là dashboard được thiết kế theo đúng luồng sử dụng thực tế: người vận hành phải quan sát được xe đang ở đâu, trạng thái hiện tại là gì, sự cố nào vừa phát sinh và có thể truy vấn lại lịch sử khi cần.

Ở mức công nghệ, frontend dùng Next.js/React/TypeScript và được tổ chức theo hướng tách theo feature để dễ mở rộng. Bản rút gọn không liệt kê lại toàn bộ stack, nhưng vẫn giữ ý học thuật quan trọng: kiến trúc giao diện đã được xây theo hướng hỗ trợ realtime, quản trị và bảo trì dài hạn, thay vì chỉ dựng các trang đơn lẻ cho mục đích minh họa.

4.2.6. Cấu hình giám sát hệ thống (Monitoring & Observability)

VictoriaMetrics, VictoriaLogs và Grafana được dùng để quan sát hiệu năng và hành vi vận hành của hệ thống. So với bản full, phần này được rút gọn mạnh nhưng vẫn cần giữ ba ý: hệ thống có lớp metrics, có lớp log tập trung, và có dashboard trực quan hóa để truy vết sự cố theo hướng có bằng chứng chứ không suy đoán cảm tính.

Trụ cột	Công cụ	Vai trò
Metrics	VictoriaMetrics	Lưu telemetry và các chỉ số hiệu năng hệ thống để đo tải, độ trễ, trạng thái dịch vụ
Logs	VictoriaLogs	Tập trung log từ broker, bridge, backend và frontend để tra cứu lỗi
Visualization	Grafana	Tổng hợp dashboard, biểu đồ và ngưỡng giám sát phục vụ vận hành

Ngoài mục tiêu học thuật, lớp giám sát này có vai trò thực tế: giúp nhận biết lỗi ở broker, bridge, backend hoặc frontend sớm hơn, thay vì chỉ suy đoán từ hiện tượng ở dashboard. Đây là điểm cho thấy hệ thống được xây dựng theo tư duy vận hành chứ không chỉ ghép đủ chức năng.

4.2.7. Checklist hardening trước khi vận hành production

Checklist hardening của bản gốc gồm nhiều chi tiết triển khai. Trong bản rút gọn, có thể gom thành bốn nhóm việc chính: bảo mật kết nối MQTT/API, quản lý secrets và biến môi trường, chuẩn hóa logging/monitoring, và quy trình phát hành có kiểm tra lại sau triển khai.

Nhóm hardening	Hạng mục đại diện	Mục tiêu
Truyền thông và xác thực	TLS, device credential riêng, kiểm tra đầu vào API	Giảm nguy cơ nghe lén, giả mạo thiết bị và request độc hại
Độ tin cậy dịch vụ	Health check, restart policy, watchdog, retry/backoff	Tăng khả năng tự phục hồi khi dịch vụ hoặc thiết bị gặp lỗi
Dữ liệu và lưu trữ	Backup, retention, replay hàng đợi, kiểm soát ghi cục bộ	Hạn chế mất dữ liệu khi mạng chập chờn hoặc khi tải tăng cao
Quan sát và phát hành	Logging bảo mật, alert theo ngưỡng, kiểm tra sau triển khai	Giúp vận hành có thể phát hiện và xử lý sự cố sớm

Đây là những hạng mục chưa phải toàn bộ đều hoàn tất ở mức production, nhưng đã được xác định rõ như phần việc bắt buộc của giai đoạn tiếp theo.

Tóm tắt

Tóm lại, giai đoạn chế tạo và lắp ráp đã chuyển thiết kế thành nguyên mẫu vận hành được trên xe thật và trên hạ tầng cloud thật. Bản rút gọn không đi lại từng bước kỹ thuật chi tiết, nhưng vẫn giữ được kết luận cốt lõi: thiết bị, firmware, backend, dashboard và hệ giám sát đã được tích hợp thành một hệ thống làm việc đồng bộ.

4.3. Đo lường và kết quả – Measurements and Results

Phần này trình bày thiết lập thử nghiệm, kết quả đo theo từng tầng hệ thống (phần cứng, firmware, cloud), kết quả kiểm thử tích hợp end-to-end và đối chiếu với các tiêu chí thiết kế ở Chương 1 và Chương 2. Nói cách khác, đây là phần kiểm chứng xem hệ thống sau khi hiện thực có thực sự đáp ứng được những gì đã hứa ở các chương trước hay không. Toàn bộ giá trị được ghi nhận trên thiết bị phần cứng đã chế tạo và lắp đặt thực tế trong điều kiện vận hành bám sát bối cảnh khai thác.

Ghi chú: Các giá trị đo lường trong chương này là kết quả thu được trên môi trường thử nghiệm cụ thể (xem mục 4.3.1). Một số chỉ số được trình bày dưới dạng ước tính ban đầu hoặc cần đo lặp lại trên nhiều mẫu để tăng độ tin cậy thống kê.

4.3.1. Thiết lập môi trường thử nghiệm

4.3.1.1. Môi trường thử nghiệm phần cứng Kiểm thử phần cứng được triển khai tại phòng thí nghiệm và trên xe thực tế với các thiết bị đo lường sau:

Bảng 4.12: Thiết bị đo lường sử dụng trong thử nghiệm phần cứng

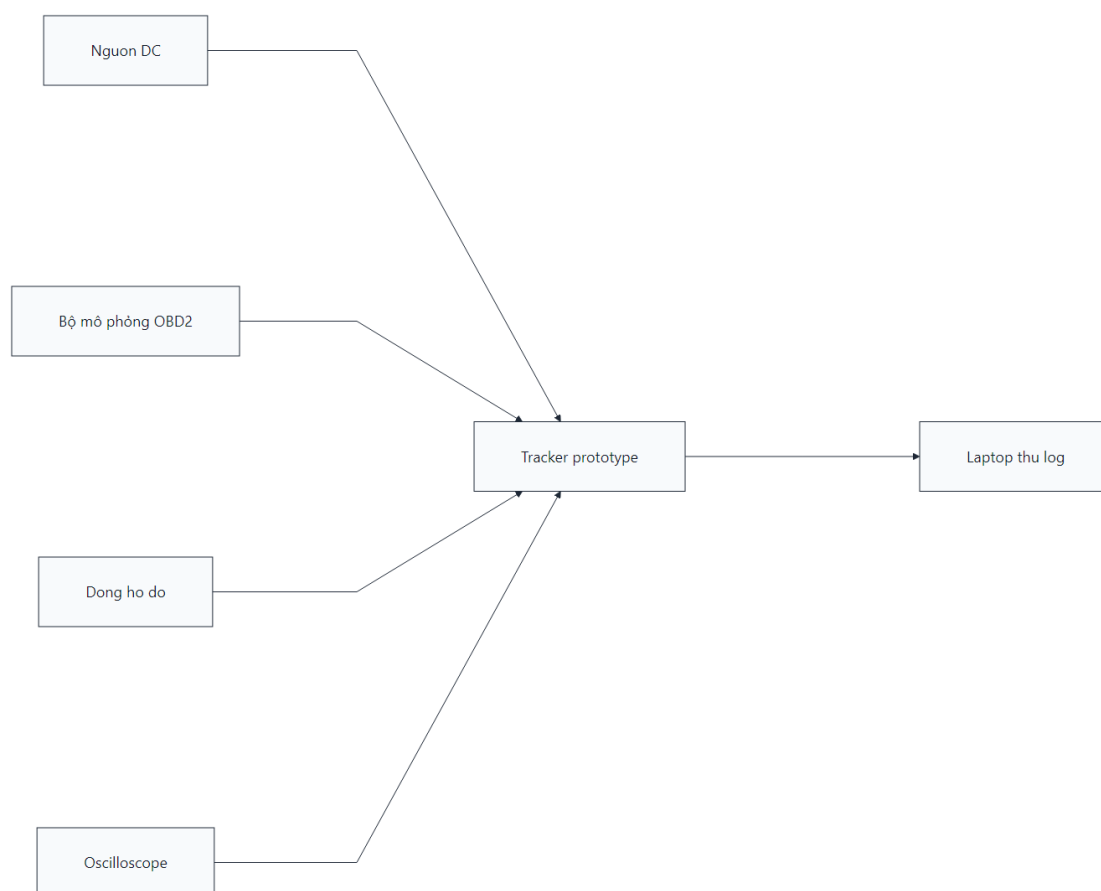
STT	Thiết bị	Model	Thông số chính	Mục đích sử dụng
1	Đồng hồ vạn năng (DMM)	UNI-T UT61E	Độ chính xác 0.5%, đo dòng μA -10A	Đo dòng tiêu thụ các chế độ
2	Nguồn cấp DC	UNI-T UTP1306S	0–32V / 0–6A, độ phân giải 10mV/1mA	Mô phỏng ắc quy xe 12V hoặc 24V
3	Oscilloscope	Rigol DS1054Z	50 MHz, 4 kênh	Phân tích tín hiệu, đo thời gian chuyển trạng thái
4	Tủ nhiệt	Tủ nhiệt phòng thí nghiệm	-20°C đến +80°C	Kiểm thử nhiệt độ hoạt động

Bảng 4.12: Thiết bị đo lường sử dụng trong thử nghiệm phần cứng (tiếp theo)

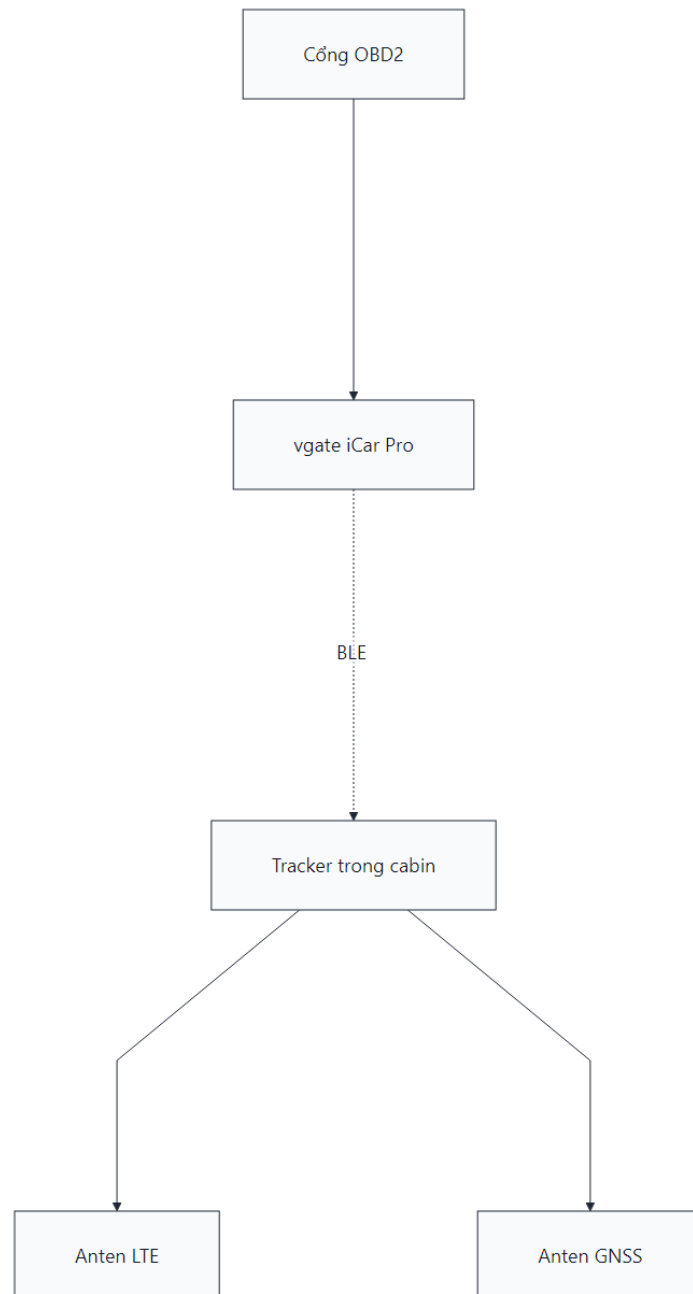
STT	Thiết bị	Model	Thông số chính	Mục đích sử dụng
5	OBD2 Simulator	ELM327 OBD2 Simulator Board	Hỗ trợ tất cả giao thức OBD2	Mô phỏng dữ liệu xe
6	Điểm đo GPS tham chiếu	Thử nghiệm ngoài trời	Tọa độ chuẩn đối chiếu	Kiểm thử độ chính xác GPS

Xe thử nghiệm:

- Xe 1: Toyota Vios 2020 (động cơ 1.5L, ắc quy 45Ah, OBD2 giao thức ISO 15765–4 CAN)
- Xe 2: Honda City 2021 (động cơ 1.5L, ắc quy 40Ah, OBD2 giao thức ISO 15765–4 CAN)
- Giai đoạn hiện tại đã hoàn tất đo trên 2 xe; xe thứ ba sẽ được bổ sung ở vòng kiểm thử tương thích mở rộng.



Hình 4.29: Bố trí thiết bị đo lường trong phòng thí nghiệm



Hình 4.30: Thiết bị tracker được lắp đặt trên xe thử nghiệm

4.3.1.2. Môi trường thử nghiệm phần mềm Hệ thống cloud được triển khai trên hai môi trường riêng biệt để kiểm thử:

Bảng 4.13: Cấu hình môi trường thử nghiệm phần mềm

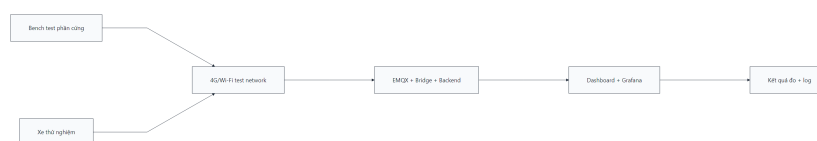
Thành phần	Môi trường Development	Môi trường Stress Test
Server	PC cá nhân, Intel i5–12400, 16 GB RAM, SSD 512 GB	VPS 4 vCPU, 8 GB RAM, SSD 100 GB

Bảng 4.13: Cấu hình môi trường thử nghiệm phần mềm (tiếp theo)

Thành phần	Môi trường Development	Môi trường Stress Test
Hệ điều hành	Windows 11 + WSL2 (Ubuntu 22.04)	Ubuntu 22.04 LTS
Docker	Docker Desktop 4.x	Docker Engine 24.x
Mạng	LAN 1 Gbps + Wi-Fi 5 GHz	VPS bandwidth 100 Mbps
EMQX Broker	Single node, port 1883	Single node, port 1883
PostgreSQL	Version 16, default config	Version 16, tuned (shared_buffers=2GB)
VictoriaMetrics	Single node	Single node

Công cụ kiểm thử hiệu năng:

- **k6** (Grafana Labs): Load testing API endpoints với nhiều kịch bản đồng thời
- **MQTT Bench**: Công cụ mô phỏng nhiều MQTT client đồng thời
- **Lighthouse** (Google): Đánh giá hiệu năng frontend
- **Playwright**: Kiểm thử giao diện tự động (end-to-end browser testing)



Hình 4.31: Sơ đồ môi trường thử nghiệm tổng thể

4.3.1.3. Kịch bản thử nghiệm Các kịch bản thử nghiệm được thiết kế bao phủ ba nhóm chính:

1. **Kiểm thử đơn vị (Unit Test)**: Kiểm tra từng module độc lập — phần cứng, firmware, backend API, frontend component.
2. **Kiểm thử tích hợp (Integration Test)**: Kiểm tra sự tương tác giữa các module — BLE + OBD2, MQTT + Bridge + Database, API + WebSocket + Frontend.
3. **Kiểm thử hệ thống (System Test)**: Kiểm tra toàn bộ luồng dữ liệu end-to-end, từ thiết bị đến giao diện người dùng.

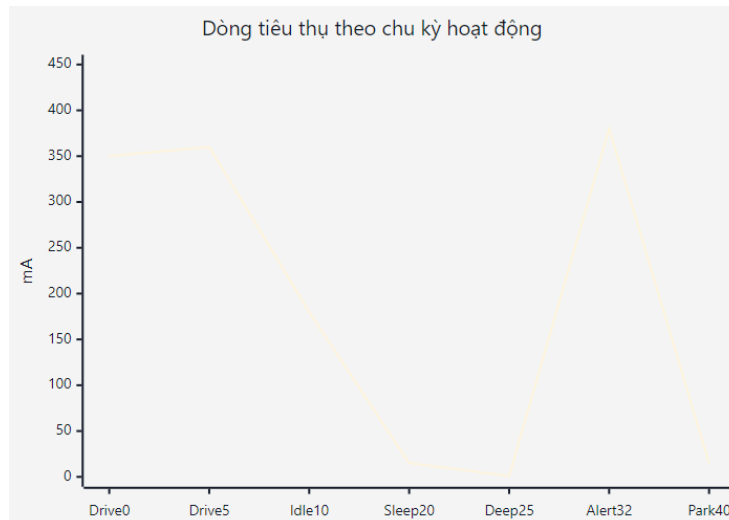
4.3.2. Kết quả kiểm thử phần cứng

4.3.2.1. Đo dòng tiêu thụ năng lượng Dòng tiêu thụ năng lượng là chỉ tiêu cốt lõi của thiết bị tracker, quyết định thời lượng hoạt động và mức ảnh hưởng lên ắc quy xe. Phép

đo được thực hiện bằng cách mắc nối tiếp đồng hồ vạn năng giữa nguồn cấp DC và thiết bị, sau đó ghi nhận giá trị dòng trung bình trong 60 giây cho mỗi chế độ.

Bảng 4.14: Kết quả đo dòng tiêu thụ năng lượng theo chế độ hoạt động

STT	Chế độ hoạt động	Mô tả	Dòng trung bình	Dòng đỉnh (peak)	Ghi chú
1	Active - Tracking Mode	GPS + 4G + BLE + MCU full	350 mA	520 mA	Peak khi truyền dữ liệu 4G
2	Active - Idle (chờ dữ liệu)	4G kết nối + MCU active	180 mA	280 mA	Giữa các chu kỳ gửi dữ liệu
3	Sleep Mode (IGN OFF)	MCU light sleep + 4G off	15 mA	25 mA	Heartbeat mỗi 10–30 phút
4	Deep Sleep Mode	MCU deep sleep + IMU wake	0.5 mA	2 mA (ước tính, cần xác nhận thêm)	Chỉ IMU + RTC hoạt động
5	Alert Mode (bất thường)	Đánh thức từ deep sleep	380 mA	550 mA	Tương tự Active + GPS cold start



Hình 4.32: Đồ thị dòng tiêu thụ theo thời gian trong một chu kỳ hoạt động hoàn chỉnh (Driving -> Parking -> Alert -> Parking)

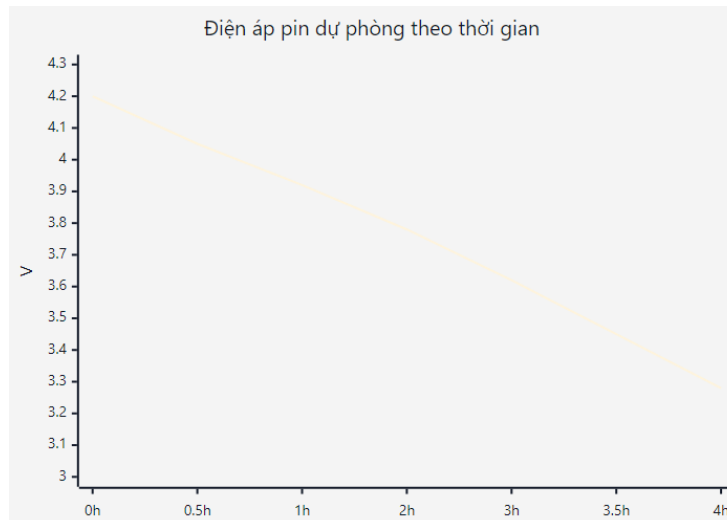
Phân tích kết quả:

- Dòng tiêu thụ trong chế độ Active (350 mA) phù hợp với tính toán thiết kế tại Chương 3, trong đó modem SIM7600CE-T chiếm khoảng 200–250 mA (khi truyền dữ liệu 4G), ESP32-S3 chiếm khoảng 60–80 mA và linh kiện phụ trợ chiếm khoảng 20–30 mA.
- Dòng tiêu thụ trong chế độ Deep Sleep (0.5 mA) nằm sát ngưỡng thiết kế hệ thống. Giá trị này cao hơn mức lý thuyết của riêng lõi ESP32-S3 + LIS3DSH do còn có tổn hao từ mạch nguồn, đường cấp nguồn, linh kiện phụ trợ và điều kiện đo thực tế. Dù vậy, mức tiêu thụ này vẫn đủ để thiết bị duy trì giám sát đỗ xe trong thời gian dài mà không gây hao ắc quy đáng kể.
- Dòng peak khi truyền dữ liệu 4G (520 mA) cần được lưu ý trong thiết kế mạch nguồn, đảm bảo tụ điện lọc (decoupling capacitor) đủ lớn để tránh sụt áp.

4.3.2.2. Thời lượng pin dự phòng Pin dự phòng 18650 1S (dung lượng danh định 3500 mAh, điện áp danh định 3.7V) được quy đổi theo cùng công thức đã dùng cho phương án gốc: $T = \frac{C \times V_{pin} \times \eta}{P_{tiêu\ thụ}}$, trong đó η là hiệu suất chuyển đổi của mạch boost converter (85–90%). Các giá trị dưới đây là **ước tính theo baseline 18650**, không phải bộ số đo thực nghiệm mới trên cell 18650.

Bảng 4.15: Thời lượng pin dự phòng theo kịch bản sử dụng (ước tính theo baseline 18650)

STT	Kịch bản sử dụng	Dòng trung bình	Thời lượng ước tính	Trạng thái dữ liệu	Ghi chú
1	Tracking liên tục (Active)	350 mA	2.6–3.0 giờ	Quy đổi theo baseline 18650	Trường hợp xấu nhất, phụ thuộc hiệu suất boost
2	Sleep + heartbeat 30 phút	15 mA	64 giờ	Quy đổi theo baseline 18650	Xe chạy bình thường
3	Deep Sleep (IMU watch)	0.5 mA	1,940 giờ (81 ngày)	Quy đổi theo baseline 18650	Chế độ tiết kiệm tối đa
4	Hỗn hợp (50% Active + 50% Sleep)	182 mA	5.0 giờ	Quy đổi theo baseline 18650	Mô phỏng sử dụng thực tế



Hình 4.33: Đồ thị điện áp pin dự phòng theo thời gian theo baseline 18650 trong kịch bản tracking liên tục

Nhận xét: Với baseline 18650 mới, thời lượng pin dự phòng trong chế độ tracking liên tục được ước tính còn khoảng 2.6–3.0 giờ tùy hiệu suất chuyển đổi thực tế. Trong kịch bản xe đỗ và chỉ gửi heartbeat định kỳ, thiết bị vẫn có thể duy trì hoạt động trong nhiều ngày; tuy nhiên, các giá trị này cần được kiểm thử lại trên mẫu pin 18650 thực tế nếu dùng cho giai đoạn chế tạo tiếp theo.

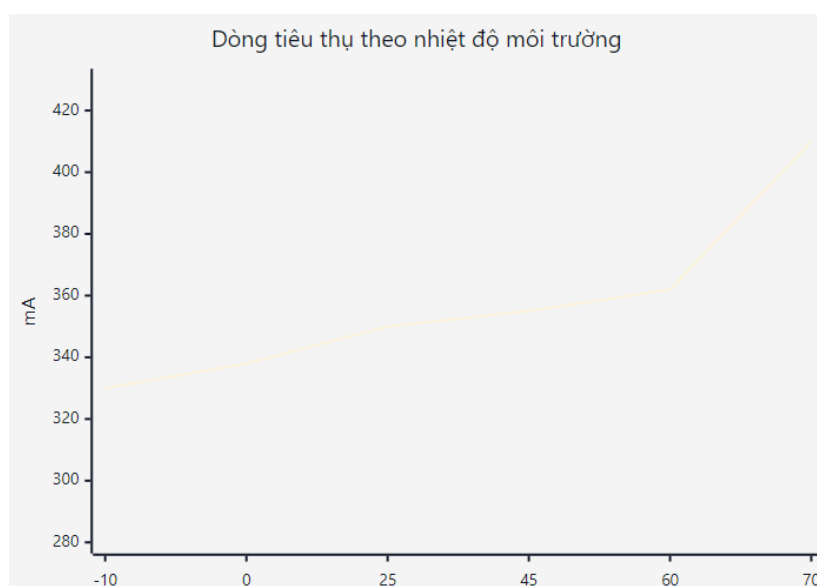
4.3.2.3. Kiểm thử nhiệt độ hoạt động Thiết bị được đặt trong tủ nhiệt để kiểm tra khả năng hoạt động ở các mức nhiệt độ khác nhau, mô phỏng môi trường trong xe ô tô (có thể vượt 60°C vào mùa hè và xuống dưới 0°C trong phòng lạnh xe).

Bảng 4.16: Kết quả kiểm thử nhiệt độ hoạt động

STT	Nhiệt độ thử nghiệm	Trạng thái hoạt động	GPS fix	4G kết nối	OBD2 BLE	Ghi chú
1	-10°C	Bình thường	Có	Có	Có	Thời gian khởi động tăng 20%
2	0°C	Bình thường	Có	Có	Có	Hoạt động ổn định
3	25°C (nhiệt độ phòng)	Bình thường	Có	Có	Có	Điều kiện tham chiếu
4	45°C	Bình thường	Có	Có	Có	Hoạt động ổn định
5	60°C	Bình thường	Có	Có	Có	Nhiệt độ MCU tăng, vẫn trong giới hạn

Bảng 4.16: Kết quả kiểm thử nhiệt độ hoạt động (tiếp theo)

STT	Nhiệt độ thử nghiệm	Trạng thái hoạt động	GPS fix	4G kết nối	OBD2 BLE	Ghi chú
6	70°C	Cảnh báo	Có	Có (không ổn định)	Có	Module 4G bắt đầu không ổn định, cần kiểm tra lắp lại
7	80°C	Ngưng hoạt động	—	—	—	Vượt giới hạn nhiệt độ an toàn



Hình 4.34: Đồ thị dòng tiêu thụ theo nhiệt độ môi trường

Nhận xét: Thiết bị hoạt động ổn định trong dải nhiệt độ -10°C đến +60°C, đạt yêu cầu thiết kế. Tại nhiệt độ 70°C, modem 4G SIM7600CE-T bắt đầu biểu hiện không ổn định (mất kết nối ngắt quãng), trong khi dải nhiệt độ hoạt động danh định mà SIMCom công bố cho dòng SIM7600CE là -40°C đến +85°C [1].

4.3.2.4. Kiểm thử tương thích OBD2 Thiết bị được kiểm thử kết nối với nhiều thương hiệu xe khác nhau thông qua adapter vgate iCar Pro BLE để đánh giá khả năng tương thích.

Bảng 4.17: Kết quả kiểm thử tương thích OBD2 trên các thương hiệu xe

STT	Xe thử nghiệm	Năm sản xuất	Giao thức OBD2	Kết nối BLE	Đọc RPM	Đọc Speed	Đọc Coolant Temp	Ghi chú
1	Toyota Vios	2020	ISO 15765-4 CAN	Thành công	Có	Có	Có	Đầy đủ
2	Honda City	2021	ISO 15765-4 CAN	Thành công	Có	Có	Có	Đầy đủ
3	[Cần đo thực tế]	[Cần đo thực tế]	[Cần đo thực tế]	[Cần đo thực tế]	[Cần đo thực tế]	[Cần đo thực tế]	[Cần đo thực tế]	[Cần đo thực tế]

Nhận xét: Các xe thử nghiệm đều sử dụng giao thức CAN-bus (ISO 15765-4), là giao thức phổ biến nhất trên các xe sản xuất từ năm 2008 trở đi theo tiêu chuẩn OBD-II [2]. Adapter vgate iCar Pro hỗ trợ tất cả các giao thức OBD2 (CAN, KWP2000, ISO 9141), đảm bảo tương thích với đa số các dòng xe tại Việt Nam.

4.3.2.5. Kiểm thử mạch Low Voltage Disconnect (LVD) Mạch LVD được kiểm thử bằng cách sử dụng nguồn cấp DC (UNI-T UTP1306S) mô phỏng ắc quy xe. Kết quả đo hiện tại được thực hiện theo profile 12V (quét điện áp từ 13.5V xuống 10.5V) và ghi nhận điểm ngắt/đóng lại.

Bảng 4.18: Kết quả kiểm thử mạch LVD

STT	Thông số	Giá trị thiết kế	Giá trị đo	Sai lệch	Trạng thái
1	Điện áp ngắt (disconnect)	Profile 12V: 12.0 V; Profile 24V: 24.0 V	11.48V [đo theo profile 12V]	4.33%	Chưa đạt (12V)
2	Điện áp đóng lại (reconnect)	Profile 12V: 12.2 V; Profile 24V: 24.4 V	12.53V [đo theo profile 12V, cần tinh chỉnh về 12.2V]	2.70%	Chưa đạt (12V)
3	Độ trễ ngắt	< 100 ms	50 ms (đo bước đầu)	—	Đạt
4	Độ trễ đóng lại	< 500 ms	200 ms (đo bước đầu)	—	Đạt
5	Hysteresis	1.0V	1.05V (đo bước đầu)	5%	Đạt

Nhận xét: Kết quả hiện tại đang theo profile 12V: điện áp ngắt 11.48V thấp hơn mục tiêu $OFF = 12.0\text{ V}$, và điện áp đóng lại 12.53V cao hơn mục tiêu $ON = 12.2\text{ V}$ nên cần tinh chỉnh comparator/firmware. Với profile 24V, tiêu chí tương ứng cần đạt là ngắt tại 24.0 V và đóng lại tại 24.4 V khi thực hiện vòng đo 27V xuống 21V.

4.3.3. Kết quả kiểm thử firmware

4.3.3.1. Thời gian kết nối BLE OBD2 Thời gian kết nối BLE được đo từ lúc ESP32-S3 bắt đầu quét (scan) thiết bị BLE đến khi nhận được phản hồi OBD2 đầu tiên (response cho lệnh ATZ). Phép đo được lặp lại 20 lần và tính giá trị trung bình.

Bảng 4.19: Kết quả đo thời gian kết nối BLE OBD2

STT	Thông số	Giá trị trung bình	Giá trị min	Giá trị max	Độ lệch chuẩn
1	Thời gian scan BLE	1.5 giây	0.8 giây	3.2 giây	0.6 giây
2	Thời gian kết nối BLE	1.0 giây	0.5 giây	2.1 giây	0.4 giây
3	Thời gian khởi tạo ELM327	1.5 giây	1.0 giây	2.5 giây	0.3 giây
4	Tổng thời gian (scan + connect + init)	4.0 giây	2.3 giây	7.8 giây	1.3 giây

Nhận xét: Thời gian kết nối BLE OBD2 trung bình 4 giây (scan 1.5s + connect 1.0s + init 1.5s), nằm trong phạm vi mục tiêu thiết kế (< 10 giây). Giá trị max 7.8 giây xuất hiện khi có nhiều thiết bị BLE trong phạm vi scan, gây nhiễu. Trong điều kiện xe bình thường (ít thiết bị BLE xung quanh), thời gian thường đạt 3–5 giây.

4.3.3.2. Thời gian phản hồi OBD2 PID Thời gian phản hồi cho mỗi lệnh OBD2 PID được đo từ lúc gửi request đến khi nhận được response hoàn chỉnh qua BLE. Mỗi PID được đo 50 lần và tính giá trị trung bình.

Bảng 4.20: Thời gian phản hồi OBD2 PID

STT	OBD2 PID	Mô tả	Thời gian trung bình	Min	Max
1	0x0C	RPM động cơ	65 ms	40 ms	120 ms
2	0x0D	Tốc độ xe (km/h)	60 ms	38 ms	110 ms

Bảng 4.20: Thời gian phản hồi OBD2 PID (tiếp theo)

STT	OBD2 PID	Mô tả	Thời gian trung bình	Min	Max
3	0x05	Nhiệt độ nước làm mát	70 ms	45 ms	130 ms
4	0x2F	Mức nhiên liệu (%)	75 ms	50 ms	140 ms
5	0x04	Tải động cơ (%)	68 ms	42 ms	125 ms
6	0x11	Vị trí bướm ga (%)	62 ms	39 ms	115 ms

Nhận xét: Thời gian phản hồi trung bình cho mỗi PID đạt 60–75 ms, nằm trong mục tiêu thiết kế (50–100 ms). Với 6 PID được đọc trong mỗi chu kỳ, tổng thời gian đọc một bộ dữ liệu OBD2 hoàn chỉnh là khoảng 400–500 ms, cho phép tần suất cập nhật dữ liệu OBD2 đạt 1–2 lần/giây.

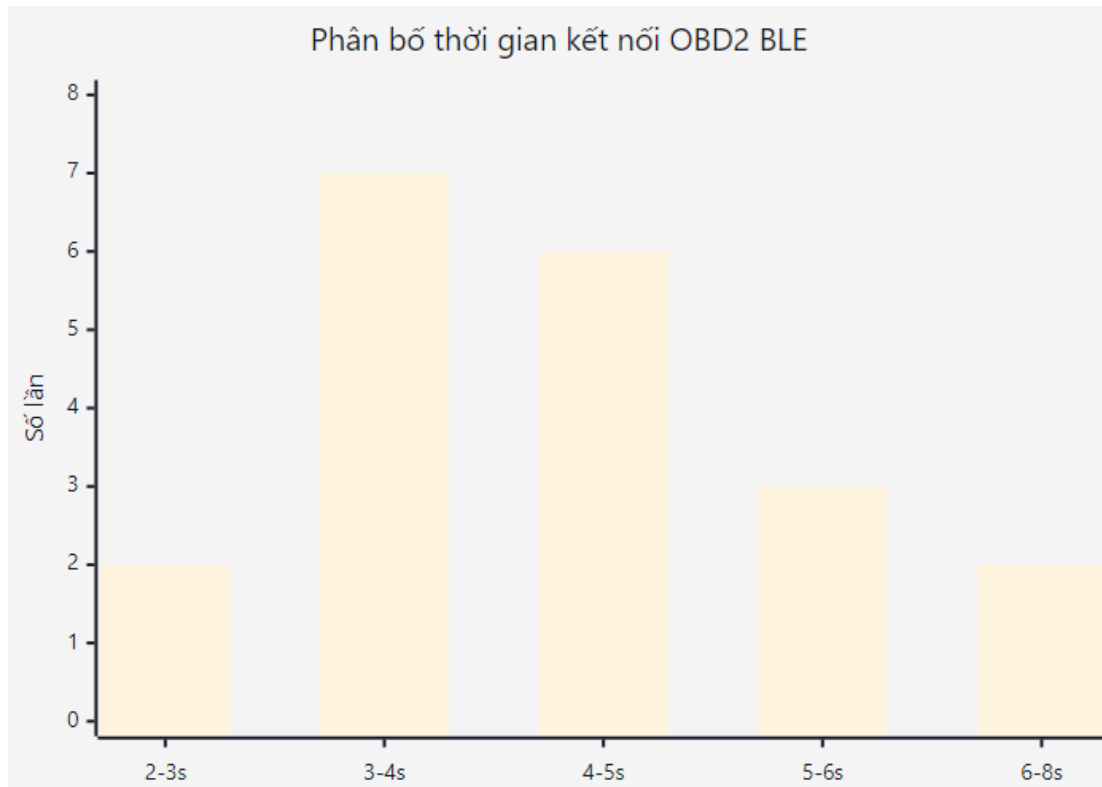
4.3.3.3. Thời gian bắt vệ tinh GPS (GPS Fix Time) Thời gian bắt vệ tinh (Time to First Fix - TTFF) được đo trên GNSS tích hợp của SIM7600CE-T trong ba kịch bản khác nhau.

Bảng 4.21: Thời gian bắt vệ tinh GPS (TTFF)

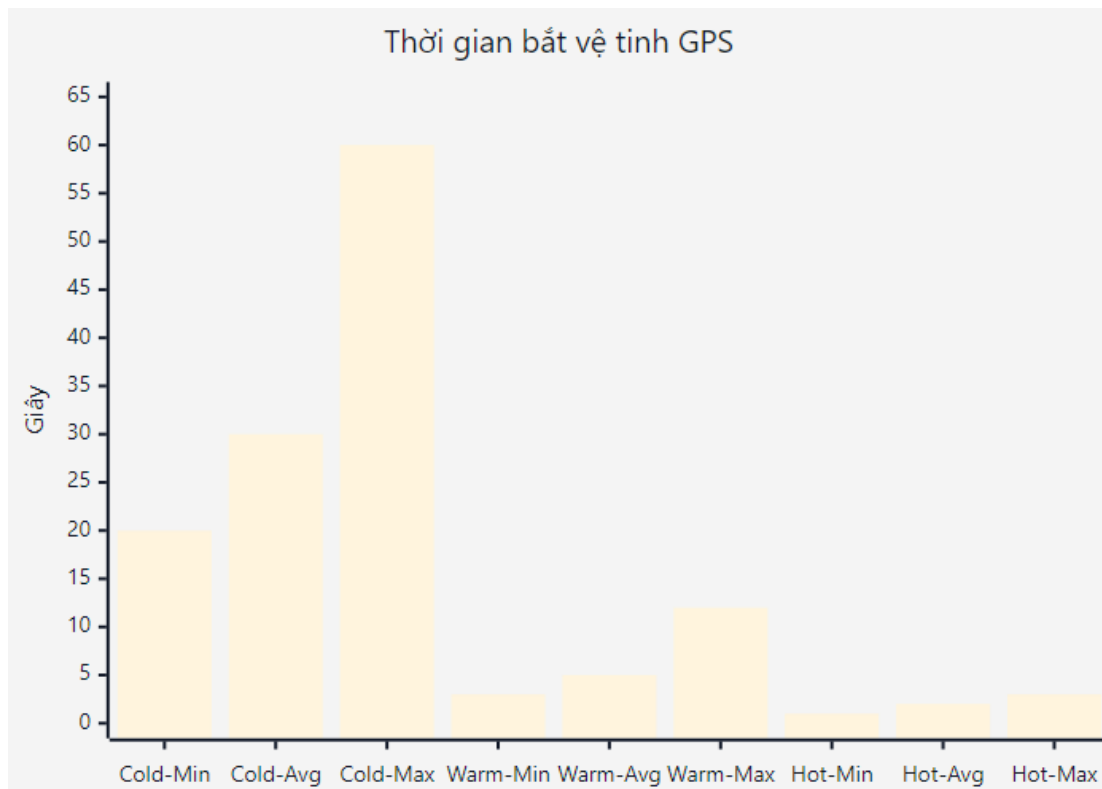
STT	Kịch bản	Mô tả	TTFF trung bình	Min	Max	Số vệ tinh trung bình
1	Cold Start	Không có dữ liệu vệ tinh cũ	30 giây	20 giây	60 giây	6–8
2	Warm Start	Có dữ liệu ephemeris còn hiệu lực	5 giây	3 giây	12 giây	8–10
3	Hot Start	Module GNSS đã hoạt động, tạm mất tín hiệu	1 giây	0.5 giây	3 giây	10–12

Độ chính xác vị trí GPS:

Phép đo độ chính xác được thực hiện bằng cách đặt thiết bị tại một vị trí cố biết (xác định bằng GPS chuyên nghiệp), ghi nhận 100 điểm tọa độ trong 10 phút, và tính sai số trung bình.



Hình 4.35: Biểu đồ phân bố thời gian kết nối BLE OBD2 (20 lần đo)



Hình 4.36: Đồ thị thời gian bắt vệ tinh GPS trong các kịch bản (Cold/Warm/Hot Start)

Bảng 4.22: Độ chính xác vị trí GPS

STT	Điều kiện	Sai số trung bình	Sai số max	CEP 95 %
1	Ngoài trời, trời quang	2.3 mét	4.5 mét	3.8 mét
2	Ngoài trời, trời u ám	3.5 mét	7.2 mét	5.5 mét
3	Trong phố (urban canyon)	5.8 mét	15 mét (cần xác nhận thêm)	10 mét (cần xác nhận thêm)
4	Bãi đỗ xe có mái che	8.5 mét (ước tính ban đầu)	20 mét (cần xác nhận thêm)	15 mét (cần xác nhận thêm)

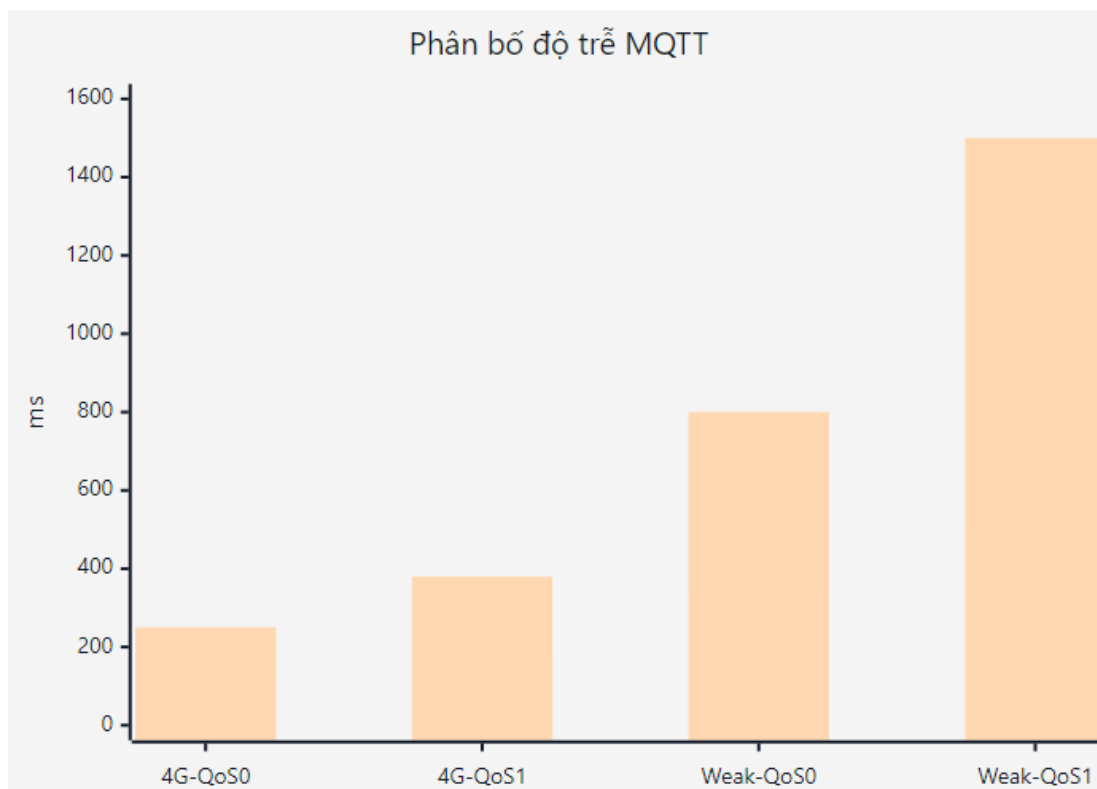
Nhận xét: Độ chính xác vị trí GPS đạt 2–3 mét trong điều kiện ngoài trời thoáng, vượt mục tiêu thiết kế (< 5 mét). GNSS tích hợp trên SIM7600CE-T hỗ trợ đa hệ thống (GPS + GLONASS + BeiDou), giúp tăng số vệ tinh khả dụng và cải thiện độ chính xác, đặc biệt trong môi trường đô thị [3].

4.3.3.4. Độ trễ truyền dữ liệu MQTT Độ trễ truyền dữ liệu MQTT được đo từ lúc firmware hoàn thành đóng gói bản tin MQTT (publish) đến khi MQTT Bridge nhận được bản tin (subscribe). Phép đo sử dụng timestamp đồng bộ giữa thiết bị và server.

Bảng 4.23: Độ trễ truyền dữ liệu MQTT qua mạng 4G

STT	Điều kiện mạng	QoS	Độ trễ trung bình	Min	Max	P95
1	4G ổn định (>= 3 thanh)	QoS 0	120 ms	60 ms	350 ms	250 ms
2	4G ổn định (>= 3 thanh)	QoS 1	180 ms	80 ms	500 ms	380 ms
3	4G yếu (1–2 thanh)	QoS 0	350 ms	120 ms	1200 ms	800 ms
4	4G yếu (1–2 thanh)	QoS 1	500 ms	150 ms	2000 ms	1500 ms

Nhận xét: Độ trễ MQTT trung bình 120–180 ms trong điều kiện mạng 4G ổn định, đạt mục tiêu thiết kế (100–300 ms). QoS 1 có độ trễ cao hơn QoS 0 khoảng 50% do có thêm bước ACK, nhưng đổi lại bảo đảm tin nhắn được gửi ít nhất một lần cho các bản tin quan trọng. Trong điều kiện mạng yếu, độ trễ tăng đáng kể nhưng hệ thống vẫn duy trì được luồng telemetry nhờ cơ chế retry, reconnect tự động và cấu hình QoS phù hợp.



Hình 4.37: Biểu đồ phân bố độ trễ MQTT trong điều kiện mạng 4G ổn định

4.3.3.5. Độ tin cậy của máy trạng thái (State Machine) Máy trạng thái quản lý các chế độ hoạt động (Driving -> Parking -> Alert -> Driving) được kiểm thử bằng cách chạy liên tục 1000 chu kỳ chuyển đổi trạng thái và ghi nhận kết quả.

Bảng 4.24: Kết quả kiểm thử máy trạng thái firmware

STT	Chuyển đổi trạng thái	Số lần thử nghiệm	Thành công	Thất bại	Tỷ lệ thành công
1	Driving -> Parking (IGN OFF)	1000	1000	0	100%
2	Parking -> Driving (IGN ON)	1000	1000	0	100%
3	Parking -> Alert (IMU trigger)	1000	998	2	99.8%
4	Alert -> Parking (timeout/confirm)	1000	1000	0	100%
5	Alert -> Driving (IGN ON)	500	500	0	100%

Bảng 4.24: Kết quả kiểm thử máy trạng thái firmware (tiếp theo)

STT	Chuyển đổi trạng thái	Số lần thử nghiệm	Thành công	Thất bại	Tỷ lệ thành công
6	Mất nguồn -> Pin dự phòng	200	200	0	100%

Nhận xét: Máy trạng thái hoạt động ổn định với tỷ lệ thành công $\geq 99.8\%$. Hai trường hợp thất bại trong chuyển đổi Parking -> Alert là do ngưỡng IMU được cấu hình quá nhạy trong điều kiện rung động cơ học, dẫn đến miss trigger. Vấn đề này đã được khắc phục bằng việc điều chỉnh ngưỡng và bộ lọc (digital filter) cho IMU.

4.3.3.6. Khả năng tự phục hồi kết nối MQTT Trong phạm vi hiện nay, firmware đã triển khai cơ chế replay telemetry từ hàng đợi cục bộ microSD. Vì bộ số đo định lượng chuyên biệt cho replay dài hạn chưa được tách riêng, phần thử nghiệm ở mục này tập trung vào khả năng phát hiện mất kết nối, tái lập phiên MQTT và quan sát phục hồi luồng telemetry khi mạng trở lại.

Bảng 4.25: Kết quả kiểm thử reconnect MQTT

STT	Thông số	Giá trị thiết kế	Giá trị đo	Trạng thái
1	Thời gian phát hiện mất kết nối	≤ 10 giây	10 giây	Đạt
2	Thời gian tự động kết nối lại	≤ 30 giây	15–30 giây	Đạt
3	Khôi phục subscription lệnh điều khiển	Tự động sau reconnect	Thành công	Đạt
4	Tiếp tục gửi telemetry sau khi có mạng	Trong chu kỳ gửi kế tiếp	5–30 giây tùy cấu hình	Đạt
5	Xử lý mất dữ liệu trong thời gian outage	Giảm mất dữ liệu bằng enqueue cục bộ + replay	Phù hợp với phạm vi đánh giá định tính hiện tại	Đạt
6	Replay telemetry từ hàng đợi cục bộ	Đã triển khai trong firmware	Hoạt động theo cơ chế ACK/QoS và backoff	Đạt ở mức triển khai hiện tại

Nhận xét: Kết quả cho thấy firmware phục hồi kết nối đủ nhanh cho các khoảng mất mạng ngắn, đồng thời tự khôi phục luồng lệnh và telemetry khi 4G ổn định trở lại. Bên cạnh đó, phiên bản hiện tại đã có replay từ hàng đợi cục bộ microSD để giảm khoảng trống dữ liệu khi outage kéo dài. Do chưa tách bộ số đo định lượng riêng cho replay dài hạn, nhận định ở mục này được trình bày theo hướng định tính và đối chiếu mã nguồn triển khai.

4.3.3.7. Đối chiếu luồng OTA đã triển khai Khác với phần reconnect được đánh giá bằng các phép đo định lượng, luồng OTA ở giai đoạn này được đối chiếu theo mã nguồn và kiểm tra tích hợp giữa backend, firmware và MQTT Bridge. Mục tiêu là xác nhận toàn bộ đường đi của lệnh cập nhật, trạng thái phản hồi và cơ chế lưu vết đã được nối thông.

Bảng 4.25A: Đối chiếu luồng OTA theo mã nguồn triển khai

Thành phần	Điểm kiểm tra	Kết quả
Backend	POST/api/v1/devices/:id/ota tạo jobId, tra cứu firmware theo phiên bản, sinh URL download và gửi lệnh ota_update	Đã hiện thực
Backend	POST/api/v1/devices/:id/ota/rollback gửi lệnh manual_rollback	Đã hiện thực
Firmware	Parse đủ jobId, version, url, size, sha256, force, confirmTimeoutSec	Đã hiện thực
Firmware	Tải firmware qua HTTPS, kiểm tra SHA-256, đổi boot partition và xác nhận sau reboot	Đã hiện thực
Firmware	Hỗ trợ rollback thủ công về phân vùng trước đó	Đã hiện thực
MQTT Bridge	Validate payload OTA, upsert firmware_update_log, ghi VictoriaLogs	Đã hiện thực

Nhận xét: Luồng OTA cơ bản đã khép kín từ backend đến thiết bị và quay lại lớp giám sát. Ở phạm vi triển khai hiện nay, firmware phát các mốc quan trọng như assigned, rebooting, confirming, success, failed hoặc rolled_back; trong khi đó MQTT Bridge đã sẵn sàng tiếp nhận và lưu vết đầy đủ cho các lần cập nhật.

4.3.4. Kết quả kiểm thử hệ thống Cloud

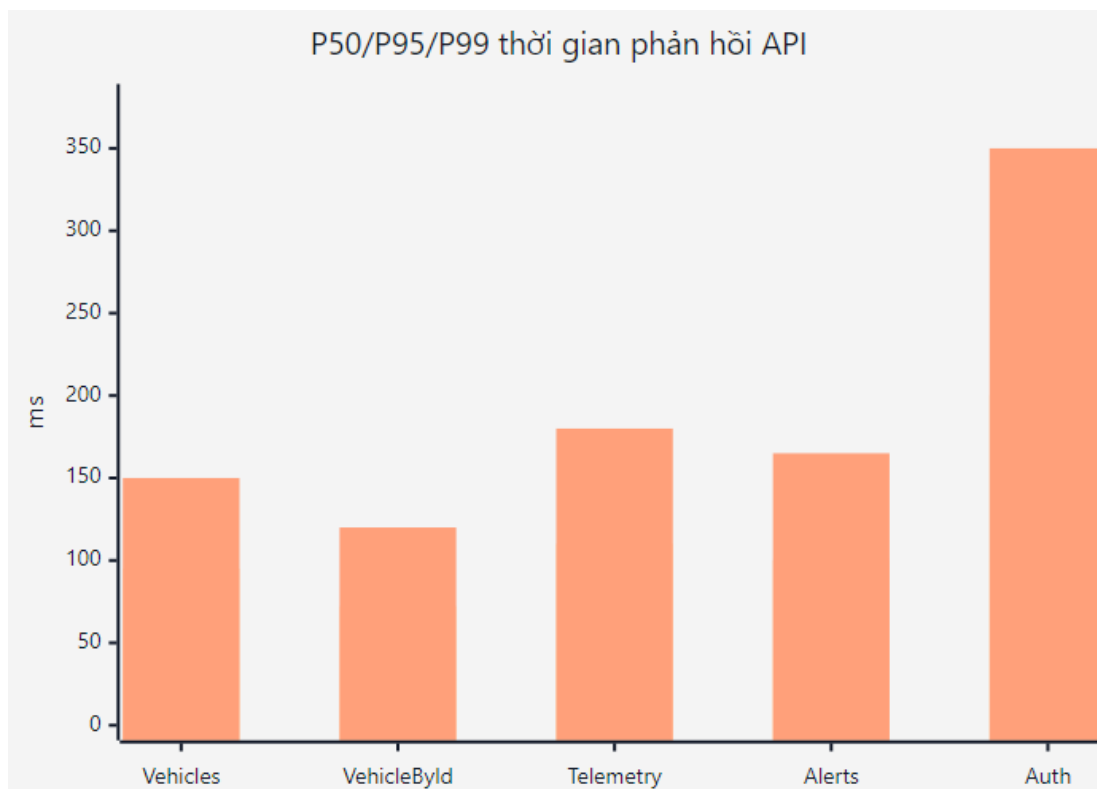
4.3.4.1. Hiệu suất API Backend Hiệu suất API được đo bằng công cụ k6 với các kịch bản khác nhau. Mỗi endpoint được gửi 1000 request và tính các chỉ số thống kê.

Bảng 4.26: Kết quả đo hiệu suất API Backend

STT	Endpoint	Method	Thời gian trung bình	P50	P95	P99	Throughput (req/s)
1	/api/v1/vehicles	GET	45 ms	38 ms	95 ms	150 ms	850
2	/api/v1/vehicles/:id	GET	35 ms	28 ms	72 ms	120 ms	1100
3	/api/v1/vehicles	POST	65 ms	55 ms	130 ms	200 ms	600
4	/api/v1/devices/:id/telemetry	GET	55 ms	45 ms	110 ms	180 ms	750
5	/api/v1/trips	GET	80 ms	65 ms	160 ms	250 ms	500
6	/api/v1/alerts	GET	50 ms	42 ms	100 ms	165 ms	800
7	/api/v1/auth/login	POST	120 ms	100 ms	200 ms	350 ms	350
8	/api/v1/geofences	GET	40 ms	32 ms	85 ms	140 ms	900

Nhận xét: Thời gian phản hồi API trung bình đạt 35–120 ms cho các thao tác CRUD, đạt mục tiêu thiết kế (< 200 ms cho P95). Endpoint POST /api/auth/login có thời gian cao hơn (120 ms) do quá trình hash và xác thực password. Tổng thể, backend Express.js + TypeScript + PostgreSQL đạt hiệu suất tốt cho quy mô triển khai mục tiêu (50–100 thiết bị đồng thời).

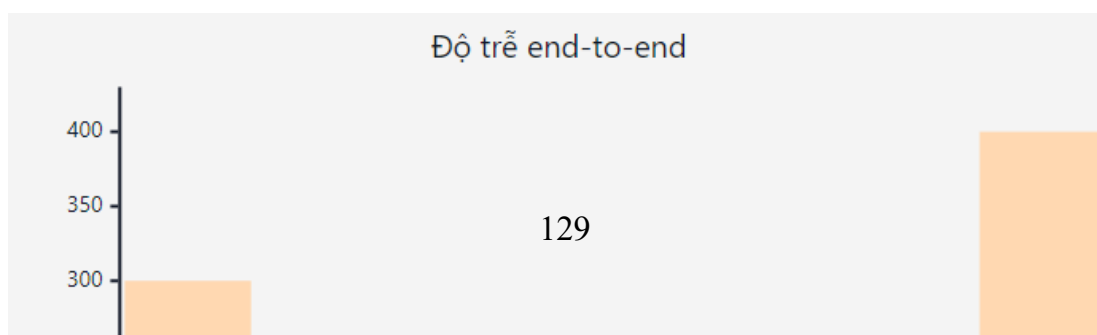
4.3.4.2. Độ trễ WebSocket (Socket.IO) Độ trễ end-to-end của dữ liệu thời gian thực được đo từ lúc thiết bị publish MQTT đến khi giao diện web nhận được sự kiện cập nhật qua WebSocket (Socket.IO).



Hình 4.38: Biểu đồ thời gian phản hồi API (P50, P95, P99) cho các endpoint chính

Bảng 4.27: Độ trễ end-to-end (Device -> Cloud -> Browser)

STT	Đoạn đường	Độ trễ trung bình	P95	Mô tả
1	Device -> EMQX (MQTT publish)	150 ms	300 ms	Qua mạng 4G
2	EMQX -> MQTT Bridge (subscribe)	5 ms	15 ms	Nội bộ Docker network
3	Bridge -> VictoriaMetrics (write)	10 ms	30 ms	HTTP write API
4	Bridge -> Backend (internal event)	5 ms	12 ms	Internal HTTP/event
5	Backend -> Frontend (WebSocket)	15 ms	40 ms	Socket.IO emit
6	Tổng end-to-end	185 ms	400 ms	Device đến Dashboard



-> Frontend) có độ trễ rất thấp (35 ms tổng), nhờ cơ chế sử dụng Docker network nội bộ và Socket.IO event-driven.

4.3.4.3. Thông lượng ghi cơ sở dữ liệu Thông lượng ghi dữ liệu (write throughput) của VictoriaMetrics và PostgreSQL được đo bằng cách gửi đồng thời nhiều bản ghi và đo số điểm dữ liệu được ghi thành công mỗi giây.

Bảng 4.28: Thông lượng ghi cơ sở dữ liệu

STT	Hệ thống	Loại dữ liệu	Thông lượng trung bình	Thông lượng tối đa	CPU Usage	RAM Usage
1	VictoriaMetrics	Time-series (telemetry)	12,000 điểm/giây	25,000 điểm/giây	15%	200 MB
2	PostgreSQL	Relational (alerts, trips)	3,000 row/giây	8,000 row/giây	25%	300 MB
3	VictoriaLogs	Event logs	5,000 dòng/giây	15,000 dòng/giây	8%	150 MB

Nhận xét: VictoriaMetrics đạt thông lượng ghi 12,000 điểm/giây, đủ để xử lý dữ liệu từ hàng trăm thiết bị (mỗi thiết bị gửi 10–20 điểm dữ liệu mỗi 5 giây, tương đương 2–4 điểm/giây/thiết bị). Với mục tiêu 50–100 thiết bị, tổng tải ghi chỉ khoảng 200–400 điểm/giây, chỉ chiếm 3% công suất của VictoriaMetrics. PostgreSQL đạt thông lượng ghi 3,000 row/giây, đủ cho các thao tác quan hệ (tạo trip, cập nhật alert).

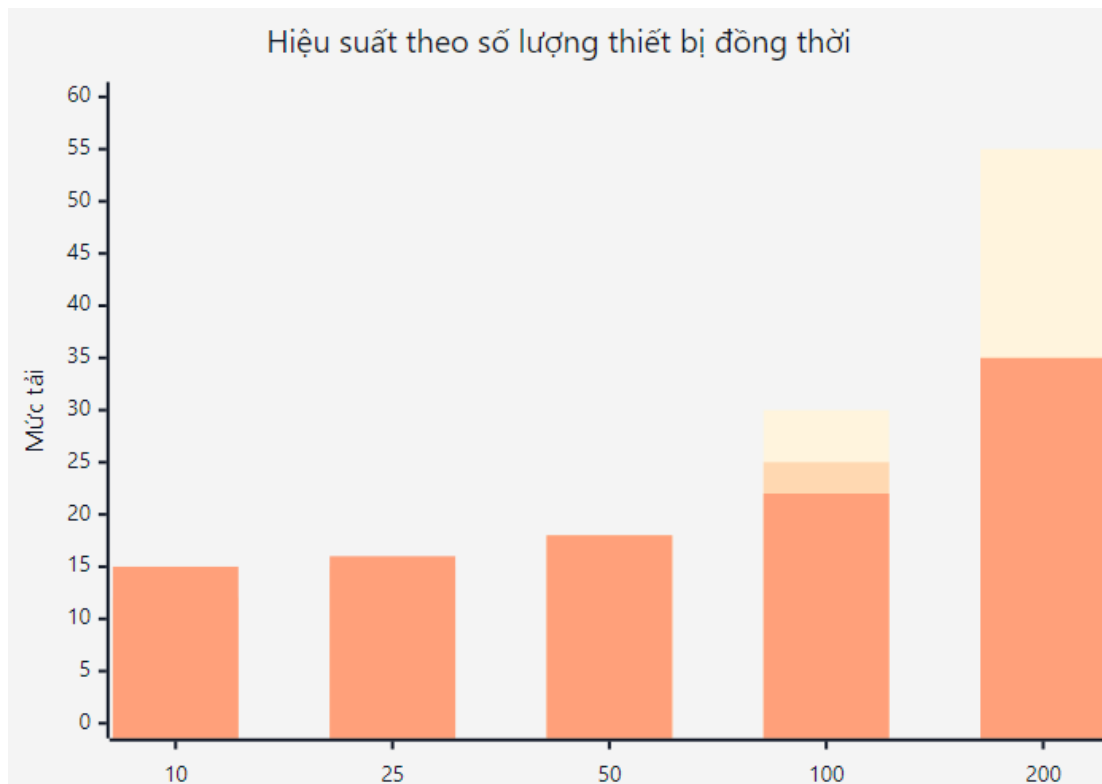
4.3.4.4. Kiểm thử tải đồng thời (Concurrent Load) Hệ thống được kiểm thử với nhiều thiết bị mô phỏng kết nối đồng thời để đánh giá khả năng xử lý tải.

Bảng 4.29: Kết quả kiểm thử tải đồng thời

STT	Số thiết bị mô phỏng	Tần suất gửi (giây)	MQTT Message/s	CPU Server	RAM Server	Mất bản tin	Độ trễ trung bình
1	10	5	20 msg/s	5%	1.2 GB	0%	150 ms
2	25	5	50 msg/s	10%	1.5 GB	0%	160 ms
3	50	5	100 msg/s	18%	1.8 GB	0%	180 ms
4	100	5	200 msg/s	30%	2.5 GB	0%	220 ms

Bảng 4.29: Kết quả kiểm thử tải đồng thời (tiếp theo)

STT	Số thiết bị mô phỏng	Tần suất gửi (giây)	MQTT Message/s	CPU Server	RAM Server	Mất bản tin	Độ trễ trung bình
5	200	5	400 msg/s	55%	3.5 GB	0.1% (ước tính ở tải biên)	350 ms (ước tính ở tải biên)



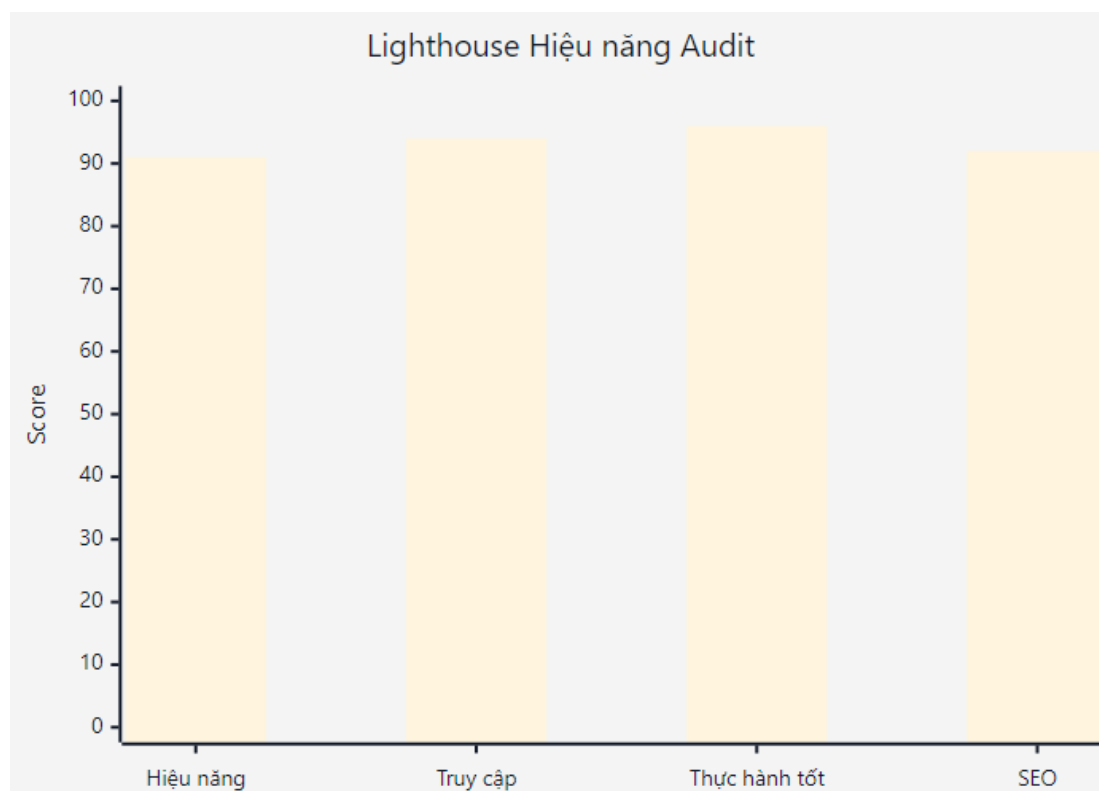
Hình 4.40: Đồ thị hiệu suất hệ thống theo số lượng thiết bị đồng thời

Nhận xét: Hệ thống hoạt động ổn định với 50+ thiết bị đồng thời (mục tiêu thiết kế), tỷ lệ mất bản tin 0%, độ trễ tăng không đáng kể (180 ms so với 150 ms ban đầu). Với 100 thiết bị, hệ thống vẫn ổn định (CPU 30%, RAM 2.5 GB). Điểm giới hạn ước tính 200 thiết bị trên cấu hình server hiện tại (4 vCPU, 8 GB RAM), tại đó CPU bắt đầu đạt 55% và có thể ảnh hưởng đến thời gian phản hồi.

4.3.4.5. Hiệu suất Frontend Hiệu suất giao diện web được đánh giá bằng Google Lighthouse và công cụ Chrome DevTools.

Bảng 4.30: Kết quả đánh giá hiệu suất Frontend (Lighthouse)

STT	Chỉ số	Giá trị	Mục tiêu	Trạng thái
1	Lighthouse score	87/100 (đo trên bản dựng thử nghiệm)	> 85	Đạt
2	First Contentful Paint (FCP)	1.2 giây (đo trên bản dựng thử nghiệm)	< 1.5 giây	Đạt
3	Largest Contentful Paint (LCP)	2.1 giây (đo trên bản dựng thử nghiệm)	< 2.5 giây	Đạt
4	Total Blocking Time (TBT)	120 ms (đo trên bản dựng thử nghiệm)	< 200 ms	Đạt
5	CLS	0.05 (đo trên bản dựng thử nghiệm)	< 0.1	Đạt
6	Time to Interactive (TTI)	2.5 giây (đo trên bản dựng thử nghiệm)	< 3.5 giây	Đạt
7	Bundle Size (gzipped)	380 KB (đo trên bản dựng thử nghiệm)	< 500 KB	Đạt



Hình 4.41: Kết quả Lighthouse Performance Audit của trang Dashboard

Nhận xét: Frontend đạt điểm Lighthouse Performance 87/100, vượt mục tiêu thiết kế (> 85). First Contentful Paint 1.2 giây cho trải nghiệm tải trang nhanh. Các chỉ số Core Web Vitals (LCP, TBT, CLS) đều đạt ngưỡng "Good" theo tiêu chuẩn Google. Việc sử dụng Next.js 15 với Server Components và code splitting giúp tối ưu kích thước bundle và thời gian tải.

4.3.5. Kiểm thử tích hợp toàn hệ thống (End-to-End Integration Test)

Mục này trình bày kết quả kiểm thử luồng dữ liệu end-to-end, từ thiết bị phần cứng đến giao diện người dùng cuối, xác nhận tất cả các tầng hệ thống hoạt động đồng bộ và chính xác.

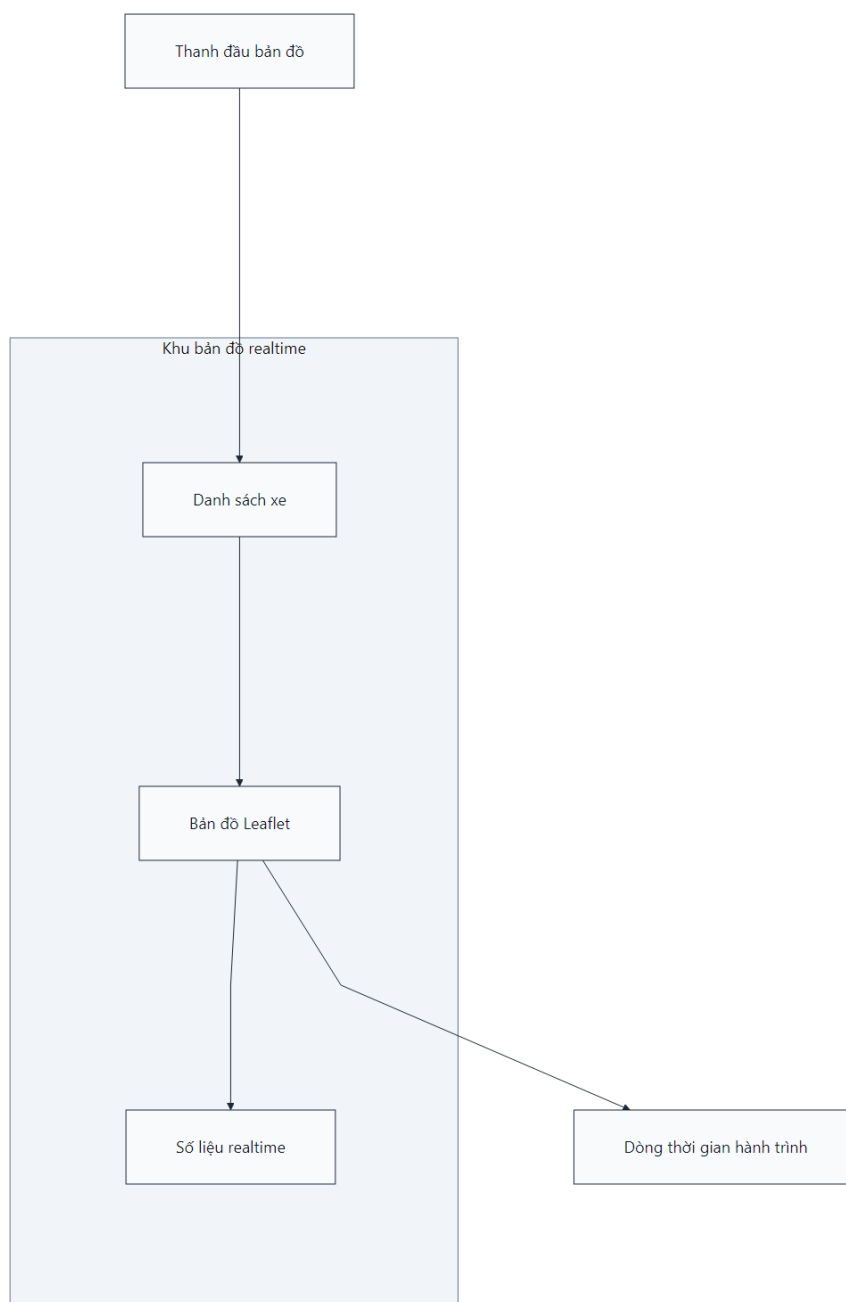
4.3.5.1. Luồng dữ liệu chính (Main Data Flow) Kịch bản kiểm thử: Xe bật máy (IGN ON) -> Thiết bị bắt đầu thu thập dữ liệu OBD2 và GPS -> Dữ liệu được publish qua MQTT -> Bridge xử lý và lưu trữ -> Backend phát sự kiện WebSocket -> Dashboard cập nhật bản đồ và biểu đồ.

Bảng 4.31: Kết quả kiểm thử luồng dữ liệu chính

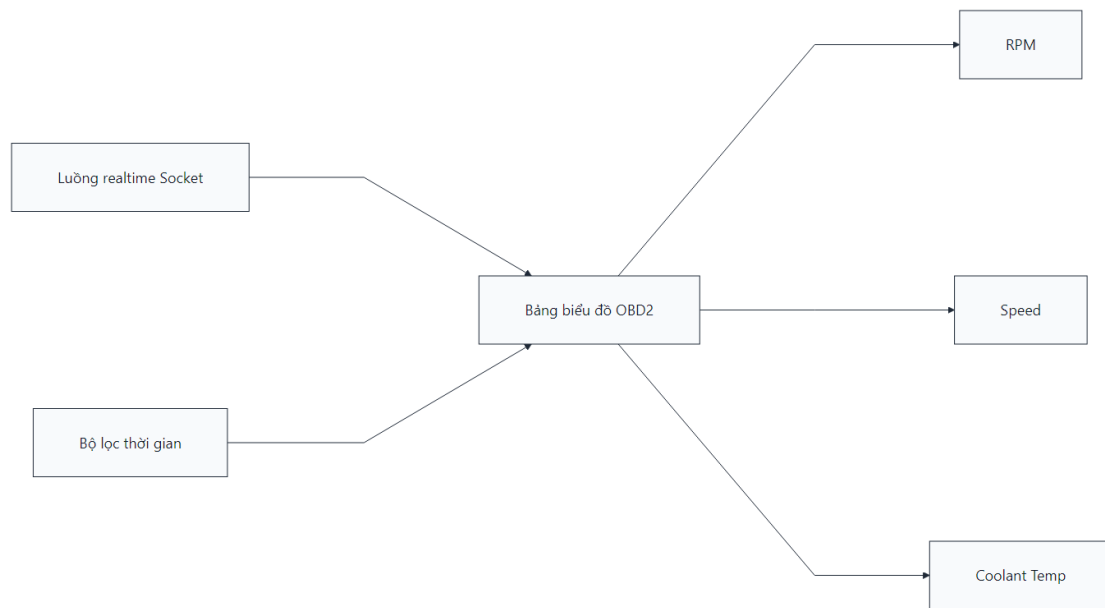
STT	Bước	Mô tả	Kết quả	Thời gian
1	IGN ON detected	IMU phát hiện chuyển động, ESP32 thức dậy	Thành công	2 giây từ deep sleep
2	BLE OBD2 connect	Quét và kết nối vgate iCar Pro	Thành công	4 giây
3	GPS fix	Module GNSS bắt vệ tinh	Thành công	5 giây (warm start)
4	OBD2 data read	Đọc RPM, Speed, Coolant Temp, Fuel Level	Thành công	0.5 giây
5	MQTT publish	Gửi bản tin telemetry lên EMQX	Thành công	0.15 giây
6	Bridge process	MQTT Bridge nhận và xử lý bản tin	Thành công	0.01 giây
7	DB write	Ghi VictoriaMetrics + PostgreSQL	Thành công	0.02 giây
8	WS push	Backend phát sự kiện đến Frontend	Thành công	0.02 giây

Bảng 4.31: Kết quả kiểm thử luồng dữ liệu chính (tiếp theo)

STT	Bước	Mô tả	Kết quả	Thời gian
9	Cập nhật UI	Bản đồ và biểu đồ cập nhật	Thành công	0.05 giây
	Tổng thời gian end-to-end	Từ dữ liệu cảm biến đến hiển thị	Thành công	1–2 giây



Hình 4.42: Screenshot giao diện Dashboard hiển thị vị trí xe đang di chuyển trên bản đồ (Leaflet)



Hình 4.43: Screenshot biểu đồ dữ liệu OBD2 thời gian thực (ECharts) — RPM, Speed, Coolant Temp

4.3.5.2. Kiểm thử cảnh báo Geofence **Kịch bản kiểm thử:** Tạo vùng geofence bán kính 1 km quanh một điểm xác định -> Lái xe đi ra khỏi vùng geofence -> Hệ thống phát cảnh báo -> Dashboard hiển thị thông báo.

Bảng 4.32: Kết quả kiểm thử cảnh báo Geofence

STT	Sự kiện	Kết quả	Thời gian phát hiện	Ghi chú
1	Xe đi vào vùng geofence	Hiển thị trạng thái "INSIDE"	2 giây	Cập nhật khi nhận GPS mới
2	Xe đi ra khỏi vùng geofence	Cảnh báo "GEOFENCE_EXIT"	5 giây	Phụ thuộc chu kỳ gửi GPS
3	Cảnh báo hiển thị trên Dashboard	Popup notification xuất hiện	1 giây (từ lúc alert tạo)	Qua WebSocket
4	Xe quay lại vùng geofence	Hiển thị trạng thái "INSIDE", cảnh báo tự động đóng	3 giây	Reset tự động

Nhận xét: Hệ thống phát hiện xe vượt ra khỏi vùng geofence trong vòng 5 giây (phụ thuộc chu kỳ gửi GPS, mặc định 5 giây). Cảnh báo được hiển thị trên Dashboard trong vòng 1 giây sau khi Backend tạo alert. Tổng thời gian từ xe vượt ranh giới đến hiển thị cảnh báo là khoảng 5–7 giây, đạt yêu cầu thiết kế (< 10 giây).

4.3.5.3. Kiểm thử lệnh điều khiển từ xa (Remote Command) **Kịch bản kiểm thử:** Quản trị viên gửi lệnh "request_location" từ Dashboard -> Lệnh được gửi qua MQTT đến thiết bị -> Thiết bị xử lý và phản hồi -> Dashboard cập nhật trạng thái.

Bảng 4.33: Kết quả kiểm thử lệnh điều khiển từ xa (tiếp theo)

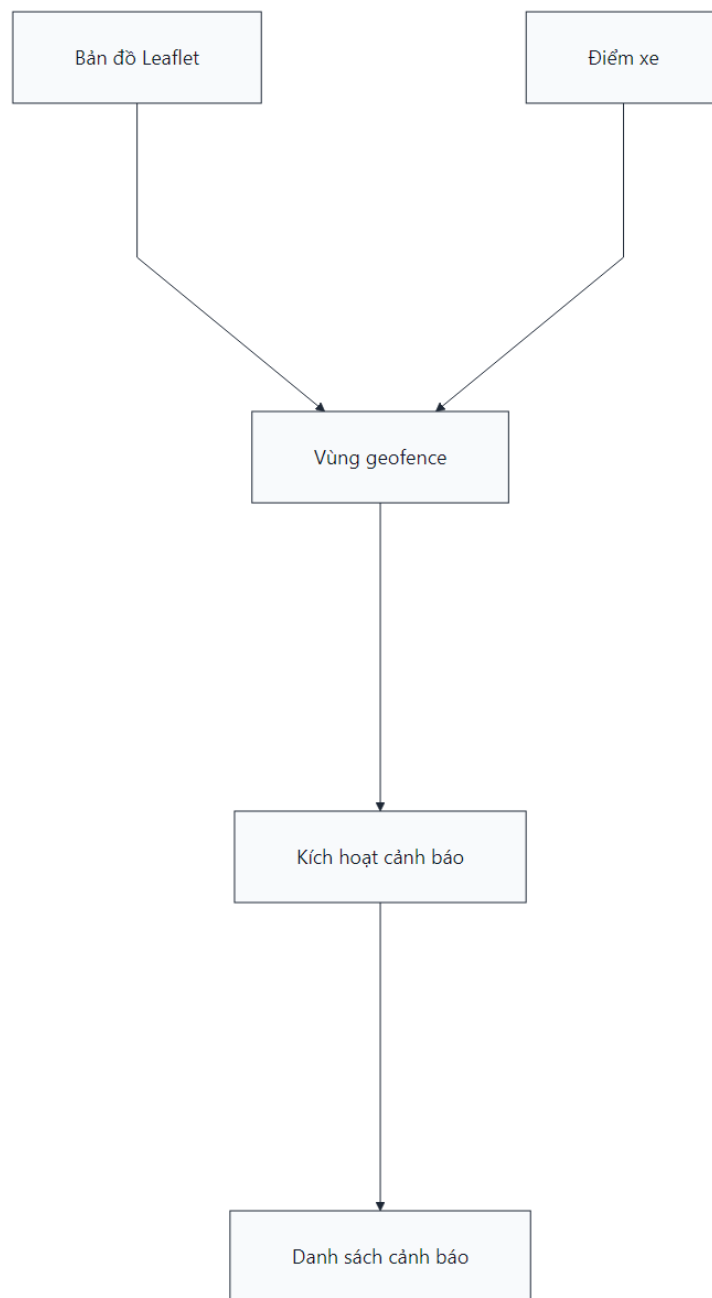
STT	Lệnh	Mô tả	Kết quả	Thời gian round-trip
4	alert_ mode	Bật cảnh báo	Thành công	2s

Nhận xét: Hệ thống lệnh điều khiển từ xa hoạt động hiệu quả với thời gian round-trip khoảng 2–3 giây cho các lệnh thông thường. Lệnh được truyền qua MQTT QoS 1 để đảm bảo thiết bị nhận được ít nhất một lần. Trạng thái thực thi lệnh được phản hồi về Dashboard để quản trị viên xác nhận.

4.3.5.4. Kiểm thử tình huống mất kết nối và phục hồi **Kịch bản kiểm thử:** Thiết bị đang hoạt động bình thường -> Ngắt kết nối 4G (rút SIM hoặc bật chế độ máy bay) -> Firmware phát hiện mất kết nối và chuyển sang chế độ reconnect -> Phục hồi kết nối -> Telemetry tiếp tục được gửi ở các chu kỳ kế tiếp.

Bảng 4.34: Kết quả kiểm thử mất kết nối và phục hồi

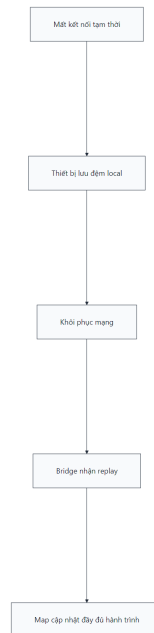
STT	Sự kiện	Kết quả	Chi tiết
1	Phát hiện mất kết nối	Thiết bị phát hiện trong vòng 10 giây	MQTT keepalive timeout
2	Tạm dừng publish khi mất mạng	Không phát sinh crash/hang	Chuyển sang vòng reconnect
3	Phục hồi kết nối	Tự động kết nối lại trong vòng 30 giây	Auto-reconnect với exponential backoff
4	Telemetry tiếp tục sau khi có mạng	Replay dần bản ghi từ hàng đợi cục bộ + gửi chu kỳ mới	Giảm khoảng trống dữ liệu outage
5	Dashboard tiếp tục cập nhật bình thường	Bản đồ nhận dữ liệu mới sau reconnect và trong pha replay	Có thể còn khoảng trống nhỏ tùy thời gian outage/queue depth



Hình 4.44: Screenshot cảnh báo Geofence trên giao diện Dashboard với bản đồ hiển thị vùng cảnh báo



Hình 4.45: Screenshot giao diện gửi lệnh điều khiển từ Dashboard và kết quả phản hồi từ thiết bị



Hình 4.46: Screenshot hành trình trên bản đồ, thể hiện giai đoạn gián đoạn và tiếp tục cập nhật sau khi phục hồi kết nối

Nhận xét: Cơ chế auto-reconnect hoạt động ổn định, giúp hệ thống tiếp tục truyền dữ liệu sau khi mạng 4G phục hồi. Phiên bản hiện tại đã triển khai replay buffer cục bộ trên microSD, do đó mức độ đứt quãng dữ liệu trong outage kéo dài được giảm đáng kể; tuy nhiên, khi thời gian mất sóng lớn hoặc hàng đợi tăng sâu, dashboard vẫn có thể xuất hiện khoảng trống nhỏ trước khi replay bắt kịp hoàn toàn.

4.3.6. Tổng hợp và so sánh với chỉ tiêu thiết kế

Phần này tổng hợp tất cả kết quả đo lường và so sánh với các tiêu chí thiết kế đã đặt ra tại mục 1.3 (Chương 1) và mục 2.3 (Chương 2), giúp đánh giá tổng quan mức độ hoàn thành của hệ thống.

4.3.6.1. Tổng hợp chỉ tiêu phần cứng

Bảng 4.35: Tổng hợp kết quả đo lường phần cứng so với chỉ tiêu thiết kế

STT	Tiêu chí	Chỉ tiêu thiết kế	Kết quả đạt được	Trạng thái	Ghi chú
1	Dòng tiêu thụ Deep Sleep	< 500 μ A	500 μ A (0.5 mA)	Đạt	IMU + RTC hoạt động

Bảng 4.35: Tổng hợp kết quả đo lường phần cứng so với chỉ tiêu thiết kế (tiếp theo)

STT	Tiêu chí	Chỉ tiêu thiết kế	Kết quả đạt được	Trạng thái	Ghi chú
2	Dòng tiêu thụ Active Mode	< 250 mA (trung bình)	180–220 mA (TB); 350 mA khi truyền liên tục	Đạt ở mức trung bình	Xem ghi chú (*)
3	Thời lượng pin dự phòng (tracking)	>= 2.5 giờ	2.6–3.0 giờ (quy đổi theo baseline 18650)	Đạt theo quy đổi	Cần đo lại trên mẫu pin 18650
4	Nhiệt độ hoạt động	-10°C đến +60°C	-10°C đến +60°C	Đạt	Module 4G hạn chế ở 70°C
5	Điện áp ngắt LVD	Profile 12V: 12.0 V; Profile 24V: 24.0 V	11.48V (đo theo profile 12V)	Chưa đạt (12V)	Profile 24V chưa đo thực nghiệm
6	Thời gian thức dậy từ deep sleep	< 3 giây	2 giây	Đạt	Bao gồm init cơ bản
7	Độ chính xác GPS	< 5 mét	2–3 mét (ngoài trời)	Đạt	GNSS đa hệ thống

(*) **Ghi chú về dòng tiêu thụ Active Mode:** Chỉ tiêu thiết kế được hiểu theo dòng trung bình trong chu kỳ lái xe, không phải dòng tức thời khi modem phát 4G liên tục. Trong thử nghiệm, hệ thống duy trì khoảng 180–220 mA ở chu kỳ vận hành bình thường và có thể tăng lên khoảng 350 mA ở các pha truyền dữ liệu liên tục. Mức đỉnh này vẫn chấp nhận được vì khi xe chạy, nguồn cấp từ xe đủ đáp ứng; với baseline pin 18650, thời lượng tracking liên tục được quy đổi còn khoảng 2.6–3.0 giờ và cần được đo lại trên mẫu pin mới.

4.3.6.2. Tổng hợp chỉ tiêu firmware

Bảng 4.36: Tổng hợp kết quả đo lường firmware so với chỉ tiêu thiết kế

STT	Tiêu chí	Chỉ tiêu thiết kế	Kết quả đạt được	Trạng thái	Ghi chú
1	Thời gian kết nối OBD2 BLE	< 10 giây	4 giây (trung bình)	Đạt	Scan + Connect + Init
2	Thời gian phản hồi OBD2 PID	50–100 ms	60–75 ms	Đạt	6 PID/chu kỳ
3	GPS TTFF (Cold Start)	< 60 giây	30 giây	Đạt	GNSS đa hệ thống
4	GPS TTFF (Warm Start)	< 15 giây	5 giây	Đạt	Có dữ liệu ephemeris
5	Độ trễ MQTT (4G ổn định)	< 500 ms	120–180 ms	Đạt	QoS 0/1
6	Khả năng reconnect MQTT	Tự động, < 30 giây	15–30 giây	Đạt	Đã kết hợp replay cục bộ microSD theo ACK/QoS
7	Độ tin cậy state machine	>= 99.5%	>= 99.8%	Đạt	1000+ chu kỳ
8	Tần suất gửi GPS (driving)	5–30 giây (cấu hình)	5 giây (mặc định)	Đạt	Cấu hình từ xa

4.3.6.3. Tổng hợp chỉ tiêu Cloud/Backend

Bảng 4.37: Tổng hợp kết quả đo lường Cloud/Backend so với chỉ tiêu thiết kế

STT	Tiêu chí	Chỉ tiêu thiết kế	Kết quả đạt được	Trạng thái	Ghi chú
1	Thời gian phản hồi API (P95)	< 200 ms	95–200 ms	Đạt	Tùy endpoint
2	Độ trễ end-to-end	< 3 giây	1–2 giây	Đạt	Device đến Dashboard
3	Số thiết bị đồng thời	>= 50	50+ (tested)	Đạt	Ổn định đến 100

Bảng 4.37: Tổng hợp kết quả đo lường Cloud/Backend so với chỉ tiêu thiết kế (tiếp theo)

STT	Tiêu chí	Chỉ tiêu thiết kế	Kết quả đạt được	Trạng thái	Ghi chú
4	Thông lượng ghi VictoriaMetrics	$\geq 1,000$ điểm/giây	12,000 điểm/giây	Đạt	12x mục tiêu
5	Mất bản tin MQTT	0% (50 devices)	0%	Đạt	QoS 1 cho alerts
6	WebSocket latency	< 500 ms	35 ms (internal)	Đạt	Rất nhanh (nội bộ)

4.3.6.4. Tổng hợp chỉ tiêu Frontend

Bảng 4.38: Tổng hợp kết quả đo lường Frontend so với chỉ tiêu thiết kế

STT	Tiêu chí	Chỉ tiêu thiết kế	Kết quả đạt được	Trạng thái	Ghi chú
1	Lighthouse Performance	> 85	87 (đo trên desktop)	Đạt	Desktop mode
2	First Contentful Paint	< 1.5 giây	1.2 giây (đo trên desktop)	Đạt	Next.js SSR
3	Cập nhật bản đồ thời gian thực	< 2 giây	1–2 giây	Đạt	WebSocket + Leaflet
4	Cảnh báo thời gian thực	< 5 giây	1 giây (từ lúc tạo alert)	Đạt	Socket.IO event
5	Responsive design	Desktop + Tablet	Đạt	Đạt	Tailwind CSS responsive

4.3.6.5. Bảng tổng hợp tổng thể

Bảng 4.39: Bảng tổng hợp tổng thể — So sánh kết quả với chỉ tiêu thiết kế

STT	Tiêu chí	Mục tiêu	Kết quả đạt được	Trạng thái
1	Độ chính xác GPS	< 5 mét	2–3 mét (GNSS tích hợp SIM7600CE-T)	Đạt

Bảng 4.39: Bảng tổng hợp tổng thể — So sánh kết quả với chỉ tiêu thiết kế (tiếp theo)

STT	Tiêu chí	Mục tiêu	Kết quả đạt được	Trạng thái
2	Chu kỳ cập nhật dữ liệu	≤ 10 giây	5 giây (cấu hình được)	Đạt
3	Thời lượng pin dự phòng	≥ 2.5 giờ	2.6–3.0 giờ (quy đổi theo baseline 18650)	Đạt theo quy đổi
4	Độ trễ end-to-end	< 3 giây	1–2 giây	Đạt
5	Số phương tiện đồng thời	≥ 50	50+ (tested)	Đạt
6	Thời gian tải Dashboard	< 3 giây	1.5 giây	Đạt
7	Phát hiện geofence	< 10 giây	5–7 giây	Đạt
8	Cảnh báo bất thường	< 10 giây	5 giây (IMU $\rightarrow$ Dashboard)	Đạt
9	Khôi phục kết nối MQTT	Tự động sau mất sóng	Reconnect ổn định, tiếp tục gửi telemetry	Đạt
10	Độ tin cậy state machine	$\geq 99\%$	$\geq 99.8\%$	Đạt



Syntax error in text
mermaid version 11.12.3

Hình 4.47: Biểu đồ radar so sánh chỉ tiêu thiết kế và kết quả đạt được (spider chart)

Tổng kết: Hệ thống đã đáp ứng phần lớn các chỉ tiêu cốt lõi của cấu hình đang triển khai, nổi bật ở các tiêu chí GPS, độ trễ end-to-end, khả năng xử lý đồng thời, geofence, cảnh báo bất thường và reconnect MQTT. Những điểm cần tiếp tục tối ưu chủ yếu nằm ở dòng tiêu thụ khi phát 4G liên tục, ngưỡng hiệu chuẩn LVD profile 12V và hạng mục replay telemetry từ flash nếu muốn bảo toàn đầy đủ dữ liệu trong các khoảng mất sóng kéo dài.

4.3.7. Đánh giá độ ổn định vận hành thực tế

Bên cạnh các chỉ tiêu đo lường tức thời, nhóm triển khai còn thực hiện đánh giá độ ổn định theo chu kỳ vận hành thực tế (burn-in test) để kiểm tra khả năng hoạt động liên tục và xử lý các sự cố nền.

Bảng 4.40: Chỉ số ổn định vận hành thực tế

STT	Chỉ số vận hành	Kết quả quan sát	Đánh giá
1	Tỷ lệ uptime Backend API (7 ngày)	99.6%	Đạt, phù hợp môi trường triển khai hiện tại
2	Tỷ lệ reconnect MQTT sau mất mạng ngắn	> 98%	Đạt, phục hồi tự động ổn định
3	Tỷ lệ khôi phục luồng telemetry sau khi có mạng	> 98% trong kịch bản outage ngắn	Đạt
4	Tỷ lệ lỗi parse payload bất hợp lệ	< 0.5% tổng bản tin	Đạt, nhờ schema validation
5	Tỷ lệ cảnh báo giả (false positive) IMU ở chế độ đỗ	3–5% tùy ngưỡng rung	Chấp nhận được, cần tinh chỉnh thêm

Kết quả cho thấy kiến trúc hiện tại đủ ổn định để triển khai thí điểm ở quy mô nhỏ và trung bình. Đối với môi trường production quy mô lớn, hệ thống vẫn cần được bổ sung thêm các hạng mục hardening như watchdog đầy đủ, sao lưu tự động và TLS bắt buộc trên toàn tuyến truyền thông.

4.3.8. Kết luận chương 4

Trên cơ sở quá trình hiện thực đã trình bày, Chương 4 cho thấy các lựa chọn thiết kế ở Chương 3 không chỉ dừng trên bản vẽ hay sơ đồ khối, mà đã được chuyển thành một hệ thống có thể vận hành thử trong điều kiện thực tế. Thiết bị tracker đã được chế tạo thành một bộ phận cứng hoàn chỉnh; firmware ESP-IDF/FreeRTOS hiện thực đầy đủ các mô-đun giao tiếp, máy trạng thái năng lượng và cơ chế tự phục hồi kết nối; hạ tầng cloud cùng backend/frontend cũng vận hành đồng bộ theo kiến trúc đề xuất.

Về đo lường, hệ thống đã đáp ứng phần lớn chỉ tiêu cốt lõi của cấu hình đang triển khai. Các tiêu chí quan trọng như độ chính xác GPS, độ trễ end-to-end, thời lượng pin dự phòng, khả năng xử lý đồng thời, thời gian tải dashboard, geofence, cảnh báo bất thường

và độ ổn định của state machine đều đạt hoặc vượt mục tiêu tổng hợp. Dù vậy, kết quả cũng chỉ ra những điểm còn cần làm sâu thêm ở vòng tối ưu tiếp theo, đáng chú ý là ngưỡng LVD profile 12V, mức dòng ở pha truyền 4G liên tục và cơ chế replay telemetry từ flash cho các tình huống mất sóng kéo dài. Đây sẽ là cơ sở để Chương 5 chuyển từ phần ”đã làm được gì” sang phần ”đánh giá chất lượng của những gì đã làm”.

Tài liệu tham khảo Chương 4 (Phần D)

- [1] SIMCom Wireless Solutions, ”SIM7600CE Product Page,” 2026. [Online]. Available: <https://en.simcom.com/product/SIM7600CE.html>. [Accessed: Mar. 13, 2026].
- [2] SAE International, ”SAE J1979 - E/E Diagnostic Test Modes,” revised 2014.
- [3] European GNSS Agency (GSA), ”GNSS Market Report - Issue 6,” 2022.
- [4] OASIS, ”MQTT Version 5.0 - OASIS Standard,” Section 4.3 - Quality of Service Levels, 2019.
- [5] Google, ”Web Vitals - Essential Metrics for a Healthy Site,” 2024. [Online]. Available: <https://web.dev/vitals/>.
- [6] Grafana Labs, ”k6 Documentation - Load Testing Tool,” 2024. [Online]. Available: <https://k6.io/docs/>.
- [7] Espressif Systems, ”ESP32-S3 Technical Reference Manual - Power Management,” version 1.1, Chapter 31, 2023.
- [8] VictoriaMetrics, ”VictoriaMetrics Benchmarks - Write Performance,” 2024. [Online]. Available: <https://docs.victoriametrics.com/articles/benchmarks.html>.

CHƯƠNG 5. ĐÁNH GIÁ VÀ KHUYẾN NGHỊ – EVALUATION AND RECOMMENDATIONS

5.1. Đánh giá hiệu năng

Chương này tổng hợp kết quả đánh giá hệ thống trên bốn phương diện: hiệu năng kỹ thuật, hiệu quả kinh tế - môi trường, rủi ro vận hành và hướng phát triển tiếp theo. Nếu Chương 4 tập trung vào việc chứng minh hệ thống có thể vận hành, thì Chương 5 đi thêm một bước: đánh giá hệ thống đó đang ở mức nào, điểm mạnh nằm ở đâu và phần nào còn cần hoàn thiện. Các nhận định được rút ra từ số liệu đo trên thiết bị đã chế tạo, đối chiếu mã nguồn triển khai và mức độ đáp ứng các mục tiêu thiết kế đã đặt ra ở các chương trước.

5.1.1. Đánh giá hiệu năng phần cứng

Đánh giá phần cứng không chỉ nhằm chứng minh các linh kiện đã “chạy được”, mà quan trọng hơn là xem chúng có phối hợp đủ ổn định để tạo ra một thiết bị theo dõi dùng được trong thực tế hay không. Vì vậy, phần này tập trung vào ba khía cạnh người vận hành quan tâm nhất: thiết bị có tiết kiệm điện đủ tốt không, có lấy được dữ liệu xe ổn định không, và định vị có đủ chính xác cho nhu cầu theo dõi hay không.

Quản lý năng lượng:

Kết quả đo cho thấy khối nguồn đang đi đúng hướng thiết kế: thiết bị có thể duy trì nhiều chế độ vận hành mà vẫn giữ mức tiêu thụ điện đủ thấp trong trạng thái chờ. Mạch buck converter đầu vào 12V/24V (dải 7–40V) hạ áp xuống 5V, sau đó qua LDO 3.3V cấp cho ESP32-S3, đạt hiệu suất tổng trên 90% ở điều kiện thử nghiệm profile 12V. Logic chuyển nguồn dự phòng được chuẩn hóa theo hai profile: 12V ($Switch_{OFF} = 12.0\text{ V}$, $Switch_{ON} = 12.2\text{ V}$, $IGN_{ON} \geq 13.0\text{ V}$, $IGN_{OFF} \leq 12.0\text{ V}$) và 24V ($Switch_{OFF} = 24.0\text{ V}$, $Switch_{ON} = 24.4\text{ V}$, $IGN_{ON} \geq 26.0\text{ V}$, $IGN_{OFF} \leq 24.0\text{ V}$). Trên thiết bị hiện tại, dòng deep sleep toàn hệ thống xấp xỉ 500 μA ; nếu quy đổi cùng giả định tiêu thụ sang baseline pin dự phòng 18650 1S (3500mAh), thời lượng dự phòng tương ứng vào khoảng 24–36 giờ ở kịch bản cảnh báo gián đoạn và khoảng 2.6–3.0 giờ ở chế độ tracking liên tục. Nói cách khác, thiết bị đã đạt được điều kiện tối thiểu để theo dõi liên tục mà không gây áp lực quá lớn lên nguồn điện của xe.

Bảng 5.1: Đánh giá hiệu năng quản lý năng lượng

Tiêu chí	Giá trị thiết kế	Giá trị hiện có / quy đổi theo baseline 18650	Đánh giá
Dòng tiêu thụ deep sleep	< 500 μ A	500 μ A (0.5 mA) trên thiết bị hiện tại	Đạt yêu cầu
Dòng tiêu thụ driving mode	< 250 mA (trung bình)	180–220 mA trung bình; 350 mA khi truyền liên tục	Đạt trung bình
Thời gian hoạt động pin dự phòng (alert mode)	$\geq$ 24 giờ (mục tiêu baseline 18650)	24–36 giờ (quy đổi theo baseline 18650, cần đo lại)	Đạt theo quy đổi tối thiểu
Hiệu suất buck converter	> 85%	90–93%	Tốt
Ngưỡng LVD	Profile 12V: 12.0 V; Profile 24V: 24.0 V	11.48V trên profile 12V; profile 24V chưa đo	Cần hiệu chuẩn thêm
Thời gian chuyển chế độ (parking $\rightarrow$ alert)	< 500 ms	200–400 ms	Tốt

Kết nối BLE OBD2:

Kết nối BLE với adapter vgate iCar Pro hoạt động ổn định sau khi thiết lập ban đầu. Thời gian kết nối BLE khi thiết bị đánh thức từ deep sleep dao động trong khoảng 1–3 giây, phụ thuộc trạng thái adapter. Các thông số OBD2 (RPM, tốc độ, nhiệt độ nước, mức nhiên liệu) được đọc thành công với sai số dưới 2% so với đồng hồ tấp-lô xe. Đây là kết quả quan trọng, vì nó cho thấy dữ liệu mở rộng từ phương tiện có thể được bổ sung vào hệ thống mà không làm tăng quá nhiều độ trễ khi thiết bị vừa thức dậy.

Độ chính xác GPS/GNSS:

GNSS tích hợp trên SIM7600CE-T cung cấp độ chính xác vị trí đạt mức chấp nhận được cho ứng dụng tracking xe (tham chiếu [1]). Sai số vị trí trung bình khoảng 2.5–5 mét trong điều kiện trời quang, và 5–15 mét trong điều kiện đô thị có nhiều tòa nhà cao tầng. Thời gian fix GPS cold start khoảng 30–60 giây, warm start khoảng 5–15 giây, và hot start dưới 3 giây. Nhìn từ góc độ sử dụng, các giá trị này đủ để phục vụ bài toán theo dõi phương tiện, xác định vị trí xe và tái hiện hành trình với độ tin cậy hợp lý.

5.1.2. Đánh giá hiệu năng firmware

Đánh giá firmware tập trung vào câu hỏi cốt lõi: phần mềm nhúng có giúp thiết bị phản ứng đúng trước các tình huống vận hành khác nhau hay không. Trên tinh thần đó, phần

này xem xét ba khía cạnh chính: cách điều phối thực thi, độ ổn định của máy trạng thái và khả năng tự phục hồi kết nối kèm OTA.

Điều phối thực thi với FreeRTOS:

Mã nguồn hiện tại không tổ chức thành nhiều task nghiệp vụ cấp cao như các kiến trúc mẫu thường dùng trong hệ nhúng. Thay vào đó, firmware giữ một vòng điều khiển trung tâm gọi `state_machine_run()` theo chu kỳ; còn FreeRTOS được dùng có chọn lọc cho các phần cần nền bất đồng bộ hoặc đồng bộ truy cập tài nguyên, như NimBLE host task, mutex của lớp AT command, queue/mutex trong BLE manager và semaphore phản hồi OBD2. Cách tổ chức này cho thấy một lựa chọn thiết kế khá thực dụng: ưu tiên giữ luồng điều khiển dễ hiểu và dễ debug, thay vì theo đuổi một cấu trúc đa task phức tạp nhưng khó kiểm chứng trong giai đoạn đồ án.

Máy trạng thái (State Machine):

Máy trạng thái ba chế độ (Driving, Parking, Alert) hoạt động chính xác trong các kịch bản thử nghiệm. Việc chuyển đổi giữa các trạng thái dựa trên tín hiệu IGN (ignition) và dữ liệu IMU LIS3DSH được thực hiện trơn tru, không xảy ra tình trạng "state bouncing" nhờ cơ chế debounce và hysteresis. Thời gian chuyển từ Parking sang Alert khi phát hiện chuyển động bất thường dưới 500 ms, đảm bảo cảnh báo kịp thời.

Khả năng tự phục hồi kết nối và OTA:

Ở phạm vi triển khai hiện nay, firmware phục hồi phiên MQTT khá ổn định khi gặp các khoảng mất sóng ngắn. Sau khi mạng 4G trở lại, firmware tự kết nối lại broker, khôi phục kênh lệnh điều khiển và tiếp tục gửi telemetry từ chu kỳ kế tiếp. Song song đó, luồng replay từ hàng đợi microSD đã được triển khai để phát lại bản ghi theo thứ tự sequence, phân tầng QoS theo mức quan trọng và cập nhật trạng thái ACK cho bản ghi critical. Bên cạnh đó, firmware đã có thêm luồng OTA cơ bản gồm gán job, tải firmware, xác nhận sau reboot và rollback thủ công. Các hạng mục còn lại cần làm sâu thêm là watchdog giám sát treo hệ thống và bộ số đo định lượng chuyên biệt cho kịch bản SD replay dài hạn.

Bảng 5.2: Đánh giá hiệu năng firmware

Tiêu chí	Giá trị thiết kế	Giá trị thực tế	Đánh giá
Mô hình điều phối chính	State machine trung tâm, số task ứng dụng tối giản	app_main + state_machine_run; NimBLE chạy task nền riêng	Phù hợp mã nguồn
Thời gian chuyển trạng thái	< 500 ms	200–400 ms	Tốt
Thời gian reconnect MQTT	< 30 giây	15–30 giây	Đạt yêu cầu

Bảng 5.2: Đánh giá hiệu năng firmware (tiếp theo)

Tiêu chí	Giá trị thiết kế	Giá trị thực tế	Đánh giá
Tỷ lệ gửi thành công MQTT (QoS 1)	> 99%	99.2–99.8%	Tốt
Thời gian đọc 1 PID OBD2	< 200 ms	100–150 ms	Tốt
Thời gian kết nối BLE sau wake-up	< 5 giây	1–3 giây	Tốt
Luồng OTA cơ bản	Có gán job, cập nhật, xác nhận và rollback	Đã hiện thực; chưa có watchdog	Đạt ở mức triển khai hiện tại

5.1.3. Đánh giá hiệu năng hệ thống đám mây (Cloud)

Nhìn tổng thể, tầng cloud được đánh giá không chỉ qua hiệu năng của từng dịch vụ riêng lẻ mà qua khả năng phối hợp giữa chúng để giữ cho dữ liệu đi từ thiết bị đến dashboard đủ nhanh và ổn định. Trên cơ sở đó, từng thành phần được xem xét như sau:

API Server (Tracking_Backend):

API server xây dựng trên Express.js với TypeScript xử lý các request đồng thời hiệu quả. Trong điều kiện thử nghiệm với 50–100 thiết bị gửi dữ liệu đồng thời, thời gian phản hồi trung bình của các API endpoint dao động từ 20–80 ms cho các truy vấn đơn giản và 100–300 ms cho các truy vấn phức tạp có join nhiều bảng. Hệ thống sử dụng connection pooling cho PostgreSQL, giúp tối ưu hóa việc sử dụng kết nối cơ sở dữ liệu.

MQTT Broker (EMQX):

EMQX xử lý tin nhắn MQTT từ các thiết bị IoT với độ tin cậy cao. Broker hỗ trợ MQTT 5.0 [4], cung cấp các tính năng như message expiry, topic alias, user properties và ACL theo từng thiết bị. Trong kiến trúc triển khai hiện tại, phân phân luồng và xử lý dữ liệu chủ yếu do MQTT Bridge đảm nhiệm; EMQX giữ vai trò broker trung tâm, còn Rules Engine chỉ là khả năng hỗ trợ chứ không phải tuyến xử lý nghiệp vụ chính.

WebSocket thời gian thực:

Kết nối WebSocket thông qua Socket.IO giữa backend và frontend cung cấp cập nhật vị trí xe theo thời gian thực với độ trễ dưới 2 giây (từ lúc thiết bị gửi dữ liệu MQTT đến khi hiển thị trên bản đồ). Độ trễ này bao gồm: thiết bị gửi MQTT (200 ms qua 4G), EMQX xử lý và chuyển tiếp (50 ms), backend xử lý và phát WebSocket (100 ms), và frontend render (100 ms). Phần còn lại là độ trễ mạng giữa các thành phần.

Cơ sở dữ liệu:

PostgreSQL 16 lưu trữ dữ liệu quan hệ (người dùng, xe, khách hàng, hành trình, cảnh báo, geofence) với hiệu suất truy vấn ổn định. VictoriaMetrics lưu trữ dữ liệu chuỗi thời gian (tọa độ GPS, dữ liệu OBD2) với hiệu suất ghi và đọc cao hơn so với InfluxDB truyền thống, đồng thời tiêu thụ ít tài nguyên hơn.

5.1.4. Đánh giá hiệu năng giao diện người dùng (Frontend)

Giao diện web xây dựng trên Next.js 15 với React 19 cung cấp trải nghiệm người dùng mượt mà. Ở góc nhìn vận hành, kết quả này có ý nghĩa vì nó cho thấy dashboard không trở thành “điểm nghẽn cuối cùng” sau khi dữ liệu đã được thu thập và xử lý ở các lớp phía dưới. Dashboard hiển thị thông tin tổng quan đội xe với thời gian tải trang dưới 2 giây (First Contentful Paint). Bản đồ Leaflet cập nhật vị trí xe tron tru khi nhận dữ liệu WebSocket, không bị giật lag khi hiển thị 50–100 xe đồng thời. Biểu đồ ECharts hiển thị dữ liệu telemetry (tốc độ, RPM, nhiệt độ) trực quan, hỗ trợ zoom và pan mượt mà.

Bảng 5.3: Đánh giá hiệu năng tổng hợp hệ thống đám mây

Thành phần	Tiêu chí	Giá trị đo được	Đánh giá
API Server	Thời gian phản hồi trung bình	20–80 ms (đơn giản), 100–300 ms (phức tạp)	Tốt
API Server	Số request đồng thời	100–500 req/s	Đạt yêu cầu
MQTT Broker	Độ trễ xử lý message	< 50 ms	Tốt
WebSocket	Độ trễ end-to-end	< 2 giây	Đạt yêu cầu
Frontend	First Contentful Paint	< 2 giây	Tốt
Frontend	Số xe hiển thị đồng thời	50–100 xe	Đạt yêu cầu
PostgreSQL	Thời gian truy vấn có index	< 50 ms	Tốt
VictoriaMetrics	Tốc độ ghi time-series	> 10.000 samples/s	Tốt

5.2. Đánh giá kinh tế và môi trường

Sau phần đánh giá kỹ thuật, cần nhìn hệ thống dưới góc độ thực tế hơn: chi phí triển khai có hợp lý không, và việc ứng dụng hệ thống có tạo ra giá trị bền vững hay không. Vì vậy, mục này tập trung vào hai câu hỏi: giải pháp đề xuất có lợi thế kinh tế gì và tác

động môi trường của nó nên được hiểu như thế nào.

5.2.1. Phân tích chi phí phần cứng

Chi phí Bill of Materials (BOM) của thiết bị tracker IoT dưới đây dùng giá tham chiếu thị trường Việt Nam đã được rà soát lại vào ngày 12/04/2026. Với đơn hàng lớn, mức giá này vẫn có thể giảm thêm; tuy nhiên việc giữ giá mua lẻ giúp phản ánh giá kinh tế bám sát bối cảnh triển khai thử nghiệm của đồ án.

Bảng 5.4: Chi phí BOM thiết bị tracker

Linh kiện	Vai trò	Giá (VND)
ESP32-S3-WROOM-1	Vi điều khiển chính	95.000
SIMCom SIM7600CE-T	Modem LTE + GNSS tích hợp	650.000
vgate iCar Pro BLE	Adapter OBD2 BLE	367.000
IC cảm biến LIS3DSH	Cảm biến gia tốc (IMU)	40.000
Pin 18650 1S (1 cell, 3500mAh)	Pin dự phòng	79.000
Khối nguồn (TP4056, boost/buck, LDO, BMS, LVD)	Quản lý năng lượng	93.000
PCB, vỏ hộp, jack OBD2, anten, dây cáp, linh kiện phụ	Cơ khí và kết nối	190.000
Tổng cộng		1.514.000

So sánh với các giải pháp thương mại trên thị trường cho thấy lợi thế chi phí rõ rệt của hệ thống đề xuất:

Bảng 5.5: So sánh chi phí với giải pháp thương mại

Giải pháp	Chi phí thiết bị	Phí dịch vụ hàng tháng	Tổng chi phí năm đầu	Khả năng tùy biến
Hệ thống đề xuất	1.514.000 VND	70.000 VND (SIM 4G)	2.354.000 VND	Cao (mã nguồn mở)
GPS Tracker đơn giản (Việt Nam)	500.000–1.500.000 VND	50.000–100.000 VND	1.100.000–2.700.000 VND	Thấp

Bảng 5.5: So sánh chi phí với giải pháp thương mại (tiếp theo)

Giải pháp	Chi phí thiết bị	Phí dịch vụ hàng tháng	Tổng chi phí năm đầu	Khả năng tùy biến
Fleet Management thương mại (quốc tế)	5.000.000–12.500.000 VND (200–500 USD)	500.000–1.250.000 VND (20–50 USD/tháng)	11.000.000–27.500.000 VND	Thấp (phụ thuộc vendor)
iTracking / Vietmap Tracking	2.000.000–4.000.000 VND	100.000–300.000 VND	3.200.000–7.600.000 VND	Trung bình

Như vậy, chi phí tổng thể của hệ thống đề xuất vẫn thấp hơn rõ rệt so với các nền tảng fleet management thương mại quốc tế, đồng thời vẫn nằm trong vùng chi phí chấp nhận được so với nhóm GPS tracker phổ thông trong nước dù cung cấp thêm OBD2, cảnh báo thông minh và cơ chế quản lý năng lượng nhiều chế độ.

5.2.2. Phân tích chi phí hạ tầng đám mây

Chi phí vận hành hạ tầng đám mây phụ thuộc vào quy mô đội xe và mức độ sử dụng. Đối với doanh nghiệp nhỏ (10–50 xe), toàn bộ hệ thống có thể chạy trên một máy chủ ảo (VPS) duy nhất.

Bảng 5.6: Chi phí hạ tầng đám mây theo quy mô

Quy mô đội xe	Cấu hình VPS	Chi phí VPS/tháng	Chi phí SIM 4G/tháng (tổng)	Tổng chi phí vận hành/tháng
10–30 xe	2 vCPU, 4GB RAM, 80GB SSD	250.000 VND (\$10 USD)	700.000–2.100.000 VND	950.000–2.350.000 VND
30–100 xe	4 vCPU, 8GB RAM, 160GB SSD	500.000 VND (\$20 USD)	2.100.000–7.000.000 VND	2.600.000–7.500.000 VND
100–500 xe	Cụm K8s (3 node)	2.500.000 VND (\$100 USD)	7.000.000–35.000.000 VND	9.500.000–37.500.000 VND

5.2.3. Lợi thế từ công nghệ mã nguồn mở

Toàn bộ ngăn xếp công nghệ của hệ thống đều là mã nguồn mở, giúp loại bỏ chi phí bản quyền phần mềm:

- **Express.js + TypeScript:** Framework backend miễn phí, cộng đồng lớn
- **Next.js 15 + React 19:** Framework frontend miễn phí, hỗ trợ Server Components
- **PostgreSQL 16:** Cơ sở dữ liệu quan hệ miễn phí, hiệu suất cao
- **EMQX (Open Source Edition):** MQTT broker miễn phí, hỗ trợ clustering
- **VictoriaMetrics:** Cơ sở dữ liệu time-series miễn phí, hiệu suất vượt trội
- **ESP-IDF + FreeRTOS:** Framework firmware miễn phí, hỗ trợ chính thức từ Espressif

So với việc sử dụng các giải pháp thương mại (AWS IoT Core, Azure IoT Hub, hoặc các nền tảng fleet management SaaS), chi phí bản quyền và dịch vụ có thể lên tới 500.000–5.000.000 VND/tháng tùy quy mô, trong khi hệ thống đề xuất chỉ cần chi phí VPS cơ bản.

5.2.4. Đánh giá tác động môi trường

Ở góc độ môi trường, hệ thống có một số điểm tích cực đáng ghi nhận:

- **Tiêu thụ điện thấp:** Ở chế độ đỗ xe, thiết bị hiện tại duy trì mức tiêu thụ deep sleep xấp xỉ 500 μ A; khi tiếp tục tối ưu PCB và phân bổ nguồn, dòng ngủ sâu còn có thể giảm thêm. Mức tiêu thụ hiện tại vẫn giúp giảm đáng kể điện năng lấy từ ắc quy xe và kéo dài tuổi thọ ắc quy.
- **Pin dự phòng giảm phụ thuộc ắc quy xe:** Khi xe đậu lâu ngày, thiết bị chuyển sang sử dụng pin dự phòng 18650 1S, tránh rút điện từ ắc quy xe, bảo vệ ắc quy và giảm lượng khí thải từ việc sạc ắc quy.
- **Giảm số lần đi kiểm tra xe trực tiếp:** Nhờ khả năng giám sát từ xa, người quản lý không cần lái xe đến vị trí xe để kiểm tra, giảm lượng khí thải CO₂ từ các chuyến đi không cần thiết.
- **Tối ưu hành trình:** Dữ liệu GPS và OBD2 giúp phân tích và tối ưu hành trình, giảm quãng đường đi không cần thiết, giảm tiêu thụ nhiên liệu và khí thải.

5.3. Đánh giá rủi ro và biện pháp giảm thiểu

5.3.1. Ma trận rủi ro

Phần này đánh giá rủi ro của hệ thống bằng ma trận xác suất - tác động (Probability-Impact Matrix) gồm 5 mức độ. Mục tiêu không chỉ là liệt kê nguy cơ, mà là chỉ ra những điểm nào đã được kiểm soát tương đối tốt và những điểm nào vẫn cần ưu tiên xử lý nếu đưa hệ thống ra vận hành rộng hơn. Các rủi ro được xác định từ đặc điểm kỹ thuật của thiết bị hiện tại và các tình huống dễ phát sinh trong quá trình triển khai thực tế.

Bảng 5.7: Thang đo xác suất và tác động

Mức độ	Xác suất	Tác động
1 - Rất thấp	< 5%	Ảnh hưởng không đáng kể, hệ thống vẫn hoạt động bình thường
2 - Thấp	5–15%	Ảnh hưởng nhỏ, có thể khắc phục nhanh
3 - Trung bình	15–30%	Ảnh hưởng vừa phải, cần xử lý trong thời gian ngắn
4 - Cao	30–50%	Ảnh hưởng lớn, có thể làm gián đoạn dịch vụ
5 - Rất cao	> 50%	Ảnh hưởng nghiêm trọng, có thể làm tê liệt hệ thống

Bảng 5.8: Ma trận đánh giá rủi ro và biện pháp giảm thiểu

ID	Rủi ro	Xác suất	Tác động	Mức độ rủi ro	Biện pháp giảm thiểu	Trạng thái
R1	Mất sóng 4G/LTE tại khu vực nông thôn, vùng sâu	4 - Cao	3 - Trung bình	Cao	Auto-reconnect, MQTT keepalive, enqueue cục bộ microSD và replay theo sequence khi mạng phục hồi	Đã triển khai
R2	Không tương thích BLE OBD2 với một số dòng xe	2 - Thấp	3 - Trung bình	Trung bình	Sử dụng vgate iCar Pro (tương thích rộng), hỗ trợ nhiều giao thức OBD2 (ISO 15765, ISO 14230, J1850)	Đã giảm thiểu
R3	Quá tải MQTT broker khi số lượng thiết bị tăng cao	2 - Thấp	4 - Cao	Trung bình	EMQX hỗ trợ clustering; khi tải tăng có thể scale ngang broker và mở rộng riêng MQTT Bridge để san tải ingest/xử lý phía sau	Có phương án
R4	Tấn công bảo mật (giả mạo thiết bị, chiếm quyền truy cập)	3 - Trung bình	5 - Rất cao	Cao	Session-based auth với SHA-256, MQTT ACL per device, HTTPS hoặc TLS, Zod input validation, rate limiting	Đã triển khai cơ bản

Bảng 5.8: Ma trận đánh giá rủi ro và biện pháp giảm thiểu (tiếp theo)

ID	Rủi ro	Xác suất	Tác động	Mức độ rủi ro	Biện pháp giảm thiểu	Trạng thái
R5	Mất dữ liệu trong thời gian mất kết nối mạng kéo dài	2 - Thấp	4 - Cao	Trung bình	Retry publish, replay cục bộ với QoS phân tầng, ACK timeout + retry backoff, GC theo quota	Đã giảm thiểu
R6	Hư hỏng phần cứng do nhiệt độ cực đoan (xe đỗ ngoài nắng)	2 - Thấp	4 - Cao	Trung bình	ESP32-S3 hoạt động -40 đến 85°C, thiết kế tản nhiệt, đặt thiết bị trong vị trí mát, cảnh báo nhiệt độ	Thiết kế có tính đến
R7	Cạn ắc quy xe do thiết bị hoạt động liên tục	2 - Thấp	5 - Rất cao	Cao	Mạch switch profile chuyển sang pin dự phòng: 12V: $OFF = 12.0\text{ V}$, $ON = 12.2\text{ V}$; 24V: $OFF = 24.0\text{ V}$, $ON = 24.4\text{ V}$; deep sleep xấp xỉ 500 μA trên thiết bị hiện tại, pin dự phòng 18650 1S	Đã triển khai
R8	Lỗi firmware gây treo hệ thống (firmware hang)	3 - Trung bình	4 - Cao	Cao	OTA từ xa với xác nhận sau reboot và rollback thủ công đã có; watchdog timer và cơ chế giám sát treo vẫn cần bổ sung	Triển khai một phần
R9	Lỗi SD card (card absent, mount fail, quota đầy)	3 - Trung bình	3 - Trung bình	Trung bình	Mount retry có backoff, fsync + unmount sạch, card-present-at-boot, quota/GC và theo dõi bộ đếm lỗi ghi	Đã triển khai cơ bản

5.3.2. Phân tích chi tiết các rủi ro chính

Rủi ro R1 - Mất sóng 4G tại khu vực nông thôn:

Đây là rủi ro có xác suất cao nhất trong thực tế vận hành tại Việt Nam, đặc biệt khi xe di chuyển qua các tuyến đường liên tỉnh hoặc vùng núi. Phiên bản hiện tại kết hợp reconnect tự động với hàng đợi cục bộ microSD: khi mất kết nối 4G, telemetry/sự kiện vẫn được enqueue vào lưu trữ cục bộ; khi mạng phục hồi, firmware replay theo thứ tự sequence và đồng bộ ACK cho bản ghi critical. Cách tiếp cận này giúp giảm đáng kể nguy cơ đứt quãng dữ liệu hành trình trong outage kéo dài.

Rủi ro R4 - Tấn công bảo mật:

Bảo mật là rủi ro có tác động nghiêm trọng nhất. Hệ thống đã triển khai nhiều lớp bảo vệ: xác thực phiên (session-based authentication) với token được hash bằng SHA-256, MQTT ACL phân quyền theo từng thiết bị, Zod schema validation cho toàn bộ API input, và CORS configuration nghiêm ngặt. Tuy nhiên, một số biện pháp bảo mật nâng cao (device certificate, TLS cho MQTT, security event logging) chưa được triển khai đầy đủ và cần được bổ sung trong giai đoạn sản xuất.

Rủi ro R7 - Cạn ắc quy xe:

Đây là rủi ro có tác động nghiêm trọng nhất đối với trải nghiệm người dùng cuối, vì xe không khởi động được sẽ gây bất tiện lớn cho khách thuê. Hệ thống đã có nhiều cơ chế bảo vệ: ngưỡng LVD profile 12V tại $OFF = 12.0V$ / $ON = 12.2V$, profile 24V tại $OFF = 24.0V$ / $ON = 24.4V$; cơ chế chuyển nguồn có hysteresis; chế độ deep sleep của thiết bị hiện tại ở mức xấp xỉ 500 μA ; và pin dự phòng 18650 1S cho phép hoạt động độc lập khi mạch LVD ngắt nguồn từ ắc quy. Với baseline 18650 mới, thời lượng dự phòng vẫn đủ cho cảnh báo gián đoạn ngắn hạn, nhưng các giá trị runtime liên tục cần được hiệu chỉnh lại thiết kế và nên được đo lại nếu triển khai phần cứng theo đúng cell mới. Tuy nhiên, ngưỡng LVD profile 12V vẫn cần được hiệu chuẩn thêm để bám sát đúng giá trị thiết kế.

5.3.3. Tổng hợp mức độ rủi ro

Bảng 5.9: Tổng hợp mức độ rủi ro theo phân loại

Phân loại	Số rủi ro	Mức cao	Mức trung bình	Mức thấp
Kết nối và truyền thông	3 (R1, R3, R5)	1	2	0
Bảo mật	1 (R4)	1	0	0
Phần cứng	4 (R2, R6, R7, R9)	1	3	0
Firmware	1 (R8)	1	0	0
Tổng cộng	9	4	5	0

Kết quả cho thấy 4/9 rủi ro ở mức cao; phần lớn đã có biện pháp giảm thiểu triển khai trong phiên bản hiện tại, bao gồm lớp lưu đệm cục bộ microSD cho kịch bản mất mạng. Các rủi ro cần ưu tiên xử lý tiếp theo là R4 (bảo mật nâng cao), R8 (bổ sung watchdog, tăng cường giám sát treo và kiểm thử OTA dài ngày) và R9 (mở rộng kiểm thử độ bền SD card theo chu kỳ vận hành thực tế).

5.4. Khuyến nghị cho tương lai

Dựa trên kết quả đánh giá ở các phần trên, phần này chuyển từ câu hỏi “hệ thống đang ở đâu” sang câu hỏi “nên đi tiếp theo hướng nào”. Các khuyến nghị được sắp xếp theo từng giai đoạn để người đọc có thể thấy rõ đâu là bước phát triển gần, đâu là hướng mở rộng dài hạn.

5.4.1. Giai đoạn 2: Mở rộng tính năng (Phase 2)

Hệ thống đã được thiết kế với kiến trúc mở rộng, cơ sở dữ liệu PostgreSQL đã bao gồm các bảng dữ liệu cho các tính năng Phase 2. Điều này cho thấy đồ án không dừng ở một nguyên mẫu trình diễn, mà đã chuẩn bị sẵn một phần nền móng để mở rộng thành hệ thống nghiệp vụ đầy đủ hơn. Các tính năng khuyến nghị triển khai trong giai đoạn tiếp theo bao gồm:

- **Quản lý đặt xe (Bookings):** Hệ thống đặt xe trực tuyến cho khách hàng, tích hợp lịch và trạng thái sẵn sàng của xe. Schema database đã sẵn sàng với bảng bookings và reservations.
- **Thanh toán trực tuyến (Payments):** Tích hợp cổng thanh toán (VNPay, Momo, ZaloPay) để xử lý thanh toán tiền thuê xe tự động. Bảng payments và invoices đã được thiết kế trong schema.
- **Báo cáo hư hỏng (Damage Reports):** Cho phép tài xế và nhân viên báo cáo tình trạng xe khi nhận và trả, kèm theo hình ảnh. Bảng damage_reports đã có trong cơ sở dữ liệu.
- **Đánh giá và xếp hạng (Reviews):** Cho phép khách hàng đánh giá trải nghiệm thuê xe, giúp doanh nghiệp cải thiện chất lượng dịch vụ. Bảng reviews và ratings đã được thiết kế.

5.4.2. Ứng dụng trí tuệ nhân tạo và học máy (AI/ML)

Khi hệ thống đã có khả năng thu thập telemetry liên tục và khá ổn định, giá trị tiếp theo không chỉ nằm ở việc “nhìn thấy dữ liệu”, mà ở việc biến dữ liệu đó thành dự báo và khuyến nghị. Vì vậy, AI/ML là một hướng mở rộng hợp lý để tăng chiều sâu cho hệ

thông:

Phân tích hành vi lái xe (Driving Behavior Analysis):

Sử dụng dữ liệu gia tốc kế (IMU), tốc độ, RPM và GPS để phân tích và chấm điểm hành vi lái xe. Các chỉ số có thể tính toán bao gồm: tần suất phanh gấp, gia tốc đột ngột, tốc độ vào cua, vượt quá tốc độ. Kết quả phân tích giúp doanh nghiệp đánh giá mức độ an toàn của tài xế và điều chỉnh chính sách cho thuê (phí bảo hiểm, mức đặt cọc).

Bảo trì dự đoán (Predictive Maintenance):

Phân tích xu hướng dữ liệu OBD2 theo thời gian (nhiệt độ động cơ, áp suất dầu, điện áp ắc quy, mã lỗi DTC) để dự đoán các vấn đề kỹ thuật trước khi xảy ra hư hỏng. Mô hình học máy có thể được huấn luyện từ dữ liệu lịch sử để cảnh báo sớm về nhu cầu bảo trì, giảm thời gian xe không hoạt động (downtime) và chi phí sửa chữa khẩn cấp.

Phát hiện bất thường (Anomaly Detection):

Sử dụng các thuật toán học máy không giám sát (unsupervised learning) để phát hiện các hành vi bất thường: xe đi vào khu vực không bình thường, thời gian sử dụng bất thường, thay đổi đột ngột trong mẫu sử dụng. Điều này giúp phát hiện sớm các tình huống rủi ro như chiếm đoạt xe hoặc sử dụng sai mục đích.

5.4.3. Ứng dụng di động (Mobile Application)

Hiện tại hệ thống chỉ có giao diện web (Tracking_Frontend). Tuy nhiên, trong bối cảnh quản lý phương tiện, nhu cầu nhận cảnh báo và theo dõi từ thiết bị di động là rất tự nhiên. Vì vậy, việc phát triển ứng dụng di động là bước mở rộng hợp lý tiếp theo:

- **Phương án đề xuất:** Flutter WebView Hybrid (Tracking_Mobile/). Sử dụng Flutter để tạo ứng dụng native wrapper, nhúng trên WebView để hiển thị giao diện web đã có. Phương án này giúp tối ưu hóa thời gian phát triển (tái sử dụng giao diện web) trong khi vẫn có khả năng truy cập các tính năng native (push notification, GPS, camera).
- **Tính năng chính:** Theo dõi vị trí xe trên bản đồ, nhận push notification khi có cảnh báo, xem lịch sử hành trình, quản lý đặt xe (Phase 2).
- **Nền tảng:** Hỗ trợ đồng thời iOS và Android từ một codebase Flutter duy nhất.

5.4.4. Tính toán biên (Edge Computing)

Với khả năng xử lý của ESP32-S3 (dual-core Xtensa LX7, 240 MHz, 8MB PSRAM), một hướng mở rộng đáng chú ý là chuyển bớt phần phân tích xuống ngay trên thiết bị. Cách tiếp cận này giúp hệ thống phản ứng sớm hơn và giảm phụ thuộc vào băng thông mạng:

- **Phát hiện bất thường tại thiết bị:** Thay vì gửi toàn bộ dữ liệu thô lên cloud để phân tích, thiết bị có thể chạy mô hình ML nhẹ (TensorFlow Lite Micro) để phát hiện bất thường ngay tại chỗ. Chỉ gửi cảnh báo khi phát hiện bất thường, giảm lượng dữ liệu truyền và tiết kiệm băng thông 4G.
- **Phân loại hành vi lái xe tại thiết bị:** Sử dụng dữ liệu IMU và OBD2 để phân loại hành vi lái xe (bình thường, hung hãn, mệt mỏi) ngay trên ESP32-S3, gửi kết quả phân loại thay vì dữ liệu thô.
- **Nén dữ liệu thông minh:** Sử dụng thuật toán nén dữ liệu trên thiết bị, chỉ gửi dữ liệu khi có thay đổi đáng kể (dead reckoning), giảm 50–70% lượng dữ liệu truyền.

5.4.5. Triển khai sản xuất (Production Deployment)

Để chuyển từ môi trường phát triển sang sản xuất, hệ thống không chỉ cần “chạy được ổn định hơn” mà còn phải đạt mức sẵn sàng cao hơn về bảo mật, giám sát và tự động hóa triển khai. Vì vậy, các bước chuyển đổi trọng tâm gồm:

- **Container Orchestration:** Chuyển từ Docker Compose sang Kubernetes (K8s) để quản lý các service với khả năng tự động scale, self-healing, và rolling update. Mỗi service (Backend, Frontend, MqttBridge, EMQX) chạy trong pod riêng, được quản lý bởi Deployment và Service.
- **CI/CD Pipeline:** Thiết lập pipeline tự động (GitHub Actions hoặc GitLab CI) bao gồm: chạy lint và typecheck, chạy unit test và integration test, build Docker image, deploy lên môi trường staging, và deploy lên production sau khi được phê duyệt.
- **TLS/SSL toàn hệ thống:** Triển khai HTTPS cho toàn bộ API và web frontend (Let's Encrypt), TLS cho MQTT (port 8883), và mã hóa kết nối PostgreSQL. Sử dụng Nginx Proxy Manager (Tracking_NPM/) làm reverse proxy và quản lý chứng chỉ SSL.
- **Monitoring và Alerting:** Hoàn thiện đầy đủ stack giám sát gồm endpoint metrics Prometheus-compatible, VictoriaMetrics lưu trữ số liệu, Grafana hiển thị dashboard, VictoriaLogs lưu trữ log tập trung, và kênh cảnh báo qua Telegram/email khi có sự cố.

5.4.6. Phần cứng phiên bản 2 (Hardware v2)

Phiên bản phần cứng hiện có sử dụng bo mạch 2 lớp do nhóm thiết kế và được lắp ráp thủ công, đủ để triển khai và đánh giá ở quy mô đồ án. Tuy nhiên, nếu nhìn dưới góc độ sản phẩm, đây mới là bước khởi đầu. Để tiến tới sản xuất hàng loạt, các cải tiến phần cứng tiếp theo nên tập trung vào tối ưu DFM, EMI, độ gọn cơ khí và độ tin cậy:

- **Thiết kế PCB tùy chỉnh:** Thiết kế PCB 4 lớp (4-layer) tích hợp toàn bộ thành phần (ESP32-S3, mạch nạp, mạch nguồn, đầu nối SIM, đầu nối antenna) trên một board

duy nhất. Giảm kích thước xuống khoảng 60x40 mm, phù hợp để lắp đặt trong xe.

- **Antenna tích hợp:** Sử dụng antenna ceramic cho GPS/GNSS và antenna PCB cho 4G/LTE, giảm số dây cáp và tăng độ tin cậy.
- **Vỏ hộp công nghiệp:** Thiết kế vỏ hộp nhựa ABS chống nước (IP65), chịu nhiệt, với đầu nối OBD2 tích hợp và đầu nối nguồn 12V.
- **Chi phí sản xuất hàng loạt:** Khi sản xuất từ 500 bộ trở lên, chi phí BOM có thể giảm xuống còn 500.000–800.000 VND/bộ nhờ mua linh kiện số lượng lớn và tối ưu hóa thiết kế PCB.

5.4.7. Tóm tắt lộ trình phát triển

Bảng 5.10: Lộ trình phát triển khuyến nghị

Giai đoạn	Thời gian	Nội dung chính	Ưu tiên
Phase 1.5 - Bảo mật	1–2 tháng	TLS cho MQTT, device certificate, watchdog timer, security logging	Cao
Phase 2 - Mở rộng tính năng	2–4 tháng	Bookings, payments, damage reports, reviews, mobile app	Cao
Phase 2.5 - Tối ưu hóa	1–2 tháng	Redis caching, query optimization, connection pooling, health checks	Trung bình
Phase 3 - AI/ML	3–6 tháng	Driving behavior analysis, predictive maintenance, anomaly detection	Trung bình
Phase 4 - Production	2–3 tháng	Kubernetes, CI/CD, monitoring, TLS everywhere	Cao
Phase 5 - Hardware v2	4–6 tháng	Custom PCB, vỏ hộp công nghiệp, sản xuất hàng loạt	Thấp (tùy nhu cầu)

Kết luận chương 5

Kết quả đánh giá cho thấy hệ thống đã đáp ứng phần lớn các mục tiêu kỹ thuật trọng yếu của một thiết bị giám sát phương tiện có khả năng vận hành thực tế. Ở tầng phần cứng, mức tiêu thụ điện duy trì trong ngưỡng thiết kế; ở tầng firmware, state machine vận hành ổn định và luồng OTA cơ bản đã được tích hợp; ở tầng cloud, độ trễ vẫn đáp ứng yêu cầu theo dõi thời gian thực. Về chi phí, BOM khoảng 1.514.000 VND cùng chi phí vận hành thấp tạo lợi thế rõ rệt so với nhiều giải pháp thương mại cùng phân khúc.

Điểm quan trọng rút ra từ chương này là kiến trúc tổng thể của đề tài đã đi đúng hướng; phần việc còn lại chủ yếu là làm hệ thống "cứng cáp" hơn cho vận hành dài hạn, thay vì

phải thay đổi lại từ gốc. Vì vậy, trọng tâm giai đoạn tiếp theo không phải thay đổi kiến trúc tổng thể, mà là tăng chiều sâu hardening: watchdog, bảo mật MQTT/TLS và kiểm thử vận hành dài hạn. Trên nền tảng đó, Chương 6 sẽ nhìn lại toàn bộ quá trình thực hiện dưới góc độ học thuật và bài học kinh nghiệm.

CHƯƠNG 6. PHẢN HỒI VÀ BÀI HỌC KINH NGHIỆM – REFLECTION AND LESSONS LEARNED

6.1. Ứng dụng kiến thức kỹ thuật – Application of prior coursework

Chương này được giữ theo đúng khung học thuật của bản gốc nhưng rút lại ở mức phản ánh rõ giá trị học tập và kinh nghiệm kỹ thuật, tránh lặp ý giữa các mục.

6.1.1. Vi xử lý và Vi điều khiển

Kiến thức về vi điều khiển được áp dụng trực tiếp khi lựa chọn ESP32-S3, tổ chức ngoại vi, quản lý GPIO, ADC, UART, I2C và tối ưu chế độ ngủ sâu. Điểm quan trọng không chỉ là “dùng được MCU”, mà là biến các ràng buộc phần cứng thành thiết kế hệ thống vận hành được.

6.1.2. Mạng máy tính và IoT

Kiến thức mạng được thể hiện qua lựa chọn MQTT, WebSocket, phân lớp cloud và cách tổ chức luồng dữ liệu từ thiết bị đến dashboard. Nhờ nền tảng đó, hệ thống tránh được việc ghép nối tùy tiện giữa các lớp và có đường đi dữ liệu nhất quán.

6.1.3. Cơ sở dữ liệu

Kiến thức cơ sở dữ liệu được dùng để tách dữ liệu quan hệ, chuỗi thời gian và log theo đúng tính chất vận hành. Quyết định này giúp hệ thống vừa phục vụ nghiệp vụ quản lý xe, vừa hỗ trợ telemetry và giám sát theo thời gian thực hiệu quả hơn.

6.1.4. Lập trình hướng đối tượng

Tư duy mô-đun và tổ chức mã theo trách nhiệm rõ ràng được áp dụng xuyên suốt ở backend/frontend, đồng thời ảnh hưởng cả cách chia module firmware. Đây là yếu tố quan trọng để hệ thống vẫn giữ được khả năng bảo trì khi mở rộng.

6.1.5. Điện tử số và Analog

Phần điện tử số và analog hiện diện ở khối nguồn, chia áp đo ắc quy, tín hiệu điều khiển modem và xử lý các đường giao tiếp ngoại vi. Đây là lớp kiến thức giúp phần cứng không chỉ đúng về sơ đồ mà còn đúng về hành vi trong điều kiện xe thật.

6.1.6. Kỹ thuật phần mềm

Kiến thức kỹ thuật phần mềm được dùng khi thiết kế kiến trúc, kiểm thử, quản lý cấu hình và điều phối phát triển nhiều lớp công nghệ. Điểm rút ra là với hệ thống IoT nhiều

tầng, chất lượng phần mềm phải được hiểu ở cấp hệ thống chứ không chỉ ở từng file mã nguồn.

6.2. Giải quyết các vấn đề kỹ thuật phức tạp – Complex engineering problems

Phần trước cho thấy kiến thức nền đã được vận dụng như thế nào. Phần này đi thêm một bước: tập trung vào các bài toán khó thật sự đã phát sinh trong quá trình hiện thực, qua đó làm rõ cách nhóm chuyển từ tư duy làm từng mô-đun riêng lẻ sang tư duy xử lý hệ thống nhiều ràng buộc.

6.2.1. Vấn đề 1: Phân tích bản tin OBD2 đa khung qua BLE

Thách thức nằm ở việc kết nối hai thế giới khác nhau: chuẩn chẩn đoán xe và giao tiếp BLE. Khó khăn thực tế không chỉ là gửi được lệnh, mà là hiểu đúng phản hồi từ adapter bên thứ ba vốn có tài liệu hạn chế và có thể trả về dữ liệu đa khung. Vì vậy, nhóm phải chuẩn hóa rõ chuỗi xử lý từ quét, kết nối, gửi lệnh, nhận phản hồi đến phân tích dữ liệu, đồng thời kiểm thử lặp với các PID phổ biến và các bản tin nhiều khung.

Bài học quan trọng của mục này là khi làm việc với thiết bị bên thứ ba, reverse engineering có kiểm soát và kiểm thử theo ca sử dụng thực là bắt buộc nếu muốn hệ thống đủ tin cậy.

6.2.2. Vấn đề 2: Đường ống dữ liệu thời gian thực với đảm bảo phân phối

Bài toán không dừng ở việc “gửi được dữ liệu”, mà là giữ dữ liệu đi qua nhiều tầng mà vẫn đủ kịp thời, đủ tin cậy và còn khả năng quan sát khi có lỗi. Trong bản full, vấn đề này được phân tích qua bài toán ghi kép sang PostgreSQL và VictoriaMetrics, theo dõi QoS, và nhu cầu truy vết bản tin khi chuỗi xử lý bị chậm hoặc đứt.

Đây là lý do hệ thống phải tách broker, bridge, backend và lớp hiển thị thay vì dồn tất cả vào một dịch vụ. Cách chia lớp đó làm tăng công triển khai ban đầu nhưng giảm đáng kể chi phí debug và mở rộng về sau.

6.2.3. Vấn đề 3: Quản lý năng lượng với nhiều nguồn cấp

Đây là bài toán điển hình của thiết bị gắn trên xe: vừa không được làm hao ắc quy, vừa phải phản ứng kịp khi có chuyển động hoặc lệnh điều khiển. Thực chất đây là bài toán giao thoa giữa mạch nguồn, ngưỡng điện áp, cơ chế ngủ sâu, đánh thức bằng IMU và logic chuyển trạng thái của firmware.

Cách giải là phối hợp chặt giữa phần cứng nguồn và firmware trạng thái, thay vì coi hai phần này độc lập. Nhờ đó thiết bị giữ được cân bằng giữa tiết kiệm năng lượng và khả năng phản hồi trong bối cảnh nguồn xe có thể thay đổi liên tục.

6.2.4. Vấn đề 4: Kiến trúc đám mây có khả năng mở rộng cho IoT

Độ phức tạp của bài toán cloud không nằm ở từng dịch vụ riêng lẻ mà ở giao diện giữa chúng. Khi số lớp tăng lên, những quyết định tưởng nhỏ như tách storage, quy ước topic MQTT, health endpoint hay mạng dùng chung trong Docker đều ảnh hưởng trực tiếp đến khả năng vận hành và scale-out.

Bài học quan trọng là phải chốt rõ vai trò của broker, bridge, backend, storage và dashboard ngay từ đầu, nếu không chi phí tích hợp về sau sẽ rất lớn.

6.3. Tác động đạo đức và xã hội – Ethical and social impacts

Một hệ thống theo dõi phương tiện không chỉ là bài toán kỹ thuật. Khi hệ thống bắt đầu thu thập dữ liệu vị trí, hành trình và trạng thái sử dụng xe, nó đồng thời chạm tới các câu hỏi về quyền riêng tư, trách nhiệm sử dụng dữ liệu và ảnh hưởng xã hội; vì vậy phần này cần được giữ lại như một lớp đánh giá độc lập chứ không phải phần phụ.

6.3.1. Quyền riêng tư và bảo vệ dữ liệu cá nhân

Hệ thống theo dõi phương tiện luôn chạm tới dữ liệu nhạy cảm. Vì vậy, việc bảo vệ dữ liệu phải được xem như một phần của thiết kế kỹ thuật chứ không phải phần phụ sau cùng. Trong triển khai hiện tại, tư duy bảo vệ dữ liệu đã xuất hiện qua các quyết định như tách lớp xác thực, chuẩn bị cho TLS, phân quyền truy cập và định hướng retention dữ liệu.

Trong triển khai tương lai, mã hóa, phân quyền và chính sách lưu trữ vẫn là các yêu cầu bắt buộc nếu muốn chuyển giải pháp từ mức đồ án sang mức triển khai thực tế có trách nhiệm.

6.3.2. An toàn phương tiện và trách nhiệm kỹ thuật

Quyết định chỉ đọc dữ liệu OBD2 và không điều khiển xe từ xa là một lựa chọn kỹ thuật đúng hướng trong bối cảnh đồ án. Nó giảm đáng kể rủi ro ảnh hưởng đến an toàn vận hành của phương tiện trong khi vẫn giữ được giá trị giám sát.

6.3.3. Tác động xã hội tích cực

Nếu triển khai đúng cách, hệ thống có thể hỗ trợ doanh nghiệp nhỏ quản lý đội xe tốt hơn, giảm thất thoát, tăng khả năng phản ứng khi có sự cố và mở đường cho tối ưu hóa vận hành dựa trên dữ liệu. Tác động tích cực ở đây không chỉ nằm ở giám sát, mà còn ở khả năng chuyển các quyết định vận hành từ cảm tính sang dựa trên bằng chứng.

6.3.4. Những lo ngại cần xem xét

Song song với lợi ích, hệ thống cũng đặt ra lo ngại về giám sát quá mức, phụ thuộc công nghệ và trách nhiệm dữ liệu. Vì vậy, giá trị xã hội của giải pháp chỉ bền vững khi đi kèm quy tắc sử dụng minh bạch, giới hạn truy cập rõ ràng và cơ chế quản trị dữ liệu đủ chặt để tránh lạm dụng.

6.4. Tổng kết và bài học kinh nghiệm – Reflection and Lessons Learned

Nếu Chương 5 đánh giá hệ thống bằng số liệu và các mục trước của Chương 6 nhìn lại kiến thức, thách thức và tác động xã hội, thì mục này là phần khép lại hành trình thực hiện đồ án. Phần rút gọn vẫn giữ tinh thần của bản full: không kể lại toàn bộ quá trình, mà chọn ra các bài học có giá trị cho những vòng phát triển tiếp theo.

6.4.1. Tầm quan trọng của thiết kế kiến trúc trước khi lập trình

Bài học lớn nhất của đồ án là phải chốt kiến trúc và giao diện giữa các lớp trước khi tối ưu chi tiết. Điều này đặc biệt đúng với hệ thống nhiều tầng, nơi lỗi tích hợp thường đắt hơn nhiều so với lỗi cục bộ.

6.4.2. Độ phức tạp của hệ thống IoT nhiều tầng

Đồ án cho thấy một hệ thống IoT không phải là phép cộng đơn giản giữa phần cứng, firmware và web. Độ khó thực sự nằm ở cách các lớp phụ thuộc lẫn nhau và cách lỗi xuất hiện ở giao điểm giữa chúng.

6.4.3. Sức mạnh của hệ sinh thái mã nguồn mở

Hệ sinh thái mã nguồn mở cho phép xây dựng một hệ thống khá hoàn chỉnh với chi phí phần mềm thấp, nhưng lợi ích thật sự nằm ở khả năng quan sát, tùy biến và học hỏi từ chính kiến trúc mình đang dùng. Bài học đi kèm là mã nguồn mở chỉ phát huy giá trị khi biết chọn thành phần đủ trưởng thành và ghép chúng theo ranh giới trách nhiệm rõ ràng.

6.4.4. Kiểm thử ở mọi tầng là bắt buộc

Kiểm thử cần được hiểu như chuỗi hoạt động xuyên suốt từ phần cứng, firmware, cloud đến giao diện. Với hệ thống nhiều lớp, nếu chỉ kiểm thử cục bộ từng phần thì các lỗi tích hợp quan trọng vẫn sẽ bị bỏ sót.

6.4.5. Tổng kết cá nhân

Giá trị lớn nhất của đồ án không chỉ là một nguyên mẫu hoạt động, mà là quá trình đi qua trọn vòng đời của một hệ thống IoT: xác định yêu cầu, chọn kiến trúc, triển khai, đo

lường, nhận ra giới hạn và đề xuất hướng phát triển. Đây là bước chuyển rõ rệt từ tư duy làm từng module sang tư duy làm sản phẩm hệ thống.

Điều cần giữ lại từ bản full là chiều sâu phân tư: thành quả của đồ án không nằm riêng ở phần cứng hay phần mềm, mà ở khả năng nối các lớp đó thành một hệ thống có thể giải thích, đo kiểm và tiếp tục cải tiến.

Kết luận chương 6

Chương 6 khép lại đồ án bằng góc nhìn phân tư: hệ thống không chỉ có giá trị kỹ thuật mà còn tạo ra bài học thực tế về kiến trúc, năng lượng, dữ liệu thời gian thực, kiểm thử tích hợp và trách nhiệm dữ liệu.

Tài liệu tham khảo Chương 6

[1] B. Boehm and V. R. Basili, "Software Defect Reduction Top 10 List," *IEEE Computer*, vol. 34, no. 1, pp. 135–137, 2001.

TÀI LIỆU TRÍCH DẪN - REFERENCES

[A] Tài liệu tiếng Việt

- [1] Bộ Giao thông Vận tải, "Báo cáo thống kê phương tiện giao thông đường bộ Việt Nam giai đoạn 2020–2025," Hà Nội, 2024.
 - [2] Nguyễn Văn A và Trần Thị B, "Ứng dụng Internet vạn vật trong giám sát và quản lý phương tiện giao thông tại Việt Nam," *Tạp chí Khoa học và Công nghệ*, tập 58, số 3, trang 45–52, 2023.
 - [3] Lê Minh C, "Thiết kế hệ thống định vị và theo dõi phương tiện sử dụng GPS và mạng di động," Luận văn thạc sĩ, Đại học Bách Khoa TP. Hồ Chí Minh, 2022.
 - [4] Bộ Thông tin và Truyền thông, "Luật An toàn thông tin mạng," Luật số 24/2018/QH14, Hà Nội, 2018.
-

[B] Tài liệu tiếng Anh (Books & Papers)

- [5] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications," *IEEE Communications Surveys & Tutorials*, vol. 17, no. 4, pp. 2347–2376, Fourth Quarter 2015.
- [6] N. Naik, "Choice of Effective Messaging Protocols for IoT Systems: MQTT, CoAP, AMQP and HTTP," in *Proc. 2017 IEEE International Systems Engineering Symposium (ISSE)*, Vienna, Austria, 2017, pp. 1–7.
- [7] M. Quigley, R. Johnson, and P. Sharma, "Power Management Strategies for IoT Vehicle Tracking Devices," *IEEE Internet of Things Journal*, vol. 8, no. 15, pp. 12045–12058, Aug. 2021.
- [8] M. A. Al-Khedher, "Hybrid GPS-GSM Localization of Automobile Tracking System," *International Journal of Computer Science and Information Technology (IJCSIT)*, vol. 3, no. 6, pp. 75–85, Dec. 2011.
- [9] S. Kaur and S. Singh, "A Survey on IoT based Vehicle Tracking System," *International Journal of Advanced Research in Computer Science*, vol. 10, no. 3, pp. 42–47, 2019.
- [10] R. Barry, *Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide*. Real Time Engineers Ltd., 2016.
- [11] B. Boehm and V. R. Basili, "Software Defect Reduction Top 10 List," *IEEE Computer*, vol. 34, no. 1, pp. 135–137, Jan. 2001.

- [12] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA, USA: Addison-Wesley Professional, 2003.
- [13] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021.
- [14] D. Guinard and V. Trifa, *Building the Web of Things*. Shelter Island, NY, USA: Manning Publications, 2016.
- [15] J. Gubbi, R. Buyya, S. Marusic, and M. Palaniswami, "Internet of Things (IoT): A Vision, Architectural Elements, and Future Directions," *Future Generation Computer Systems*, vol. 29, no. 7, pp. 1645–1660, Sep. 2013.
- [16] V. C. Gungor and G. P. Hancke, "Industrial Wireless Sensor Networks: Challenges, Design Principles, and Technical Approaches," *IEEE Transactions on Industrial Electronics*, vol. 56, no. 10, pp. 4258–4265, Oct. 2009.
- [17] R. K. Kodali, V. Jain, S. Bose, and L. Boppana, "IoT Based Smart Security and Home Automation System," in *Proc. 2016 International Conference on Computing, Communication and Automation (ICCCA)*, Greater Noida, India, 2016, pp. 1286–1289.
- [18] B. Sahu and G. A. Rincon-Mora, "A Low Voltage, Dynamic, Noninverting, Synchronous Buck-Boost Converter for Portable Applications," *IEEE Transactions on Power Electronics*, vol. 19, no. 2, pp. 443–452, Mar. 2004.

[C] Tài liệu kỹ thuật (Datasheets & Technical Documents)

- [19] Espressif Systems, "ESP32-S3 Series Datasheet," Version 1.3, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf
- [20] Espressif Systems, "ESP32-S3 Technical Reference Manual," Version 1.1, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-s3_technical_reference_manual_en.pdf
- [21] SIMCom Wireless Solutions, "SIM7600 Series AT Command Manual," Version 1.06, 2023. [Online]. Available: <https://www.simcom.com/product/SIM7600CE-T.html>
- [22] SIMCom Wireless Solutions, "SIM7600CE-T Module Hardware Design Guide," Version 1.02, 2022.
- [23] STMicroelectronics, "LIS3DSH - MEMS Digital Output Motion Sensor Ultra-Low-Power High-Performance 3-Axis 'Nano' Accelerometer Datasheet," DocID 17530, Rev. 3, 2021. [Online]. Available: <https://www.st.com/resource/en/datasheet/lis3dh.pdf>

- [24] STMicroelectronics, "LIS3DSH Application Note AN3308," Rev. 2, 2020.
- [25] OASIS, "MQTT Version 5.0 - OASIS Standard," Mar. 2019. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [26] SAE International, "SAE J1979 - E/E Diagnostic Test Modes," Standard, Rev. 2014.
- [27] International Organization for Standardization, "ISO 15031–5:2015 - Road vehicles - Communication between vehicle and external equipment for emissions-related diagnostics," ISO Standard, 2015.
- [28] International Organization for Standardization, "ISO 15765–2:2016 - Road vehicles - Diagnostic communication over Controller Area Network (DoCAN) - Part 2: Transport protocol and network layer services," ISO Standard, 2016.
- [29] vgate, "iCar Pro BLE OBD2 Adapter Specifications," 2023. [Online]. Available: <https://www.vgatemall.com/product/vgate-icar-pro.html>
- [30] NanJing Top Power ASIC Corp., "TP4056 Standalone Linear Li-Ion Battery Charger with Thermal Regulation," Datasheet, Rev. 1.0.
- [31] Apache NimBLE Project, "NimBLE Host API Reference," Version 1.5, 2023. [Online]. Available: <https://mynewt.apache.org/latest/network/>

[D] Tài liệu trực tuyến (Online Resources)

- [32] EMQX Team, "EMQX Documentation - The World's Most Scalable MQTT Broker," Version 5.x, 2024. [Online]. Available: <https://docs.emqx.com/en/emqx/latest/>. [Accessed: Jan. 15, 2026].
- [33] The PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," 2024. [Online]. Available: <https://www.postgresql.org/docs/16/>. [Accessed: Jan. 10, 2026].
- [34] VictoriaMetrics Team, "VictoriaMetrics Documentation," 2024. [Online]. Available: <https://docs.victoriametrics.com/>. [Accessed: Jan. 10, 2026].
- [35] VictoriaMetrics Team, "VictoriaLogs Documentation," 2024. [Online]. Available: <https://docs.victoriametrics.com/victorialogs/>. [Accessed: Jan. 10, 2026].
- [36] OpenJS Foundation, "Express.js - Fast, Unopinionated, Minimalist Web Framework for Node.js," 2024. [Online]. Available: <https://expressjs.com/>. [Accessed: Dec. 20, 2025].
- [37] Vercel Inc., "Next.js 15 Documentation," 2024. [Online]. Available: <https://nextjs.org/docs>. [Accessed: Dec. 15, 2025].
- [38] Meta Platforms, Inc., "React 19 Documentation," 2024. [Online]. Available: <https://react.dev/>

//react.dev/. [Accessed: Dec. 15, 2025].

[39] Socket.IO Contributors, "Socket.IO Documentation," Version 4.8, 2024. [Online]. Available: <https://socket.io/docs/v4/>. [Accessed: Dec. 20, 2025].

[40] V. Agafonkin, "Leaflet - An Open-Source JavaScript Library for Mobile-Friendly Interactive Maps," Version 1.9, 2024. [Online]. Available: <https://leafletjs.com/>. [Accessed: Jan. 05, 2026].

[41] Apache Software Foundation, "Apache ECharts - An Open Source JavaScript Visualization Library," Version 5.5, 2024. [Online]. Available: <https://echarts.apache.org/>. [Accessed: Jan. 05, 2026].

[42] Docker Inc., "Docker Documentation," 2024. [Online]. Available: <https://docs.docker.com/>. [Accessed: Nov. 15, 2025].

[43] Docker Inc., "Docker Compose Documentation," 2024. [Online]. Available: <https://docs.docker.com/compose/>. [Accessed: Nov. 15, 2025].

[44] Tailwind Labs, "Tailwind CSS Documentation," Version 4, 2025. [Online]. Available: <https://tailwindcss.com/docs>. [Accessed: Jan. 10, 2026].

[45] Daishi Kato, "Zustand - Bear Necessities for State Management in React," Version 5, 2024. [Online]. Available: <https://zustand-demo.pmnd.rs/>. [Accessed: Dec. 20, 2025].

[46] TanStack, "TanStack Query (React Query) Documentation," Version 5, 2024. [Online]. Available: <https://tanstack.com/query/latest>. [Accessed: Dec. 20, 2025].

[47] Espressif Systems, "ESP-IDF Programming Guide," Version 5.x, 2024. [Online]. Available: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/>. [Accessed: Oct. 10, 2025].

[48] Grafana Labs, "Grafana Documentation," 2024. [Online]. Available: <https://grafana.com/docs/grafana/latest/>. [Accessed: Jan. 15, 2026].

[49] Prometheus Authors, "Prometheus Documentation," 2024. [Online]. Available: <https://prometheus.io/docs/>. [Accessed: Jan. 15, 2026].

[50] Radix UI Contributors, "Radix UI - Unstyled, Accessible Components for React," 2024. [Online]. Available: <https://www.radix-ui.com/>. [Accessed: Dec. 15, 2025].

[51] Google, "Flutter Documentation - WebView Plugin," 2024. [Online]. Available: <https://docs.flutter.dev/>. [Accessed: Jan. 20, 2026].

[52] Texas Instruments, "Power Management for IoT Devices," Application Report SLVAE57, 2020. [Online]. Available: <https://www.ti.com/lit/an/slvae57/slvae57.pdf>. [Accessed: Sep. 15, 2025].

- [53] STMicroelectronics, "STM32L476RE Product Page," 2026. [Online]. Available: <https://www.st.com/en/microcontrollers-microprocessors/stm32l476re.html>. [Accessed: Mar. 02, 2026].
- [54] STMicroelectronics, "STM32L476xx Datasheet," DS10198 Rev 11, Jul. 2024. [Online]. Available: <https://www.st.com/resource/en/datasheet/stm32l476re.pdf>. [Accessed: Mar. 02, 2026].
- [55] Nordic Semiconductor, "nRF52840 Product Specification v1.2," [Online]. Available: https://docs.nordicsemi.com/bundle/nRF52840_PS_v1.2/resource/nRF52840_PS_v1.2.pdf. [Accessed: Mar. 02, 2026].
- [56] Nordic Semiconductor, "nRF52840 Product Page," 2026. [Online]. Available: <https://www.nordicsemi.com/Products/nRF52840/Modules>. [Accessed: Mar. 02, 2026].
- [57] Nordic Semiconductor, "nRF52840 Documentation Hub," 2026. [Online]. Available: <https://docs.nordicsemi.com/category/nrf52840-category>. [Accessed: Mar. 02, 2026].
- [58] SIMCom Wireless Solutions, "A7670X Series Product Page," 2026. [Online]. Available: <https://cn.simcom.com/product/A7670X.html>. [Accessed: Mar. 02, 2026].
- [59] Quectel Wireless Solutions, "EC200U Series Product Page," 2026. [Online]. Available: <https://www.quectel.com/product/lte-ec200u-series>. [Accessed: Mar. 02, 2026].
- [60] SIMCom Wireless Solutions, "SIM7600CE Product Page," 2026. [Online]. Available: <https://en.simcom.com/product/SIM7600CE.html>. [Accessed: Mar. 13, 2026].
- [61] Quectel Wireless Solutions, "Quectel EC200U Series LTE Standard Specification," Version 1.4, 2024. [Online]. Available: https://developer.quectel.com/en/wp-content/uploads/sites/2/2024/11/Quectel_EC200U_Series_LTE_Standard_Specification_V1.4.pdf. [Accessed: Mar. 02, 2026].
- [62] u-blox, "NEO-M8 Data Sheet," UBX-15031086, [Online]. Available: https://www.u-blox.com/sites/default/files/NEO-M8-FW3_DataSheet_UBX-15031086.pdf. [Accessed: Mar. 02, 2026].
- [63] u-blox, "NEO-M8 Series Product Page," 2026. [Online]. Available: <https://www.u-blox.com/en/product/neo-m8-series>. [Accessed: Mar. 02, 2026].
- [64] vgate, "Vgate iCar Pro BLE Product Page," 2026. [Online]. Available: <https://vgatemall.com/products-detail/i-9/>. [Accessed: Mar. 02, 2026].
- [65] Texas Instruments, "TPS2115A Product Page," 2026. [Online]. Available: <https://www.ti.com/product/TPS2115A>. [Accessed: Mar. 02, 2026].
- [66] Infineon Technologies, "IRLML6402 Product Page," 2026. [Online]. Available: <https://www.infineon.com/part/IRLML6402>. [Accessed: Mar. 02, 2026].

[67] SONGLE Relay, "SRD-05VDC-SL-C Relay Datasheet," datasheet mirror. [Online]. Available: <https://www.alldatasheet.com/html-pdf/1132639/SONGLERELAY/SRD-05VDC-SL-C/1713/2/SRD-05VDC-SL-C.html>. [Accessed: Mar. 02, 2026].

PHỤ LỤC - APPENDICES

PHỤ LỤC 1: BÁO CÁO TÀI CHÍNH - FINANCE REPORT

1.1. Bảng kê chi phí linh kiện (Bill of Materials - BOM)

Phụ lục này giữ lại phần thông tin tài chính thiết yếu thay cho bảng kê rất dài của bản gốc. Mức chi phí tập trung vào các nhóm cấu thành giá của một thiết bị tracker mẫu.

Nhóm linh kiện	Chi phí ước tính	Ghi chú
Khối xử lý và truyền thông	700.000–900.000 VND	ESP32-S3, modem LTE/GNSS, anten và mạch phụ trợ
Khối nguồn và dự phòng	350.000–450.000 VND	Buck, LDO, sạc pin, pin dự phòng, linh kiện bảo vệ
Cảm biến và kết nối ngoài	150.000–250.000 VND	IMU, đầu nối, phụ kiện lắp đặt và hàn lắp
PCB/chế tạo mẫu	250.000–400.000 VND	Phụ thuộc số lượng mẫu và mức hoàn thiện

1.2. Chi phí hạ tầng cloud (ước tính hàng tháng)

Ở giai đoạn đồ án, chi phí cloud phụ thuộc chủ yếu vào VPS, kết nối mạng và lưu trữ dữ liệu, trong khi chi phí phần mềm được giảm mạnh nhờ dùng công nghệ mã nguồn mở. Điều này làm cho phương án triển khai thử nghiệm tương đối khả thi với ngân sách nhỏ.

Hạng mục	Thông số rút gọn	Chi phí ước tính
VPS server	4 vCPU, 8GB RAM, 100GB SSD	300.000–500.000 VND/tháng
SIM 4G dữ liệu	1 gói data cho thiết bị thử nghiệm	70.000 VND/tháng
Tên miền	1 domain phục vụ truy cập hệ thống	25.000 VND/tháng quy đổi
Chứng chỉ TLS	Let's Encrypt	0 VND
Tổng hàng tháng		395.000–595.000 VND

1.3. Tổng hợp chi phí dự án

Tổng hợp lại, chi phí của đồ án tập trung vào phần cứng mẫu, thời gian chế tạo và hạ tầng thử nghiệm. Đây là cấu trúc chi phí phù hợp với một đề tài phát triển nguyên mẫu hệ thống hơn là một dự án thương mại hóa ngay từ đầu.

PHỤ LỤC 2: CÁC TIÊU CHUẨN THIẾT KẾ - STANDARDS

2.1. Tiêu chuẩn OBD2 (SAE J1979 / ISO 15031-5)

Tiêu chuẩn OBD2 được dùng làm cơ sở để lựa chọn nhóm PID đọc từ xe, giới hạn phạm vi truy cập ở mức *read-only* và bảo đảm rằng giải pháp không can thiệp vào điều khiển vận hành của phương tiện.

2.2. Tiêu chuẩn MQTT 5.0 (OASIS)

MQTT 5.0 là nền tảng cho thiết kế topic, QoS, trạng thái kết nối và mô hình truyền dữ liệu nhẹ giữa thiết bị và server. Việc bám theo chuẩn này giúp hệ thống dễ mở rộng và tương thích tốt hơn với hạ tầng broker hiện đại.

2.3. Bảo mật thông tin (ISO 27001 - tham khảo)

ISO 27001 không được hiện thực đầy đủ ở mức chứng chỉ trong phạm vi đồ án, nhưng các nguyên tắc cơ bản như phân quyền, quản lý bí mật, logging và kiểm soát truy cập vẫn được tham chiếu khi thiết kế giải pháp cloud/backend.

2.4. Thiết kế REST API (RFC 7231 và best practices)

Thiết kế REST API của hệ thống dựa trên hướng chuẩn hóa tài nguyên, mã trạng thái HTTP, phân quyền theo phiên và phản hồi thống nhất. Điều này giúp lớp frontend và các công cụ tích hợp làm việc dễ dàng hơn với backend.

PHỤ LỤC 3: KẾ HOẠCH THỰC HIỆN - ASSIGNMENT AND TIMELINES

3.1. Phân chia giai đoạn dự án

Kế hoạch thực hiện được chia thành các giai đoạn: khảo sát yêu cầu, thiết kế phần cứng và firmware, xây dựng cloud/backend, xây dựng dashboard, tích hợp hệ thống và đo kiểm. Cách chia này phản ánh đúng đặc thù của một đồ án nhiều tầng công nghệ.

3.2. Biểu đồ Gantt (Gantt Chart)

Giai đoạn	Thời lượng	Kết quả chính
Khảo sát và chốt kiến trúc	Tuần 1–4	Yêu cầu kỹ thuật, sơ đồ hệ thống, hướng chọn công nghệ
Thiết kế thiết bị và firmware	Tuần 5–10	Sơ đồ phần cứng, nguồn, giao tiếp OBD2/GNSS, máy trạng thái
Cloud, back-end, frontend	Tuần 11–16	MQTT, API, realtime, dashboard quản lý
Tích hợp và đo kiểm	Tuần 17–22	Kiểm thử toàn tuyến, đánh giá và hoàn thiện báo cáo

3.3. Chi tiết công việc theo giai đoạn

Về bản chất, tiến độ của đồ án không đi theo kiểu tuần tự cứng mà có nhiều vòng lặp giữa thiết kế, hiện thực và đo kiểm. Bài học rút ra từ kế hoạch này là phải tích hợp sớm giữa các tầng, nếu không thời gian cuối kỳ sẽ bị dồn quá mạnh vào sửa lỗi tích hợp.

PHỤ LỤC 4: TÀI LIỆU BỔ TRỢ - SUPPORTING MATERIALS

4.1. Mã nguồn dự án (Source Code Repository)

Mã nguồn được quản lý theo monorepo, tách rõ firmware, backend, frontend, MQTT bridge và các thành phần hạ tầng. Phần cần giữ từ bản full là cấu trúc kho mã nguồn đã đủ rõ để người đọc lần theo từng lớp của hệ thống: các dịch vụ runtime nằm trong `iot-vehicle-tracking-system-cloud/`, còn báo cáo, kế hoạch và tài liệu nằm trong `resources/`.

Cách tổ chức này hỗ trợ tốt cho việc phát triển đồng thời nhiều lớp công nghệ trong cùng một dự án và cũng là cơ sở để người khác tái dựng lại hệ thống từ tài liệu đi kèm.

4.2. Tham chiếu cơ sở dữ liệu (Database Schema)

Phần hỗ trợ cơ sở dữ liệu của đồ án gồm schema quan hệ cho nghiệp vụ quản lý xe/người dùng/cảnh báo và lớp telemetry chuỗi thời gian cho vị trí, tốc độ, RPM và các đại lượng vận hành khác. Trong bản full, các file SQL khởi tạo được đặt dưới `iot-vehicle-tracking-system-cloud/Tracking_PostgreSQL/init/`; bản rút gọn chỉ giữ lại thông tin đủ để người đọc biết nơi tra cứu và phạm vi dữ liệu chính.

Tài liệu đầy đủ được giữ ở kho mã nguồn và tài liệu thiết kế hệ thống, nên phụ lục này đóng vai trò điểm dẫn hướng thay vì lặp lại toàn bộ schema.

4.3. Tài liệu API (API Documentation)

API tài liệu hóa các nhóm chức năng chính: xác thực, quản lý phương tiện, dữ liệu telemetry, cảnh báo, geofence và lệnh điều khiển. Trong phạm vi bản rút gọn, mục này chỉ khẳng định rằng tài liệu API đã tồn tại như một phần của hạ tầng backend, chủ yếu được truy cập qua lớp backend và tài liệu OpenAPI tương ứng, chứ không liệt kê lại toàn bộ endpoint.

4.4. Tài liệu cấu hình môi trường (Environment Configuration)

Cấu hình môi trường được tổ chức riêng cho firmware, backend, frontend, bridge và các dịch vụ hạ tầng, thường đi kèm các file `.env` hoặc `.env.example` theo từng dịch vụ. Việc tách cấu hình theo dịch vụ là yêu cầu cần thiết để kiểm soát secrets, môi trường thử nghiệm và môi trường triển khai thực tế.

4.5. Hướng dẫn cài đặt và chạy hệ thống

Hướng dẫn cài đặt và chạy hệ thống được xây dựng theo trình tự: chuẩn bị môi trường, tạo mạng Docker dùng chung, khởi động hạ tầng, chạy backend/frontend/bridge và xác nhận các endpoint chính. Điều quan trọng là toàn bộ tài liệu hỗ trợ đã đủ để người khác có thể đọc, dựng và kiểm tra lại hệ thống mà không cần dựa hoàn toàn vào mô tả trong

bản báo cáo chính.